

Notes for 3611: Programming in Quantitative Economics

C. Dolleris

2025-05-29

Table of contents

Preface	3
I Lectures	4
1 Introduction to the Course	5
1.1 Standard functions in R	5
Small Exercise 1	5
Small Exercise 2	7
1.2 External packages in R	7
1.3 Types of variables	7
1.3.1 Vectors, Matrices, Arrays, Data frames	8
1.3.2 Access elements	8
1.4 How to get help	9
Small Exercise 3	9
Small Exercise 4	11
1.5 Additional code in R	11
2 Getting Started with R	14
2.1 Working directory	14
2.2 Arithmetic	16
2.3 Variable types	16
2.3.1 Defining variables	16
2.3.2 Variable types and syntax	17
2.3.3 Variable names and Hungarian notation	17
2.3.4 Integers	18
2.3.5 Doubles	18
2.3.6 Precision	18
2.4 Built-in	19
2.4.1 Character/strings	19
2.4.2 Built-in functions	19
2.4.3 Functions	20
2.4.4 Arguments of functions	20
2.5 Vectors	21
2.5.1 Vector operations	21

2.6	Missing values	22
2.6.1	Missing date: NA	22
2.6.2	Searching for NAs	23
2.7	Small Exercise	23
2.8	Logical	26
2.8.1	Logical expression	26
2.8.2	Logical expressions: & and 	27
2.8.3	Sequential && and 	27
2.9	Matrices	28
2.9.1	Multiplication with matrices	29
2.10	Factors	30
2.11	Lists	30
2.12	Dataframes	32
2.12.1	Dataframes: Extract	32
2.12.2	Dataframes: Assign	33
2.13	Small exercises with logical operators, matrices, lists and dataframes	33
2.14	Additional code in R	35
3	Basic Programming	38
3.1	The if statement	38
3.1.1	Small Exercise	39
3.1.2	Nested if statements	40
3.1.3	An extension to the exercise solution above	40
3.2	The for loop	41
3.2.1	Exercise: a combination of for loops and if conditions	42
3.3	The while loop	42
3.3.1	Exercise with while loops	43
3.4	Vector-based programming	45
3.4.1	Example	45
3.5	Load data in R	46
3.5.1	Load MAERSK historical prices	47
3.5.2	Descriptive statistics of the MAERSK log-returns	47
3.5.3	Output to a file	47
3.6	Plotting	48
3.6.1	Basic arguments for plot	50
3.6.2	Example:	50
3.6.3	Exercise	51
3.7	Additional code in R	52
4	Programming with Functions	55
4.1	Definition of functions in R	55
4.1.1	Running a Function	56
4.1.2	Examples	56

4.1.3	Notes on Functions	57
4.1.4	Small Exercises	57
4.1.5	Example: roots of a quadratic function	58
4.1.6	Advantages of coding with functions	60
4.1.7	Variables inside a function	60
4.1.8	The scope of a variable	61
4.2	Arguments of a function	61
4.2.1	Default arguments	62
4.2.2	Default arguments: Partial matching	63
4.3	Ellipsis	64
4.4	Vector-based programming using functions	64
4.4.1	<code>sapply</code>	65
4.4.2	<code>sapply</code> and <code>lapply</code>	66
4.4.3	<code>mapply</code>	66
4.4.4	<code>apply</code>	67
4.4.5	Exercise	68
4.4.6	Recursive programming	69
4.4.7	Multiple outputs	69
4.5	Comments	70
4.5.1	Exercise	71
5	Program Efficiency and Parallel Computing	72
5.1	Good practice for programming	72
5.1.1	Programming in theory - plan ahead	72
5.1.2	Closer to practice:	72
5.1.3	Use of Functions and computational efficiency	73
5.2	Code execution time	73
5.3	Loops versus vectors	74
5.3.1	But... If you have to use a loop in R...	75
5.3.2	The flexibility vs speed trade-off	75
5.3.3	Arithmetic is not always the fastest	76
5.3.4	Vectorization: Tips and Tricks for Speed	76
5.3.5	Summing the columns of a matrix	76
5.3.6	Comparison	78
5.3.7	Vectorized if/else statement	78
5.3.8	Subsetting/subscripting: One of the most powerful tools!	80
5.3.9	Exercises in vectorization	81
5.4	Parallel processing	82
5.4.1	Warning before parallellization	82
5.4.2	Parallel Processing	82
5.4.3	Creating a cluster	83
5.4.4	Running jobs in parallel	83
5.4.5	Kill your cluster	84

5.4.6	Small exercises on the parallel universe	84
5.5	Memory issues	85
5.6	Small hints	86
6	Introduction to Simulation	87
6.1	Seeding	87
6.2	Discrete Random variables	88
6.2.1	Simulate Discrete Random Variables	88
6.3	Inversion	91
6.3.1	Inversion Method	91
6.3.2	Small Exercise	94
6.4	Acceptance-Rejection	95
6.5	Simulate Normals	99
6.5.1	Simulate Normals: Using uniformly distributed random variable	99
6.5.2	Simulate Normals: Acceptance-Rejection with Exponential Envelope	101
6.5.3	Simulate Normals: Box-Muller Algorithm	102
6.6	Monte Carlo Integration	104
6.6.1	Monte Carlo Integration - a brief introduction	104
6.6.2	Monte Carlo Integration - the code	105
6.6.3	Monte Carlo Integration - examples	105
6.6.4	Small Exercise	105
7	Root-finding	107
7.1	Main problem	107
7.1.1	The Roots of a Function	108
7.1.2	Example: Loan repayments	108
7.2	Numerical Methods for Root-finding	110
7.3	The Newton-Raphson Method	110
7.3.1	Notes for the Newton-Raphson method	111
7.3.2	The Newton-Raphson algorithm	112
7.3.3	Drawbacks of the Newton-Raphson method	112
7.3.4	Loan example	113
7.4	The Secant method	115
7.5	The bisection method	116
7.5.1	The bisection method: Implementation in R	118
7.6	The <code>uniroot</code> function	119
7.6.1	Example for the loan repayment function	120
7.7	Numerical Derivatives in R	120
7.7.1	Warnings	121
8	Numerical Optimization 1	122
8.1	Numerical vs Analytical Optimization	122
8.1.1	Optimization and Root-finding	122

8.1.2	Maximum: Definition	124
8.1.3	Numerical optimization: local search techniques	124
8.1.4	Numerical optimization: general structure	124
8.1.5	Stopping criteria	125
8.2	The Newton-Raphson Method - and Method for Optimization.	125
8.2.1	Newton's Method for Optimization	126
8.2.2	Newton's algorithm convergence	126
8.2.3	Newton's method: Example	127
8.2.4	Newton's method: implementation in R	129
8.2.5	Exercise	135
8.3	The Golden-section method	138
8.3.1	The idea of the Golden-section method	138
8.3.2	The Golden ratio	139
8.4	New algorithm	140
8.4.1	Notes	140
8.4.2	The golden-section method: R implementation	141
8.5	Optimization in R: univariate	142
9	Numerical Optimization 2	144
9.1	Multivariate Optimization	144
9.2	Steepest ascent method	145
9.2.1	Steepest ascent method: Implementation in R	145
9.2.2	Line search	146
9.2.3	Line-search: Implementation in R	146
9.2.4	Example	149
9.3	Newton's method in higher dimensions	151
9.3.1	Newton's method: Implementation in R	152
9.3.2	Example	152
9.4	Disadvantages of steepest ascent and Newton's method	156
9.5	Gradient and Hessian in R	156
9.6	Optimization in R: multivariate	156
9.6.1	Example	157
10	Constrained Optimization and Pitfalls in Optimization	158
10.1	Constrained Optimization	158
10.2	Penalization functions	159
10.3	Barrier functions - (Interior-point)	159
10.4	Parameter constraints	160
10.5	Reparameterization	161
10.6	Differentiation after reparameterization: Delta method	161
10.7	Transforming parameters	162
10.8	Transform: Common transformations	162
10.9	Code for constrained optimization	162

10.10	Pitfalls	173
10.11	Problematic Hessian?	176
10.12	Recap of Newton's method in higher dimensions	176
10.13	Broyden, Fletcher, Goldfarb and Shanno (BFGS)	177
10.14	Example: Regression Likelihood	178
10.15	optim continued	179
10.16	Code for pitfalls and BFGS	179
11	Integration of C++ Code with Rcpp	182
11.1	Introduction	182
11.2	Rcpp	182
11.3	R versus C++	182
11.4	Basic C++ in R	183
11.5	C++ functions in R	183
11.5.1	Example: No inputs, scalar output	184
11.6	Variables declation	184
11.6.1	Example: scalar input, scalar output	185
11.7	For loops in C++	186
11.7.1	Example: vector input, scalar output	186
11.7.2	Efficiency of loops: C++ vs. R	187
11.7.3	Example: vector input, vector output	187
11.7.4	Example: matrix input, vector output	188
11.8	Using sourceCpp	189
11.9	The Fibonacci sequence	191
11.9.1	Fibonacci in R	191
11.9.2	Fibonacci in C++	192
11.9.3	Comparing fibR and fibC	192
11.10	Vector, Matrices and Linear Algebra in C++	192
11.10.1	Example	193
11.11	Inverse of Matrix Product	194
11.12	Calling R Functions from C++	195
11.12.1	Warning	196
11.13	The Rcpp Family	197
11.14	A simple model for daily financial returns	197
11.15	Estimation of the GARCH(1,1) model	198
11.15.1	GARCH practical issues	198
11.16	Simulation of GARCH(1,1) in C++	198
12	Creating R packages with RStudio	202
12.1	Introduction	202
12.2	Why a package?	202
12.3	Setting up	203
12.4	Code in Packages	203

12.5	Object documentation	205
12.5.1	Documentation for -package	205
12.6	Description	207
12.7	Different types of R packages	208
12.8	Beyond the basics	208
12.9	Coding Style and Swirl	208
II	Problem sets	210
13	Exercise set 1	211
13.1	(1)	211
13.2	(2)	212
13.3	(3)	212
13.4	(4)	213
13.5	(5)	214
13.6	(6)	216
14	Exercise set 2	217
14.1	(1)	217
14.2	(2)	218
14.3	(3)	219
14.4	(4)	219
14.5	(5)	220
14.6	(6)	221
14.7	(7)	221
14.8	(8)	222
14.9	(9)	222
14.10	(10)	222
14.11	(11)	223
15	Exercise set 3	225
15.1	(1)	225
15.2	(2)	225
15.3	(3)	229
15.4	(4)	230
15.5	(5)	231
15.6	(6)	231
16	Exercise set 4	233
16.1	(1)	233
16.2	(2)	235

17 Exercise set 5	240
17.1 (1)	240
17.2 (2)	242
18 Exercise set 6	244
19 Ordinary exam 2020	248
19.1 Problem 1:	248
19.2 Problem 2:	253
20 Ordinary exam 2021	259
20.1 Problem 1:	259
20.2 Problem 2: (Constrained) Optimization, C++, and Packaging	264
21 Retake exam 2021	271
21.1 Problem 1:	271
21.2 Problem 2: (Constrained) Optimization, C++, and Packaging	275
22 Ordinary exam 2022	283
22.1 Problem 1:	283
22.2 Problem 2: (Constrained) Optimization, C++, and Packaging	286
23 Retake exam 2022	293
23.1 Problem 1:	293
23.2 Problem 2: (Constrained) Optimization, C++, and Packaging	298
24 Ordinary exam 2023	308
24.1 Problem 1:	308
24.2 Problem 2: (Constrained) Optimization, C++, and Packaging	310
25 Retake exam 2023	317
25.1 Problem 1:	317
25.2 Problem 2: (Constrained) Optimization, C++, and Packaging	320
26 Ordinary exam 2024	327
26.1 Problem 1	327
26.1.1 Problem 1.1	327
26.1.2 Problem 1.2	328
26.1.3 Problem 1.3	329
26.1.4 Problem 1.4	330
26.2 Problem 2: (Constrained) Optimization, C++, and Packaging	332
26.2.1 Problem 2.1	332
26.2.2 Problem 2.2	333
26.2.3 Problem 2.3	334

26.2.4	Problem 2.4	336
26.2.5	Problem 2.5	337
26.2.6	Problem 2.6	338
27	Retake exam 2024	339
27.1	Problem 1	339
27.1.1	Problem 1.1	339
27.1.2	Problem 1.2	340
27.1.3	Problem 1.3	340
27.1.4	Problem 1.4	341
27.2	Problem 2: (Constrained) Optimization, C++, and Packaging	342
27.2.1	Problem 2.1	343
27.2.2	Problem 2.2	344
27.2.3	Problem 2.3	348
27.2.4	Problem 2.4	350
27.2.5	Problem 2.5	350
28	Extra exercises	352
29	Exercise Set 1: R Basics, Data Structures, and Control Flow	353
29.1	(1)	353
29.2	(2)	353
29.3	(3)	354
29.4	(4)	354
29.5	(5)	355
29.6	(6)	355
29.7	(7)	356
29.8	(8)	356
29.9	(9)	357
29.10	(10)	357
30	Exercise Set 2: Functions, Scope, and Efficiency	359
30.1	(1)	359
30.2	(2)	360
30.3	(3)	360
30.4	(4)	361
30.5	(5)	361
30.6	(6)	362
30.7	(7)	363
30.8	(8)	363
30.9	(9)	364
30.10	(10)	364

31 Exercise Set 3: Simulation	366
31.1 (1)	366
31.2 (2)	367
31.3 (3)	368
31.4 (4)	370
31.5 (5)	373
31.6 (6)	373
31.7 (7)	375
32 Exercise Set 4: Root-Finding	377
32.1 (1)	377
32.2 (2)	379
32.3 (3)	380
32.4 (4)	383
32.5 (5)	384
32.6 (6)	385
33 Exercise Set 5: Numerical Optimization (Univariate and Multivariate)	388
33.1 (1)	388
33.2 (2)	390
33.3 (3)	392
33.4 (4)	395
33.5 (5)	397
33.6 (6)	397
33.7 (7)	399
33.8 (8)	399
34 Exercise Set 6: C++ with Rcpp and RcppArmadillo	401
34.1 (1)	401
34.2 (2)	402
34.3 (3)	403
34.4 (4)	404
34.5 (5)	406
34.6 (6)	407
35 Exercise Set 7: Creating R Packages	408
35.1 (1)	408
35.2 (2)	408
35.3 (3)	409
35.4 (4)	409
35.5 (5)	410

36 Exercise Set 8: Parallel Computing	411
36.1 (1)	411
36.2 (2)	412
36.3 (3)	413
36.4 (4)	414
37 Advanced exercises	415
38 Exercise Set 9: Advanced R, Data Manipulation, and Debugging	416
38.1 (1)	416
38.2 (2)	416
38.3 (3)	417
38.4 (4)	418
38.5 (5)	418
39 Exercise Set 10: Advanced C++ and Package Development	420
39.1 (1)	420
39.2 (2)	421
39.3 (3)	421
39.4 (4)	422
40 Exercise Set 11: Advanced Simulation and Optimization	423
40.1 (1)	423
40.2 (2)	424
40.3 (3)	425
41 Exercise Set 12: Exam-Style Problems and Combinations	427
41.1 (1)	427
41.2 (2)	428
41.3 (3)	430
42 Extra extra exercises	432
43 Exercise Set 13: R Fundamentals and Data Structures (Review)	433
43.1 (1)	433
43.2 (2)	433
43.3 (3)	434
43.4 (4)	435
43.5 (5)	435
43.6 (6)	436
43.7 (7)	436
43.8 (8)	437

44 Exercise Set 14: Functions, Scope, and Vectorization Focus	438
44.1 (1)	438
44.2 (2)	438
44.3 (3)	439
44.4 (4)	440
44.5 (5)	440
44.6 (6)	441
44.7 (7)	441
44.8 (8)	442
45 Exercise Set 15: Likelihood Functions and Optimization (Medium/Hard)	443
45.1 (1) Likelihood of a Poisson Process	443
45.2 (2) Optimization of a Bimodal Likelihood (Reparameterization) . . .	445
45.3 (3) Likelihood for Linear Regression with Known Variance	447
45.4 (4) Optimization with Constraints using constrOptim (or Penalty) . .	448
45.5 (5) Profile Likelihood	448
46 Likelihood Exercises	451
47 Exercise Set 16: Advanced Econometric Likelihoods and Optimization	452
47.1 (1) Probit Model Likelihood (Medium-Hard)	452
47.2 (2) GARCH(1,1) Model Likelihood (Hard)	455
47.3 (3) Heckman Selection Model Likelihood (Extremely Hard)	458
47.4 (4) Vector Autoregression (VAR) Likelihood (Multivariate Normal) (Hard)	461
48 Exercise Set 17: Integrated Econometric Modeling Problems	464
48.1 (1) Logistic Regression: Full Implementation (Hard)	464
48.2 (2) AR(1) Model with Student's t-errors (Extremely Hard)	469
48.3 (3) Zero-Inflated Poisson (ZIP) Regression Likelihood (Extremely Hard)	473

Preface

This is a collection of notes for the course “**Programming in Quantitative Economics**”.

Part I

Lectures

1 Introduction to the Course

1.1 Standard functions in R

R comes with a lot of standard functions. We can print stuff to the screen, invert matrices, perform OLS, generate random numbers, etc., so there is no need to reinvent the wheel every time!

These functions are also available in R packages. However, these are ‘special’ packages that are included by default in R and don’t need to be loaded. Examples are ‘stats’, ‘base’, ‘graphics’,...

These functions generally have sensible names:

- `print` to print something to the screen.
- `plot` to plot figures.
- `solve` to invert a matrix.
- `lm` to perform OLS (‘linear model’ → ‘lm’)
- `sum`, `median`, `mean`,...

Obviously, there are too many to list here. R manuals are freely available online <https://cran.r-project.org/>

Small Exercise 1

- *Open and run the .R file called `Becoming_a_userR`.*

 Click to view “Becoming_a_useR.R”

```
1 # 1. Add and remove elements from the environment
2 # Clear the environment: Type ctrl+L for clearing the console
3 rm(list = ls())
4 Sys.setenv(lang = "en_US")
5 # Define an integer
6 iN <- 8
7
8 # Remove the object from the environment
9 rm(iN)
10
11 # Reintroduce the integer
12 iN <- 8
13
14 # 2. There is difference between "=" and "<-" in R. Let's discuss an example
15 # A matrix with zero elements can be generated by a syntax: matrix(1, ncol = Nc, nrow = Nr)
16
17 mE <- matrix(0, ncol = 2) #Saying to R the number of columns is 2
18 ncol #Comment
19 mR <- matrix(0, ncol <- 2) #Saying to R the number of ?rowss is 2: 2 is assigned to ncol a
20 ncol #Comment
21 mE # Print matrix E
22 mR # Print matrix R
23
24 # 3. Some error messages
25 # changing the value of iN
26 iN <- 10
27 cat("... but now it is ", iN, "\n")
28 cat("What am I printing now? ", iN==10, "\n") # Comment
29 cat("Now I get an error... ", iS, "\n") # Why?
```

- Can you understand what each line of the code does?
- What is the output?
- What are the differences in the matrices defined in line 17-19?
 - The matrix `mE` is defined properly with 2 columns. The matrix `mR` has 2 rows.
- Try to run lines 27-29. Can you comment the output?
 - The error is received since the variable `iS` is not defined.

Small Exercise 2

- Open and run the .R file called `Exercise_error`.

 Click to view “Exercise_error.R”

```
1 # 0. Clear the environment: CTRL + L to clear the Console
2 rm(list=ls())
3 Sys.setenv(lang = "en_US")
4 # Define vector X
5 vX <- c(-1,2)
6
7 # Why did the following output produce NAN output?
8 cat("This is square root of X: ", sqrt(vX), "\n")
9 # Why?
```

- Why do you get an error message?
 - There is no error message. There is a warning message.
- Which output do you get from $\sqrt{-1}$?
 - Naturally, you get an error as the square root of a negative number isn't possible.

1.2 External packages in R

More than 12,000 external packages are available on CRAN: <https://cran.r-project.org/web/packages/>.

Knowing the name of a package, say ("rugarch"), this can be installed in R with `install.packages("rugarch")`.

After a package has been installed, in order to load it into the R environment, we need to run: `library("rugarch")`.

1.3 Types of variables

The function `class()` allows you to find the class of an R object. For instance:

```
cfoo = "sun"
class(cfoo)
#> [1] "character"
```

```
bfoo = TRUE
class(bfoo)
#> [1] "logical"
```

```
dfoo = 1.785632458
class(dfoo)
#> [1] "numeric"
```

The “foo” terminology stands for a variable which is not important, i.e., a variable where intermediate results are stored. We will later see that “c”, “b” , and “d” before the “foo” actually mean something!

1.3.1 Vectors, Matrices, Arrays, Data frames

Data can be organized in different ways in R depending on the particular needs. If we plan to do linear algebra, we want to use:

```
vY = c(1, 2, 5.48652) # vectors
mY = matrix(c(1, 2, 3, 4, 5, 6, 7, 8, 9.485), ncol = 3) # matrices
aY = array(1:27, dim = c(3, 3, 3)) # arrays
```

If we want to organize data (numeric/character or mixed), we use:

```
d = data.frame(x = 1, y = 1:10, fac = LETTERS[1:10]) # data frame
```

Vectors, matrices and arrays can be: “numeric” or “character” i.e., they can contain only one of the two types of variables (we cannot have a vector with some elements “numeric” and others “character”). Data.frames can contain both.

1.3.2 Access elements

To access elements of vectors, matrices and arrays, we use the square brackets:

```
1 vY[1]
2 #> [1] 1
3 vY[1:2]
4 #> [1] 1 2
5 vY[c(3,1)]
6 #> [1] 5.48652 1.00000
```

```

7 mY[1, 1]
8 #> [1] 1
9 mY[1, ]
10 #> [1] 1 4 7
11 mY[1, c(1,3)]
12 #> [1] 1 7
13 aY[1, 1, 1]
14 #> [1] 1
15 aY[1, 1, ]
16 #> [1] 1 10 19

```

1.4 How to get help

When you are in trouble the `help()` function is your friend! Whenever you want to understand the functioning of any function, say `list`, you type:

```
help(list)
```

This is equivalent to:

```
?list
```

If you don't remember the full function name, but only a part of it (or something related to it), you can use the “??” operator, for example:

```
??download
```

Small Exercise 3

- *Open and run the .R file called `Get_2_know_the_help.R`.*

 Click to view “Get_2_know_the_helpeR.R”

```
1 #Clear the environment
2 rm(list = ls())
3 Sys.setenv(lang = "en_US")
4
5 # Define a vector A
6 vA <- c(1, 2, 3, 4, 5, 6)
7
8 # Define a 2x3 matrix A
9 mA <- matrix(vA, nrow = 2, ncol = 3, byrow = TRUE)
10
11 # Define a 2x2 matrix B
12 mB <- matrix(c(9, 8, 7, 6), ncol = 2, byrow = TRUE, nrow = 2)
13
14 # Print matrix B
15 mB
16
17 # Find out what the function crossprod (from the "base" package) does via the help() function
18 mC <- crossprod(mA,mB)
19 mC
20
21 # The below matrix will give you an error. Try to reproduce the matrix mC via linear algebra
22 mD <- mA %*% mB
23 # Hint: Try ??transpose and look for Matrix Transpose.
24 mD <- t(mA) %*% mB
25 mD # Print matrix D
26
27 # Use one of the below to test if mC and mD are equal
28 all.equal(mC,mD) # Check if all mC and mD are equal, returns "TRUE" if this is mC = mD.
29 vCheck <- c(mC) == c(mD) # Form a vector of booleans, look up '?'=='
30 all(vCheck) # Check if all mC and mD are equal, returns "TRUE" if mC = mD.
```

- Run the code line by line. Do you understand it?
- Try to look up some of the functions with the `help`-function.
- Discover how to transpose `mA` to obtain `mD`?

```
md <- t(mA) %*% mB
```

- Evaluate whether your `mD` is equal to `mC`.

```
all.equal(mC,mD) # Check if all mC and mD are equal, returns "TRUE" if this is mC = mD.
vCheck <- c(mC) == c(mD) # Form a vector of booleans, look up ?'=='
all(vCheck) # Check if all mC and mD are equal, returns "TRUE" if mC = mD.
```

Small Exercise 4

- Try to install the package “Rfast”. (Hint: Go back a couple of slides to find the relevant function).

```
install.packages("Rfast")
```

- Load the package and try the following code:

```
library("Rfast") # Loads the package
mA <- matrix(1:100,10,10)
rowVars(mA)
```

- There are only a finite amount of names to call functions (especially if it has to make sense). What happens if you have two packages that have a similar name? E.g., `rowVars` from `Rfast` and `matrixStats`?

Either use:

```
detach("package:Rfast", unload=TRUE)
```

Or by the use of namespaces, i.e., `Rfast::rowVars(mA)` to run the function from the package we want. (This syntax also carries over to other programming languages, e.g., directly to C++)

1.5 Additional code in R

 Click to view “Week1_lecture1.R”

2 Getting Started with R

2.1 Working directory

When you run R, it nominates one of the directories on your hard drive as a working directory, which is where it looks for user-written programs and data files.

You can determine the current working directory using the command `getwd()`. You can do this using the command `setwd("sDir")`, where `dir` is the directory address.

```
getwd()
sDir <- "G:/foo" # this path needs to exist
setwd(sDir)
getwd()
```

If the path does not exist, R can create the directory (“folder”) for you via the `dir.create` built-in function and then change to the new directory.

```
# Assign a new path to create the new directory
sPath <- getwd()
sDir <- file.path(sPath, "new_dir2")
dir.create(sDir) # Creates a new directory
setwd(sDir) # set WD to sDir
```

You can also just pass the directory path directly, i.e., `dir.create("C:/user/foo")`.

Note that R will return an error message if “\” is used instead of “/” in the path as this is reserved for other operations, e.g., the line change used in [Small Exercise 1](#) ; `cat("\n")`

When the working directory is set, we can call other scripts from the currently specified “WD” without specifying the path via the `source()` function, i.e.:

```
source("foo.r") # tell R to Source/Run the .R file/script
```

Calling e.g. scripts from another directory is also possible, but requires a specific path, i.e., `"C:/wd/foo.r"`.

Furthermore, this feature is incredibly useful when writing longer scripts containing several user written functions.

This is demonstrated in `Example_1.R`:

 Click to view “Example_1.R”

```
1  rm(list = ls()) # clear the environment
2  cat("\014")      # Clear console:equivalent to ctrl + L
3
4
5  getwd() # Check working directory is correct
6
7  dir() # Check that the directory contain the desired files.
8
9  # Source the functions contained in the script "example1_fun.r"
10 source("example1_fun.r")
11
12 squaring(-3) # Calling the function squaring with input -3
13
14 squaring(3)  # Calling the function squaring with input 3
15
16 squaring("a") # Calling the function squaring with input "a"
17
18 clc() # Use the newly defined clc() function to clear console
```

Note for the script above: The sourced file, `example1_fun.r`, can be seen below:

 Click to view “example1_fun.R”

```
1 # This file contains two functions.
2 # At this stage it is not necessary that you completely understand the functions, they will
3
4 ## Function 1: squaring
5 # input numeric types: dX
6 # Output the square of the input
7 squaring <- function(dX) {
8   dX^2
9 }
10
11
12 ## Function 2: clc
13 # non-input function to clear console
14 clc <- function() {
15   cat("\014") # "\014" is equivalent to ctrl + L (Windows) or cmd + L (MacOS)
16 }
```

In addition, `source` will check your program for completeness before executing it. That is, it will stop processing the call if an error is reported.

2.2 Arithmetic

R uses the usual symbols for addition `+`, subtraction `-`, multiplication `*`, division `/`, and exponentiation `^`. Parentheses `()` can be used to specify the order of operations.

```
(1 + 1/100)^100
#> [1] 2.704814
```

Notice that by default, R prints 7 significant digits. You can change the display to `x` digits using `options(digits = x)`. See `help(options)` for other options.

2.3 Variable types

2.3.1 Defining variables

To assign a value to a variable, we use (almost equivalently) the assignment commands `=` and `<-`. For example:

```
iX <- 100
iX = 100
# These are equivalent
```

With some exceptions as shown last time in [Becoming_a_useR.R](#). We can perform operations in `iX`:

```
iX
#> [1] 100
(1 + 1/iX)^iX
#> [1] 2.704814
iX <- iX + 1 # is allowed
iX
#> [1] 101
```

2.3.2 Variable types and syntax

If you try to pass the string “a” to the function `squaring`, you get an error. Functions only take sensible inputs: in this case “numeric” class objects such as double or integers (and even vectors or matrices...)

In addition, R has several object type specific functions, i.e., `length(vY)` will give you the length of a vector, but `mY` will return the length of `vY <- c(mY)`. Hence, keeping track of variable types or variable classes therefore seems worthwhile.

- To R, everything is an “object”.
- We’ve seen some examples of objects as the atomic types/classis previously:
 - *Integers*, such as `3L` or `as.integer(3)`.
 - *Numeric*, such as `3`, `-3`, `9.01` (So in principle both doubles and “integers”).
 - *Logicals*, such as `TRUE` or `FALSE`, “booleans”:
 - *Characters*, such as `“asdf”`.
 - And data structures as *Matrices*, such as $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$
- We’ll study these types, and a few other common ones in detail later on.

2.3.3 Variable names and Hungarian notation

Hungarian notation was developed for more restrictive languages, but also has its uses in R.

- Use variable names consisting of multiple characters.

- The first one of two indicate the type of the variable.
- The remainder forms a descriptive name.

Example: suppose that you are estimating a linear model: $y = X\beta + \varepsilon$. So, the input consists of a vector y and a matrix X . Variable names could be `vY` and `mX`. The output is probably a vector of estimated regression coefficients, which you could call `vBeta` or `vBetaHat` or `vBetaOLS` or something. And perhaps there's also a `vEpsilon` or `vEps`.

R doesn't care what you call your variables. As soon as a variable is given a value, R will figure out what type it is automatically.

So, if you calculate $(X'X)^{-1}X'y$ and want to call the result `apple`, this is not technically wrong. But it's not particularly nice to people reading your code either.

2.3.4 Integers

The simplest value type is an *integer*. Its value is always a whole number, minimum $\approx -2^{31}$, maximum $2^{31} - 1$ (other bounds in other programming languages). The Hungarian prefix is `i`. The R command `iN <- 5L` results in the creation of a variable that has name `iN`, type integer, and value 5. If we subsequently execute command `iN <- 7L`, the variable still has the same name and type, but it has a different value. R forgets the value before the change.

A logical, prefix `b`, is a special type of integer that holds boolean values. It can only take two different values: `TRUE` (`=1`) or `FALSE` (`=0`).

2.3.5 Doubles

For a real number that's not an integer, we generally use a *double*, Hungarian: `d`. We can write things like, `dPi <- 3.14159`. Note, there's nothing that prevents a double variable having an integer value. It would be weird if `dX` could be 4.9, 4.99, 4.999, 5.1, 5.01, 5.001, but not exactly 5. However, redefining the variable as `dX <- 5` will create an integer, not a double. Instead, define `dX <- 5.0`.

2.3.6 Precision

Note, that $(1/3 + 1/3 + 1/3 + 1/3 + 1/3 + 1/3) - 2$ is not exactly zero in R; it's something like -0.0000000000002220446.

Integers aren't rounded, so integer arithmetic is done exactly. To avoid unnecessary rounding errors, always use the integer type if you know you're only going to be dealing with integers.

For the most part, R will automatically set the type for you as needed, so no need to put too much emphasis on `5` vs `5L`. Use Hungarian "i" notation for both.

2.4 Built-in

2.4.1 Character/strings

Characters are the R objects for storing strings. If we're interested in storing text (a character string), we use a *character*. A string needs to be enclosed in quotation marks. Thus, the command to create a string variable that contains *Hello world* is `sText <- "Hello world"` or `sText <- 'Hello world'`. We can use functions such as `substr()` to access part of a string, e.g.:

```
sText <- "Hello world"
substr(sText, start = 6, stop = nchar(sText))
#> [1] " world"
```

Note that R has built-in functions such as `is.character()`, `as.character()` but not `is.string()` built.in. For comparability with other languages, we use Hungarian `s`, but `c` is also fine.

2.4.2 Built-in functions

R has a number of built-in functions, for example `sin(x)`, `cos(x)`, `tan(x)`, (all in radians), `exp(x)`, `log(x)`, and `sqrt(x)`. Some special constants such as `pi` (π) are also predefined.

```
exp(1)
#> [1] 2.718282
options(digits = 16)
exp(1)
#> [1] 2.718281828459045
pi
#> [1] 3.141592653589793
sin(pi/6)
#> [1] 0.499999999999999
```

The functions `floor(x)` and `ceiling(x)` round down and up, respectively, to the nearest integer.

2.4.3 Functions

In mathematics, a function takes one or more arguments (or inputs) and produces one or more outputs (or return values). Functions in R work in an analogous way. Consider the `seq` function. This function allows you to create a sequence of numbers:

```
seq(from = 1, to = 9, by = 2)
#> [1] 1 3 5 7 9
```

Notice that by default (see `help(seq)`), the `by` argument is equal to 1, such that:

```
seq(from = 1, to = 3)
#> [1] 1 2 3
```

For a sequence with `by` equal to 1, R has a shorthand notation, so we may use `to:from`;

```
1:4
#> [1] 1 2 3 4
```

2.4.4 Arguments of functions

Every function has a default order for the arguments. For `seq` this is: `from`, `to`, `by`. If you provide arguments in this order, then they do not need to be named explicitly.

```
seq(1, 9, 2)
#> [1] 1 3 5 7 9
```

But you can choose to give the arguments out of order provided you give them names in the format `argument_name = expression`.

```
seq(by = 2, to = 9, from = 1)
#> [1] 1 3 5 7 9
# Is different from
seq(2, 9, 1)
#> [1] 2 3 4 5 6 7 8 9
```

2.5 Vectors

We have already seen the `c` function. However, vectors can be created by any function that returns vectors as output. For instance, `seq` and `rep`. The `rep(vX, n)` function exactly replicates the vector `vX`, n times.

```
vX = c(1, 3, 4)
rep(vX, 3)
#> [1] 1 3 4 1 3 4 1 3 4
```

The function `c` also accepts vectors as inputs:

```
vX <- seq(1, 20, by = 2)
vY <- rep(3, 4)
vZ <- c(vY, vX)
vZ
#> [1] 3 3 3 3 1 3 5 7 9 11 13 15 17 19
```

2.5.1 Vector operations

All algebraic operations are defined for vectors and act on each element separately, that is, element-wise:

```
vX <- c(1, 2, 3)
vY <- c(4, 5, 6)
vX * vY
#> [1] 4 10 18
vX + vY
#> [1] 5 7 9
vY ^ vX
#> [1] 4 25 216
```

However, care should be taken when vectors of unequal length are used:

```
c(1, 2, 3) + c(1, 2)
#> Warning in c(1, 2, 3) + c(1, 2): longer object length is not a multiple of
#> shorter object length
#> [1] 2 4 4
```

When you apply an algebraic expression to two vectors of unequal length, R automatically repeats the shorter vector until it has something the same length as the longer vector.


```
c(1, 2, 3, 4) + c(1, 2)
#> [1] 2 4 4 6
(1:10) ^ c(1, 2)
#> [1] 1 4 3 16 5 36 7 64 9 100
```

This happens even when the shorter vector is of length 1, allowing the shorthand notation:

```
2 + c(1, 2, 3)
#> [1] 3 4 5
2 * c(1, 2, 3)
#> [1] 2 4 6
```

For example, using the modulus operator: $a \bmod b$ (`%%`):

```
1:20 %% 3
#> [1] 1 2 0 1 2 0 1 2 0 1 2 0 1 2 0 1 2 0 1 2
```

2.6 Missing values

2.6.1 Missing data: NA

In real experiments, it is often the case, for one reason or another, that certain observations are missing. Depending on the statistical analysis involved, missing data can be ignored or invented (a process called imputation). R presents missing observations through the data value NA. They can be mixed in with all other kinds of data:

```
vA <- c(11, NA, 13)
vA
#> [1] 11 NA 13
```

Performing analysis with NAs can be problematic:

```
mean(vA) # NAs can propagate
#> [1] NA
```

The logical argument `na.rm` is often specified to deal with NAs.

```
mean(vA, na.rm = TRUE) # NAs can be removed
#> [1] 12
```

2.6.2 Searching for NAs

Sometimes you want to search for NAs in your dataset to prevent misleading results at the end of your analysis. The function `is.na` searches for NAs inside vectors and returns a logical output of the same size of the input provided:

```
vA <- c(11, NA, 13)
is.na(vA) # identify missing elements
#> [1] FALSE TRUE FALSE
```

When `length(vA)` is very large, we might want to write:

```
any(is.na(vA)) # are any missing?
#> [1] TRUE
```

In order to remove NAs we can use:

```
na.omit(vA)
#> [1] 11 13
#> attr("na.action")
#> [1] 2
#> attr("class")
#> [1] "omit"
```

2.7 Small Exercise

- Define a vector `vX` containing the double 5.01, the string “hello”, the logical/boolean `FALSE` and the integer 10L.
 - `vX <- c(5.01, "hello", FALSE, 10L)`
- Determine the type of this vector? (Use the `typeof()` built-in function or look at values in the environment).
 - The vector is of type `character`.
- Do you have an explanation for why it is this type? (Try instead to assign all the above values to `vX` but not “hello” i.e., `c(5.01, FALSE, 10L)` and repeat the second bullet point).

- Types are coerced in a fixed order: character, double, integer and logical. This only occurs for vectors. Lists are able to retain the correct datatypes.
- *Define a vector `vY` as `(vY <- c(NaN, NA, 6L))`*
- *compare the output for `is.na(vY)` and `is.nan(vY)`.*
 - `is.nan` does not recognize the NA.
- *What is the difference between `NaN` and `NA`? Try to print `0/0` in the console*
 - NaN: “Not a Number”, which refers to things that cannot be calculated.
 - NA: “Not Available”, referring to values that are unknown or missing.

 Click to view the complete solution to the exercise

```
1 rm(list = ls()) # clear environment
2 cat("\014")      # Clear console:equivalent to ctrl + L
3
4 # Small exercise with vectors and missing values
5 # Exercises 1
6 s <- c(5.01,"hello",FALSE,10L)
7 typeof(s) # Note the vector is of mode "character"
8 s <- c(5.01,FALSE,10L)
9 typeof(s) # Without the character element the vector is now of mode "double"
10 s <- c(FALSE,10L)
11 typeof(s) # ... Now of mode integer
12 s <- c(FALSE)
13 typeof(s) # ... Now of mode logical
14
15 # Exercise 2
16 vY <- c(NaN,NA,6L) # define vector
17 vY
18 is.na(vY) # look for NA-value
19 is.nan(vY) # look for NaN-value
20 # NaN: "Not a Number", used for things that cannot be calculated
21 # NA: "Not Available", values that are unknown or missing
22
23 # NaN examples:
24 0/0      # is NaN
25 sqrt(-1) # is NaN
26 Inf-Inf  # is NaN
27 NaN/NA   # is NaN
28
29 ## Extra question..
30 # Can NA matrices be used for more memory efficient
31 # usage as predefined matrix/vectors prior to i.e. a loop?
32 object.size(matrix(NA,1000,1000)) # Size: 4000216 bytes
33 object.size(matrix(0,1000,1000))  # Size: 8000216 bytes
```

2.8 Logical

2.8.1 Logical expression

A logical expression is formed using the comparison operators:

- $<$ lower than
- $>$ greater than
- $\leq$ lower than or equal to
- $\geq$ greater than or equal to
- $==$ equal to
- $!=$ not equal to
- $\&$ and
- $|$ or
- $!$ not

The operators return a logical output:

```
vX = c(1, 2, 3, 4)
vX == 2
#> [1] FALSE TRUE FALSE FALSE
vX != 2
#> [1] TRUE FALSE TRUE TRUE
```

Vectors (but also matrices and arrays) can be accessed also using logical indicators:

```
vX <- c(1, 3, 4, 18)
vX > 2
#> [1] FALSE TRUE TRUE TRUE
vX[vX > 2 & vX < 10]
#> [1] 3 4
```

The subset function can be used for a similar scope:

```
subset(vX, subset = vX > 2)
#> [1] 3 4 18
```

2.8.2 Logical expressions: & and |

Take two logical objects:

```
vX = 4
vY = vX > 2
vY
#> [1] TRUE
vZ = vX < 3
vZ
#> [1] FALSE
```

The & operator returns TRUE if both vY and vZ are TRUE, and FALSE otherwise:

```
vY & vZ
#> [1] FALSE
```

The | operator returns TRUE if y or z are TRUE, and FALSE otherwise:

```
vY | vZ
#> [1] TRUE
```

2.8.3 Sequential && and ||

The logical operators && and || are sequentially evaluated version of & and |, respectively.

To evaluate `vX & vY`, R first evaluates `vX` and `vY`, then returns TRUE, if `vX` and `vY` are both TRUE, and FALSE otherwise.

To evaluate `vX && vY`, R first evaluates `vX`. If `vX` is FALSE then R returns FALSE without evaluating `vY`. If `vX` is TRUE, R evaluates `vY` and returns TRUE if `vY` is TRUE, and FALSE otherwise.

Sequential evaluation of `vX` and `vY` is useful when `vY` is not always well defined, or when `vY` takes a long time to compute.

As an example of the first instance, suppose we wish to know if $dX * \sin(1/dX) = 0$:

```
dX <- 0
dX * sin(1/dX) == 0
#> Warning in sin(1/dX): NaNs produced
#> [1] NA
(dX == 0) | (sin(1/dX) == 0)
```

```
#> Warning in sin(1/dX): NaNs produced
#> [1] TRUE
(dX == 0) || (sin(1/dX) == 0)
#> [1] TRUE
```

Note that `%%` and `||` only work on scalars, whereas `&` and `|` work on vectors on a element-by-element basis:

```
vX <- c(1:10) # Note: c() is not necessary here.
vX
#> [1] 1 2 3 4 5 6 7 8 9 10
bTest <- FALSE
(vX == 1 & bTest == TRUE)
#> [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
(vX == 1 && bTest == TRUE) # Not even possible to run
#> Error in vX == 1 && bTest == TRUE: 'length = 10' in coercion to 'logical(1)'
(vX == 2 && bTest == FALSE)
#> Error in vX == 2 && bTest == FALSE: 'length = 10' in coercion to 'logical(1)'
(vX == 1 && bTest == FALSE)
#> Error in vX == 1 && bTest == FALSE: 'length = 10' in coercion to 'logical(1)'
```

2.9 Matrices

We have already seen how to build matrices in R:

```
mA <- matrix(1:6, nrow = 2, ncol = 3, byrow = TRUE)
```

To retrieve the dimension of a matrix, use `dim`:

```
dim(mA)
#> [1] 2 3
```

Useful functions for matrices:

- `diag`, extract the diagonal elements and return a vector or create a diagonal matrix, see `help(diag)`.
- `rbind`, join matrices with rows of the same length (stacking vertically).
- `cbind`, join matrices with columns of the same length (stacking horizontally).

- `solve`, invert a matrix.
- `eigen`, extract eigenvalues and associated eigenvectors of a matrix.
- `t`, transpose of matrix.

2.9.1 Multiplication with matrices

Define:

```
mA <- matrix(c(3, 5, 2, 3), nrow = 2, ncol = 2)
mB <- matrix(c(9, 4, 1, 2), nrow = 2, ncol = 2)
mA
#>      [,1] [,2]
#> [1,]    3    2
#> [2,]    5    3
mB
#>      [,1] [,2]
#> [1,]    9    1
#> [2,]    4    2
```

Element-wise multiplication, $A \circ B$ (Hadamard product):

```
mA * mB
#>      [,1] [,2]
#> [1,]   27    2
#> [2,]   20    6
```

Matrix multiplication, AB :

```
mA %*% mB
#>      [,1] [,2]
#> [1,]   35    7
#> [2,]   57   11
```

Kronecker product, $A \otimes B$:

```
kronecker(mA, mB)
#>      [,1] [,2] [,3] [,4]
#> [1,]   27    3   18    2
#> [2,]   12    6    8    4
#> [3,]   45    5   27    3
#> [4,]   20   10   12    6
```


2.10 Factors

R has a special data structure for Categorical variables that take ordinal or cardinal values.

- Ordinal (rankable): Low, medium, high.
- Cardinal (Unrankable values): Colour, marital status, names, etc...

In R, we (may) deal with categorical variables in factor variables; each category is called a level.

- Packages or statistical models may facilitate easy use of the factor class, .e.g, in `lm` to create dummies from the categorical variable.
- Memory efficient data storage.

```
phys.act <- c("L", "H", "H", "L", "M", "M")
phys.act[2] > phys.act[1]
#> [1] FALSE
phys.act <- factor(phys.act, levels = c("L", "M", "H"), ordered = TRUE)
is.ordered(phys.act)
#> [1] TRUE
phys.act[2] > phys.act[1]
#> [1] TRUE
```

2.11 Lists

We have seen that all the elements of a vector have to be of the same type: **numeric**, **character**, or **logical**. The type of the vector is called: **mode**.

A list is an indexed set of objects (and so has a length) whose elements can be of different types, including other lists! The mode of a list is **list**.

A list is just a generic container for other objects. The power and utility of lists comes from this generality. In R, lists are often used for collecting and storing complicated function outputs. For example, the first element of a list can be a vector, the second can be another list and the third can be a matrix.

A list is created using the `list(...)` command, with comma-separated arguments:

```
my.list <- list("one", TRUE, 3, c("f", "o", "u", "r"))
```

Double-square brackets are used to extract a single element:

```
my.list[[1]]
#> [1] "one"
mode(my.list[[1]])
#> [1] "character"
```

Single-square brackets are used to select a sublist:

```
my.list[1]
#> [[1]]
#> [1] "one"
mode(my.list[1])
#> [1] "list"
```

When displaying a list, R uses double-square brackets `[[1]]`, and `[[2]]`, etc. to indicate list element. The single-square brackets `[1]`, `[2]`, etc. incide vector elements within the list.

The elements of a list can be named when the list is created, using arguments of the form `name1 = x1, name2 = x2`, etc.:

```
my.list <- list(first = "one", second = TRUE, third = 3, fourth = c("f", "o", "u", "r"))
names(my.list)
#> [1] "first" "second" "third" "fourth"
my.list$second
#> [1] TRUE
```

Note the use of the `$` operator to access named lists. Alternatively, a list element can be named later by assigning a value to the `names` attribute:

```
my.list <- list("one", TRUE, 3, c("f", "o", "u", "r"))
names(my.list) <- c("first", "second", "third", "fourth")
```

To flatten a list `x`, i.e., convert it to a vector, we use `unlist(x)`:

```
x <- list(1, c(2, 3), c(4, 5, 6))
unlist(x)
#> [1] 1 2 3 4 5 6
```

If the list object itself comprises lists, then these lists are also flattened, unless the argument `recursive = FALSE` is set in `unlist`.

2.12 Dataframes

We have already seen how to work in R with numbers, strings, and logical values. We have also worked with homogeneous collections of such objects, grouped into numeric, character, or logical vectors.

The defining characteristic of the vector data structure in R is that all components must be of the same mode. Obviously, to work with datasets from real experiments, we need a way to group data of differing modes.

Suppose to have the following dataset representing a forestry experiment in which we randomly selected a number of plots and then from each plot selected a number of trees.

Plot	Tree	Species	Diameter (cm)	Height (m)
2	1	DF	39	20.5
2	2	WL	48	33.0
3	2	GF	52	30.0
3	5	WC	36	20.7
3	8	WC	38	22.5
:	:	:	:	:

For each tree, we measured its height and diameter (which are **numeric**), and also the species of tree (which is a **character** string).

As experimental data collected in a table looks like an array, you may be tempted to represent it in R as a matrix. But in R, matrices cannot contain heterogeneous data (data of different modes, like **numeric** and **character**)

Lists and dataframes are able to store much more complicated data structures than matrices. A dataframe is a list of vectors restricted to be of equal length. Each vector (column of the dataframe) can be of any of the basic modes of object. To create a dataframe, we write:

```
mData <- data.frame(Plot = c(1, 2, 2, 5, 8, 8),
                    Tree = c(2, 2, 3, 3, 3, 2),
                    Species = c("DF", "WL", "GF", "WC", "WC", "GF"),
                    Diameter = c(39, 48, 52, 35, 37, 30),
                    Height = c(20.5, 33.0, 30.0, 20.7, 22.5, 20.1))
```

2.12.1 Dataframes: Extract

Each column, or variable, in a **dataframe** has a unique name. We can extract that variable by means of the dataframe name, the column name, the a dollar sign, or as we do for a matrix:

```
mData$Diameter
#> [1] 39 48 52 35 37 30
mData[["Diameter"]]
#> [1] 39 48 52 35 37 30
mData[[4]]
#> [1] 39 48 52 35 37 30
mData[, 4]
#> [1] 39 48 52 35 37 30
mData[, "Diameter"]
#> [1] 39 48 52 35 37 30
```

2.12.2 Dataframes: Assign

To assign a new variable to a dataframe, we write:

```
mData$newdata <- c(1, 2, 3, 4, 5, 6)
```

If the new variable is the same across all the observations, we can write:

```
mData$newdata2 <- TRUE
```

This also works as long as the number of rows of the dataframe is a multiple of the length of the new variable (if it is not a multiple, we get an error):

```
mData$newdata3 <- c("one", "two")
mData
#>   Plot Tree Species Diameter Height newdata newdata2 newdata3
#> 1     1     2     DF       39   20.5       1     TRUE     one
#> 2     2     2     WL       48   33.0       2     TRUE     two
#> 3     2     3     GF       52   30.0       3     TRUE     one
#> 4     5     3     WC       35   20.7       4     TRUE     two
#> 5     8     3     WC       37   22.5       5     TRUE     one
#> 6     8     2     GF       30   20.1       6     TRUE     two
```

2.13 Small exercises with logical operators, matrices, lists and dataframes

- Define a 10×4 matrix `mA` that holds the values from a vector of the sequence from 1 to 40 that increments by 1.

- Print `mA` and visually determine if `R` fills in the matrix by column or by row.
- Assign `mA` to the variable `dfA` as a dataframe (Try `?as.data.frame`).
- Assign the column names “one”, “two”, “three” and “four” to the data frame (Try `?colnames`).
- Assign a new variable called “five” that contains the logical `FALSE` to the data frame `dfA`.
- Verify that `dfA` is indeed both considered a list and a data frame using a logical operator along with `is.list` and `is.data.frame`.
- If time permits; remove the column named “two” from the data frame using a logical operator.

Solution

```
mA <- matrix(1:40, 10, 4)
mA # it's filled by columns
#>      [,1] [,2] [,3] [,4]
#> [1,]    1   11   21   31
#> [2,]    2   12   22   32
#> [3,]    3   13   23   33
#> [4,]    4   14   24   34
#> [5,]    5   15   25   35
#> [6,]    6   16   26   36
#> [7,]    7   17   27   37
#> [8,]    8   18   28   38
#> [9,]    9   19   29   39
#> [10,]   10   20   30   40
dfA <- as.data.frame(mA) # convert to dataframe
colnames(dfA) <- c("one", "two", "three", "four") # add headers
dfA # view dataframe
#>      one two three four
#> 1     1  11   21   31
#> 2     2  12   22   32
#> 3     3  13   23   33
#> 4     4  14   24   34
#> 5     5  15   25   35
#> 6     6  16   26   36
#> 7     7  17   27   37
#> 8     8  18   28   38
#> 9     9  19   29   39
#> 10    10  20   30   40
dfA$five <- FALSE
dfA # view the new column
#>      one two three four five
#> 1     1  11   21   31 FALSE
```

```

#> 2      2  12      22      32 FALSE
#> 3      3  13      23      33 FALSE
#> 4      4  14      24      34 FALSE
#> 5      5  15      25      35 FALSE
#> 6      6  16      26      36 FALSE
#> 7      7  17      27      37 FALSE
#> 8      8  18      28      38 FALSE
#> 9      9  19      29      39 FALSE
#> 10     10  20      30      40 FALSE
is.list(dfA) # check if it's a list
#> [1] TRUE
is.data.frame(dfA) # check if it's a dataframe
#> [1] TRUE
dfA[, !(names(dfA) == "two")] # remove the column with the name two
#>      one three four  five
#> 1      1      21     31 FALSE
#> 2      2      22     32 FALSE
#> 3      3      23     33 FALSE
#> 4      4      24     34 FALSE
#> 5      5      25     35 FALSE
#> 6      6      26     36 FALSE
#> 7      7      27     37 FALSE
#> 8      8      28     38 FALSE
#> 9      9      29     39 FALSE
#> 10     10      30     40 FALSE

```

2.14 Additional code in R

 Click to view “Week1_lecture1.R”

3 Basic Programming

This lecture introduces a set of basic programming constructs, which are the building blocks of most programs.

Some of these tools are used by practically all programming languages, for example, conditional execution by `if` statements, and looped execution by `for` and `while` statements.

Notice that code that seems to be efficient in another language may not be efficient in R. For example, R is known to be quite slow in computing `for` and `while` loops. On the other hand, R is fast in doing vector-based operations.

A program or script is just a list of commands, which are executed one after the other. A program is usually composed by three parts:

- Input
- Computations
- Output

Usually, we write the list of commands in separate files, called scripts, which we can save on our hard drive.

Suppose we have a program saved as `prog.r` in the working directory (recall the `getwd` and `setwd` functions). In order to run or execute the program use the command:

```
source("prog.r")
```

3.1 The `if` statement

It is often useful to choose the execution of some or other part of a program to depend on a condition. The `if` function has the form:

```
if (logical_expression) {  
  expression_1 #code to run if logical_expression is TRUE  
  ...  
}
```

A natural extension of the `if` command includes an `else` part:

```
if (logical_expression) {  
  expression_1  
  ...  
} else {  
  expression_2  
  ...  
}
```

When an `if` expression is evaluated, if `logical_expression` is `TRUE` then the first group of expressions is executed and the second group of expressions is not executed. Conversely, if `logical_expression` is `FALSE` then only the second group of expressions is executed.

Example:

```
dX <- rnorm(1) # random number from standard normal distribution  
if (dX > 0) {  
  print("x is positive")  
} else {  
  print("x is non-positive")  
}  
#> [1] "x is positive"
```

Note: be careful with the use of the curly brackets {}!

3.1.1 Small Exercise

- Generate a random number from a standard Gaussian distribution.
- If the number is negative, then compute the square of it and print the value of `x` *along with the output, e.g., “the value of `x` is __ and its square is __” (Hint: use the* `cat`function).
- Otherwise, compute the cube and print the value of `x` along with the output.

Solution:

```
dX <- rnorm(1) # Generates 1 observation, mean 0 and standard deviation 1  
if (dX < 0) {  
  cat("The value of x is ", dX, " and its square is ", dX^2)  
} else {  
  cat("The value of x is ", dX, " and its cube is ", dX^3)  
}  
#> The value of x is  0.4822965  and its cube is  0.112187
```

3.1.2 Nested if statements

Nested if statements are constructed using the `else if` command:

```
if (logical_expression_1) {
  expression_1 # code to run if logical_expression_1 is TRUE
  ...
} else if (logical_expression_2) {
  expression_2 # code to run if logical_expression_1 is FALSE and logical_expression_2 is TRUE
  ...
} else if (logical_expression_3) {
  expression_3 # code to run if logical_expression_1 is FALSE and logical_expression_2 is FALSE and logical_expression_3 is TRUE
  ...
} else {
  expression_4 # code to run if all logicals are FALSE
}
```

Example:

```
dX - runif(1) # Random number from the uniform distribution
#> [1] 0.2034026
if (dX < 0.5) {
  print("x is less than 0.5")
} else if (dX < 0.75) {
  print("x is less than 0.75")
} else {
  print("x is greater than or equal to 0.75")
}
#> [1] "x is less than 0.5"
```

3.1.3 An extension to the exercise solution above

Extending the code from above to also include a nested `else if` such that it includes an action for the event of a non-negative value of `x`.

```
dX <- rnorm(1) # random number from the Gaussian distribution
if (dX < 0) {
  cat("the value of x is ", dX, " and its square is ", dX^2)
} else if (dX >= 0) {
  cat("x is non-negative, taking the value ", dX, " and its square is ", dX^2)
} # Note the tautological nature of the logical expression in the else if part...
#> the value of x is -0.6360399 and its square is 0.4045468
```

3.2 The for loop

The `for` command has the following form, where `x` is a simple variable and `vector` is a vector.

```
for (x in vector) {  
  expression_1  
  ...  
}
```

When executed, the `for` command executes the group of expressions within the braces `{}` once for each element of `vector`:

```
for (iX in 1:50) {  
  print(iX)  
}
```

Note that `vector` can also be a `list`.

Example:

```
vX <- seq(1, 9, by = 2)  
dSum_x <- 0  
for (iX in vX) {  
  dSum_x <- dSum_x + iX  
  cat("The current loop element is ", iX, "\n")  
  cat("The cumulative total is ", dSum_x, "\n")  
}  
#> The current loop element is 1  
#> The cumulative total is 1  
#> The current loop element is 3  
#> The cumulative total is 4  
#> The current loop element is 5  
#> The cumulative total is 9  
#> The current loop element is 7  
#> The cumulative total is 16  
#> The current loop element is 9  
#> The cumulative total is 25  
sum(vX)  
#> [1] 25  
cumsum(vX)  
#> [1] 1 4 9 16 25
```

3.2.1 Exercise: a combination of for loops and if conditions

- Within a `for` loop, that runs from 1 to 100, generate a standard normal random variable.
- For each draw, check whether it is positive or negative.
- Calculate the fraction of the positive and negative realizations.
- Store the simulated variables in a vector

Solution:

```
iPos <- 0
iNeg <- 0 # This integer is technically redundant, as the knowledge of the positive values and
vSimVals <- vector()
for (iX in 1:100) {
  dX <- rnorm(1)
  if (dX > 0) {
    iPos <- iPos + 1
  } else {
    iNeg <- iNeg + 1
  }
  vSimVals[iX] <- dX
}
cat("The fraction of positive realizations is: ", iPos, "/", (iNeg + iPos), " or ", iPos / (iNeg + iPos), "\n")
#> The fraction of positive realizations is: 48 / 100 or 0.48
cat("The first of the simulated values are: ", vSimVals[1:5], "...")
#> The first of the simulated values are: -0.6349054 0.9983947 0.005114277 1.182877 0.7805101
```

3.3 The while loop

Often, we do not know beforehand how many times we need to go around a loop. That is, each time we go around the loop, we check some condition to see if we are done yet.

In this situation, we use a `while` loop, which has the form:

```
while (logical_expression) {
  # This should have an impact on logical expression
  expression_1
  ...
}
```

When a `while` command is executed, `logical_expression` is evaluated first. If it is true, then the group of expressions in braces `{}` is executed.

Until `logical_expression` becomes false, the `while` command executes the group of expressions in braces `{}`. **warning: while loops can run forever, if a stopping criterion has not been specified!**

Example:

Consider the following example where we use the `while` loop to compute the square of numbers until 6:

```
iN <- 1
while (iN <= 6) {
  print(iN ^ 2)
  iN <- iN + 1
}
#> [1] 1
#> [1] 4
#> [1] 9
#> [1] 16
#> [1] 25
#> [1] 36
```

3.3.1 Exercise with while loops

- Generate a random number from a Gaussian distribution with mean 100 and variance 100.
- Start a `while` loop that at each iteration divides the result obtained at the previous iteration by two
- Set the stopping criterion when the result at the previous iteration is lower than 1.
- Print the number of iterations when the `while` loop has terminated.
- Could you accomplish the same with a `for`-loop? How?

Solution:

```
dCurrent <- rnorm(1, mean = 100, sd = sqrt(100))
dIte <- 0
while (dCurrent >= 1) {
  dCurrent <- dCurrent / 2
  dIte <- dIte + 1
}
print(dIte)
#> [1] 7
```

The same cannot be accomplished with a `for`-loop, because the `for`-loop requires a pre-determined number of iterations which is not possible to know beforehand with a random number.

i Click to view an example of nested while and if statements

```
1 Want.2.bake.cake <- FALSE
2 bowl <- matrix(0,1,1)
3
4 while(Want.2.bake.cake == FALSE) {
5
6     Want.2.bake.cake <- (runif(1) < 0.5)
7
8     if (Want.2.bake.cake == TRUE) {
9
10        print("You start baking, find the appropriate bowl")
11        Sys.sleep(2)
12
13        print("looking for bowl...")
14
15        while ((prod(dim(bowl)) != 100 | nrow(bowl) != ncol(bowl))) {
16
17            bowl <- matrix(0,sample(1:100,1), sample(1:100,1))
18
19            if (!(prod(dim(bowl)) != 100 | nrow(bowl) != ncol(bowl))){
20                Sys.sleep(2)
21
22                print("Wrong bowl... look for another one..")
23                print("looking for another...")
24
25            }
26        }
27    }
28 }
```

Please note that the code is actually wrong! The logical expression in the `if` condition after sampling of a bowl is only true if the correct bowl is found, not when the wrong bowl is found.

3.4 Vector-based programming

It is often necessary to perform an operation upon each of the elements of a vector.

R is set up such that these programming tasks can be accomplished using vector operations rather than looping.

Using vector operations is more efficient computationally as well as more concise literally. For example, consider the `while` loop example of computing the square of the first 6 numbers from before, now computed using vector based programming.

```
vX <- (1:6) ^ 2
vX
#> [1] 1 4 9 16 25 36
```

3.4.1 Example

If we want to find the sum of the first $n = 100$ squares, we might use a `for` loop:

```
iN <- 100
dS <- 100
for (i in 1:iN) {
  dS <- dS + i ^ 2
}
dS
#> [1] 338450
```

Or use vector-based programming:

```
sum((1:iN)^2)
#> [1] 338350
```

Comparing the computational time, we see that vector-based programming is more efficient.

```
iN <- 1e8
system.time({
  dS <- 0
  for (i in 1:iN) {
    dS <- dS + i ^2
  }
  dS
})
```



```
#>   bruger   system forløbet
#>    2.30    0.00    2.31
system.time({sum((1:iN) ^ 2)})
#>   bruger   system forløbet
#>    0.33    0.10    0.42
```

3.5 Load data in R

R implements functionalities to read from text files like: `.txt` and `.csv`. In order to read excel files external libraries are required, such as: **excel**, **gdata**, **rodbc**, **XLConnect**, **xls**, **xlsReadWrite**, **xlsx**.

R provides a number of ways to read data from a file: `scan`, `read.table`, `read.csv`. In this course, we will mostly use `read.table`.

```
read.table(file, header = FALSE, sep = "", dec = ".", row.names, col.names, na.strings = "NA")
```

The dots ... means that other arguments are available. See `help(read.table)`.

file: the name of the file which the data are to be read from. Each row of the table appears as one line of the file. If it does not contain an absolute path, the file name is relative to the current working directory, `getwd()`.

header: a logical value indicating whether the file contains the names of the variables as its first line.

sep: the field separator character. Values on each line of the file are separated by this character. If `sep = ""` (the default), the separator is ‘white space’.

dec: the character used in the file for decimal points.

row.names: a vector of row names. This can be a vector giving the actual row names, or a single number giving the column of the table which contains the row names, or character string giving the name of the table column containing the row name.

col.names: a vector of optional names for the variables. The default is to use “V” followed by the column number.

na.string: a character vector of strings which are to be interpreted as NA values.

skip: the number of lines of the data file to skip before beginning to read data.

3.5.1 Load MAERSK historical prices

The data set with historical prices for MAERSK is contained in the `MAERSK-B.CO.csv`-file. In the case below, the file is in the folder named 'data'.

```
mData <- read.table(file = "data/MAERSK-B.CO.csv", sep = ",", dec = ".", header = TRUE, row.names = NULL)
head(mData)
#>           Open      High      Low      Close Adj.Close Volume
#> 2000-02-01 7640.00 7666.67 7333.33 7466.67 2267.703 4590
#> 2000-02-02 7466.67 7666.67 7400.00 7566.67 2298.074 5640
#> 2000-02-03 7741.07 7800.00 7659.00 7800.00 2368.939 3090
#> 2000-02-04 7800.00 7800.00 7525.80 7600.00 2308.197 1185
#> 2000-02-07 7660.00 7666.67 7566.67 7600.00 2308.197 1035
#> 2000-02-08 7600.00 8200.00 7600.00 8110.80 2463.332 12990
dim(mData)
#> [1] 4571 6
```

3.5.2 Descriptive statistics of the MAERSK log-returns

```
any(is.na(mData[, "Adj.Close"])) # Search for NAs
#> [1] TRUE
length(which(is.na(mData[, "Adj.Close"]))) # How many NAs?
#> [1] 59
# Compute financial log-returns omitting the NAs:
vY <- diff(log(na.omit(mData[, "Adj.Close"])))
# Compute descriptive statistics
c("mean" = mean(vY), "sd" = sd(vY), "median" = median(vY))
#>           mean           sd           median
#> 0.0003439318 0.0220065696 0.0000000000
```

On a slightly more advanced note, very quick and efficient summary statistics can be obtained with `summary()`.

3.5.3 Output to a file

R provides a number of commands for writing output to a file.

We will generally use `write.table` for writing numeric values and `cat` for writing text, or a combination of numeric and character values.

The command `write.table` has the form:

```
write.table(x, file = "", quote = TRUE, sep = " ", na = "NA", dec = ".", row.names = TRUE, c
```

x: the object to be written, preferably a matrix or dataframe.

file: character string naming a file.

quote: a logical value (TRUE or FALSE). If TRUE, any character columns will be surrounded by double quotes (“”).

sep and **dec** work as in `read.table`.

na: the string to use for missing values in the data.

row.names: either a logical value indicating whether the row names of **x** are to be written along with **x**, or a character vector of row names to be written.

col.names: either a logical value indicating whether the column names of **x** are to be written along with **x**, or a character vector of column names to be written.

Example:

```
write.table(vY, file = "MAERSK_returns.txt", row.names = FALSE, col.names = FALSE, dec = ".")
```

3.6 Plotting

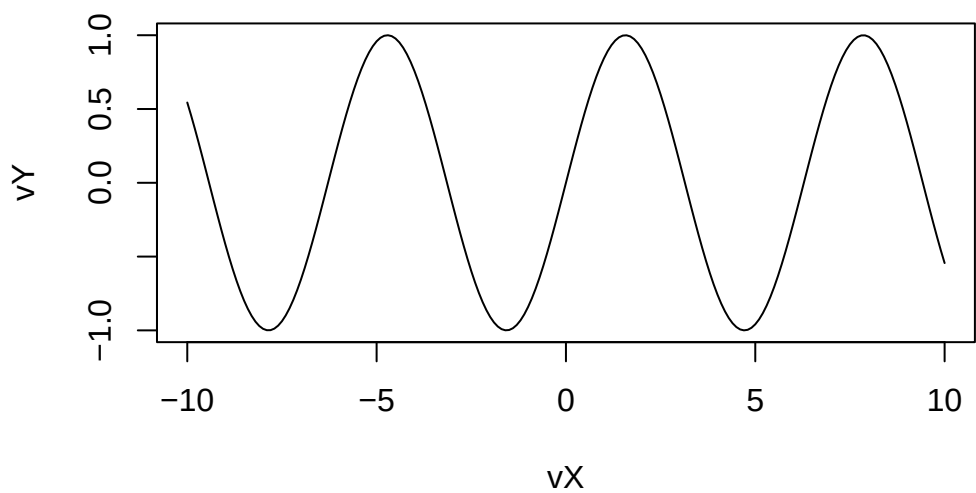
R provides a huge number of plotting routines. The generic function for plotting of R objects is `plot`.

External packages can also be used for plotting (e.g, **ggplot2**, see <http://ggplot2.org/> if you're interested), however, we will focus on the built-in function `plot` in this course.

R generally has good graphical capabilities. Get an initial overview of these capabilities by typing `demo(graphics)` in your R console.

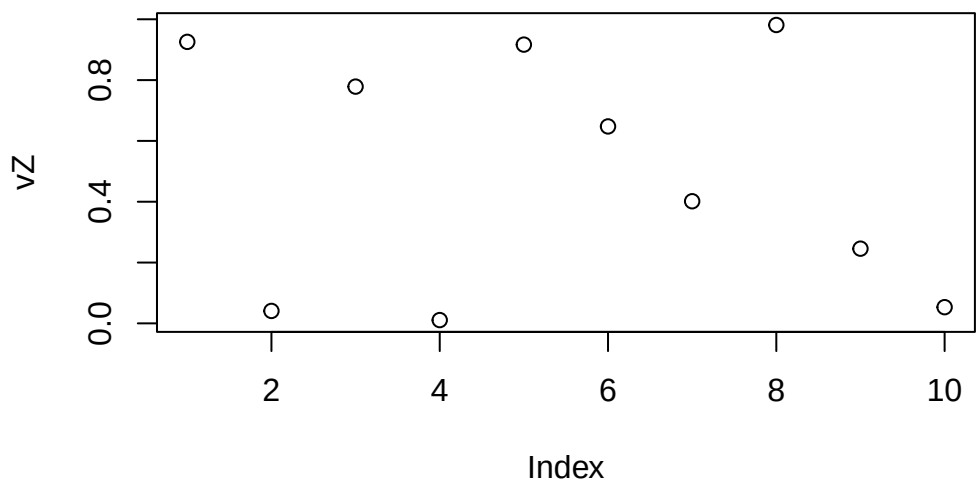
`plot` can be used to represent a pair of data points **x** and **y**, e.g.:

```
vX <- seq(-10, 10, 0.1)
vY <- sin(vX)
plot(vX, vY, type = "l")
```



Or a single vector of points, e.g.:

```
vZ <- runif(10)
plot(vZ)
```



3.6.1 Basic arguments for plot

The `plot` command offerse a wide variety of options for customizing the graphic. Some of these are:

- `type` determines the type of the plot, e.g., `type = "p"` for points (the default), `type = "l"` for lines, `type = "o"` for both lines and points, `type = "s"` for a step function, `type = "n"` for no plotting, etc. etc...
- `xlim = c(a,b)` and `ylim = c(a,b)` will set the lower and upper limits of the x- and y-axis to be a and b, respectively.
- `main = "Plot title goes here"` provides the plot title.
- `xlab = "X axis label goes here"` provides the label for the x-axis.
- `ylab = "Y axis label goes here"` provides the label for the y-axis.
- `col = "black"` colour for lines and points. Type `colours()` for a list of colours known to R (there are a lot!).
- `lty` determines the line type, e.g., 0=blank, 1=solid (default), 2=dashed, 3=dotted, etc...
- `lwd` determines the line width.
- `pch = k` determines the shape of points with `k` taking a value from 1 to 25.
- To add points `(x[1], y[1])`, `(x[2], y[2])`, ... to the current plot, use `points(x, y)`.
- To add lines instead use `lines(x, y)`.
- To add text use `text(x, y, "test")`.
- Vertical and horizontal lines can be drawn using `abline(v = xpos)` and `abline(h = ypos)`.
- The functions `points`, `lines`, `abline` also accept the `col`, `lty`, `lwd` arguments.
- The command `par` is used to set many different parametrs that control how graphics are produced (see `help(par)`). E.g., the command `par(mfrow = c(nr, nc))` creates a grid of plots with `nr` rows and `nc` columns, which is filled row by row. `par(mfcol = c(nr, nc))` is similar but fills the plots column by column.

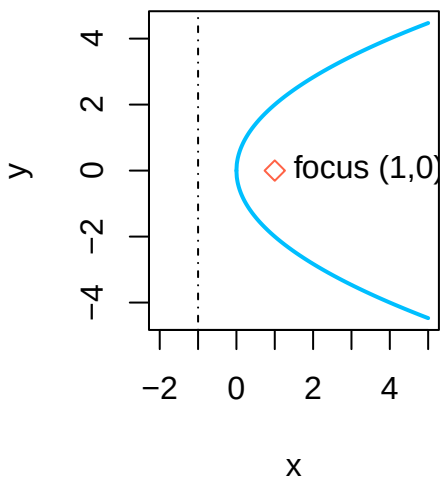
3.6.2 Example:

```

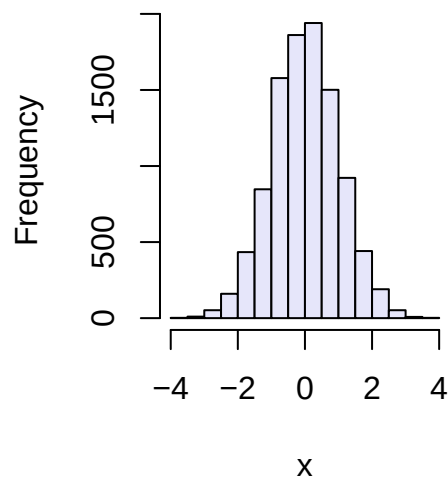
1 par(mfrow = c(1, 2))
2 x <- seq(0, 5, by = 0.01)
3 y.upper <- 2 * sqrt(x)
4 y.lower <- -2 * sqrt(x)
5 y.max <- max(y.upper)
6 y.min <- min(y.lower)
7 plot(c(-2, 5), c(y.min, y.max), type = "n", xlab = "x", ylab = "y", main = "Parabola Graph")
8 lines(x, y.upper, lwd = 2, col = "deepskyblue")
9 lines(x, y.lower, lwd = 2, col = "deepskyblue")
10 abline(v = -1, lty = 4)
11 points(1, 0, col = "tomato", pch = 5)
12 text(1, 0, "focus (1,0)", pos = 4)
13
14 x = rnorm(10000)
15 hist(x, col = "lavender", main = "Normal Histogram")

```

Parabola Graph



Normal Histogram



3.6.3 Exercise

- Generate a 1000 standard uniforms sort (in ascending order) your pseudo data. (Hint: ?runif and ?sort).
- Plot the random uniforms using the plot-function (Add any arguments you find necessary, i.e., legend or axis text)

- Then save the plot as a .pdf in your working directory (Check your WD with `getwd()`) using the following wrapper:

```
pdf("./myplot.pdf") # Your plot function with input  
dev.off()
```

Solution:

```
x <- sort(runif(1000), decreasing = FALSE)  
plot(x, main = "Uniform sample", xlab = "Ordering", ylab = "Value") # A simple scatterplot  
points(x, cex = .5, col = "dark red") # Colors
```



3.7 Additional code in R

 Click to view “Week1_lecture1.R”


```
1 #0.1. clear the workspace
2 rm(list = ls())
3
4 #0.2. set the working directory
5 setwd("...Exercises")
6
7 #1. If statement exercise
8 # Simulate a standard random normal variable
9 dX <- rnorm(1)
10
11 # Plain if
12 if (dX < 0) {
13   dX_2 <- dX^2
14   print(dX_2)
15 } else {
16   dX_3 <- dX^3
17   print(dX_3)
18 }
```

4 Programming with Functions

We have already seen and used a number of different pre-defined functions, such as:

- `matrix(data = NA, nrow = 1, ncol = 1, byrow = FALSE, dimnames = NULL)`
- `mean(x), sd(x), median(x)`
- `read.table(file, header = FALSE, sep = "", dec = ".", row.names, col.names, na.strings = "NA", skip = 0, ...)`
- `write.table(x, file = "", quote = TRUE, sep = " ", na = "NA", dec = ".", row.names = TRUE, col.names = TRUE, ...)`
- `plot(x, y, ...)`

4.1 Definition of functions in R

A function is a piece of code that takes input (in the form of variables), performs a specified action using the input, and produces output (variables, or a table of numbers, or a graph, etc.).

A function has the general form:

```
name <- function(argument_1, argument_2, ...) {  
  expression_1  
  expression_2  
  #<some other expression>  
  return(output)  
}
```

- `argument_1`, `argument_2`, etc., are the names of variables (the input).
- `expression_1`, `expression_2`, and `output` are all regular R expressions.
- `name` is the name of the function.

4.1.1 Running a Function

To call or run a function, we type (as an example):

```
name(x1, x2)
```

The value of this expression is the value of the expression output.

To calculate the value of output, the function first copies the value of `x1` to `argument_1`, the value of `x2` to `argument_2`, and so on. The arguments have then been passed to the function and act as variables within the function.

Next, the function evaluates the grouped expressions contained in the braces `{}`, and the value of the expression output is returned as the value of the function.

4.1.2 Examples

Example 1

Consider the following example of a very simple function that computes the value of `x` to the power of `y`:

```
power_fct <- function(vX, power) {  
  return(vX ^ power)  
}
```

The function is then carried out as:

```
power_fct(vX = 2, power = 3)  
#> [1] 8
```

Example 2

Consider the following example of a function that swaps the values of the first and second elements of a vector:

```
swap <- function(vZ) {  
  dTemp <- vZ[2]  
  vZ[2] <- vZ[1]  
  vZ[1] <- dTemp  
  return(vZ)  
}
```

The function is then carried out as:

```
swap(vZ = c(7, 8, 9))  
#> [1] 8 7 9
```

4.1.3 Notes on Functions

Note that some functions have no arguments, e.g., `getwd()`. If there is no `return(output)` statements then the value returned by the function is the value of the last expression in the braces.

4.1.4 Small Exercises

- Write a function that returns the sum of the first N natural numbers.

```
## There are multiple solutions to this:  
# The efficient  
fEfficientSum <- function(iN) {  
  # Input validation: Check if n is a non-negative integer  
  if (!is.numeric(iN) || iN < 0 || iN %% 1 != 0) {  
    stop("Input must be a non-negative integer.")  
  }  
  return(iN * (iN + 1) / 2)  
}  
# The for loop  
fForSum <- function(iN) {  
  sum <- 0  
  for (iIndex in 1:iN) {  
    sum = sum + iIndex  
  }  
  return(sum)  
}  
# The recursive  
fRecursiveSum <- function(iN) {  
  if (iN <= 0) {  
    return(iN)  
  } else {  
    return(iN + fRecursiveSum(iN - 1))  
  }  
}  
fEfficientSum(5)
```

```
#> [1] 15
fForSum(5)
#> [1] 15
fRecursiveSum(5)
#> [1] 15
```

- Create a function that removes missing values from its vector input if it contains any.

```
fRemoveMissing <- function(vInput) {
  if (any(is.na(vInput)) == TRUE) {
    return(na.omit(vInput))
  } else {
    return(vInput)
  }
}
vVector <- c(1, NA, 2)
vVector <- fRemoveMissing(vVector)
#> 1 2
```

4.1.5 Example: roots of a quadratic function

Assume we want to find the roots of:

$$ax^2 + bx + c = 0$$

The analytic solution is:

$$(x_1, x_2) = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The associated code can be written as a function:

```
rootfinder <- function(dA, dB, dC) {
  vOut = numeric(2)
  vOut[1] = (-dB + sqrt(dB ^ 2 - 4 * dA * dC)) / (2 * dA)
  vOut[2] = (-dB - sqrt(dB ^ 2 - 4 * dA * dC)) / (2 * dA)
  return(vOut)
}
```

But there are some problems related to the efficiency of the code. We are computing the algebraic operations too many times, and we are not exploiting vector-operations.

- What if $a = b = c = 0$?
- What if $a = b = 0$?

- What if $a = 0$?
- What if $b^2 - 4ac = 0$?
- What is $b^2 - 4ac < 0$?

Reformulate the analytical solution as:

$$(x_1, x_2) = \begin{cases} \mathbb{R}, & \text{if } a = b = c = 0 \\ \emptyset, & \text{if } a = b = 0 \wedge c \neq 0 \\ -\frac{c}{b}, & \text{if } a = 0 \wedge b \neq 0 \wedge c \neq 0 \\ \frac{-b \pm \sqrt{|\Delta|} \sqrt{\text{sgn}(\Delta)}}{2a}, & \text{otherwise} \end{cases}$$

Where $\Delta = b^2 - 4ac$. Note that:

$$\sqrt{\text{sgn}(\Delta)} = \begin{cases} 1, & \text{if } \Delta \geq 0 \\ i, & \text{if } \Delta < 0 \end{cases}$$

Thus, we modify our function:

```
rootfinder <- function(dA, dB, dC) {
  if (dA == 0 && dB == 0 && dC == 0) {
    vRoots <- Inf
  } else if (dA == 0 && dB == 0) {
    vRoots <- NULL
  } else if (dA == 0) {
    vRoots <- -dC / dB
  } else {
    # calculate the discriminant
    dDelta <- dB^2 - 4 * dA * dC
    if (dDelta > 0) {
      vRoots <- (-dB + c(1, -1) * sqrt(dDelta)) / (2 * dA)
    } else if (dDelta == 0) {
      vRoots <- rep(-dB / (2 * dA), 2)
    } else {
      di <- complex(1, 0, 1)
      vRoots <- (-dB + c(1, -1) * sqrt(-dDelta) * di) / (2 * dA)
    }
  }
  return(vRoots)
}
```

Suppose we have saved the function `rootfinder` in the script `rootfinder.R` which is located in the script folder inside our working directory.

To use the function, we first load it (using `source` or by copying and pasting into R), then call it, supplying suitable arguments.

```
# source("./scripts/rootfinder.r")
rootfinder(dA = 1, dB = 1, dC = 0)
#> [1] 0 -1
rootfinder(dA = 1, dB = 0, dC = -1)
#> [1] 1 -1
rootfinder(dA = 1, dB = -2, dC = 1)
#> [1] 1 1
rootfinder(dA = 1, dB = 1, dC = 1)
#> [1] -0.5+0.8660254i -0.5-0.8660254i
```

4.1.6 Advantages of coding with functions

Once a function is loaded, it can be used again and again without having to reload it.

User-defined functions can be used in the same way as predefined functions are used in R. In particular, they can be used within other functions.

The use of functions allows you to break down a programming task into smaller, logical units.

Large programs are typically made up of a number of smaller functions, each of which does a simple, well-defined task.

4.1.7 Variables inside a function

The arguments and variables that are defined within a function exist only within that function.

If you define and use a variable `x` inside a function, it does not exist outside the function.

```
test <- function(x) {
  y <- x + 1
  return(y)
}
test(1)
x # This will return an error
y # This will return an error
```

4.1.8 The scope of a variable

If variables with the same name exist inside and outside a function, then they are separate and do not interact at all.

You can think of a function as a separate environment that communicates with the outside world only through the values of its arguments and its output expression.

That part of a program in which a variable is defined is called its scope. Restricting the scope of variables within a function provides an assurance that calling the function will not modify variables outside the function, except by assigning the returned value.

Beware, however, the scope of a variable is not symmetric!

Variables defined inside a function cannot be seen outside, but variables defined outside the function can be seen inside the function, *provided there is not a variable with the same name defined inside*.

Consider for example:

```
test2 <- function(x) {  
  y <- x + z  
  return(y)  
}  
z <- 1  
test2(1)  
#> [1] 2  
z <- 2  
test2(1)  
#> [1] 3
```

Best practice is always to pass variables to be used inside the function as additional arguments:

```
test2 <- function(x, z) {  
  y <- x + z  
  return(y)  
}
```

4.2 Arguments of a function

The arguments of a function can be mandatory or optional depending on the function specification.

The arguments of an existing function can be obtained by calling the `formals` function:


```
formals(rootfinder)
#> $dA
#>
#>
#> $dB
#>
#>
#> $dC
```

4.2.1 Default arguments

In order to simplify calling functions, some arguments may be assigned default values. Default values are used in the case where the argument is not provided in the call to the function:

```
test3 <- function(x = 1) {
  return(x)
}
test3(2)
#> [1] 2
test3()
#> [1] 1
```

Calling `formals` on a function with default arguments, the value of the default arguments will be stated as well

```
formals(matrix)
#> $data
#> [1] NA
#>
#> $nrow
#> [1] 1
#>
#> $ncol
#> [1] 1
#>
#> $byrow
#> [1] FALSE
#>
#> $dimnames
#> NULL
matrix()
```

```
#>      [,1]
#> [1,]    NA
```

If an argument is omitted in a function call, R automatically assigns arguments to variables from the left and uses the default values for the unassigned arguments:

```
test4 <- function(x = 1, y = 1, z = 1) {
  return(x * 100 + y * 10 + z)
}
test4(2, 2)
#> [1] 221
test4(y = 2, z = 2)
#> [1] 122
```

In general, naming the arguments in the function call is good practice, because it increases the readability and eliminates one potential source of errors.

4.2.2 Default arguments: Partial matching

Sometimes you will want to define arguments so that they can take only a small number of different values, and the function will stop informatively if an inappropriate value is passed.

When writing the function, we include a vector of the permissible values for any such argument, and then check them using the `match.arg` function. For example:

```
MyFunc <- function(vX, method = c("add", "multiply")) {
  method <- match.arg(method)
  if (method == "add") {
    return(sum(vX))
  } else {
    return(prod(vX))
  }
}
MyFunc(c(1, 2, 3, 4), method = "multiply")
#> [1] 24
MyFunc(c(1, 2, 3, 4), method = "add")
#> [1] 10
```

The R function `ar`, for example, which fits an autoregressive time series model to data, takes (among others) the argument specifying the method by which the model is estimated.

```
method = c("yule-walker", "burg", "ols", "mle", "yw")
```

4.3 Ellipsis

R provides a very useful means of passing arguments, unaltered, from the function that is being called to the functions that are called within it.

These arguments do not need to be named explicitly in the outer function, hence providing great flexibility.

To use this facility, you need to include ... in your argument list. These three dots (an ellipsis) act as a placeholder for any extra arguments given to the function.

Consider for example the function ‘square of the mean’:

```
SquareMean <- function(vX, ...) {  
  dSM <- mean(vX, ...)~2  
  return(dSM)  
}  
SquareMean(c(1, 3, 5, NA))  
#> [1] NA  
SquareMean(c(1, 3, 5, NA), na.rm = TRUE)  
#> [1] 9
```

4.4 Vector-based programming using functions

Many R functions are vectorized, meaning that given vector input the function acts on each element separately, and a vector output is returned, e.g.:

```
vX <- c(1, NA, 3, 8)  
is.na(vX)  
#> [1] FALSE TRUE FALSE FALSE
```

This is a very powerful aspect of R that allows for compact, efficient, and readable code. Moreover, for many R functions, applying the function to a vector is much faster than if we were to write a loop to apply it to each element one at a time.

To further facilitate vector-based programming, R provides a family of powerful and flexible functions that make it easier for user-defined functions to handle vector inputs. These belong to the **apply** family of functions.

Here we cover, `apply`, `sapply`, `lapply`, and `mapply`. Other functions belonging to this family are: `tapply`, `vapply`, and `eapply`, which require a bit of advanced R programming knowledge.

4.4.1 `sapply`

The effect of `sapply(vX, FUN)` is to apply the function `FUN` to every element of vector `vX`. That is, `sapply` returns a vector whose i -th element is the value of the expression `FUN(vX[i])`.

Consider the function, `f`, “sum of all integers lower than `X`”:

```
f <- function(dX) {  
  if (dX < 0) {  
    stop("dX needs to be positive.")  
  }  
  dSum = 0.0  
  iC = 0  
  while (iC <= dX) {  
    dSum = dSum + iC  
    iC = iC + 1  
  }  
  return(dSum)  
}
```

Suppose we want to apply the function `f` to the following vector of observations `vX = c(1, 4, 9.478, 6, 75, 0.48)`. We can of course write a `for` loop:

```
vX <- c(1, 4, 9.478, 6, 75, 0.48)  
vSum <- numeric(length(vX))  
for (i in 1:length(vSum)) {  
  vSum[i] <- f(vX[i])  
}  
vSum  
#> [1]      1     10     45     21 2850      0
```

Or we can use the `sapply` function:

```
vSum <- sapply(vX, f)  
vSum  
#> [1]      1     10     45     21 2850      0
```

Note that R performs a loop over the elements of `vX`, so execution of this code is not faster than execution of an equivalent loop - it is just faster to type.

If `FUN` has arguments other than `X[i]`, then they can be included using the three dots (an ellipsis) as shown above. That is, `sapply(X, FUN, ...)` returns `FUN(X[i]; ...)` as the i -th element.

```
g <- function(dX, iN) {  
  if (dX < 0) {  
    stop("dX needs to be positive.")  
  }  
  dSum <- 0.0  
  iC <- 0  
  while (iC <= dX) {  
    dSum <- dSum + iC^iN  
    iC <- iC + 1  
  }  
  return(dSum)  
}  
sapply(vX, g, iN = 2)  
#> [1]      1      30     285     91 143450      0
```

`lapply` does the same as `sapply` but always returns a `list`.

4.4.2 `sapply` and `lapply`

That is, the arguments `...` are passed directly from `sapply` to `FUN`, thus allowing you to use a function with more than one argument.

Note that the values of the arguments `...` are the same for each of the different arguments of `vx`. In order to vectorize over more than one argument (over both `vX` and `...`) we need to use `mapply`.

`lapply` does the same as `sapply`, but always returns a `list`.

4.4.3 `mapply`

`mapply` is a multivariate version of `sapply` and can be used to call a function `FUN` over vectorized arguments one index at a time. In other words, the function is first called over elements at index 1 of all vectors (or lists), it is then called over all elements at index 2 and so on.

Consider for example the function `rep` which takes arguments: `x` and `times`. `mapply` allows you to write:

```

mapply(rep, times = c(4, 3), x = c(2, 4))
#> [[1]]
#> [1] 2 2 2 2
#>
#> [[2]]
#> [1] 4 4 4

```

Or using `mapply` on our `power_fct(vX, power)`:

```

mapply(power_fct, vX = 1:4, power = c(2, 2, 3, 3))
#> [1] 1 4 27 64

```

4.4.4 `apply`

If you wish to apply a function that takes a vector argument to each of the rows (or columns) of a matrix (or array), then use the function `apply`. Its formula is:

```

apply(X, MARGIN, FUN, ...)

```

Where:

- `X` is an **array**.
- `MARGIN` is the index/indices of the **array** to which apply `FUN`.
- `FUN` is the function to apply.
- ... are extra arguments for `FUN`.

Note that `MARGIN` can be a vector of indices. If `X` is a **matrix** (an array of 2 dimensions), then `MARGINS = 1` indicates rows, while `MARGINS = 2` indicates columns.

Example with `apply`

To compute the cumulative sum over the columns of a matrix `mX` we write:

```

mX <- matrix(1:16, 4, 4)
mX
#>      [,1] [,2] [,3] [,4]
#> [1,]    1    5    9   13
#> [2,]    2    6   10   14
#> [3,]    3    7   11   15
#> [4,]    4    8   12   16

```

```

apply(mX, 2, cumsum)
#>      [,1] [,2] [,3] [,4]
#> [1,]    1    5    9   13
#> [2,]    3   11   19   27
#> [3,]    6   18   30   42
#> [4,]   10   26   42   58

```

4.4.5 Exercise

- Define the 3 x 4 matrix:

$$mA = \begin{bmatrix} 1 & 10 & 7 & 4 \\ 8 & 5 & 2 & 11 \\ 6 & 3 & 9 & 12 \end{bmatrix}$$

- Use the `apply` function to compute the row sums and the column sums of your matrix.
- Use the `apply` function to sort the columns of `mA` in a decreasing order and the rows of `mA` in an increasing order.
- Use the `sapply` function to find sine of all elements of the first row of `mA` and cosine of all elements of the second column of `mA`. Is this the easiest way to find the sine and cosine of a row/column of `mA`?

Solution

```

mA <- matrix(c(1, 10, 7, 4, 8, 5, 2, 11, 6, 3, 9, 12), 3, 4, byrow = TRUE)
mA
#>      [,1] [,2] [,3] [,4]
#> [1,]    1   10    7    4
#> [2,]    8    5    2   11
#> [3,]    6    3    9   12
t(apply(mA, 1, cumsum)) # compute the row sums
#>      [,1] [,2] [,3] [,4]
#> [1,]    1   11   18   22
#> [2,]    8   13   15   26
#> [3,]    6    9   18   30
apply(mA, 2, cumsum) # compute the column sums
#>      [,1] [,2] [,3] [,4]
#> [1,]    1   10    7    4
#> [2,]    9   15    9   15
#> [3,]   15   18   18   27
apply(mA, 2, sort, decreasing = TRUE) # sort columns in decreasing order

```

```

#>      [,1] [,2] [,3] [,4]
#> [1,]    8   10    9   12
#> [2,]    6    5    7   11
#> [3,]    1    3    2    4
t(apply(mA, 1, sort, decreasing = FALSE)) # sort rows in increasing order
#>      [,1] [,2] [,3] [,4]
#> [1,]    1    4    7   10
#> [2,]    2    5    8   11
#> [3,]    3    6    9   12
sapply(mA[1, ], sin) # sine of first row using sapply
#> [1]  0.8414710 -0.5440211  0.6569866 -0.7568025
sapply(mA[, 2], cos) # cosine of second column using sapply
#> [1] -0.8390715  0.2836622 -0.9899925
# the easier way is to do 'sin(mA[, 1])'

```

4.4.6 Recursive programming

Recursive programming is a powerful programming technique, made possible by functions. A recursive program is simply one that calls itself. This is useful because many algorithms are recursive in nature.

Consider for example the evaluation of $n!$, i.e., the factorial of the non-negative integer n . We know that $n! = n(n-1)!$, such that we can write a function like:

```

myfactorial <- function(iN) {
  if ((iN == 0) | (iN == 1)) {
    return(1)
  } else {
    return(iN * myfactorial(iN - 1))
  }
}
myfactorial(5)
#> [1] 120

```

4.4.7 Multiple outputs

A function can generate multiple output values.

Example: parameter estimates, their standard errors, and the optimized log-likelihood value.

A strategy in R is to create a `list` with all your outputs and return it as a single output of a function. For example:


```
lOut = list(Par = vParam, SE = vStdErr, LLK = dLogLik)
return(lOut)
```

4.5 Comments

Remember, you often want to re-use a code. It is often hard to read code that someone else has written, or code that you have written yourself months ago. Using sensible function and variable names, and Hungarian notation helps... somewhat.

But also: **comment your code!**

In R, `#` signals a comment to the end of the line:

```
vBeta <- c(0, 1, -5) # Initial values
```

It is best practice to declare what a script does at its beginning. Furthermore, at the start of *every* function, put something like:

```
##
## FunctionName(Inputs)
##
## Purpose:
## Description of what the function does
##
## Input:
## List of inputs, describing what they represent
##
## Output:
## List of outputs, describing what they represent
##
## Return value:
## List of return values, describing what they represent
##
```

In RStudio, you can use the **Ctrl+Shift+C** shortcut to comment multiple lines at once.

In addition, add a general description of the program at the top of the .R file. This should include your name and the date for future reference. It may seem like a lot of work, but it will pay off in the end. So: comment a lot!

4.5.1 Exercise

- Define a function, `f`, that calculates the output of the following recursive algorithm, where $n \geq 0$ is an integer.
 1. If $n < 2$, return n .
 2. If $n \geq 2$, return $f(n) = f(n-1) + f(n-2)$
- Run the function
- Now pass the function in `sapply` along with the additional vector of integers, e.g., `(1:20)`, do you recognize the pattern?
- Bonus: Comment the code

Solution

```
f <- function(iN) {  
  if (iN < 2) { # stopping criterion  
    return(iN)  
  } else {  
    return(f(iN - 1) + f(iN - 2)) # recursive part  
  }  
}  
  
sapply(1:20, f) # apply function f to a vector  
#> [1] 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610  
#> [16] 987 1597 2584 4181 6765  
# regarding the output: it's the fibonacci series
```

5 Program Efficiency and Parallel Computing

5.1 Good practice for programming

5.1.1 Programming in theory - plan ahead

- Research question: What do I want to know?
- Data: What inputs do I have?
- Output: What kind of output do I expect/need?
- Modelling:
 - What is the structure of the problem?
 - Can I write it down in equations?
- Estimation: What procedure for estimation is needed (OLS, ML, GMM, etc)?

5.1.2 Closer to practice:

- Blocks:
 - Is the project separable into blocks? What separate routines could I write?
 - Are these blocks, independent, or possibly dependent?
 - What separate routines could I write?
 - Are there any routines available, in my own old code, or from other sources?
 - Check to see if the routines are working, as well as how the program is running across multiple routines.
 - How can I name functions and variables that I am confident I will remember later (i.e., after 3 months)? Make use of Hungarian notation.
- On paper, define the following for each routine/step/function:
 - What inputs does it have? What are the outputs?

5.1.3 Use of Functions and computational efficiency

Why all the small functions when you could do everything in one?

- Avoiding the use of duplicate code. You don't want to type in the entire code for a specific operation every time.
- Helps to identify where things go wrong.
- Using previously written code.
- Code sharing.

Once you've identified a bottleneck, you must speed it up:

- Look for existing solutions.
- Vectorize.
- Parallelize.
- Avoid copies.

When using a computer to perform intensive numerical calculations, we must keep two things in mind: code efficiency and execution time. Advanced topics and other parallelization strategies not covered in this lecture are available at the “High-Performance and Parallel Computing with R” CRAN Task View: <https://CRAN.R-project.org/view=HighPerformanceComputing>

5.2 Code execution time

Ultimately, we measure how efficient a program is by how long it takes to run, which will depend on the language it is written in and the computer it is run on.

Also, computers typically are doing a number of things at once, so the time taken to run a program will also depend on what else the computer is doing at the same time.

R provides the function `system.time` to measure how many CPU (Computer Processing Unit) seconds are spent evaluating an expression:

```
system.time({  
  mA <- matrix(rnorm(1000^2), 1000, 1000)  
  solve(mA)  
})  
#>      user  system elapsed  
#>   0.83    0.00    0.83
```

We know that in R, creating or changing the size of a vector (also called redimensioning an array) is relatively slow. Consequently, operations like:

```
vX <- 0
for (i in 1:1e5) {
  vX <- c(vX, i)
}
```

Should be avoided in favour of:

```
vX <- numeric(1e5 + 1)
for (i in 1:1e5) {
  vX[i + 1] = i
}
```

Indeed, the code speeds up by a factor of about 537x:

```
system.time({
  vX <- 0
  for (i in 1:1e5) {
    vX <- c(vX, i)
  }
})
#>    user  system elapsed
#>  8.77    1.85   10.62

system.time({
  vX <- numeric(1e5 + 1)
  for (i in 1:1e5) {
    vX[i + 1] = i
  }
})
#>    user  system elapsed
#>     0      0      0
```

5.3 Loops versus vectors

In R, vectorized operations are faster than equivalent loops. The difference is because R is a very high-level language.

In R, it is relatively easy to create and manipulate variables. The price we pay for this flexibility is speed.

When you evaluate an expression in R, it is ‘translated’ into a faster lower-level language before being evaluated, then the result is translated back into R. The translation is what takes much of the time, and vectorization saved on the amount of translation required.

Take the following code to square each element of `vX`:

```
for (i in 1:length(vX)) vX[i] <- vX[i]^2
```

Each time we evaluate the expression `vX[i] <- vX[i]^2`, we translate `vX` all at once and then square it, before translating the answer back: all the work takes place in our faster lower-level language.

Many of R’s functions are vectorized, which means that if the first argument is a vector, then the output will be a vector of the same length, computed by applying the function element-wise to the input vector.

5.3.1 But... If you have to use a loop in R...

Sometimes, loops can be intuitive and more easily visualized than vectorized code or matrix operations. For instance, many algorithms stated in academic papers are listed in terms of for-loop syntax.

If vectorization simply takes too long time or simply is impossible then by all means use a loop, but...

- Pre-allocate vector/matrix prior to the loop
- Iterate as few times as possible

Basically, everything that is not needed in the loop should not be in it! But, do avoid loops in R if possible! The penalty of loops in R is simply much larger than lower level languages, i.e., C++. Note that it’s relatively easy to make R code much faster!

5.3.2 The flexibility vs speed trade-off

The `mean()` function example:

```
library("microbenchmark")
#> Warning: package 'microbenchmark' was built under R version 4.3.3
vY <- runif(1E7)
microbenchmark(sum(vY) / length(vY), mean(vY))
#> Unit: milliseconds
#>      expr      min       lq      mean  median      uq      max neval
#> sum(vY)/length(vY)  7.6299  7.66385  7.718192  7.7123  7.75485  7.9085   100
```

```
#>          mean(vY) 15.3297 15.42645 15.493443 15.4693 15.50955 16.1694 100
all.equal(sum(vY) / length(vY), mean(vY))
#> [1] TRUE
```

... also you would probably not have too hard of a time programming a tailored simple linear regression function that beats the `lm()` built-in R function. But `lm()` offers a lot of flexibility...

5.3.3 Arithmetic is not always the fastest

```
library("microbenchmark")
vY <- runif(1E5)
mX <- replicate(10, runif(1E5))
microbenchmark(vY^(1/2), sqrt(vY))
#> Unit: microseconds
#>      expr      min       lq      mean  median       uq      max neval
#>  vY^(1/2) 2464.9 2494.95 2625.742 2690.85 2708.80 2990.0   100
#>  sqrt(vY)  346.0  347.80  510.428  560.20  566.45 4494.3   100
all.equal(vY^(1/2), sqrt(vY))
#> [1] TRUE
microbenchmark(t(mX) %*% vY, crossprod(mX, vY))
#> Unit: milliseconds
#>      expr      min       lq      mean  median       uq      max neval
#>  t(mX) %*% vY 2.9399 3.1030 3.767998 3.21760 3.33095 10.4770   100
#> crossprod(mX, vY) 1.6713 1.7026 1.738440 1.71985 1.75540  2.0801   100
all.equal(t(mX) %*% vY, crossprod(mX, vY))
#> [1] TRUE
```

5.3.4 Vectorization: Tips and Tricks for Speed

- Vector-based programming (also mentioned in previous lectures)
- Vectorized functions
- Vectorized if/else?
- Subsetting/subscripting of data structures

5.3.5 Summing the columns of a matrix

Consider the problem of evaluating the sum of the columns of a 10000 x 10000 matrix `mA`:

```
mA <- matrix(rnorm(1e8), nrow = 10000)
```

To solve our problem, we have several options:

1. A double loop of summations

```
system.time({
  vColSum <- rep(NA, ncol(mA))
  for (i in 1:ncol(mA)) {
    s <- 0
    for (j in 1:nrow(mA)) {
      s <- s + mA[j, i]
    }
    vColSum[i] <- s
  }
})
#>      user  system elapsed
#>   3.42    0.00    3.44
```

2. The use of apply

```
system.time(
  vColSum <- apply(mA, 2, sum)
)
#>      user  system elapsed
#>   0.69    0.35    1.03
```

3. A single loop of sums

```
system.time({
  vColSum <- numeric(ncol(mA))
  for (i in 1:ncol(mA)) {
    vColSum[i] <- sum(mA[, i])
  }
})
#>      user  system elapsed
#>   0.26    0.33    0.59
```

4. Exploit the mathematical formulation: $\mathbf{u}'\mathbf{A}$, where $\mathbf{u}$ is a vector of ones with length equal to the number for columns of $\mathbf{u}$:


```
vU <- rep(1, ncol(mA))
system.time({
  vColSum <- vU %*% mA
})
#>      user  system elapsed
#>    0.16    0.00    0.15
```

5. Using the dedicated R function:

```
system.time({
  vColSum <- colSums(mA)
})
#>      user  system elapsed
#>    0.08    0.00    0.08
```

5.3.6 Comparison

Strategy	CPU Time	Gain over double loop
Double loop	3,7	
<code>apply</code>	1,29	287%
One loop with sum	0,77	481%
$u'A$	0,17	2176%
<code>colSums</code>	0,08	4625%

Sometimes the easiest solution is also the preferred one.

5.3.7 Vectorized if/else statement

The `if else` statement in R is one of the few things that does not take vector inputs.

One possibility is to use loops around the if-statements. Another is to use the `ifelse()` function that takes vector/matrix inputs. For a single vector, this should be faster than a for-loop approach.

A simple for-loop if-statement function:

```

fun <- function(vY) {
  iN <- length(vY) # Number of elements
  vD <- numeric(iN) # Pre-allocate vector
  for (i in 1:iN) {
    if(vY[i] < 0.5) vD[i] <- -1 else vD[i] <- 1
  }
  vD
}

```

Vs. the vectorized function `ifelse()`:

```

ifelse(vY < 0.5, -1, 1)

```

Using the `microbenchmark` package to time the two functions a 100 times. Which function is the fastest?

```

library("microbenchmark")
vY <- runif(1E5)
microbenchmark(fun(vY), ifelse(vY < 0.5, -1, 1))
#> Unit: milliseconds
#>
#>      expr      min       lq      mean  median       uq      max neval
#>  fun(vY)  6.5482  6.62155  6.716239  6.6653  6.73355 10.3179   100
#> ifelse(vY < 0.5, -1, 1) 3.0435  3.17430  3.283494  3.2508  3.37880  3.8028   100
all.equal(fun(vY), ifelse(vY < 0.5, -1, 1))
#> [1] TRUE

```

`ifelse()` is definitely faster and simpler!

Suppose:

```

mY <- matrix(4E2, 200, 200)

```

The function now is:

```

fun3 <- function(mY) {
  iN <- nrow(mY) # Number of rows of mY
  iK <- ncol(mY) # Number of columns of mY
  mD <- matrix(0, iN, iK) # Pre-allocate to avoid growing objects!
  for (i in 1:iN) { # Outer loop
    for (j in 1:iK) { # Nested loop
      if (mY[i, j] < 0.5) mD[i, j] <- -1 # If true
    }
  }
  mD
}

```

```

    else mD[i, j] <- 1 # Otherwise
  }
}
mD
}

```

While, `ifelse` is still `ifelse(mY < 0.5, -1, 1)`. And obviously, faster:

```

microbenchmark(ifelse(mY < 0.5, -1, 1), fun3(mY))
#> Unit: microseconds
#>
#>      expr      min      lq      mean  median      uq      max neval
#> ifelse(mY < 0.5, -1, 1) 410.6   892.5  859.898   928.2   973.70 1169.4   100
#>      fun3(mY) 2992.6 3086.5 3194.583 3107.9 3173.25 8455.1   100
all.equal(ifelse(mY < 0.5, -1, 1), fun3(mY))
#> [1] TRUE

```

5.3.8 Subsetting/subscripting: One of the most powerful tools!

Subsetting is usually overlooked when attempting to vectorize code in R, but is usually quite effective and nearly always faster than alternatives.

You have already used it (or at least seen it in action) plenty of times, for extracting or removing rows/columns on different data structures.

In practice, subscripting is an invaluable tool for data management, i.e., cleaning or sample selection of data.

On a 200x200 matrix `mY`:

```

fun2 <- function(mY) {
  mBool <- (mY < 0.5) # Matrix of booleans
  mY[mBool] <- -1 # If true
  mY[!mBool] <- 1 # If false
  mY # output
}
microbenchmark(ifelse(mY < 0.5, -1, 1), fun2(mY), fun3(mY))
#> Unit: microseconds
#>
#>      expr      min      lq      mean  median      uq      max neval
#> ifelse(mY < 0.5, -1, 1) 407.6   909.55  893.255   937.75   981.80 1305.6   100
#>      fun2(mY) 222.7   506.45  520.884   523.10   558.75 2640.2   100
#>      fun3(mY) 2985.6 3081.10 3121.759 3101.95 3161.05 3444.7   100
all.equal(fun2(mY), fun3(mY))
#> [1] TRUE

```

5.3.9 Exercises in vectorization

- Reformulate exercise 5 from Exercise set 2 (the if else statement one) in terms of the `ifelse()` function (Hint: some functions take functions as input).
- Suppose you use a vector of draws from a standard normal dist. as input instead. Can you beat the `ifelse()` you just made somehow? Write a function that is faster than the code you wrote in the above question. Compare using the `microbenchmark` package.
- Find a way to vectorize calculations of `rowMeans` of a 100 by 100 matrix of 10000 random draws from a uniform distribution.

Solution:

```
# Using ifelse()
slowSolution <- function(vInput) {
  # ifelse(vInput <= 0, vInput <- - vInput ^ 3, ifelse(vInput <= 1, vInput <- vInput ^ 2, sq

  for (i in 1:length(vInput)) {
    ifelse(vInput[i] <= 0, vInput[i] <- - vInput[i] ^ 3, vInput[i])
    ifelse(vInput[i] <= 1, vInput[i] <- vInput[i] ^ 2, vInput[i])
    ifelse(vInput[i] > 1, vInput[i] <- sqrt(vInput[i]), vInput[i])
  }
  return(vInput)
}

# Fast solution using indexing
fastSolution <- function(vInput) {
  vInput[vInput <= 0] <- - vInput[vInput <= 0] ^ 3
  vInput[vInput <= 1] <- vInput[vInput <= 1] ^ 2
  vInput[vInput > 1] <- sqrt(vInput[vInput > 1])
  return(vInput)
}

set.seed(1)
vX <- rnorm(10)

microbenchmark(slowSolution(vX), fastSolution(vX))
#> Unit: microseconds
#>      expr   min    lq   mean median    uq   max neval
#> slowSolution(vX) 37.2 38.1 114.232   38.7 42.15 7345.3   100
#> fastSolution(vX)  2.0  2.4  46.598    2.7  3.00 4303.9   100
all.equal(slowSolution(vX), fastSolution(vX))
#> [1] TRUE
```

```
# Vectorization of rowMeans
vX <- rnorm(10000)
mX <- matrix(vX, 100, 100)
fRowMeans <- function(mInput) {
  return(rowSums(mInput) / ncol(mInput))
}

microbenchmark(rowMeans(mX), fRowMeans(mX))
#> Unit: microseconds
#>      expr    min     lq   mean  median     uq    max neval
#> rowMeans(mX) 20.9 21.1 21.907   21.1 21.40   47.1   100
#> fRowMeans(mX) 21.7 22.0 38.891   22.1 22.25 1581.5   100
all.equal(rowMeans(mX), fRowMeans(mX))
#> [1] TRUE
```

5.4 Parallel processing

5.4.1 Warning before parallellization

Optimizing code to make it run faster is an iterative process:

1. Find the biggest bottleneck (the slowest part of your code).
2. Try to eliminate it (you may not succeed but that's ok)
3. Repeat until your code is “fast enough”.

This sounds easy, but it's not. Parallel processing should first be considered when you cannot achieve efficiency gains by any other means... Or if you are really pressured on time.

5.4.2 Parallel Processing

Consider the problem of summing a vector of length n , which requires $n - 1$ separate additions. If each addition takes 1 second, and we do them one after another, then the whole calculation takes $n - 1$ seconds.

Now, suppose we split the vector into two and give each half to a different calculator. Each calculator spend $n/2 - 1$ seconds adding up their half, which happens concurrently, then we spend 1 second adding together the two bits, for a total of $n/2$ seconds.

Note, however, that communications between the processors requires time: i.e., the total time is usually $n/2 + \varepsilon$.

Clearly, parallelization requires multiple CPU's, or cores (computers). The R community has developed tools that support the splitting of computations among different machines on a network, and among different cores on a single machine.

The base package **parallel** provides tools that will work across most of the platforms that are supported by R. We first load the package and determine how many cores can be detected on the machine:

```
library(parallel)
detectCores()
#> [1] 6
```

5.4.3 Creating a cluster

We initialize a cluster using the `makeCluster` function:

```
cluster <- makeCluster(2)
```

A *cluster* with 2 *workers* has been created and its details are collected in the object `cluster`.

If you now open the Task Manager, you will find 2 new R instances. Having made the cluster, we can then test the cluster by using a simple example that calls a function using each worker within the cluster.

```
clusterCall(cluster, function(x) print("Pick me!"))
#> [[1]]
#> [1] "Pick me!"
#>
#> [[2]]
#> [1] "Pick me!"
```

5.4.4 Running jobs in parallel

Provided that our job can run in parallel (there is nothing recursive that cannot be allocated to different workers), we can exploit the parallelised version of the **apply** family of functions. For instance:

- `apply` → `parApply`
- `lapply` → `parLapply`
- `sapply` → `parSapply`

The use of these functions is exactly analogous to the ones belonging to the `apply` family. The only difference is that they accept an extra argument `cl` which accepts a `cluster` object. For instance:

```
waste.of.time <- function(x) for (i in 1:10000) i
system.time(lapply(1:10000, waste.of.time))
#>    user  system elapsed
#>  1.12    0.00    1.12
system.time(parLapply(cl = cluster, 1:10000, waste.of.time))
#>    user  system elapsed
#>  0.00    0.00    0.49
```

5.4.5 Kill your cluster

Once the cluster is no longer necessary, we kill it with:

```
stopCluster(cluster)
```

If we omitted this command, then the cluster would be disbanded only when the R session that created it is terminated. This means that until you close R, your computer's resources will be occupied by unnecessary processes.

5.4.6 Small exercises on the parallel universe

- *Install and load the package “parallel” into your R session.*
- *Detect how many cores R can detect on your system.*
- *If more than 1, then form a cluster using all but one of the cores detected above.*
- *Use the function(s) `waste.of.time()` and/or `fibo()` (`fibo()` can be found in the script from the last exercise from last lecture)*
- *Pass your function to the `sapply()` function and pass inputs 1 to 30 by `sapply` to your function.*
- *Instead, pass the function to the `parSapply()` function. (Remember to export the function to the clusters)*
- *Is `parSapply()` faster than regular `sapply`? (Don't do too many repetitions of `sapply()/parSapply()`; only at most 1:30 with `fibo()`.)*
- *Terminate the assigned cluster!*

Solution:

```
# Install if necessary and load the library quietly
if(!require("parallel")) install.packages("parallel")
suppressMessages(library(parallel))

# Make cluster with more cores than 1 if possible
if (detectCores() > 1) cluster <- makeCluster(detectCores() - 1)

waste.of.time <- function(x) for (i in 1:10000) i
fibo <- function(n) {
  if (n < 2) return(n)
  return(fibo(n - 1) + fibo(n - 2))
}

# `fibo` is a recursive function and thus needs to be exported to the cluster
clusterExport(cluster, "fibo")

# Without a cluster
system.time(sapply(1:30, waste.of.time))
system.time(sapply(1:30, fibo))

# Using the cluster
system.time(parSapply(cl = cluster, 1:30, waste.of.time))
system.time(parSapply(cl = cluster, 1:30, fibo))

# Kill the cluster
stopCluster(cluster)
```

5.5 Memory issues

Computer memory comes in a variety of forms. For most purposes it is sufficient to think in terms of RAM (random access memory), which is fast, and the hard disk, which is slow.

Variables require memory. R stores variables in virtual memory, which is a seamless combination of RAM and hard disk space, managed by the underlying operating system.

The operating system will use R in preference to disk space, but if your variables require more memory than the amount of RAM physically installed, then it will have to use disk space as well, and your program will slow down as a result.

As soon as a vector (or list) is too large to store in RAM all at once, the speed at which you can use it will drop dramatically.

If the vector is sufficiently large, then it may not be possible to store it at all, in which case you are said to have run out of memory.

Also note that R has an absolute limit on the length of a vector of $2^{31} - 1 = 2,147,483,647$.

In the case your code runs out of memory, you will need to break your vectors down into small subvectors and deal with each in turn.

Alternatively, you can use the `rm` function to delete objects from memory and free space:

```
dA <- 5
ls()
#> [1] "dA"
rm("dA")
```

To remove all objects, you can use `rm(list = ls())`.

5.6 Small hints

- Adding / subtracting tends to be better than multiplying.
- Hence, log-likelihood $\sum \log L_i$ is better than likelihood $\prod L_i$
- Simplify your equations, minimize number of operations.
- Don't do $x = \exp(\log(z))$ if you can avoid it.
- Split your program into smaller tasks. Keep it simple!

6 Introduction to Simulation

Simulation is essential for quantitative analysis. It is widely applied in estimation, hypothesis testing, equation solving, numerical computing, etc.

6.1 Seeding

Once the seed is known, the whole sequence of random variables can be reproduced. This property facilitates repetition of experimentation.

- `set.seed(seed)` is used to specify the seed.
- The current state of the random number generator is kept in the vector `.Random.seed`.
- If the random number generator is not initialized before generating pseudo-random numbers, R initializes it using a value taken from the system clock.

For example, run first:

```
set.seed(10086)
runif(5)
#> [1] 0.708768032 0.384298612 0.003025926 0.277903935 0.714458605
```

Now run again:

```
runif(5)
#> [1] 0.3166092 0.6863317 0.4725650 0.2655531 0.6193426
```

Setting seed back to 10086, we can reproduce results in the first frame:

```
set.seed(10086)
runif(5)
#> [1] 0.708768032 0.384298612 0.003025926 0.277903935 0.714458605
```

You can also save the current state by:

```
RNG.state <- .Random.seed
runif(5)
#> [1] 0.3166092 0.6863317 0.4725650 0.2655531 0.6193426
```

Now try to implement the following codes:

```
.Random.seed <- RNG.state
runif(5)
#> [1] 0.3166092 0.6863317 0.4725650 0.2655531 0.6193426
```

In this way you reproduce the results.

6.2 Discrete Random variables

6.2.1 Simulate Discrete Random Variables

Suppose that we would like to simulate a **discrete** random variable X .

- X takes values in $\{0,1,2,\dots\}$
- X has the CDF F

Suppose m is any number from the set $0, 1, 2, \dots$ such that it satisfies the following equation, given that U follows the uniform distribution over $(0, 1)$.

$$P(X = m) = F(m) - F(m - 1) = P(F(m - 1) < U \leq F(m))$$

Hence, the algorithm for simulating X is:

1. Simulate a random variable $U \sim \text{Uniform}(0, 1)$;
2. Find the minimal m such that $F(m) \geq U$, that is,

$$m = \min\{k \geq 0 : F(k) \geq U\}$$

3. Set $X = m$.

This algorithm vectorizes the function and takes advantage of the ellipsis ..., since at times additional arguments need to be passed to the cumulative distribution function.

```

Discrete.Simulate <- function(F, size, ...) {

  # F - Cumulative distribution function
  # size - number of i.i.d. random variables to be simulated

  m <- 0
  U <- runif(size)
  X <- rep(NA, size)
  X[F(0, ...) >= U] <- 0

  while (any(F(m, ...) < U)) {
    m <- m + 1
    X[(F(m, ...) >= U) & (F(m - 1, ...) < U)] <- m
  }

  return(X)
}

```

For example, suppose that we would like to simulate 1000 i.i.d. discrete random variables whose probability density function is:

$$p(x) = \begin{cases} 0.2, & \text{if } x = 1 \\ 0.3, & \text{if } x = 2 \\ 0.5, & \text{if } x = 3 \end{cases}$$

The corresponding cumulative distribution function is:

$$F(x) = \begin{cases} 0.2, & \text{if } 1 \leq x < 2 \\ 0.5, & \text{if } 2 \leq x < 3 \\ 1, & \text{if } x \geq 3 \end{cases}$$

The following program makes use of `Discrete.Simulate` we just wrote to accomplish the task, and in addition produces the histogram:

```

# cumulative distribution function

F <- function(x) {
  return(0.2 * ((x >= 1) && (x < 2)) + 0.5 * ((x >= 2) && (x < 3)) + (x >= 3))
}

```

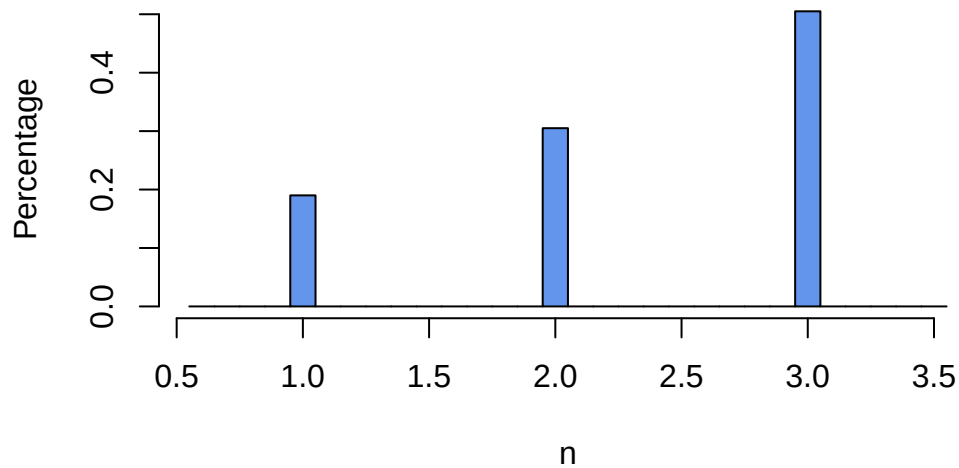
```

# draw 1000 i.i.d. random variables whose CDF is F

set.seed(10086)
X <- Discrete.Simulate(F, 1000)

# histogram
h <- hist(X, breaks = seq(0.55, 3.55, 0.1), plot = FALSE)
h$density <- h$counts / 1000
plot(h, freq = FALSE, col = "cornflowerblue",
     main = "", xlab = "n", ylab = "Percentage")

```



Simulate a binomial random variable $X \sim \text{Binomial}(n, p)$.

- The density is given as

$$p(k) = \mathbb{P}(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \text{ for } 0 \leq k \leq n$$

- The CDF is given as

$$F(x) = \sum_{k=0}^{\lfloor x \rfloor} \binom{n}{k} p^k (1-p)^{n-k}$$

The following program draws 1000 random variables with the distribution `Binomial(10, 0.5)`. Here, we see how ellipsis ... renders `Discrete.Simulate` more flexible.

```
# cumulative distribution function of binomial distribution

F <- function(x, n, p) {
  Index <- 0:floor(x)
  Density <- choose(n, Index) * p^(Index) * (1-p)^(n-Index)
  return(sum(Density))
}

# draw 1000 i.i.d. binomial random variables

set.seed(10086)
X <- Discrete.Simulate(F, 1000, 10, 0.5)
```

Alternatively, you can make use of the built-in function `rbinom`:

```
Y <- rbinom(1000, 10, 0.5)
```

Compare the histograms:

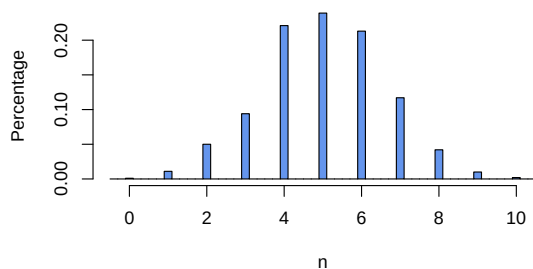
```
h1 <- hist(X, breaks = seq(-0.5, 10.5, 0.2), plot = FALSE)
h1$density <- h1$counts / 1000
plot(h1, freq = FALSE, col = "cornflowerblue",
     main = "", xlab = "n", ylab = "Percentage")
h2 <- hist(Y, breaks = seq(-0.5, 10.5, 0.2), plot = FALSE)
h2$density <- h2$counts / 1000
plot(h2, freq = FALSE, col = "cornflowerblue",
     main = "", xlab = "n", ylab = "Percentage")
```

6.3 Inversion

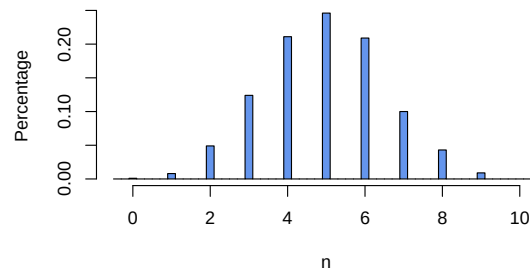
6.3.1 Inversion Method

Thus far, we have discussed about simulation of random variables taking values in $\{0, 1, \dots\}$. Next, we explain how to simulate random variable taking value in $\mathbb{R}$.

- Suppose F is a cumulative distribution function on $\mathbb{R}$ and U is a random variable uniformly distributed over $(0, 1)$, then:



(a) Use Discrete.Simulate



(b) Use rbinom

$$X = F^{-1}(U)$$

- where $F^{-1}(\cdot)$ is an inverse function of F .
- **Inversion method:** As long as you can compute F^{-1} , you can simulate a random variable using a uniformly distributed random variable.

Recall, that is $X \sim \text{Exp}(\lambda)$, its cumulative distribution function is given by:

$$F(x) = \begin{cases} 0, & \text{for } x < 0 \\ 1 - e^{-\lambda x}, & \text{for } x \geq 0 \end{cases}$$

One can show that the inverse F^{-1} is given by:

$$F^{-1}(y) = -\lambda^{-1} \log(1 - y), 0 \leq y \leq 1$$

When, $y = 1$, $F^{-1}(y) = \infty$.

By means of $-\lambda^{-1} \log(1 - U)$, we can simulate the desired exponential distribution with intensity λ . Observe that $1 - U$ also has uniform distribution over $(0, 1)$. Hence, it is equivalent to use $-\lambda^{-1} \log(U)$, which is more often used in practice.

Here is an example of drawing 1000 i.i.d. random variables with distribution $\text{exp}(0.7)$.

```
Exponential.Simulate <- function(lambda, size) {
  # lambda - intensity of the exponential distribution
  # size - number of i.i.d. random variables to be drawn
```

```

U <- runif(size)
return(-1/lambda * log(U))
}

# A test of the function
set.seed(10086)
X <- Exponential.Simulate(0.7, 1000)

```

Alternatively, we can also make use of the built-in function `rexp`, but an example is not provided.

Below is a histogram of the sample drawn with `Exponential.Simulate`. We also superimpose the theoretical density function (now, with code included!):

```

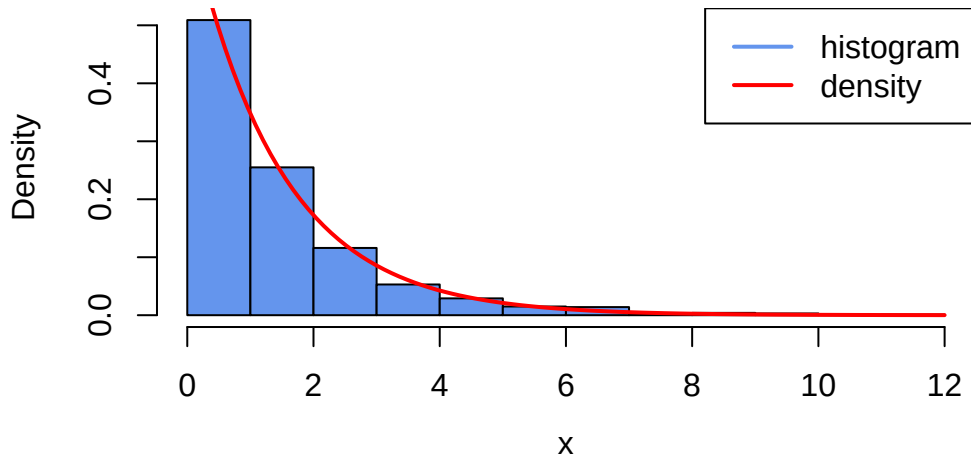
# 1. Create the Histogram
hist(X,
      freq = FALSE,          # Use density instead of frequency on y-axis
      breaks = 15,           # Adjust number of breaks for desired resolution
      col = "cornflowerblue",
      xlab = "",              # Remove default x-label (add it later with mtext)
      ylab = "Density",
      main = "",
      xlim = c(0, 12))

# 2. Superimpose the Theoretical Density Curve
curve(dexp(x, rate = 0.7), # Theoretical exponential density function
      from = 0,             # Start of the curve
      to = 12,              # End of the curve
      col = "red",
      lwd = 2,               # Adjust line thickness
      add = TRUE)           # Add to the existing plot (histogram)

# 3. Add Labels and Title (using mtext for better control)
mtext("x", side = 1, line = 2.5) # Add x-axis label below the axis
mtext("Histogram of Exponential Sample and Theoretical Density",
      side = 3, line = 1, outer = TRUE) # Title at the top margin

# 4. Add a Legend
legend("topright",          # Position of the legend
      legend = c("histogram", "density"), # Text for the legend
      col = c("cornflowerblue", "red"),   # Colors of the legend elements
      lwd = 2)                # Line width for the legend

```

6.3.2 Small Exercise

- Consider the random variable With Gumble distribution $X \sim \text{Gumbel}(0, 1)$. The CDF is given by $F_X(x) = \exp(-\exp(-x))$
- Now, suppose you want to simulate random draws from this distribution by use of the inverse transform method.
 - Derive the inverse transform $F_X^{-1}(y)$ from this cdf. (Hint: solve $\exp(-\exp(-F_X^{-1}(y))) = y$). Write a function in R implementing the derived inverse transform.
 - Set the seed to 1234 and use 100 random draws of standard uniforms.
 - The first 4 draws will be: -0.7766432 0.7458437 0.7022162 0.7495061

Solution

Note for the first part:

$$\begin{aligned} \exp(-\exp(-F_X^{-1}(y))) &= y \\ \Rightarrow F_X^{-1}(y) &= -\log(-\log(y)) \end{aligned}$$

Then, implementing the inverse transform:

```
Gumbel.Simulate <- function(min, max, size) {  
  U <- runif(size, min, max)  
  return(-log(-log(U)))  
}  
  
set.seed(1234)  
  
X <- Gumbel.Simulate(0, 1, 100)  
X[1:4]  
#> [1] -0.7766432  0.7458437  0.7022162  0.7495061
```

6.4 Acceptance-Rejection

The inversion method works fine if we can find F^{-1} analytically. It becomes less feasible when F^{-1} does not admit a closed-form representation, although it is still possible to employ (time-consuming) numerical methods to approximate F^{-1} .

Yet, there is another clever method that is often applicable, called the **acceptance-rejection method**.

Suppose we would like to simulate a random variable X whose cumulative distribution is F , and

$$F(x) = \int_{-\infty}^x f(t) dt$$

for the probability density f .

The basic idea of the acceptance-rejection method is to find another distribution G with probability density function g , such that:

- A random variable with distribution G can be simulated using some efficient method (such as inversion method).
- g is nonzero on the support of f , that is, $g(x) > 0$ as long as $f(x) > 0$.
- f/g is uniformly bounded, that is, $c = \sup_X \frac{f(x)}{g(x)} < \infty$

Once the forgoing conditions are satisfied, we shall say that g **envelopes** f .

The acceptance-rejection algorithm for generating a random variable distributed as F :

1. Generate a random variable Y distributed as G

2. Generate $U \sim \text{Uniform}(0, 1)$, which is independent from Y
3. If

$$U \leq \frac{f(Y)}{cg(Y)}$$

Then set $X = Y$ (“accept”); otherwise go back to step 1 (“reject”).

In practice, choosing the distribution function G is a kind of “art”.

First of all, G should be chosen such that the corresponding random variable could be simulated efficiently.

Secondly, G should be chosen such that $c = \sup_X \frac{f(x)}{g(x)}$ is as close to 1 as possible.

We consider the example of generating a random variable from the Gamma distribution. For¹ $k > 1$ and $\theta > 0$, the $\text{Gamma}(k, \theta)$ density is given by:

$$f(x) = \frac{\theta^k}{\Gamma(k)} x^{k-1} e^{-\theta x}, x > 0$$

To envelope f , we choose the probability density function $g(\cdot; \lambda)$ of the $\text{Exp}(\lambda)$, that is,

$$g(x; \lambda) = \lambda e^{-\lambda x}, x > 0$$

Then:

$$h(x; \lambda) = \frac{f(x)}{g(x; \lambda)} = \frac{\theta^k}{\Gamma(k)\lambda} x^{k-1} e^{(\lambda-\theta)x}$$

If $h \geq \theta$, apparently $\sup_{x>0} h(x; \lambda) = \infty$, so we require $\lambda < \theta$ below. Given such λ , $h(x; \lambda)$ is maximized at $x_\lambda = \frac{k-1}{\theta-\lambda}$, and

$$\sup_x \frac{f(x)}{g(x; \lambda)} = h(x_\lambda; \lambda) = \frac{\theta^k (k-1)^{k-1}}{\Gamma(k)\lambda(\theta-\lambda)^{k-1}} = e^{-1(k-1)}$$

As said above, we would like to choose the envelope density function such that the above supremum is as small as possible. In other words, we should choose λ to minimize $h(x_\lambda; \lambda)$. This is equivalent to maximizing $\lambda(\theta-\lambda)^{k-1}$ by optimizing the first order condition wrt. λ . This leads to:

¹Nelder, J. A. and Mead, R. (1965) A simplex algorithm for function minimization. Computer Journal 7, 308-313.

$$\lambda = \frac{\theta}{k}$$

Plugging this back to the supremum, we get:

$$c = \frac{k^k e^{-(k-1)}}{\Gamma(k)}$$

The code for the acceptance-rejection method for simulating a Gamma distributed random variable is.

```
### Auxiliary functions ###

gamma.pdf <- function(x, k, theta) {
  return(theta^k * x^(k-1) * exp(-theta*x)/gamma(k))
}

exponential.pdf <- function(x, lambda) {
  return(lambda * exp(-lambda*x))
}

Exponential.Simulate <- function(lambda, size = 1) {
  V <- runif(size)
  return(-1/lambda * log(V))
}

Gamma.Simulate <- function(k, theta, size = 1) {

  lambda <- theta/k
  c <- k^k * exp(-k+1) / gamma(k)

  U <- rep(NA, size)
  Y <- rep(NA, size)
  X <- rep(NA, size)
  Unaccepted <- rep(TRUE, size)

  while (any(Unaccepted)) {

    UnacceptedCount <- sum(Unaccepted)

    U <- runif(UnacceptedCount)
    Y <- Exponential.Simulate(lambda, UnacceptedCount)
```

```

Accepted_ThisTime <- Unaccepted[Unaccepted] &
  ( U <= ( gamma.pdf(Y, k, theta) / exponential.pdf(Y, lambda)/c ) )

X[Unaccepted][Accepted_ThisTime] <- Y[Accepted_ThisTime]
Unaccepted[Unaccepted] <- !Accepted_ThisTime

}

return(X)

}

```

We test the function by simulating 10^6 i.i.d. random variables with distribution $\text{Gamma}(2, 1)$. Compare the histogram of the simulated data and the theoretical density:

```

set.seed(10086)

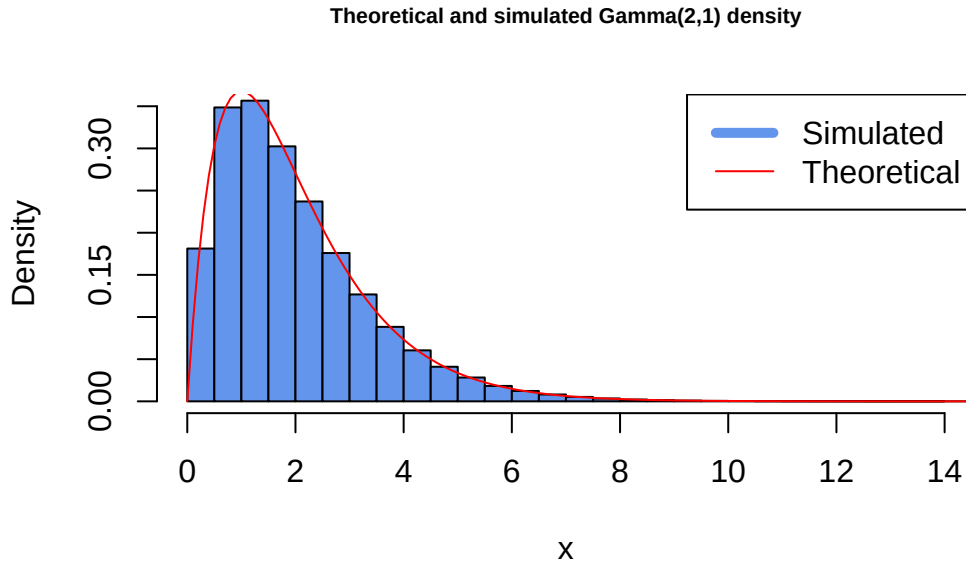
X <- Gamma.Simulate(2, 1, 10^6)

# histogram

hist(X, breaks = 30, freq = FALSE,
     main = "Theoretical and simulated Gamma(2,1) density",
     col = "cornflowerblue",
     xlim = c(0, 14), ylim = c(0, 0.35),
     xlab = "x",
     cex.main = 0.7)

xticks = seq(0, max(X), 0.1)
lines(xticks, dgamma(xticks, 2, 1), col = "red")
legend("topright", legend = c("Simulated", "Theoretical"), lty = c(1, 1), lwd = c(5, 1), col

```



6.5 Simulate Normals

If $Z \sim N(0, 1)$ then $\mu + \sigma Z \sim N(\mu, \sigma^2)$. Hence, it suffices to have an algorithm for simulating a $N(0, 1)$ random variable, i.e., a standard normal variable.

We cover three ways of simulating a standard normal random variable:

- Central limit theorem applied to a uniformly distributed random variable.
- Acceptance-Rejection method with exponential envelope
- Box-Muller algorithm

6.5.1 Simulate Normals: Using uniformly distributed random variable

Let $U_i, i = 1, 2, \dots$ be a sequence of i.i.d. random variables uniformly distributed over $(0, 1)$.

Note that $\mathbb{E}[U_i] = 1/2$ and $\text{var}[U_i] = 1/12$ respectively. The classic central limit theorem entails:

$$\frac{1}{\sqrt{n}} \sum_{i=1}^n \frac{U_i - \frac{1}{2}}{\sqrt{\frac{1}{12}}} \rightarrow N(0, 1)$$

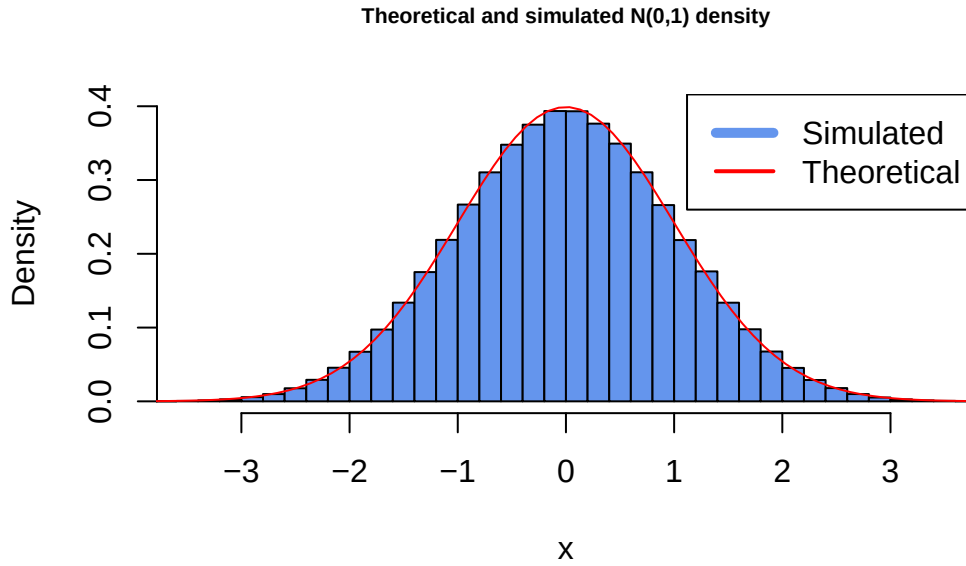
In particular, for some n a random variable $Z = \sum_{i=1}^n U_i - n/2$ is approximately a standard normal distribution.

The resulting program is very simple - set $n = 12$:

```
### Simulate standard normal random variable ###  
  
Normal.Simulate <- function(size = 1) {  
  
  Z <- rep(NA, size)  
  
  for (i in 1:size) {  
    Z[i] <- sum(runif(12)) - 6  
  }  
  return(Z)  
}
```

Despite its simpleness, this method works well in fact.

```
set.seed(10086)  
  
X <- Normal.Simulate(10^6)  
  
# histogram  
  
hist(X, breaks = 35, freq = FALSE,  
      main = "Theoretical and simulated N(0,1) density",  
      col = "cornflowerblue",  
      xlim = c(-3.5, 3.5), ylim = c(0, 0.4),  
      xlab = "x",  
      cex.main = 0.7)  
  
xticks = seq(min(X), max(X), 0.1)  
lines(xticks, dnorm(xticks, 0, 1), col = "red")  
legend("topright", legend = c("Simulated", "Theoretical"), lty = c(1, 1), lwd = c(5, 2), col
```



But it is possible to do better.

6.5.2 Simulate Normals: Acceptance-Rejection with Exponential Envelope

When $Z \sim N(0, 1)$, we let $X = |Z|$. Suppose that S is a random variable which is independent of Z and uniformly distributed over $-1, 1$, that is,

$$\mathbb{P}(S = -1) = \mathbb{P}(S = 1) = \frac{1}{2}$$

Then it is easy to check

$$SX \sim N(0, 1)$$

The probability density function of X is:

$$f(x) = \begin{cases} \sqrt{\frac{2}{\pi}} \exp(-\frac{1}{2}x^2), & \text{if } x > 0 \\ 0, & \text{if } x \leq 0 \end{cases}$$

Which is the half $N(0, 1)$ scaled by 2 to maintain a proper density.

We use the envelope density function:

$$g(x) = \begin{cases} \exp(-x), & \text{if } x > 0 \\ 0, & \text{if } x \leq 0 \end{cases}$$

Which is the probability density function of $\text{Exp}(1)$ distribution. Easily,

$$c = \sup_x \frac{f(x)}{g(x)} = \sup_x \sqrt{\frac{2}{\pi}} \exp\left(x - \frac{x^2}{2}\right) = \sqrt{\frac{2e}{\pi}}$$

This thus gives the following algorithm:

1. Draw $V \sim \text{Uniform}(0, 1)$ and set $Y = -\log(V)$
2. Draw $U \sim \text{Uniform}(0, 1)$, which is independent of Y
3. If

$$U \leq \exp\left(-\frac{1}{2}(Y-1)^2\right)$$

then generate S uniformly distributed over $-1, 1$ and return $Z = SY$; Otherwise go back to step 1.

6.5.3 Simulate Normals: Box-Muller Algorithm

The Box-Muller algorithm is grounded on the following theorem:

Theorem 6.1 (Box-Muller transform). Let U and V be two independent random variables uniformly distributed over $(0, 1)$. Define:

$$X = \sqrt{-2\log(U)} \cos(2\pi V), \quad Y = \sqrt{-2\log(U)} \sin(2\pi V)$$

Then $(X, Y)^\top$ is a standard bivariate normal random vector, i.e.,

$$(X, Y)^\top \sim N\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\right)$$

Hence, we have the Box-Muller algorithm, which draws two independent standard normal random variables at a time:

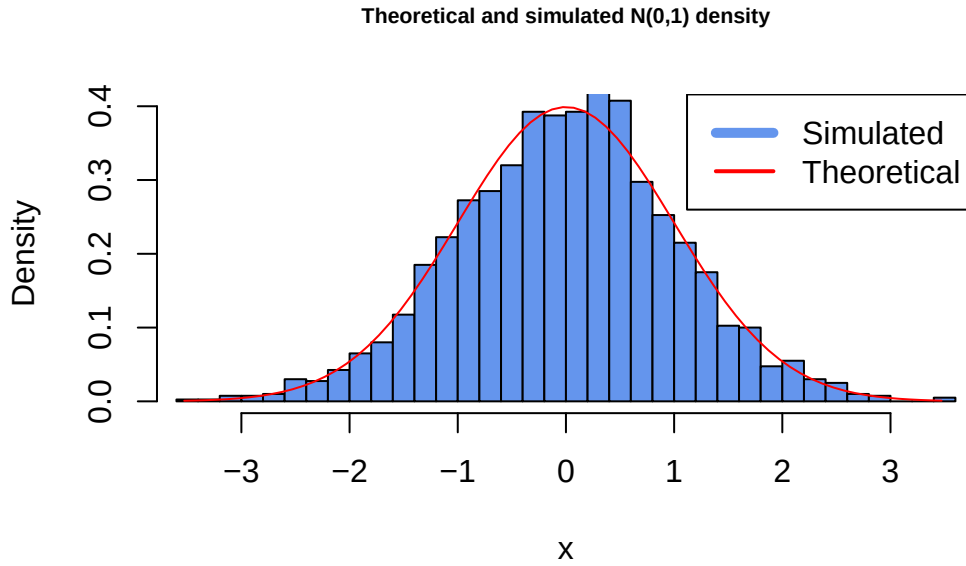
1. Draw independent $U \sim \text{Uniform}(0, 1)$ and $V \sim \text{Uniform}(0, 1)$
2. Set $R = \sqrt{-2\log(U)}$ and $\Theta = 2\pi V$
3. Return $X = R \cos \Theta$ and $Y = R \sin \Theta$

The resulting program is very simple:

```
BoxMuller <- function(size = 1) {  
  
  # size: number of standard bivariate normal random variables to simulate  
  
  U <- runif(size)  
  V <- runif(size)  
  
  X <- sqrt(-2*log(U)) * cos(2*pi*V)  
  Y <- sqrt(-2*log(U)) * sin(2*pi*V)  
  
  return(c(X,Y))  
  
}
```

The histogram of 1000 standard normal random variables simulated using Box-Muller transformation:

```
set.seed(10086)  
  
X <- BoxMuller(1000)  
  
# histogram  
  
hist(X, breaks = 35, freq = FALSE,  
      main = "Theoretical and simulated N(0,1) density",  
      col = "cornflowerblue",  
      xlim = c(-3.5, 3.5), ylim = c(0, 0.4),  
      xlab = "x",  
      cex.main = 0.7)  
  
xticks = seq(min(X), max(X), 0.1)  
lines(xticks, dnorm(xticks, 0, 1), col = "red")  
legend("topright", legend = c("Simulated", "Theoretical"), lty = c(1, 1), lwd = c(5, 2), col
```



6.6 Monte Carlo Integration

6.6.1 Monte Carlo Integration - a brief introduction

Suppose that you want to compute the integral:

$$I = \int_a^b f(x)dx$$

It is not always possible to calculate it analytically. In such cases, we can use Monte Carlo by noting that:

$$I = (b - a)\mathbb{E}[f(U)], U \sim \text{Uniform}(a, b)$$

Let $U_i : i = 1, 2, \dots$ be an i.i.d. sequence of random variables uniformly distributed over (a, b) . The strong law of large numbers implies:

$$\frac{b - a}{n} \sum_{i=1}^n f(U_i) \xrightarrow{a.s.} I, \text{ as } n \rightarrow \infty$$

Where $\xrightarrow{a.s.}$ means almost sure convergence.

6.6.2 Monte Carlo Integration - the code

Hence, you may pick a large n , draw i.i.d. $U_i \sim \text{Uniform}(a, b)$, and use $\frac{b-a}{n} \sum_{i=1}^n f(U_i)$ as an approximate of I . The following function does this:

```
MonteCarlo.Integration <- function(f, n, a, b) {  
  
  U <- runif(n, min = a, max = b)  
  return( (b-a)*mean(f(U)) )  
  
}
```

6.6.3 Monte Carlo Integration - examples

Set seed to 10086 and consider two examples.

Example 1: $\int_{-1}^1 \frac{1}{1+x^2} dx$ with the accurate value $\frac{\pi}{2} \approx 1.570796$. We get:

```
set.seed(10086)  
MonteCarlo.Integration(function(x) 1/(1+x^2), 100, -1, 1)  
#> [1] 1.589363
```

Example 2: $\int_0^1 \arctan x dx$ with the accurate value 0.4388246.

```
set.seed(10086)  
MonteCarlo.Integration(function(x) atan(x), 100, 0, 1)  
#> [1] 0.445995
```

6.6.4 Small Exercise

Suppose you want to measure the area of the unit circle via the Monte Carlo integration using the following steps:

- Simulate $U \sim U(0, 1)$ and $V \sim U(0, 1)$ to obtain $R^2 = U^2 + V^2$, 100 times.
- Define a function $f(U, V) = 1_{\{U^2+V^2 \leq 1\}}$: $1_{\text{Condition}}$ is an indicator function that returns 1 if the condition is true and 0 otherwise.
- Implement the monte carlo integration of the unit circle in R using the formula $(b-a)E[f(U)]$.
- Now, note that your result likely is somewhat lower than the area of the unit circle. Why is that? (Hint: try to multiply your obtained result by 4).

- Try to increase the number of standard uniform draws used to approximate the area.

Note that you should have successfully approximated a famous mathematical constant.

Solution

```
vU <- runif(10000)
vV <- runif(10000)

fLessThanOne <- function(vInput) {
  return(vInput < 1)
}

dResult <- (1 - 0) * mean(fLessThanOne(vU^2 + vV^2))
dResult * 4
#> [1] 3.1716
```

7 Root-finding

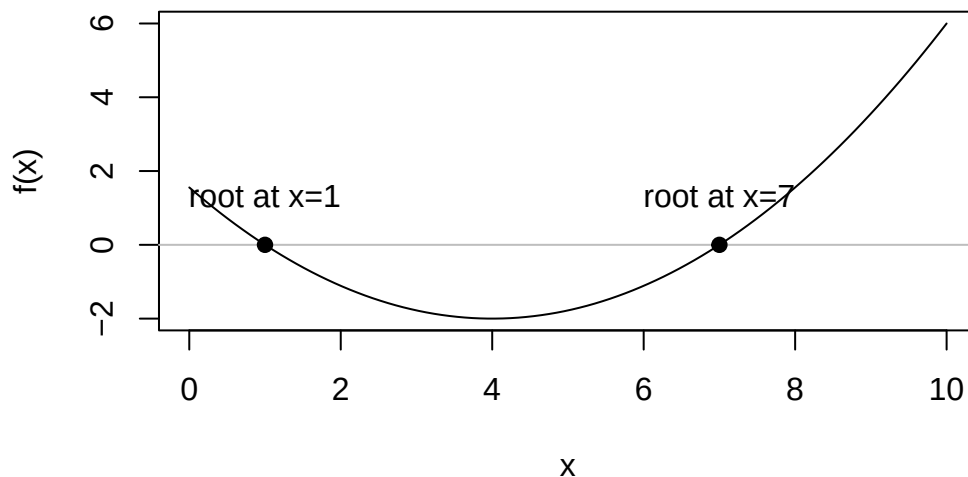
7.1 Main problem

Suppose $f : \mathbb{R} \rightarrow \mathbb{R}$ is a continuous function. A *root* of f is a solution to the equation $f(x) = 0$.

That is, a root is a number $a \in \mathbb{R}$ such that $f(a) = 0$, i.e.,

$$R = \{a \in \mathbb{R} : f(a) = 0\}$$

If we draw the graph of our function, say $y = f(x)$, which is a curve in the plane, a solution of $f(x) = 0$ is the x -coordinate of a point at which the curve crosses the x -axis.



7.1.1 The Roots of a Function

Except for some very special functions, it is not possible to find an analytic expression for the root/roots.

A commonly arising function is the polynomial function of the form:

$$P_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

We know that the roots of the quadratic function $P_2(x) = a_0 + a_1x + a_2x^2$ can be obtained analytically by the expression:

$$x = \frac{-a_1 \pm \sqrt{a_1^2 - 4a_0a_2}}{2a_2}$$

Unfortunately, such analytical formulas for the roots do not exist for polynomials of degree 5 or greater. Thus, most computational methods for root-finding are *iterative* in nature.

In general, the solution to a physical problem can often be expressed as the root of a suitable function, and we will see different methods of how to solve such problems.

But first, some examples.

7.1.2 Example: Loan repayments

A loan has an initial amount P , a monthly interest rate r , a duration of N months and a monthly repayment of A . The remaining debt after n months is given by P_n :

$$\begin{aligned} P_0 &= P \\ P_{n+1} &= P_n(1+r) - A \end{aligned}$$

After iterative substitutions, we obtain:

$$\begin{aligned} P_n &= P(1+r)^n - A \sum_{i=0}^{n-1} (1+r)^i \\ &= P(1+r)^n - A[(1+r)^n - 1]/r \end{aligned}$$

At the end of the period ($n = N$), we know that $P_n = 0$ (we have repaid our loan), we get:

$$\frac{A}{P} = \frac{r(1+r)^N}{(1+r)^N - 1}$$

Knowing the loan's amount P , the number of months N and the monthly repayment A , what is the interest rate r that we pay? The answer is the solution of $f(r) = 0$, where:

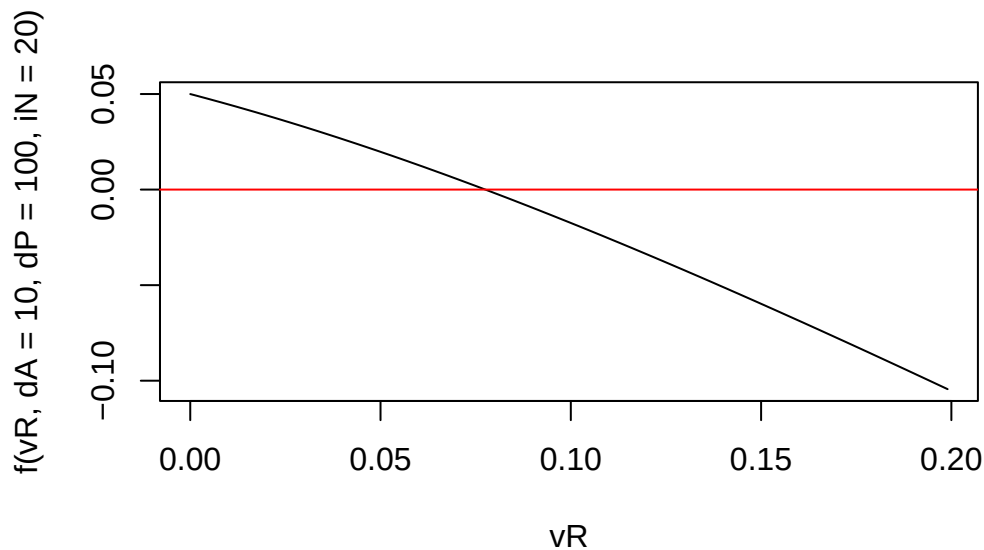
$$f(r) = \frac{A}{P} - \frac{r(1+r)^N}{(1+r)^N - 1}$$

Which is not available in closed form.

```
f <- function(dR, dA, dP, iN) {
  dOut = dA/dP - (dR * (1.0 + dR)^iN)/((1.0 + dR)^iN - 1)
  return(dOut)
}

vR <- seq(1e-5, 0.2, 0.001)

plot(vR, f(vR, dA = 10, dP = 100, iN = 20), type = "l")
abline(h = 0, col = "red")
```



Other examples:

- Estimation of Econometric Models
- Problems in Finance
- Physics (area minimization, shooting problems, orbital motions, ...)
- ...

7.2 Numerical Methods for Root-finding

As suggested above, when an analytical expression is not available for the root of some function, f , i.e., the solution to $f(x) = 0$, we have to use numerical methods that are *iterative* in nature.

The idea behind an iterative method is the following:

Starting with an initial approximation, x_0 , construct a sequence of iterates $\{x_n\}$ using an iteration formula with hope that this sequence converges to a root of $f(x) = 0$.

As concrete examples of such numerical methods, we will consider the Newton-Raphson method, the secant method and the bisection method.

7.3 The Newton-Raphson Method

The Newton-Raphson method is a root-finding algorithm that uses an iterative process to approach one root of a function.

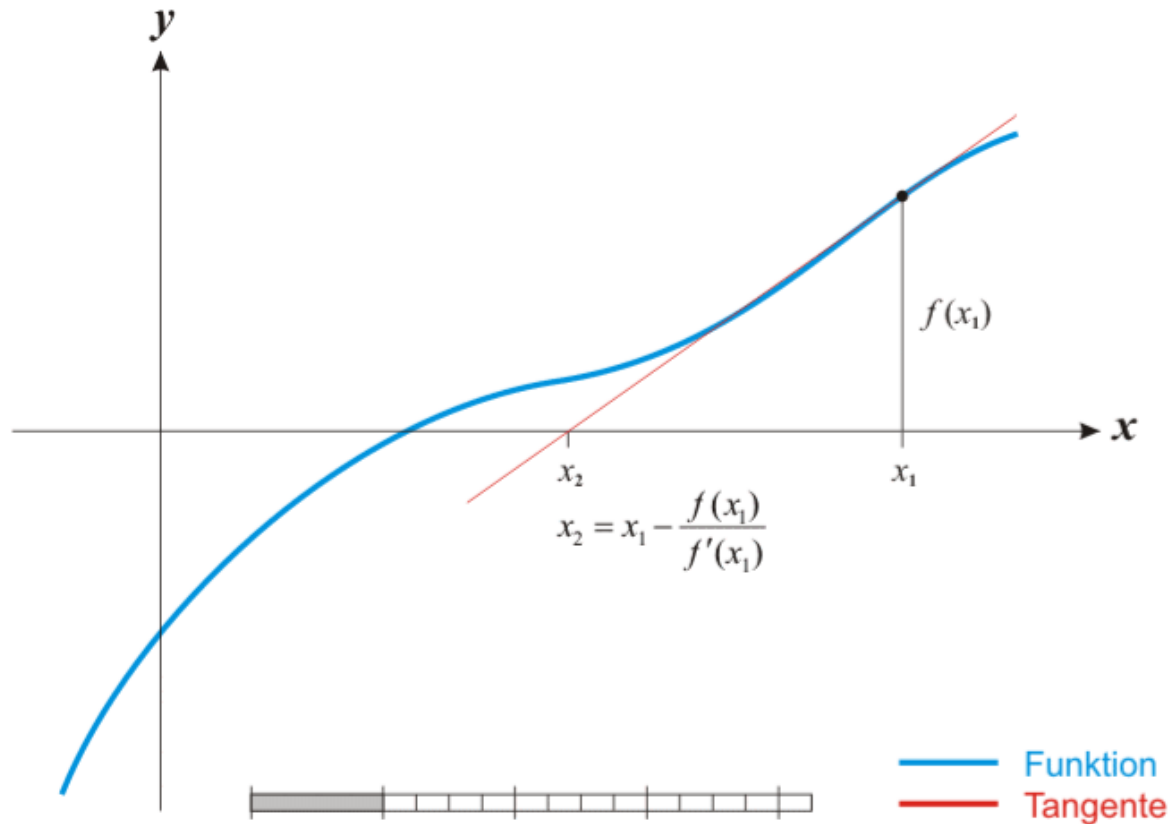
- Suppose the function f is differentiable with continuous derivative f' and a root a .
- Let $x_0 \in \mathbb{R}$ and think of x_0 as our current ‘guess’ at a .
- Now, the straight line through the point $(x_0, f(x_0))$ with slope $f'(x_0)$ is the best straight approximation to the function $f(x)$ at the point x_0 .
- The equation of this straight line is given by $y = f(x_0) + f'(x_0)(x - x_0)$
- This line crosses the x -axis at a point x_1 , which should be a better approximation than x_0 to a .
- We find x_1 by solving the equation for the straight line for x with $y = 0$:

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

i.e., we update our best guess to be x_1 which is obtained from x_0 by subtracting a correction term $f(x_0)/f'(x_0)$.

- Now that we have x_1 , we use the same procedure to get the next best guess:

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)}$$



7.3.1 Notes for the Newton-Raphson method

It can be shown that if f is ‘well-behaved’ at a (in particular, if $f'(a) \neq 0$ and f'' is finite and continuous at a) and you start with x_0 ‘close enough’ to a , then x_n will converge to a quickly.

- Unfortunately, we don’t know if f is well behaved at a until we know a , and we don’t know beforehand how close is close enough.

Since we are expecting $f(x) \rightarrow 0$, a good stopping condition for the Newton-Raphson algorithm is $|f(x_n)| \leq \varepsilon$ for some tolerance ε .

We also always want a hard limit on the maximum number of iterations, n_{max} , in case we get stuck in an infinite loop.

Implementing the Newton-Raphson method in R is straightforward; we just need the function f , its derivative f' , an initial guess x_0 , and the two stopping criteria ε (the convergence tolerance) and n_{max} (the maximum number of iterations).

7.3.2 The Newton-Raphson algorithm

The Newton-Raphson algorithm proceeds as follows:

- Define inputs: $f(x)$, x_0 , ε , n_{max} . For $n = 0, 1, \dots$ until stopping condition reached, do:
- Compute $f(x_n)$ and $f'(x_n)$.
- Compute $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$
- Stopping conditions:
 - If $|f(x_n)| \leq \varepsilon$ then set $a = x_n$ and stop; Algorithm converged.
 - If $n = n_{max}$ then stop; Maximum number of iterations has been reached, algorithm failed to converge.

7.3.3 Drawbacks of the Newton-Raphson method

The Newton-Raphson root-finding method works by producing a sequence of guesses to the root and, under favourable circumstances, converges rapidly to the root from an initial guess.

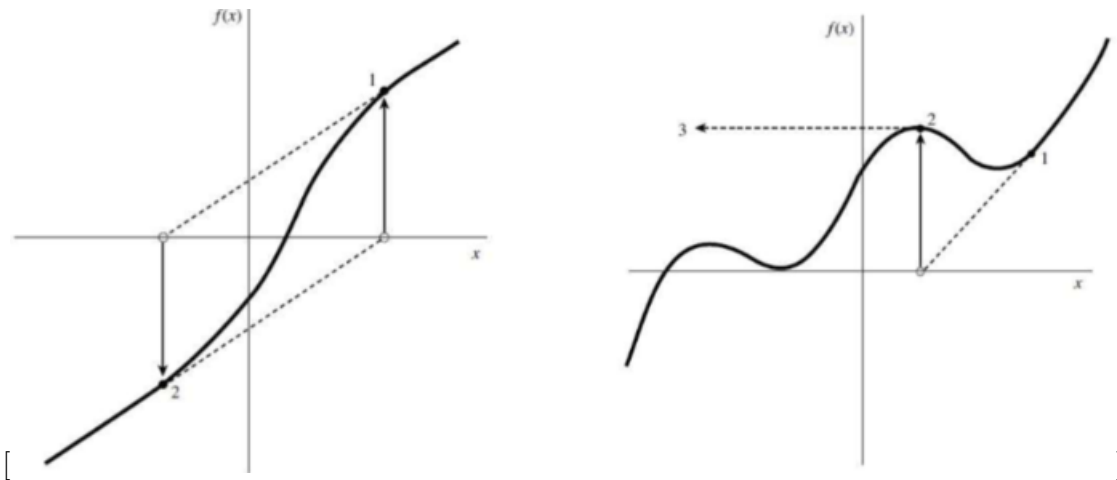
- Unfortunately, it cannot be guaranteed to work.

It can be a costly task to calculate the required derivative $f'(x_n)$ for every iteration, when we are not able to analytically derive it.

Instability: if the iteration hits a value x_n where the function is close to flat (i.e., $f'(x_n)$ close to zero), the iteration is sent far away from the current point.

No guarantee of convergence: the iteration might get caught up in an infinite loop.

In general, we need to pick a starting point relatively close to the desired root (could be more than one).



7.3.4 Loan example

Consider the loan repayments problem with objective function:

$$f(x) = \frac{A}{P} - \frac{x(1+x)^N}{(1+x)^N - 1}$$

and first derivative:

$$f'(x) = \frac{[(1+x)^N + Nx(1+x)^{N-1}][(1+x)^N - 1] - Nx(1+x)^{2N-1}}{[(1+x)^N - 1]^2}$$

```
f <- function(dX, dA, dP, iN) {
  dOut = dA/dP - (dX * (1.0 + dX)^iN)/((1.0 + dX)^iN - 1)
  return(dOut)
}

f_prime <- function(dX, dA, dP, iN) {
  dNum1 = ((1.0 + dX)^iN + iN * dX * (1.0 + dX)^(iN - 1)) * ((1.0 + dX)^iN - 1)
  dNum2 = iN * dX * (1.0 + dX)^(2 * iN - 1)
  dDen = ((1.0 + dX)^iN - 1)^2
  dOut = -(dNum1 - dNum2)/dDen
  return(dOut)
}

NR <- function(f, f_prime, dX0, dTol = 1e-9, max.iter = 1000, ...) {
  dX <- dX0
  for (i in 1:max.iter) {
    dX = dX - f(dX)/f_prime(dX)
    if (abs(f(dX)) < dTol) break
  }
  return(dX)
}
```

```

fx <- f(dX, ...)
iter <- 0
while ((abs(fx) > dTol) && (iter < max.iter)) {
  dX <- dX - f(dX, ...)/f_prime(dX, ...)
  fx <- f(dX, ...)
  iter <- iter + 1
  cat("At iteration ", iter, "value of x is: ", dX, "\n")
}
if (abs(fx) > dTol) {
  cat("Algorithm failed to converge\n")
  return(NULL)
} else {
  cat("Algorithm converged\n")
  return(dX)
}
}

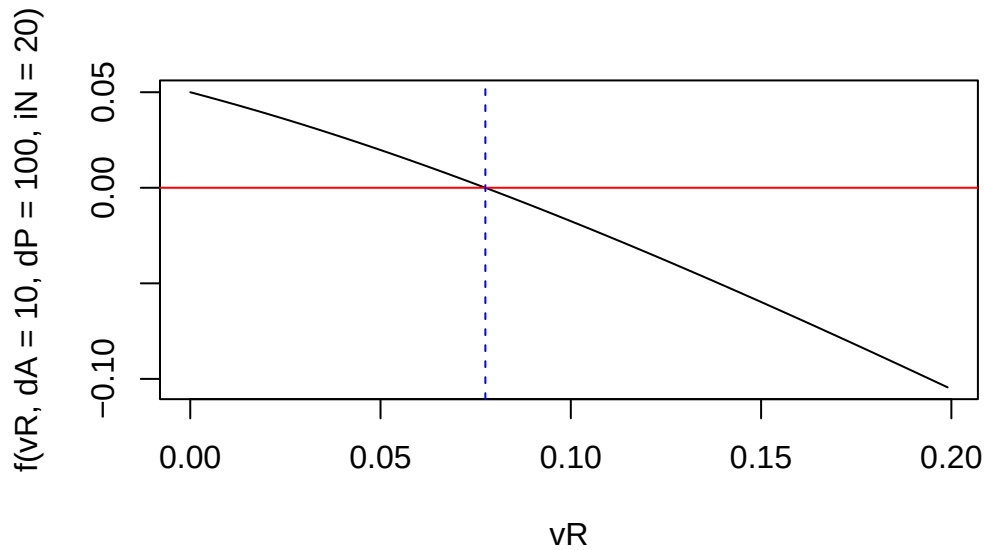
NR(f, f_prime, dX0 = 0.15, dA = 10, dP = 100, iN = 20)
#> At iteration 1 value of x is: 0.08241672
#> At iteration 2 value of x is: 0.07758319
#> At iteration 3 value of x is: 0.0775469
#> At iteration 4 value of x is: 0.0775469
#> Algorithm converged
#> [1] 0.0775469

```

```

#> [1] -3.534942e-09

```



7.4 The Secant method

If the derivative f' does not exist (or we don't want to compute it), then we can use the secant method, which only requires that the function f is continuous.

Suppose f has a root a . For this method, we assume that we have two current 'guesses', x_0 and x_1 , for the value of a .

We will think of x_0 as an older guess and we want to replace the pair x_0, x_1 by the pair x_1, x_2 , where x_2 is a new guess.

To find a new guess x_2 , we first draw the straight line from $(x_0, f(x_0))$ to $(x_1, f(x_1))$, i.e., the *secant* of the curve $y = f(x)$.

The equation of the secant is:

$$y = f(x_1) + \frac{f(x_0) - f(x_1)}{x_0 - x_1}(x - x_1)$$

As the new guess, we will use the x -coordinate x_2 of the point at which the secant crosses the x -axis, and so x_2 can be found from:

$$0 = f(x_1) + \frac{f(x_0) - f(x_1)}{x_0 - x_1}(x - x_1)$$

Which implies:

$$x_2 = x_1 - f(x_1) \frac{x_0 - x_1}{f(x_0) - f(x_1)}$$

Repeating this, we get a second-order recurrence relation (each new value depends on the previous two):

$$x_{n+1} = x_n - f(x_n) \frac{x_{n-1} - x_n}{f(x_{n-1}) - f(x_n)}$$

Note that if x_n and x_{n-1} are close together, then:

$$\frac{1}{f'(x_n)} \approx \frac{x_{n-1} - x_n}{f(x_{n-1}) - f(x_n)}$$

And we can thus view the secant method as an approximation of the Newton-Raphson method.

Similar convergence properties to the Newton-Raphson method:

- If f is well behaved at a and you start with x_0 and x_1 sufficiently close to a , then x_n will converge quickly to a , though not quite as fast as the NR method.
- We cannot guarantee convergence.
- Trade-off compared to NR: we no longer need to know f' but in return we give up some speed and have to provide two initial points, x_0 and x_1 .

7.5 The bisection method

A more reliable but slower approach is root-bracketing, which works by first isolating an interval in which the root must lie, and then successively refining the bounding interval in such a way that the root is guaranteed to always lie inside the interval.

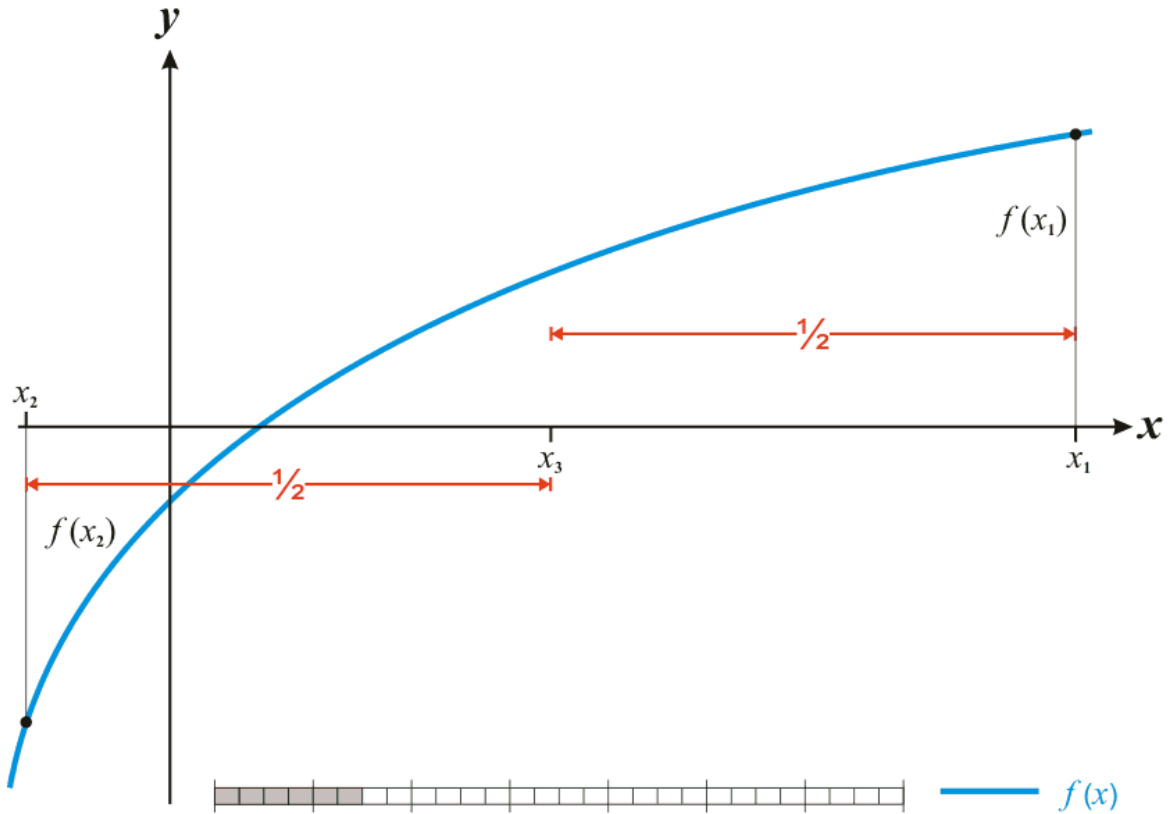
The canonical example is the *bisection method*, in which the width of the bounding interval is successively halved.

Suppose that f is a continuous function, then it is easy to see that f has at least one root in the interval (x_l, x_r) if $f(x_l)f(x_r) < 0$ (i.e., if either $f(x_l) < 0$ and $f(x_r) > 0$ or $f(x_l) > 0$ and $f(x_r) < 0$).

The bisection method works by taking an interval (x_l, x_r) that contains a root, the successively refining x_l and x_r until $x_r - x_l \leq \varepsilon$, where ε is some predefined tolerance.

The algorithm proceeds as follows:

0. Start with $x_l < x_r$ such that $f(x_l)f(x_r) < 0$
1. If $x_r - x_l \leq \varepsilon$ then stop; set $a = x_m = \frac{x_r + x_l}{2}$
2. Put $x_m = (x_l + x_r)/2$; if $f(x_m) = 0$ then stop.
3. If $f(x_l)f(x_m) < 0$ then put $x_r = x_m$; otherwise put $x_l = x_m$.
4. Go back to step 1.



Note that at every iteration of the algorithm, we know that there is a root in the interval (x_l, x_r) . Hence, provided we start with $f(x_l)f(x_r) < 0$, the algorithm is *guaranteed to converge* and we do not need to put a bound on the maximum number of iterations.

If we stop when $x_r - x_l \leq \varepsilon$, then we know that both x_l and x_r are within distance ε of a root.

The bisection method converges very slowly...

The most popular current root-finding methods use root-bracketing to get close to a root, then switch over to the Newton-Raphson or secant method when it seems safe to do so. This strategy combines the safety of bisection with the speed of the secant method.

7.5.1 The bisection method: Implementation in R

```
bisection <- function(f, dX.l, dX.r, dTol = 1e-9, max.iter = 1000, ...) {  
  
  #check inputs  
  if (dX.l >= dX.r) {  
    cat("error: x.l >= x.r \n")  
    return(NULL)  
  }  
  f.l <- f(dX.l, ...)  
  f.r <- f(dX.r, ...)  
  if (f.l == 0) {  
    return(dX.l)  
  } else if (f.r == 0) {  
    return(dX.r)  
  } else if (f.l*f.r > 0) {  
    cat("error: f(x.l)*f(x.r) > 0 \n")  
    return(NULL)  
  }  
  
  # successively refine x.l and x.r  
  iter <- 0  
  while ((dX.r - dX.l) > dTol && (iter < max.iter)) {  
    dX.m <- (dX.l + dX.r)/2  
    f.m <- f(dX.m, ...)  
    if (f.m == 0) {  
      return(dX.m)  
    } else if (f.l*f.m < 0) {  
      dX.r <- dX.m  
      f.r <- f.m  
    } else {  
      dX.l <- dX.m  
      f.l <- f.m  
    }  
    iter <- iter + 1  
  }  
}
```

```

    cat("at iteration", iter, "the root lies between", dX.l, "and", dX.r, "\n")
  }

  # return approximate root
  return((dX.l + dX.r)/2)
}

```

Let again f be the loan repayment function.

```

bisection(f, dX.l = 1e-6 , dX.r = 0.2, dTol = 1e-5, dA = 10, dP = 100, iN = 20)
#> at iteration 1 the root lies between 1e-06 and 0.1000005
#> at iteration 2 the root lies between 0.05000075 and 0.1000005
#> at iteration 3 the root lies between 0.07500063 and 0.1000005
#> at iteration 4 the root lies between 0.07500063 and 0.08750056
#> at iteration 5 the root lies between 0.07500063 and 0.08125059
#> at iteration 6 the root lies between 0.07500063 and 0.07812561
#> at iteration 7 the root lies between 0.07656312 and 0.07812561
#> at iteration 8 the root lies between 0.07734436 and 0.07812561
#> at iteration 9 the root lies between 0.07734436 and 0.07773499
#> at iteration 10 the root lies between 0.07753967 and 0.07773499
#> at iteration 11 the root lies between 0.07753967 and 0.07763733
#> at iteration 12 the root lies between 0.07753967 and 0.0775885
#> at iteration 13 the root lies between 0.07753967 and 0.07756409
#> at iteration 14 the root lies between 0.07753967 and 0.07755188
#> at iteration 15 the root lies between 0.07754578 and 0.07755188
#> [1] 0.07754883

```

7.6 The unirootfunction

R implements a general purposed unit root finder with the function `uniroot`.

`uniroot` is based on the Brent's method, which is a root-finding algorithm that combines the bisection method, the secant method and a third method called the inverse quadratic interpolation method.¹

The function `uniroot` searches the interval from lower to upper for a root (i.e., zero) of the function f with respect to its **first argument**. The function is given by:

¹Nelder, J. A. and Mead, R. (1965) A simplex algorithm for function minimization. Computer Journal 7, 308-313.

```
uniroot(f, interval, ..., lower = min(interval), upper = max(interval), tol = .Machine$double
```

7.6.1 Example for the loan repayment function

```
uniroot(f, lower = 1e-6, upper = 0.2, dA = 10, dP = 100, iN = 20)
#> $root
#> [1] 0.07752483
#>
#> $f.root
#> [1] 1.659286e-05
#>
#> $iter
#> [1] 4
#>
#> $init.it
#> [1] NA
#>
#> $estim.prec
#> [1] 6.103516e-05
```

7.7 Numerical Derivatives in R

In order to implement the Newton-Raphson method as well as other numerical algorithms, the use of the derivative of the objective function is required.

Sometimes, we are not able to analytically derive the derivative or we are just too lazy to do that. In these cases, the derivative can be computed numerically.

In R, there are several routines to evaluate numerical derivatives. One of the most used is the `grad` function in the `numDeriv` package of Gilbert and Varadhan (2016).²

`grad` calculates a numerical approximation of the first derivative of a function at the point `x`. Its formulation is:

```
grad(func, x, method = "Richardson", side = NULL, method.args=list(), ...)
```

Where:

²Byrd, R. H., Lu, P., Nocedal, J. and Zhu, C. (1995) A limited memory algorithm for bound constrained optimization. SIAM J. Scientific Computing, 16, 1190-1208.

- `func` is a function with a scalar real result.
- `x` is a real scalar or vector argument to `func`, indicating the point(s) at which the gradient is to be calculated.
- `method` is one of `Richardson`, `simple` or `complex` indicating the method to use for the approximation.

In order to check that our implementation of `f_prime` in the loans problem is correct, we can test it using `grad`:

```
grad(f, 0.2, dA = 10, dP = 100, iN = 20)
#> [1] -0.9351161

f_prime(0.2, dA = 10, dP = 100, iN = 20)
#> [1] -0.9351161

abs(grad(f, 0.2, dA = 10, dP = 100, iN = 20) - f_prime(0.2, dA = 10, dP = 100, iN = 20))
#> [1] 1.055467e-11
```

7.7.1 Warnings

Beware that:

- Numerical derivatives are not always precise.
- The computational cost generally increases considerably.

```
library(microbenchmark)
#> Warning: pakke 'microbenchmark' blev bygget under R version 4.3.3
microbenchmark(
  grad(f, 0.2, dA = 10, dP = 100, iN = 20),
  f_prime(0.2, dA = 10, dP = 100, iN = 20)
)
#> Unit: microseconds
#>
#>      expr    min      lq   mean median      uq    max
#> grad(f, 0.2, dA = 10, dP = 100, iN = 20) 36.6 36.95 40.604   37.3 43.85 101.9
#> f_prime(0.2, dA = 10, dP = 100, iN = 20)  1.0  1.20  1.529    1.5  1.60   3.9
#> neval
#>    100
#>    100
```

8 Numerical Optimization 1

8.1 Numerical vs Analytical Optimization

First, recall that any maximization problem can be converted into a minimization problem by multiplying the function by -1.

Most economic, econometric, and statistical problems can be solved by maximizing/minimizing an objective function.

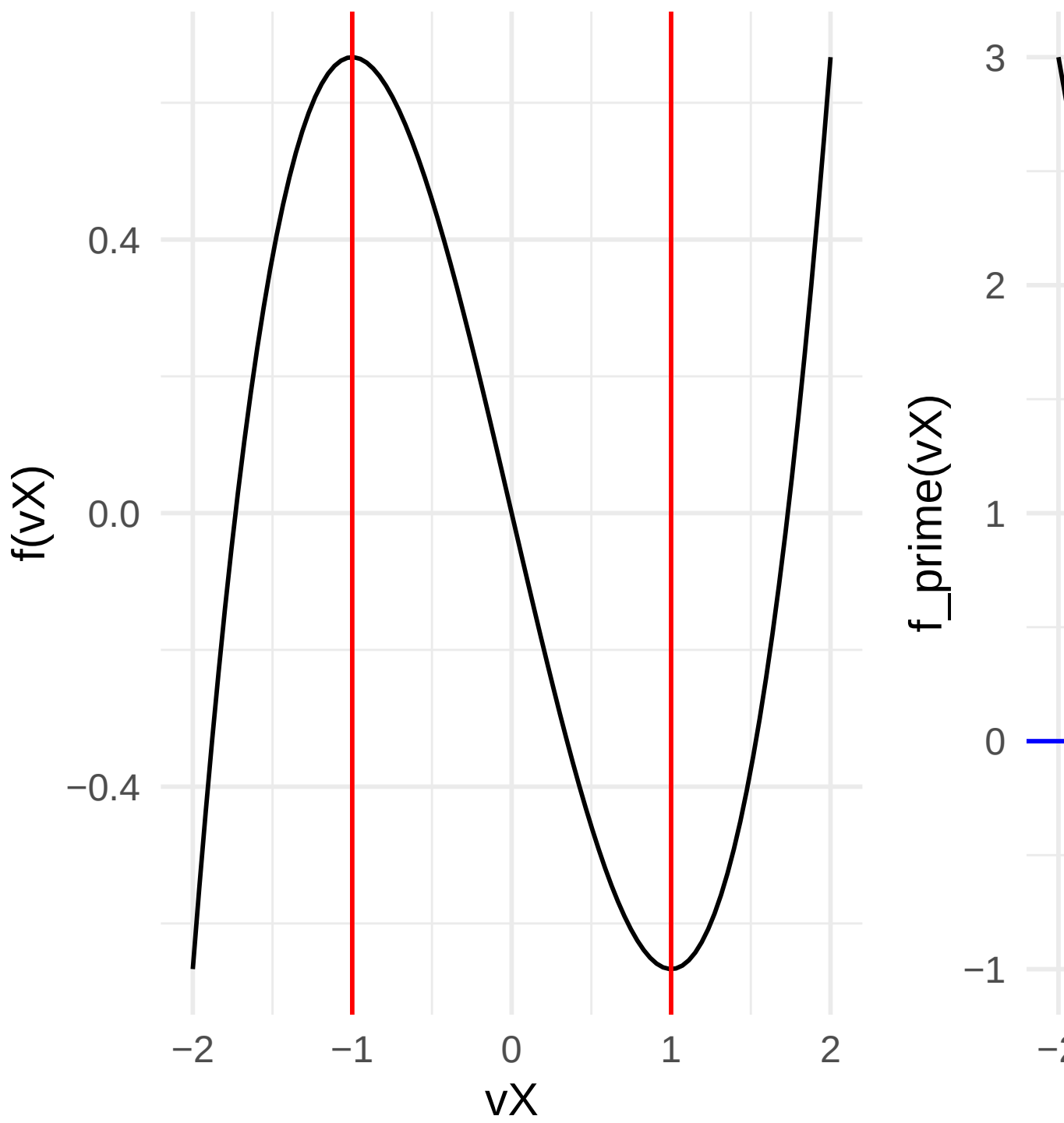
- Maximum likelihood estimation
- GMM
- Portfolio allocation
- ...

In some cases, the optimal points can be found analytically, but in most cases a closed form solution is not available. In these cases, we need numerical optimization methods.

8.1.1 Optimization and Root-finding

Root-finding and optimization are very closely related subjects which often occur in practical applications.

In fact, as you all know, optimizing a function, e.g., $f(x) = \frac{x^3}{3} - x$ is equivalent to finding the roots of its derivative, $f'(x) = x^2 - 1$.



Therefore, the methods used for numerical optimization are closely related to the numerical root-finding methods that were seen in the last lecture.

8.1.2 Maximum: Definition

In one dimension, we suppose that we have a function $f : \mathbb{R} \rightarrow \mathbb{R}$ with continuous first and second derivatives.

The function f has a *global maximum* at x^* if $f(x) \leq f(x^*)$ for all x .

The function f has a *local maximum* at x^* if $f(x) \leq f(x^*)$ for all x in a neighborhood of x^* ,

A necessary condition for x^* to be a local maximum of f is $f'(x^*) = 0$ and $f''(x^*) \leq 0$, a sufficient condition is $f'(x^*) = 0$ and $f''(x^*) < 0$.

8.1.3 Numerical optimization: local search techniques

Clearly, finding a local maximum is much easier than finding a global maximum.

All of the algorithms we consider for numerical optimization are local search techniques.

These algorithms work by generating a series of points, $x_0, x_1, x_2, \dots$, which (hopefully, but not necessarily) converge to a local maximum of f .

Given a prospective solution x_n , we look for the next prospective solution x_{n+1} in some neighborhood of x_n .

Because they never consider the whole space of possible solutions, local search techniques can only ever be guaranteed to find local maxima.

8.1.4 Numerical optimization: general structure

These search techniques are iterative procedures with the following general structure.

1. Chose a starting point, x_0 ;
2. Check for optimality in x_0 ;
3. A series of instructions to select another candidate of an optimal point;
4. Back to point 2 until an optimality criterion is satisfied;
5. Final point, x_N , is an approximation of the local optimal point, x^* , for the function $f(\cdot)$.

A search method is said to be *globally convergent* if it can reach a stationary point starting from any initial point. Otherwise, the method is said to be locally convergent.

8.1.5 Stopping criteria

All algorithms require a stopping criteria to avoid that the routines keep running forever. The purpose is:

- Limit the number of iterations.
- Obtain an accurate solution.

Typical stopping criteria are:

1. $|x_n - x_{n-1}| \leq \varepsilon$, with $\varepsilon > 0$;
2. $|f(x_n) - f(x_{n-1})| \leq \varepsilon$, with $\varepsilon > 0$;
3. $|f(x_n)| \leq \varepsilon$, with $\varepsilon > 0$;
4. Maximum number of iterations reached, $n = n_{max}$.

If the sequence $\{x_n\}_{n=1}^{\infty}$ converges to a local maximum, the criteria 1-3 will all be satisfied, but the converse is not necessarily true. Thus, even when a local search technique appears to converge, we may still need to check that the final solution really is a local maximum.

We may find that the algorithm does not converge at all; e.g., if f is unbounded then possibly $x_n \rightarrow \infty$. This is the reason that we need to specify a maximum number of iterations n_{max} .

8.2 The Newton-Raphson Method - and Method for Optimization.

As we saw last time, the Newton-Raphson method uses information on the function f and its derivative f' . The original formulation of the NR method is intended to find the *roots* of a function, and it is based on the following recursive algorithm:

- Define inputs: $f(x), x_0, \varepsilon, n_{max}$. For $n = 0, 1, \dots$ until stopping condition reached, do:
- Compute $f(x_n)$ and $f'(x_n)$.
- Compute $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$.
- Stopping conditions:
 - If $|f(x_n)| \leq \varepsilon$ then set $x_n = a$ and stop; algorithm converged.
 - If $n = n_{max}$ then stop; maximum number of iterations has been reached, algorithm failed to converge.

8.2.1 Newton's Method for Optimization

Since we deal with a problem of optimization, i.e., $f'(x) = 0$, Newton's method for optimization replaces $f(x)$ with $f'(x)$ in the original Newton-Raphson method.

Hence, the n -th step in Newton's method for optimization (for univariate functions) is:

$$x_n = x_{n-1} - \frac{f'(x_{n-1})}{f''(x_{n-1})}$$

The Newton's method is such that the convergence is *local* and *quadratic*, meaning that near the solution, the convergence is very fast!

The Newton's method is really suitable when the first- and second-order information are readily and easily calculated.

8.2.2 Newton's algorithm convergence

When the Newton algorithm converges, we can end up with a minimum, or indeed a 'flat spot', just as easily as a maximum.

The reason is that all such stationary points satisfy $f'(x^*) = 0$.

It can be shown that if:

- x^* is a local maximum.
- $f'(x^*) = 0$
- $f''(x^*) < 0$
- $\exists k \in \mathbb{R} : |f''(x) - f''(y)| \leq k|x - y| \forall (x, y) \in \mathbb{R}^2$ (that is, f'' is Lipschitz-continuous) in a neighborhood of x^* then, provided that x_0 is close enough to x^* , $x_n \rightarrow x^*$ quickly as $n \rightarrow \infty$.

We will revisit Newton's method later in higher dimensions.

8.2.3 Newton's method: Example

Consider the function $f(x) = 2x(x-1)^2(x+2)$

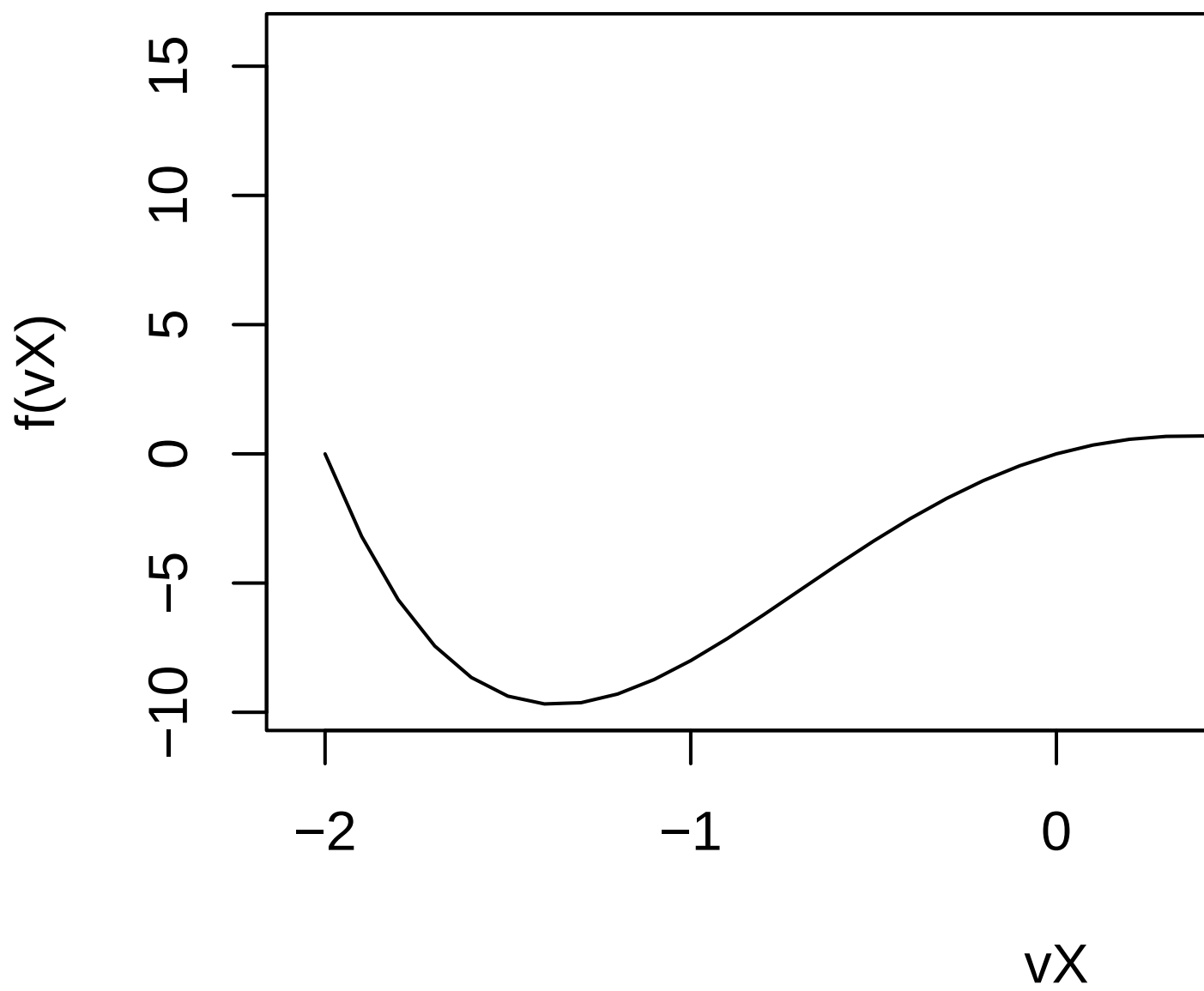
```
f <- function(dX) {  
  dOut <- 2 * dX * (dX - 1)^2 * (dX + 2)  
  return(dOut)  
}
```

with first derivative $f'(x) = 4 - 12x + 8x^3$

```
f_prime <- function(dX) {  
  dOut <- 4 - 12 * dX + 8 * dX^3  
  return(dOut)  
}
```

and second derivative $f''(x) = 24x^2 - 12$

```
f_second <- function(dX) {  
  dOut <- 24 * dX^2 - 12  
  return(dOut)  
}  
vX <- seq(-2, 2, 0.1)  
plot(vX, f(vX), type = "l")
```

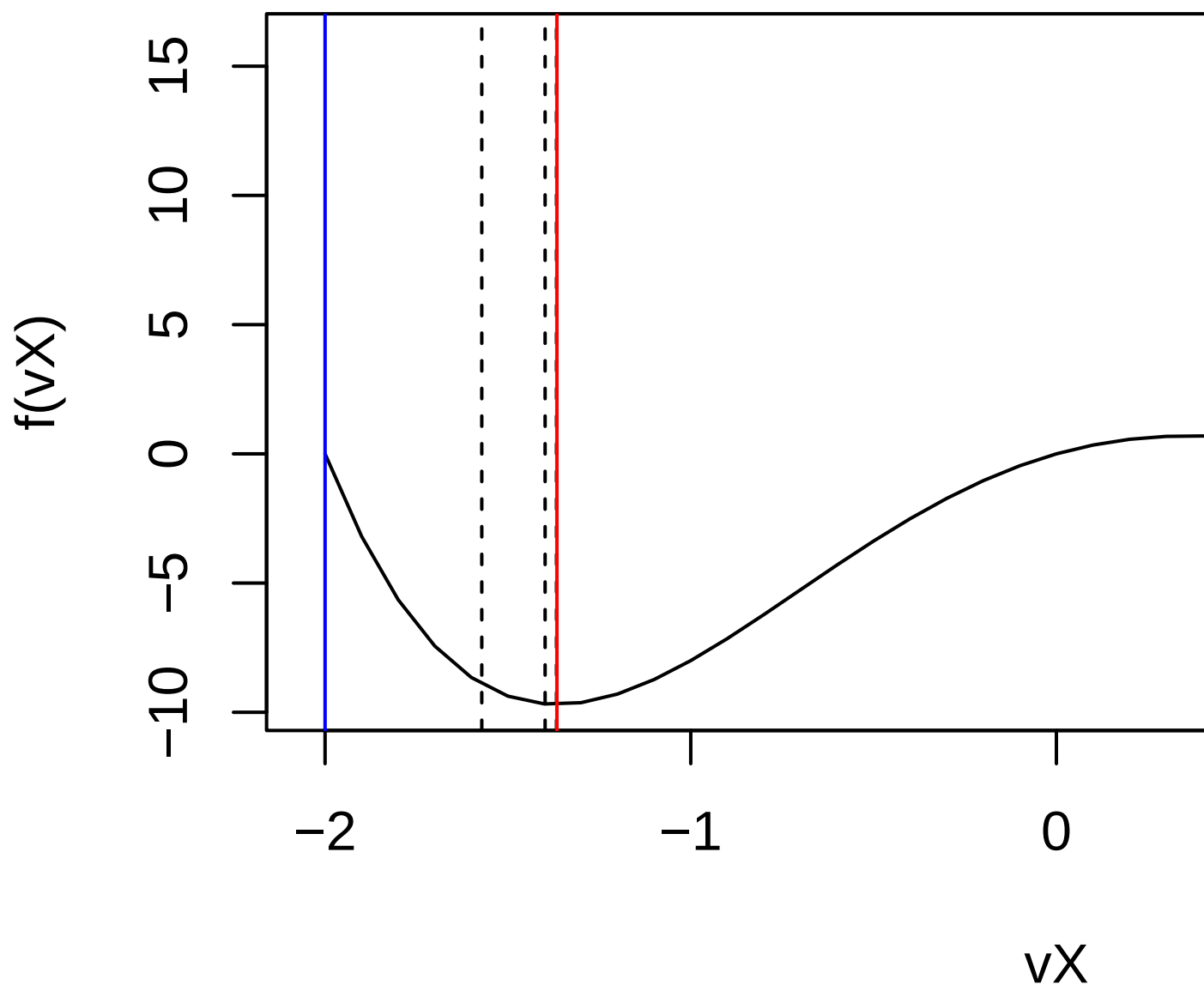


8.2.4 Newton's method: implementation in R

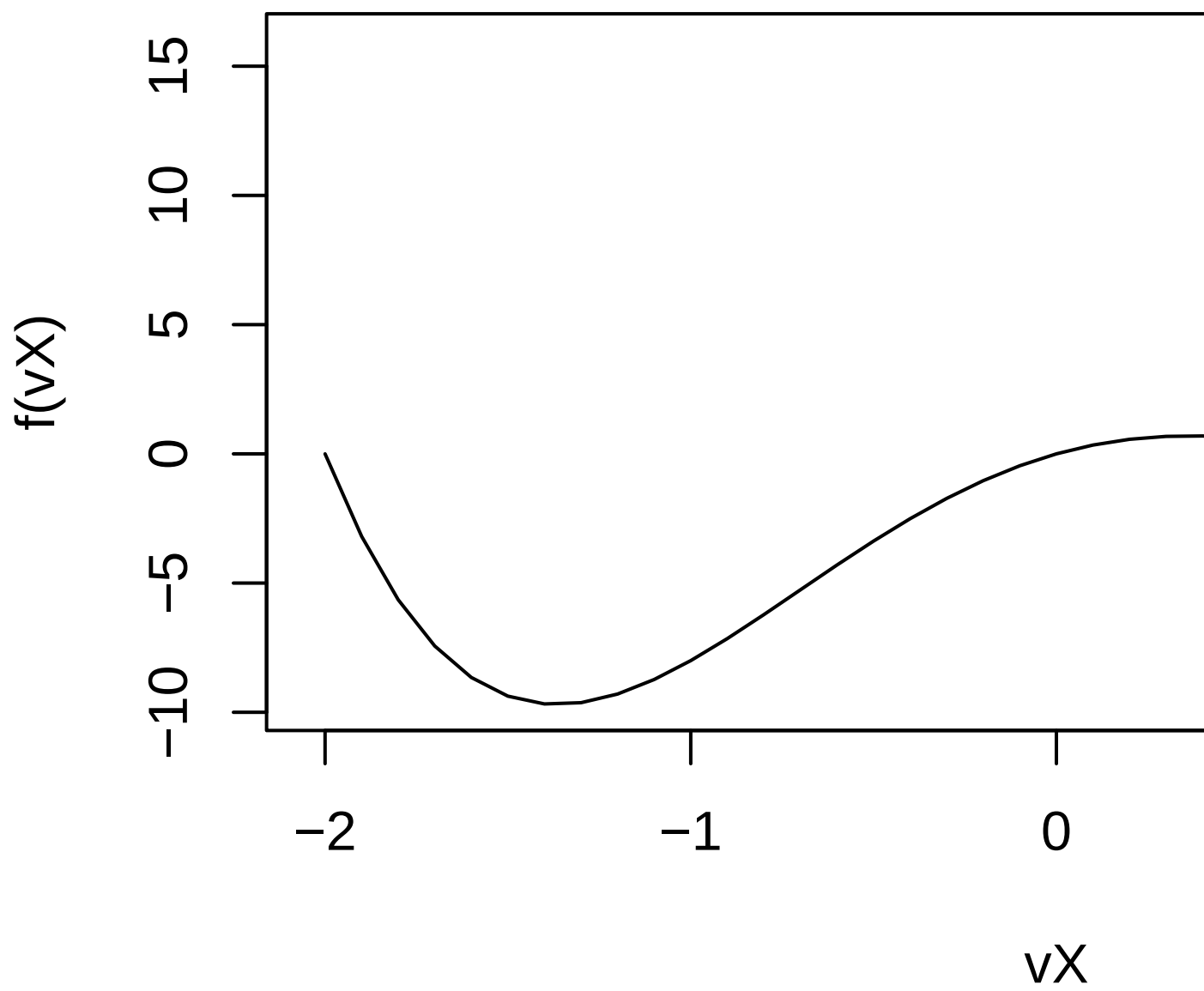
```
NM <- function(f, f_prime, f_sec, dX0, dTol = 1e-9, n.max = 1000){
  dX <- dX0
  fx <- f(dX)
  fpx <- f_prime(dX)
  fsx <- f_sec(dX)
  n <- 0
  while ((abs(fpx) > dTol) && (n < n.max)) {
    dX <- dX - fpx/fsx
    fx <- f(dX)
    fpx <- f_prime(dX)
    fsx <- f_sec(dX)
    n <- n + 1
    cat("At iteration", n, "the value of x is:", dX, "\n")
  }
  if (n == n.max) {
    cat('newton failed to converge\n')
  } else {
    return(dX)
  }
}
```

Example: starting point $x_0 = -2$:

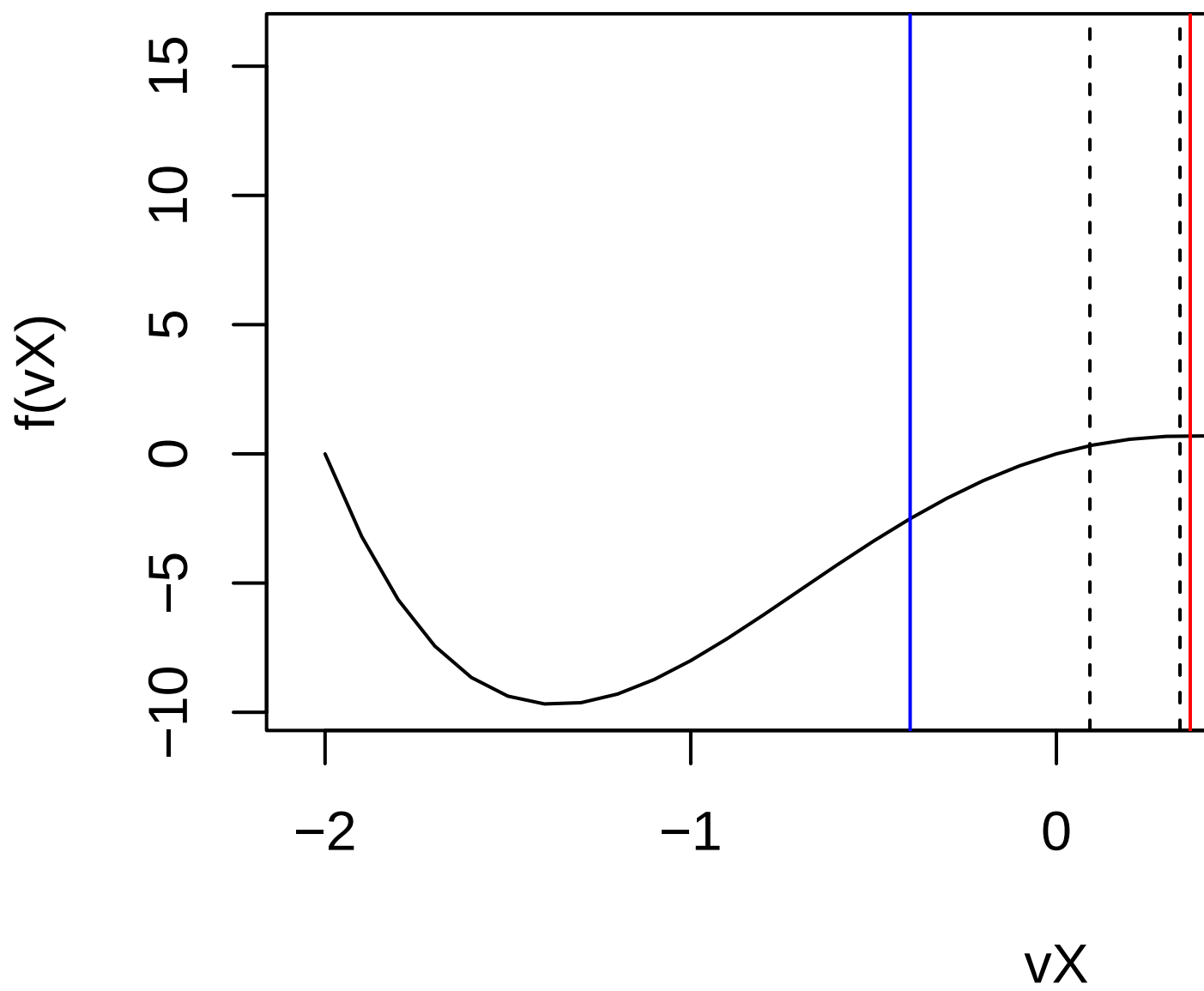
```
x1 <- NM(f, f_prime, f_second, dX0 = -2)
#> At iteration 1 the value of x is: -1.571429
#> At iteration 2 the value of x is: -1.398224
#> At iteration 3 the value of x is: -1.367014
#> At iteration 4 the value of x is: -1.366026
#> At iteration 5 the value of x is: -1.366025
vX <- seq(-2, 2, 0.1)
plot(vX, f(vX), type = "l")
abline(v=-2, col="blue") # initial guess
abline(v=-1.571429, lty=2) # first iteration
abline(v=-1.398224, lty=2) # second iteration
abline(v=-1.367014, lty=2) # third iteration
abline(v=x1, col="red") # solution
```



Example: starting point $x_0 = 1.5$:



Example: starting point $x_0 = -0.4$:



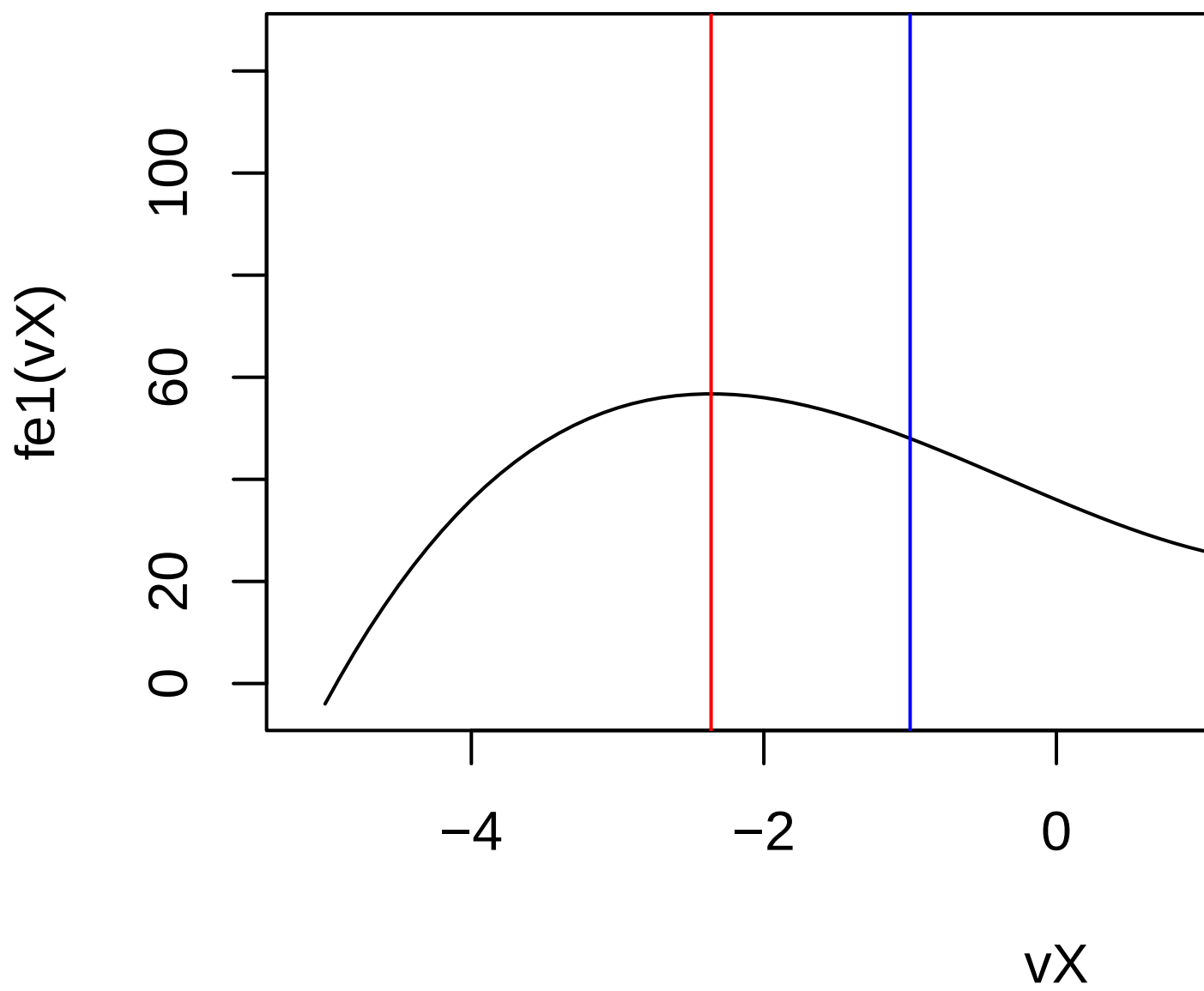
8.2.5 Exercise

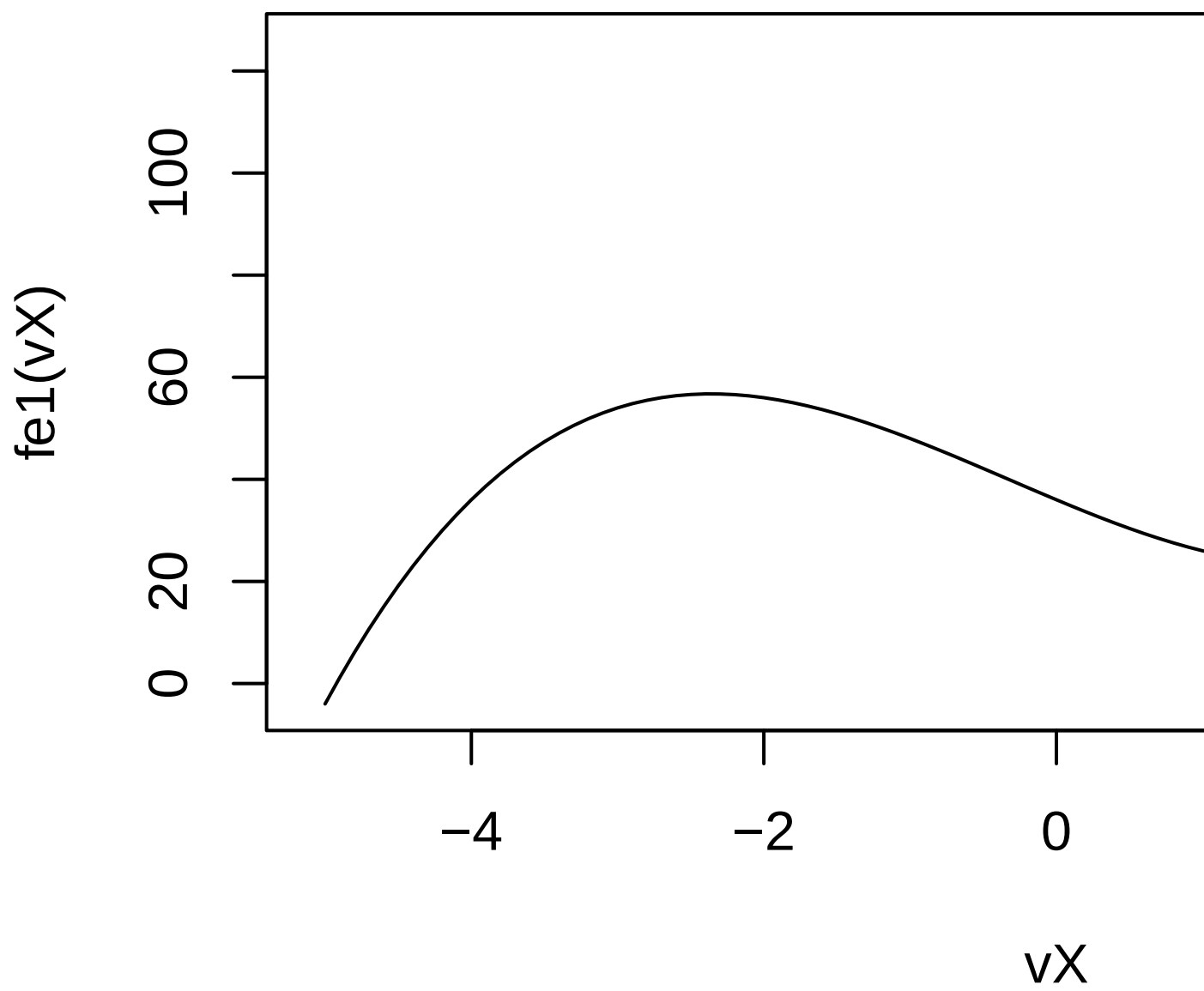
Find the local maximum and the local minimum of the function $f(x) = x^3 + (6 - x)^2$ on the interval $[-5, 5]$ using Newton's method and the `optimize` function.

Plot the function to pick starting points for Newton's method.

```
# Functions
fe1 <- function(dX) {
  dOut <- dX^3 + (6-dX)^2
  return(dOut)
}
feprime <- function(dX) {
  dOut <- 3 * dX^2 - 2*(6-dX)
  return(dOut)
}
fesecond <- function(dX) {
  dOut <- 6 * dX + 2
  return(dOut)
}
#Solution
xe1 <- NM(fe1, feprime, fesecond, dX0 = -1) # minimum
vX <- seq(-5, 5, 0.1)
plot(vX, fe1(vX), type = "l")
abline(v=-1, col="blue") # initial guess
abline(v=xe1, col="red") # solution

xe2 <- NM(fe1, feprime, fesecond, dX0 = 4) # maximum
vX <- seq(-5, 5, 0.1)
plot(vX, fe1(vX), type = "l")
abline(v=4, col="blue") # initial guess
abline(v=xe2, col="red") # solution
```





8.3 The Golden-section method

The golden-section method works in one dimension only, but does not need f' .

This method is similar to the root-bracketing technique for root-finding.

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous function (note, that we do not assume that we have a derivative).

If we have two points $x_l < x_r$ such that $f(x_l)f(x_r) \leq 0$ then we know that there is a root in the interval $[x_l, x_r]$.

To determine if we have a local maximum, we need three points: if $x_l < x_m < x_r$ and $f(x_l) \leq f(x_m)$ and $f(x_r) \leq f(x_m)$ then there must be a local maximum in the interval $[x_l, x_r]$.

8.3.1 The idea of the Golden-section method

0) Start with $x_l < x_m < x_r$ such that $f(x_l) \leq f(x_m)$ and $f(x_r) \leq f(x_m)$.

1) If $x_r - x_l \leq \varepsilon$ then stop; set $x^* = x_m$.

2) If $x_r - x_m > x_m - x_l$ then 2a.; otherwise do 2b.

2a) Choose a point $y \in (x_m, x_r)$

- If $f(y) \geq f(x_m)$ then the maximum is (x_m, x_r) and we can redefine the interval: put $x_l = x_m$ and $x_m = y$;
- Otherwise the maximum is in the interval (x_l, y) and we can redefine the interval: $x_r = y$.

2b) Choose a point $y \in (x_l, x_m)$

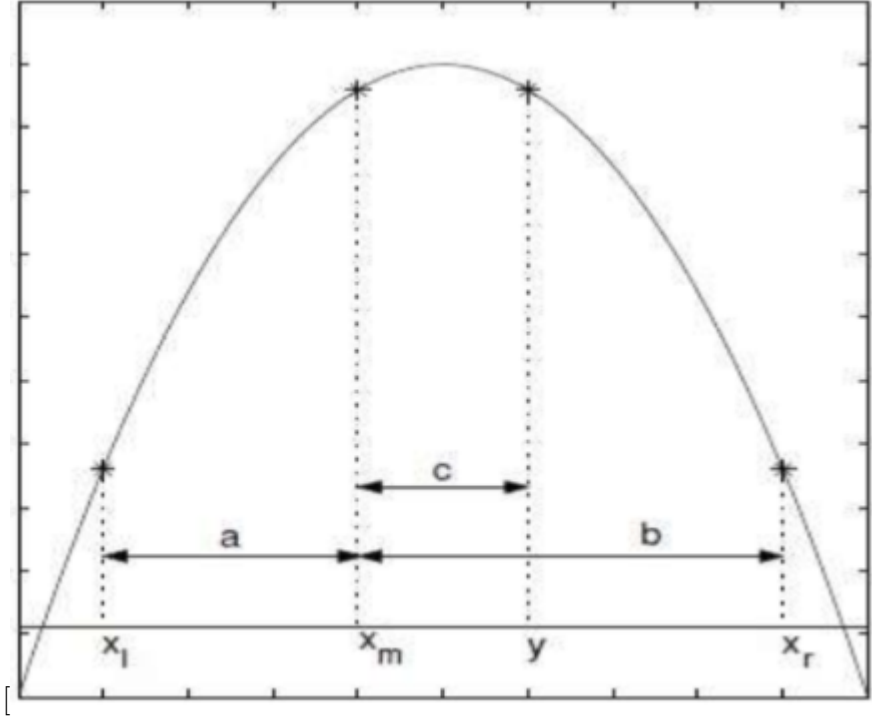
- If $f(y) \geq f(x_m)$ then the maximum is (x_l, x_m) and we can redefine the interval: put $x_r = x_m$ and $x_m = y$;
- Otherwise the maximum is in the interval (y, x_r) and we can redefine the interval: $x_l = y$.

3) Go back to step 1.

8.3.2 The Golden ratio

Other than saying that y should be in the larger of the two intervals, (x_l, x_m) and (x_m, x_r) , how do we specify y ?

Suppose (x_m, x_r) is the larger interval (as in the figure below), and let $a = x_m - x_l$, $b = x_r - x_m$, and $c = y - x_m$.



The golden-section algorithm chooses y such that the ratio of the lengths of the larger to the smaller interval stays the same at each iteration.

That is, if the new bracketing interval is $[x_l, y]$ then:

$$\frac{a}{c} = \frac{b}{a}$$

While if the new bracketing interval is $[x_m, x_r]$ then:

$$\frac{b-c}{c} = \frac{b}{a}$$

Setting $\phi = \frac{b}{a}$ and using the two equations above to solve for ϕ and we can find the famous golden ratio:

$$\phi^2 - \phi - 1 = 0 \Rightarrow \phi = \frac{1 + \sqrt{5}}{2} \approx 1.618$$

We then solve for c using the second equation:

$$\frac{b-c}{c} = \phi \Rightarrow c = b/(1+\phi)$$

Choose y :

$$y = x_m + \underbrace{c}_{y-x_m} + x_m + \underbrace{b}_{x_r-x_m}/(1+\phi) = x_m + (x_r - x_m)/(1+\phi)$$

An analogous argument applies if (x_l, x_m) is the larger interval.

Using this method for choosing y gives the following version of the algorithm:

8.4 New algorithm

0. Start with $x_l < x_m < x_r$ such that $f(x_l) \leq f(x_m)$ and $f(x_r) \leq f(x_m)$.
1. If $x_r - x_l \leq \varepsilon$ then stop; set $x^* = x_m$.
2. If $x_r - x_m > x_m - x_l$ then 2.i; otherwise do 2.ii.
 - i) Set $y = x_m + (x_r - x_m)/(1+\phi)$. If $f(y) \geq f(x_m)$ then put $x_l = x_m$ and $x_m = y$; otherwise put $x_r = y$;
 - ii) Set $y = x_m + (x_m - x_l)/(1+\phi)$. If $f(y) \geq f(x_m)$ then put $x_r = x_m$ and $x_m = y$; otherwise put $x_l = y$;
3. Go back to step 1.

With $\phi = \frac{1+\sqrt{5}}{2}$ being the famous golden ratio.

8.4.1 Notes

The length ratio of the new interval to the old is either $b/(a+b)$ or $(a+c)/(a+b)$ (see figure to verify graphically), which both work out as $\phi/(1+\phi)$.

That implies that if we start with x_m chosen so that the ratio $\underbrace{(x_r - x_m)}_b / \underbrace{(x_m - x_l)}_a = \phi$ or

$1/\phi$ then at each iteration the width of the bracketing interval is reduced by a constant factor of $\phi/(1+\phi)$ and so must eventually go to zero.

If you do not start with the ratio $(x_r - x_m)/(x_m - x_l) = \phi$ or $1/\phi$ it is not a problem because as soon as you have an iteration that puts $x_m = y$, this will be the case (by definition of how y is chosen).

Since $(x_r - x_l) \rightarrow 0$ as $n \rightarrow \infty$, to stop the golden-section algorithm it is sufficient to specify a tolerance $\varepsilon > 0$ then stop when $x_r - x_l \leq \varepsilon$.

8.4.2 The golden-section method: R implementation

```
gsection <- function(f, dX.l, dX.r, dX.m, dTol = 1e-9) {

  # golden ratio plus one
  dGR1 <- 1 + (1 + sqrt(5))/2

  # successively refine x.l, x.r, and x.m
  f.l <- f(dX.l)
  f.r <- f(dX.r)
  f.m <- f(dX.m)
  while ((dX.r - dX.l) > dTol) {
    if ((dX.r - dX.m) > (dX.m - dX.l)) { # if the right segment is wider than the left
      dY <- dX.m + (dX.r - dX.m)/dGR1 # put Y into the right segment according to the golden
      f.y <- f(dY)
      if (f.y >= f.m) {
        dX.l <- dX.m
        f.l <- f.m
        dX.m <- dY
        f.m <- f.y
      } else {
        dX.r <- dY
        f.r <- f.y
      }
    } else { #if the left segment is wider than the right
      dY <- dX.m - (dX.m - dX.l)/dGR1 # put Y into the left segment according to the golden
      f.y <- f(dY)
      if (f.y >= f.m) {
        dX.r <- dX.m
        f.r <- f.m
        dX.m <- dY
        f.m <- f.y
      } else {
        dX.l <- dY
      }
    }
  }
}
```



```

        f.l <- f.y
      }
    }
  }
  return(dX.m)
}
gsection(f, dX.l = -1, dX.r = 1, dX.m = 0.5)
#> [1] 0.3660254

```

8.5 Optimization in R: univariate

In one dimension, R provides the function `optimize`, which uses a combination of the golden-section algorithm with a technique called parabolic interpolation.

```
optimize(f, interval, ..., lower = min(interval), upper = max(interval), maximum = FALSE, to
```

Note that since `maximum = FALSE`, by default, `optimize` minimizes f .

```

optimize(f, lower = -0.4, upper = 0.8, maximum = TRUE)
#> $maximum
#> [1] 0.3660215
#>
#> $objective
#> [1] 0.6961524

```

For the example:

```

# Using built-in optimizer
optimize(fe1, lower = -5, upper = 5, maximum = TRUE) # maximum
#> $maximum
#> [1] -2.360919
#>
#> $objective
#> [1] 56.74535

optimize(fe1, lower = -5, upper = 5) # minimum
#> $minimum
#> [1] 1.694255
#>
#> $objective

```

```
#> [1] 23.4028

f2 <- function(x){-1*f1(x)}
optimize(f2, lower = -5, upper = 5)
#> $minimum
#> [1] -2.360919
#>
#> $objective
#> [1] -56.74535
```

9 Numerical Optimization 2

9.1 Multivariate Optimization

We now turn to the more useful but more difficult problem of finding local minima or maxima of a function of several variables, i.e., **multivariate optimization**.

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ and suppose that all of the first- and second-order partial derivatives of f exist and are continuous everywhere.

Denote the i -th partial derivative at $\mathbf{x} = (x_1, \dots, x_d)^\top$ as $f_i(\mathbf{x}) = \frac{\partial f(\mathbf{x})}{\partial x_i}$ and define the *gradient*:

$$\nabla f(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_d(\mathbf{x}))^\top,$$

And the *Hessian*:

$$\mathbf{H}(\mathbf{x}) = \begin{pmatrix} \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_1} & \dots & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_d \partial x_1} & \dots & \frac{\partial^2 f(\mathbf{x})}{\partial x_d \partial x_d} \end{pmatrix}$$

A necessary (but not sufficient) condition for a local maximum at $\mathbf{x}$ is $\nabla f(\mathbf{x}) = \mathbf{0}$ and the Hessian is **negative semi-definite**.

A sufficient (but not necessary) condition for a local maximum at $\mathbf{x}$ is $\nabla f(\mathbf{x}) = \mathbf{0}$ and the Hessian is **negative definite**.

As in one dimension, we will use iterative local search techniques to find local maxima.

Define $\|\mathbf{x}\|_\infty = \max_i |x_i|$. In higher dimensions, we use stopping conditions that are combinations of the following:

- $\|\mathbf{x}_n - \mathbf{x}_{n-1}\|_\infty \leq \varepsilon$;
- $|f(\mathbf{x}_n) - f(\mathbf{x}_{n-1})| \leq \varepsilon$;
- $\|\nabla f(\mathbf{x}_n)\|_\infty \leq \varepsilon$;
- And to guard against non-convergence, we also specify a maximum number of iterations n_{max} , then stop when $n = n_{max}$.

9.2 Steepest ascent method

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a function with continuous partial derivatives everywhere. We wish to find a local maximum of f in the neighborhood of some point x_0 .

In the steepest ascent method, we seek a local maximum by moving in the direction of the gradient, which is the direction with the largest slope, i.e., the steepest ascent. (The direction with the smallest slope at the point $\mathbf{x}$ is $-\nabla f(\mathbf{x})$, which you use if you are searching for a local minimum).

Thus, the steepest ascent method has the form:

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \alpha \nabla f(\mathbf{x}_n)$$

With some $\alpha \geq 0$ being the step size while $\nabla f(\mathbf{x}_n)$ is the direction of the step.

Given this form, we choose $\alpha \geq 0$ to maximize.

$$g(\alpha) = f(\mathbf{x}_n + \alpha \nabla f(\mathbf{x}_n)) = f(\mathbf{x}_{n+1}).$$

Unless we're already at a local maximum, we will always choose $\alpha > 0$ with $f(\mathbf{x}_{n+1}) > f(\mathbf{x}_n)$.

If f is bounded from above then, because $f(\mathbf{x}_{n+1}) \geq f(\mathbf{x}_n)$, the sequence $\{f(\mathbf{x}_n)\}_{n=1}^{\infty}$ must converge, which suggests that we can use the stopping condition $f(\mathbf{x}_{n+1}) - f(\mathbf{x}_n) \leq \varepsilon$ for some small tolerance $\varepsilon > 0$.

Define the function `line.search` that takes arguments f , $\mathbf{x}_n$, and $\nabla f(\mathbf{x}_n)$ and returns $\mathbf{x}_n + \alpha^* \nabla f(\mathbf{x}_n)$, where α^* is the value that maximizes $g(\alpha)$, we can sketch the implementation of the steepest ascent method.

9.2.1 Steepest ascent method: Implementation in R

```
ascent <- function(f, grad.f, vX0, dTol = 1e-9, n.max = 100) {  
  vX.old <- vX0  
  vX <- line.search(f, vX0, grad.f(vX0))  
  n <- 1  
  while ((f(vX) - f(vX.old) > dTol) & (n < n.max)) {  
    vX.old <- vX  
    vX <- line.search(f, vX, grad.f(vX))  
    cat("at iteration", n, "the coordinates of x are", vX, "\n")  
    n <- n + 1  
  }  
}
```

```

    return(vX)
}

```

9.2.2 Line search

To complete the steepest ascent algorithm, at each step n , given the optimal direction $f(\mathbf{x}_n)$, we need to find the optimal step size, α^* , that is, we need to maximize $g(\alpha) = f(\mathbf{x}_n + \alpha f(\mathbf{x}_n))$ over $\alpha > 0$.

Hence, we need to find

$$\alpha^* = \operatorname{argmax}_{\alpha} f(\mathbf{x}_n + \alpha f(\mathbf{x}_n)).$$

As we know, it is very hard to find a global maximum, so we will instead look for a local maximum, for which we will use the golden-section algorithm.

The golden-section method requires three initial points $\alpha_l < \alpha_m < \alpha_r$ such that $g(\alpha_m) \geq g(\alpha_l)$ and $g(\alpha_m) \geq g(\alpha_r)$.

Put $\alpha_l = 0$. Theoretically, if $\|\nabla f(\mathbf{x}_n)\| > 0$ then $g'(0) > 0$ and there must exist some $\varepsilon > 0$ such that $g(\varepsilon) > g(0)$, so we can put $\alpha_m = \varepsilon$.

In practice, however, if $g'(0)$ is very small, it might not be possible to numerically distinguish $g(0)$ from $g(\varepsilon)$, and in that case we set $\alpha^* = 0$.

There is no theoretical guarantee that a suitable α_r exists, because we might have g increasing over the whole interval $[0, \infty)$. To deal with this problem, we specify a maximum step size, α_{max} , such that if we cannot find $\alpha_r \leq \alpha_{max}$ such that $g(\alpha_m) \geq g(\alpha_r)$, we set $\alpha^* = \alpha_{max}$.

9.2.3 Line-search: Implementation in R

```

line.search <- function(f, vX, vG, dTol = 1e-9, dA.max = 2^5) {
  # f is a real function that takes a vector of length d
  # x and y are vectors of length d
  # line.search uses gsection to find a >= 0 such that
  # g(a) = f(x + a*y) has a local maximum at a,
  # within a tolerance of tol
  # if no local max is found then we use 0 or a.max for a
  # the value returned is x + a*y
  if (sum(abs(vG)) == 0){
    return(vX) # +0*vG
  }
}

```

```

} # g(a) constant
g <- function(dA){
  return(f(vX + dA*vG))
}
# find a triple a.l < a.m < a.r such that
# g(a.l) <= g(a.m) and g(a.m) >= g(a.r)

# choose a.l
dA.l <- 0
g.l <- g(dA.l)
# find a.m
dA.m <- 1
g.m <- g(dA.m)
while ((g.m < g.l) & (dA.m > dTol)) {
  dA.m <- dA.m/2
  g.m <- g(dA.m)
}
# if a suitable a.m was not found then use 0 for a, so just return vX as the next step
if ((dA.m <= dTol) & (g.m < g.l)){
  return(vX)
}
# find a.r
dA.r <- 2*dA.m
g.r <- g(dA.r)
while ((g.m < g.r) & (dA.r < dA.max)) {
  dA.m <- dA.r
  g.m <- g.r
  dA.r <- 2*dA.m
  g.r <- g(dA.r)
}
# if a suitable a.r was not found then use a.max for a
if ((dA.r >= dA.max) & (g.m < g.r)){
  return(vX + dA.max*vG)
}
# apply golden-section algorithm to g to find a
dA <- gsection(g, dA.l, dA.r, dA.m)
return(vX + dA*vG)
}

```

i Note: the code depends on the golden section algorithm. [Click here to view it.](#)

```
1 # golden section algorithm from last week
2 gsection <- function(f, dX.l, dX.r, dX.m, dTol = 1e-9) {
3
4   # golden ratio plus one
5   dGR1 <- 1 + (1 + sqrt(5))/2
6
7   # successively refine x.l, x.r, and x.m
8   f.l <- f(dX.l)
9   f.r <- f(dX.r)
10  f.m <- f(dX.m)
11  while ((dX.r - dX.l) > dTol) {
12    if ((dX.r - dX.m) > (dX.m - dX.l)) { # if the right segment is wider than the left
13      dY <- dX.m + (dX.r - dX.m)/dGR1 # put Y into the right segment according to the golden
14      f.y <- f(dY)
15      if (f.y >= f.m) {
16        dX.l <- dX.m
17        f.l <- f.m
18        dX.m <- dY
19        f.m <- f.y
20      } else {
21        dX.r <- dY
22        f.r <- f.y
23      }
24    } else { #if the left segment is wider than the right
25      dY <- dX.m - (dX.m - dX.l)/dGR1 # put Y into the left segment according to the golden
26      f.y <- f(dY)
27      if (f.y >= f.m) {
28        dX.r <- dX.m
29        f.r <- f.m
30        dX.m <- dY
31        f.m <- f.y
32      } else {
33        dX.l <- dY
34        f.l <- f.y
35      }
36    }
37  }
38  return(dX.m)
39 }
```

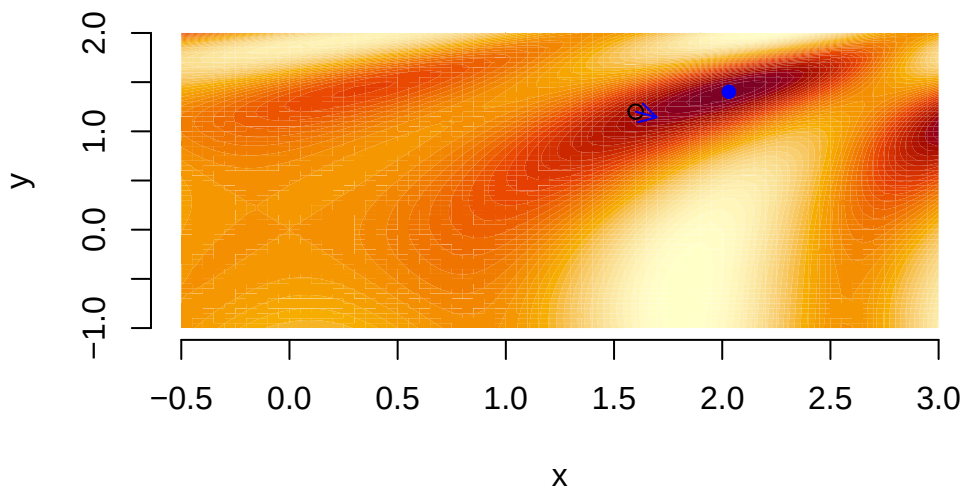
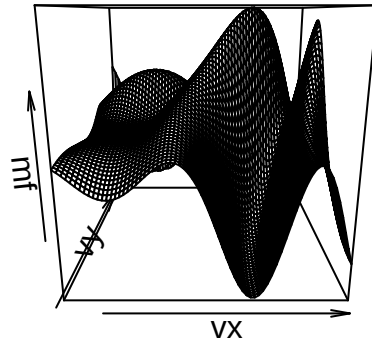
9.2.4 Example

Consider the function $f(x, y) = \sin(x^2/2 - y^2/4) \cos(2x - \exp(y))$

```
# function
f <- function(vX) {
  dOut = sin(vX[1]^2/2 - vX[2]^2/4) * cos(2*vX[1] - exp(vX[2]))
  return(dOut)
}

# gradient
grad.f <- function(vX) {
  dX_prime.1 = cos(vX[1]^2/2 - vX[2]^2/4)*vX[1] * cos(2*vX[1] - exp(vX[2]))
  dX_prime.2 = sin(vX[1]^2/2 - vX[2]^2/4) * (-sin(2*vX[1] - exp(vX[2]))*2)
  dY_prime.1 = cos(vX[1]^2/2 - vX[2]^2/4)*(-vX[2]/2) * cos(2*vX[1] - exp(vX[2]))
  dY_prime.2 = sin(vX[1]^2/2 - vX[2]^2/4) * sin(2*vX[1] - exp(vX[2]))*exp(vX[2])
  vOut = c(dX_prime.1 + dX_prime.2, dY_prime.1 + dY_prime.2)
  return(vOut)
}

ascent(f, grad.f, vX0 = c(0.1, 0.3))
#> [1] 2.030674 1.401513
ascent(f, grad.f, vX0 = c(0, 0.5))
#> [1] 0.3424608 1.4271493
```

A small difference in where you start can make a big difference to where you end up; we find different local maxima with the two different starting points.

9.3 Newton's method in higher dimensions

The steepest ascent method uses information about the gradient, i.e., the first-order derivatives of f .

By making use of the Hessian, i.e., the second-order derivatives of f , we can construct methods that converge in fewer steps.

The simplest second-order technique is Newton's method, which can be generalized from one dimension to higher dimensions relatively easily.

Newton's method looks for a point $\mathbf{x}$ such that $\nabla f(\mathbf{x}) = \mathbf{0}$.

The basis of the method is a second-order Taylor expansion of f . For any $\mathbf{x}$ and $\mathbf{y}$ close together we have:

$$f(\mathbf{y}) \approx f(\mathbf{x}) + (\mathbf{y} - \mathbf{x})' \nabla f(\mathbf{x}) + \frac{1}{2} (\mathbf{y} - \mathbf{x})' \mathbf{H}(\mathbf{x}) (\mathbf{y} - \mathbf{x}). \quad (1)$$

Taking first-order partial derivatives on both sides of (1) with respect to the components of $\mathbf{y}$, we get:

$$\nabla f(\mathbf{y}) \approx \nabla f(\mathbf{x}) + \mathbf{H}(\mathbf{x})(\mathbf{y} - \mathbf{x}).$$

If $\mathbf{y}$ is a local maximum then $\nabla f(\mathbf{y}) = 0$ and, solving the equation above, we get $\mathbf{y} = \mathbf{x} - \mathbf{H}(\mathbf{x})^{-1} \nabla f(\mathbf{x})$.

Putting $\mathbf{x} = \mathbf{x}_n$ and $\mathbf{y} = \mathbf{x}_{n+1}$, our Newton's step in higher dimensions becomes:

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \mathbf{H}(\mathbf{x}_n)^{-1} \nabla f(\mathbf{x}_n).$$

Clearly, if $\mathbf{H}(\mathbf{x}_n)$ is singular (has no inverse), then Newton's method does not work.

However, as in the one-dimensional case, even if $\mathbf{H}(\mathbf{x}_n)$ is non-singular at each step, Newton's method may not converge.

But if f has a local maximum at $\mathbf{x}^*$, f is 'nicely behaved' near $\mathbf{x}^*$, and if our initial point $\mathbf{x}_0$ is 'close enough' to $\mathbf{x}^*$, then Newton's method will converge to $\mathbf{x}^*$ quickly.

In implementing Newton's method, we assume that we have some function `f3` that takes argument $\mathbf{x}$ and returns a list containing $f(\mathbf{x})$, $\nabla f(\mathbf{x})$ and $\mathbf{H}(\mathbf{x})$, and for the stopping condition we use $\|\nabla f(\mathbf{x}_n)\|_\infty \leq \varepsilon$.

Furthermore, we make use of the fact that for an invertible matrix $\mathbf{A} \in \mathbb{R}^{d \times d}$ and a vector $\mathbf{b} \in \mathbb{R}^d$, $\mathbf{A}^{-1}\mathbf{b}$ is the solution to the system of equations $\mathbf{A}\mathbf{x} = \mathbf{b}$, which can be found by the R command `solve(A, b)`.

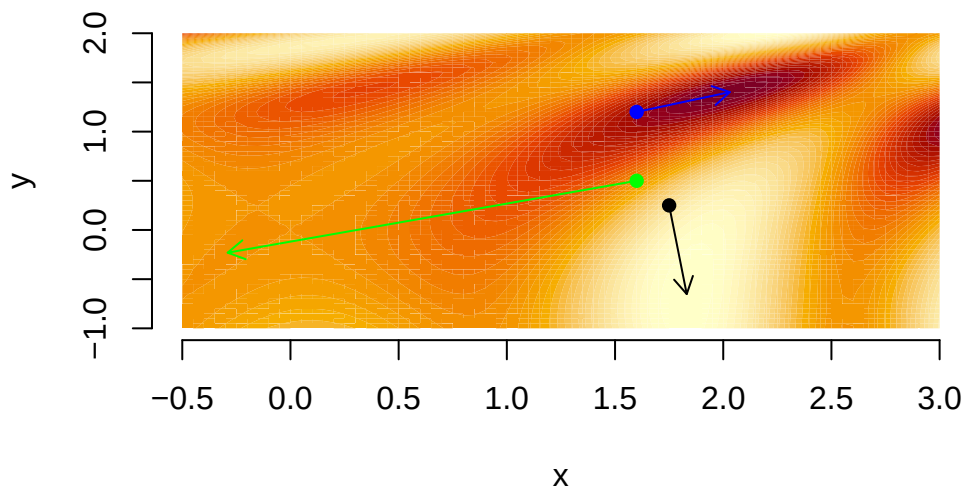
9.3.1 Newton's method: Implementation in R

```
newton <- function(f3, vX0, dTol = 1e-9, n.max = 100) {  
  # Newton's method for optimisation, starting at x0  
  # f3 is a function that given x returns the list  
  # {f(x), grad f(x), Hessian f(x)}, for some f  
  vX <- vX0  
  f3.x <- f3(vX)  
  n <- 0  
  while ((max(abs(f3.x[[2]])) > dTol) & (n < n.max)) {  
    vX <- vX - solve(f3.x[[3]], f3.x[[2]])  
    #vX <- vX - solve(f3.x[[3]])%*%f3.x[[2]]  
    f3.x <- f3(vX)  
    cat("At iteration", n, "the coordinates of x are", vX, "\n")  
    n <- n + 1  
  }  
  if (n == n.max) {  
    cat('newton failed to converge\n')  
  } else {  
    return(vX)  
  }  
}
```

9.3.2 Example

Consider again the function $f(x, y) = \sin(x^2/2 - y^2/4) \cos(2x - \exp(y))$.

```
newton(f3, vX0 = c(1.6, 1.2)) # maximum  
#> [1] 2.030697 1.401526  
newton(f3, vX0 = c(1.6, 0.5)) # saddle point  
#> [1] -0.2902213 -0.2304799  
newton(f3, vX0 = c(1.75, 0.25)) # minimum  
#> [1] 1.8313777 -0.6516928
```



Remember to check whether it is in fact a maximum with the `eigen` function, e.g.,

```
eigen(f3(n1)[[3]]) # maximum
#> eigen() decomposition
#> $values
#> [1] -2.030712 -23.079010
#>
#> $vectors
#>          [,1]      [,2]
#> [1,] -0.8429247 -0.5380316
#> [2,] -0.5380316  0.8429247

eigen(f3(n2)[[3]]) # saddle point
#> eigen() decomposition
#> $values
#> [1] -0.06666478 -1.20252229
#>
#> $vectors
#>          [,1]      [,2]
#> [1,] -0.456103 -0.889927
#> [2,] -0.889927  0.456103

eigen(f3(n3)[[3]]) # minimum
```

```
#> eigen() decomposition
#> $values
#> [1] 7.382289 0.349442
#>
#> $vectors
#>           [,1]      [,2]
#> [1,] -0.99798281 -0.06348473
#> [2,]  0.06348473 -0.99798281
```

i Click to view the function f3 and the code for the plot

```
1 #function, gradient, and hessian in list
2 f3 <- function(vX) {
3   dA <- vX[1]^2/2 - vX[2]^2/4
4   dB <- 2*vX[1] - exp(vX[2])
5   f <- sin(dA)*cos(dB)
6   f1 <- cos(dA)*cos(dB)*vX[1] - sin(dA)*sin(dB)*2
7   f2 <- -cos(dA)*cos(dB)*vX[2]/2 + sin(dA)*sin(dB)*exp(vX[2])
8   f11 <- -sin(dA)*cos(dB)*(4 + vX[1]^2) + cos(dA)*cos(dB) -
9     cos(dA)*sin(dB)*4*vX[1]
10  f12 <- sin(dA)*cos(dB)*(vX[1]*vX[2]/2 + 2*exp(vX[2])) +
11    cos(dA)*sin(dB)*(vX[1]*exp(vX[2]) + vX[2])
12  f22 <- -sin(dA)*cos(dB)*(vX[2]^2/4 + exp(2*vX[2])) - cos(dA)*cos(dB)/2 -
13    cos(dA)*sin(dB)*vX[2]*exp(vX[2]) + sin(dA)*sin(dB)*exp(vX[2])
14  return(list(f, c(f1, f2), matrix(c(f11, f12, f12, f22), 2, 2)))
15 }
16
17 n1<-newton(f3, vX0 = c(1.6, 1.2))
18 n2<-newton(f3, vX0 = c(1.6, 0.5))
19 n3<-newton(f3, vX0 = c(1.75, 0.25))
20
21 {
22   plot(NA,xlim=range(vx),
23     ylim=range(vy),xlab="x",ylab="y",
24     frame=FALSE)
25   levels = pretty(range(mf), 50)
26   color.palette = function(n) hcl.colors(n, "YlOrRd", rev = TRUE)
27   .filled.contour(x=vx, y=vy, z=mf,
28     levels=levels,
29     col=color.palette(length(levels) - 1))
30   points(1.6, 1.2, pch=16, col="blue")
31   arrows(1.6, 1.2, n1[1], n1[2], length=0.1, col="blue")
32
33   points(1.6, 0.5, pch=16, col="green")
34   arrows(1.6, 0.5, n2[1], n2[2], length=0.1, col="green")
35
36   points(1.75, 0.25, pch=16, col="black")
37   arrows(1.75, 0.25, n3[1], n3[2], length=0.1, col="black")
38 }
```

The above example has illustrated that:

- Newton's method can converge to minima or saddle points as well as maxima.
- Newton's method is faster than the steepest ascent method.
- Unless you are close to a minimum or maximum, you can move in unexpected directions.

9.4 Disadvantages of steepest ascent and Newton's method

Potential disadvantages of the steepest ascent method and Newton's method is the need to calculate the gradient and Hessian.

For functions that can be expressed in terms of polynomials and the simple transcendental functions (sin, cos, exp, log etc.) the process of calculating the gradient and Hessian should pose no problems (can use the R function `deriv` or just `D` to find the gradient and Hessian of a simple expression - check it out yourselves.)

There are, however, plenty of situations where f is available but ∇f is not; e.g. f might be the result of some numerical procedure or an approximation obtained by simulation.

Next time you will see how to deal with such situations as well as see how to do constrained optimization.

9.5 Gradient and Hessian in R

We saw that the `gradfunction` in the `numDeriv` package can help us to compute the gradient of f :

```
grad(f, x, ...)
```

The `numDeriv` package also provides us the `hessian` function:

```
hessian(f, x, ...)
```

See `help(hessian)`. (Remember to load `numDeriv` with `library(numDeriv)`).

9.6 Optimization in R: multivariate

In higher dimensions there are a variety of optimization methods in current use. The R function `optim` provides functionalities to maximize a multivariate function:

```
optim(par, fn, gr = NULL, ..., method = c("Nelder-Mead", "BFGS", "CG", "L-  
BFGS-B", "SANN", "Brent"), lower = -Inf, upper = Inf, control = list(),  
hessian = FALSE)
```

Where the `method` argument specifies the type of optimizer we want to employ. The most used are:

- `method = "Nelder-Mead"` the Nelder and Mead¹, method.
- `method = "BFGS"` is a quasi-Newton method.
- `method = "L-BFGS-B"` of Byrd et. al. (1995)² which allows box constraints.

9.6.1 Example

```
# fnscale multiplies the function by -1, such that a maximum is found.
# the default is a minimum
optim(c(1.6, 1.2), f, gr = grad.f, method = "BFGS", control = list(fnscale = -1))
#> $par
#> [1] 2.030697 1.401526
#>
#> $value
#> [1] 1
#>
#> $counts
#> function gradient
#>      22      9
#>
#> $convergence
#> [1] 0
#>
#> $message
#> NULL
```

¹Nelder, J. A. and Mead, R. (1965) A simplex algorithm for function minimization. Computer Journal 7, 308-313.

²Byrd, R. H., Lu, P., Nocedal, J. and Zhu, C. (1995) A limited memory algorithm for bound constrained optimization. SIAM J. Scientific Computing, 16, 1190-1208.

10 Constrained Optimization and Pitfalls in Optimization

10.1 Constrained Optimization

Very often we have some restrictions on the input space used for optimization. Typical examples include optimization under

- nonnegativity constraints $\mathbf{x} \geq \mathbf{0}$
- m linear restrictions $\mathbf{R}\mathbf{x} - \mathbf{q} = \mathbf{0}$ for some $\mathbf{R} \in \mathbb{R}^m \times \mathbb{R}^d$, $\mathbf{q} \in \mathbb{R}^d$
- upper and lower bounds $\mathbf{a} \leq \mathbf{x} \leq \mathbf{b}$ for some $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$

Example I: Likelihood of a stationary autoregressive AR(1) process:

$$y_t = \alpha y_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim N(0, \sigma^2)$$

For stationarity we need that $|\alpha| < 1$ or $-1 < \alpha < 1$.

Example II: Stationary GARCH Process

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

For stationarity and nonnegativity we need that $\omega > 0$ and $\alpha + \beta < 1$

In general the constrained optimization problem can be defined as

$$\begin{cases} \min_x f(x) \\ \text{s.t. } g(x) \leq 0 \end{cases} \quad (1)$$

E.g.

- $\mathbf{g}(\mathbf{x}) = -\mathbf{x} \leq \mathbf{0}$
- $\mathbf{g}(\mathbf{x}) = \begin{pmatrix} \mathbf{R}\mathbf{x} - \mathbf{q} \\ -(\mathbf{R}\mathbf{x} - \mathbf{q}) \end{pmatrix} \leq \mathbf{0}$
- $\mathbf{g}(\mathbf{x}) = \begin{pmatrix} \mathbf{x} - \mathbf{b} \\ \mathbf{a} - \mathbf{x} \end{pmatrix} \leq \mathbf{0}$

The most common techniques to transform a constrained problem into an unconstrained problem are:

- **Penalization functions**
- **Barrier functions**

10.2 Penalization functions

If we could, we'd like to optimize

$$\min_{\mathbf{x}} (f(\mathbf{x}) + p(\mathbf{x})), \quad p(\mathbf{x}) = \begin{cases} 0 & \mathbf{g}(\mathbf{x}) \leq \mathbf{0} \\ \infty & \text{otherwise} \end{cases} \quad (2)$$

Not a continuous function! Instead we can do

$$\min_{\mathbf{x}} \underbrace{(f(\mathbf{x}) + \gamma \cdot \|\max\{\mathbf{g}(\mathbf{x}), \mathbf{0}\}\|^2)}_{f_p(\mathbf{x}, \gamma)} \quad \gamma > 0 \quad (3)$$

such that every time the function exits from the boundary, its value is increased by an amount that is proportional to the distance from the admissible set. We can then solve a sequence of optimization problems

$$\mathbf{x}^{*(h)} = \min_{\mathbf{x}} f_p(\mathbf{x}, \gamma_h)$$

for an increasing sequence of positive values of γ_h . The algorithm stops when

$$\|\mathbf{x}^{*(h)} - \mathbf{x}^{*(h-1)}\| < \epsilon_\gamma$$

10.3 Barrier functions - (Interior-point)

The objective function in this case is transformed as

$$\min_{\mathbf{x}} \underbrace{(f(\mathbf{x}) + \gamma \cdot b(\mathbf{x}))}_{f_b(\mathbf{x}, \gamma)} \quad \gamma > 0 \quad (4)$$

where

$$b(\mathbf{x}) = \begin{cases} -\sum_{i=1}^m \log(-g_i(\mathbf{x})) & \mathbf{g}(\mathbf{x}) \leq \mathbf{0} \\ \infty & \text{otherwise} \end{cases}$$

We impose a very large cost on feasible points that lie close to the boundary since $-\log(-g_i(\mathbf{x})) \rightarrow \infty$ as $g_i(\mathbf{x}) \rightarrow 0^-$. We can then solve a sequence of optimization problems

$$\mathbf{x}^{*(h)} = \min_{\mathbf{x}} f_b(\mathbf{x}, \gamma_h)$$

for a **decreasing** sequence of positive values of γ_h . The algorithm stops when

$$\|\mathbf{x}^{*(h)} - \mathbf{x}^{*(h-1)}\| < \epsilon_\gamma$$

10.4 Parameter constraints

So far we have considered the case where $\mathbf{x} \in \mathbb{R}^p$, however, the most common scenario is when $\mathbf{x} \in \mathcal{X} \subset \mathbb{R}^p$.

- Newton-based method (`method = "BFGS"`) doesn't know about ranges
- Alternative optimization (`method = "L-BFGS-B"`) does but: slower/worse convergence

Consider for example the problem of estimating the location μ , scale ψ and degree of freedom parameters of a Student's t distribution. Formally let $\mathbf{y} = (y_1, \dots, y_T)'$ a sample of T iid observations from a Student's t distribution. The ML estimator for $\theta = (\mu, \psi, \nu)$ is:

$$^{ML} = \operatorname{argmax}_{\theta \in \Theta} \sum_{t=1}^T \log p_T(y_t; \theta),$$

where Θ is the restricted space $\mathbb{R} \times (0, \infty) \times (0, \infty)$.

We have two options:

- 1) Use a constrained optimizer like `optim` with `method = "L-BFGS-B"`.
- 2) Reparameterize our problem and use an unrestricted optimizer.

The second option is usually preferred and provides better results.

10.5 Reparameterization

Let $\boldsymbol{\theta} : \mathbb{R}^p \rightarrow \Omega$ a vector-valued differentiable mapping function such that $\boldsymbol{\theta}(\boldsymbol{\eta}) = \boldsymbol{\theta}$ where $\boldsymbol{\eta}$ is a reparametrization of $\boldsymbol{\theta}$. In the Student's t case $\boldsymbol{\eta}$ can be chosen as:

$$\boldsymbol{\eta} = \begin{cases} \mu = \tilde{\mu} \\ \psi = \exp(\tilde{\psi}) \\ \nu = \exp(\tilde{\nu}) \end{cases}$$

where $\tilde{\psi}$ and $\tilde{\nu}$ are reparameterization of ψ and ν , respectively.

Since $\tilde{\boldsymbol{\theta}} \in \mathbb{R}^3$, the original optimization problem can now be reformulated as:

$$\tilde{\boldsymbol{\theta}}^{ML} = \underset{\tilde{\boldsymbol{\theta}} \in \mathbb{R}^3}{\operatorname{argmax}} \sum_{t=1}^T \log p_T(y_t; \boldsymbol{\eta}),$$

and then $\boldsymbol{\theta}^{ML} = \boldsymbol{\theta}(\tilde{\boldsymbol{\theta}}^{ML})$. Which can be solved with an unconstrained optimizer like `optim` with `method = "BFGS"`.

10.6 Differentiation after reparameterization: Delta method

Note however that the evaluation of the gradient and the hessian matrix needs to be modified after the reparameterization of the problem. We have the following identities:

$$\begin{aligned} \tilde{\nabla} f(\boldsymbol{\eta}) &= \mathcal{J}(\boldsymbol{\eta})' \nabla f(\boldsymbol{\theta}) = \mathcal{J}(\boldsymbol{\eta})' \nabla f(\boldsymbol{\theta}(\boldsymbol{\eta})) \\ \tilde{\mathbf{H}}(\tilde{\boldsymbol{\theta}}) &= \mathcal{J}(\tilde{\boldsymbol{\theta}})' \mathbf{H}(\boldsymbol{\theta}) \mathcal{J}(\tilde{\boldsymbol{\theta}}) = \mathcal{J}(\tilde{\boldsymbol{\theta}})' \mathbf{H}(\boldsymbol{\theta}(\tilde{\boldsymbol{\theta}})) \mathcal{J}(\tilde{\boldsymbol{\theta}}), \end{aligned}$$

where $\mathcal{J}(\boldsymbol{\eta})$ is the Jacobian associated to the mapping $\boldsymbol{\theta}(\cdot)$. In the previous example:

$$\mathcal{J}(\boldsymbol{\eta}) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \exp(\tilde{\psi}) & 0 \\ 0 & 0 & \exp(\tilde{\nu}) \end{pmatrix}$$

10.7 Transforming parameters

Variance parameter positive? Solutions:

1. Use $\sigma \equiv |\theta|$ as parameter, ie forget the sign altogether (doesn't matter for optimisation, interpret negative σ in outcome as positive value)
2. Transform, optimise $\theta = \log \sigma \in (-\infty, \infty)$, no trouble for optimisation

Last option most common, most robust, neatest.

10.8 Transform: Common transformations

Constraint	θ	$\tilde{\theta}$
$[0, \infty)$	$\exp(\tilde{\theta})$	$\log(\theta)$
$[0, 1]$	$\frac{\exp(\tilde{\theta})}{1+\exp(\tilde{\theta})}$	$\log\left(\frac{\theta}{1-\theta}\right)$

Of course, to get a range of $[L, U]$, use a rescaled $[0, 1]$ transformation.

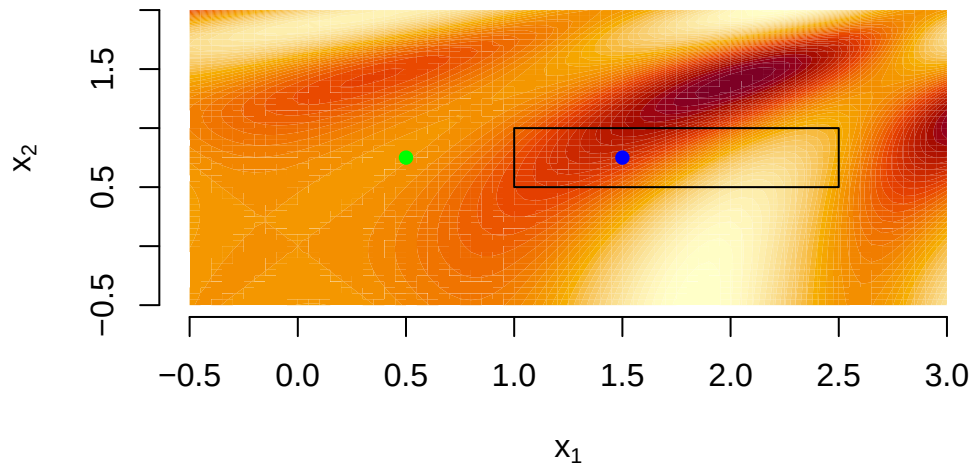
10.9 Code for constrained optimization

 Click to view full code associated with the first part

```

1  # adapted code from last lecture -----
2
3  #golden section
4  gsection <- function(f, dX.l, dX.r, dX.m, dTol = 1e-9, ...) {
5
6      # golden ratio plus one
7      dGR1 <- 1 + (1 + sqrt(5))/2
8
9      # successively refine x.l, x.r, and x.m
10     f.l <- f(dX.l, ...)
11     f.r <- f(dX.r, ...)
12     f.m <- f(dX.m, ...)
13     while ((dX.r - dX.l) > dTol) {
14         if ((dX.r - dX.m) > (dX.m - dX.l)) { # if the right segment is wider than the left
15             dY <- dX.m + (dX.r - dX.m)/dGR1 # put Y into the right segment according to the golden
16             f.y <- f(dY, ...)
17             if (f.y >= f.m) {
18                 dX.l <- dX.m
19                 f.l <- f.m
20                 dX.m <- dY
21                 f.m <- f.y
22             } else {
23                 dX.r <- dY
24                 f.r <- f.y
25             }
26         } else { #if the left segment is wider than the right
27             dY <- dX.m - (dX.m - dX.l)/dGR1 # put Y into the left segment according to the golden
28             f.y <- f(dY, ...)
29             if (f.y >= f.m) {
30                 dX.r <- dX.m
31                 f.r <- f.m
32                 dX.m <- dY
33                 f.m <- f.y
34             } else {
35                 dX.l <- dY
36                 f.l <- f.y
37             }
38         }
39     }
40     return(dX.m)
41 }
42
43 #line search
44 line.search <- function(f, vX, vG, dTol = 1e-9, dA.max = 2^5, ...) {
45     # f is a real function that takes a vector of length d
46     # x and y are vectors of length d
47     # line.search uses gsection to find a >= 0 such that
48     # g(a) = f(x + a*y) has a local maximum at a,
49     # within a tolerance of tol
50     # if no local max is found then we use 0 or a.max for a
51     # the value returned is x + a*y
52     if (sum(abs(vG)) == 0){
53         return(vX) # +0*vG

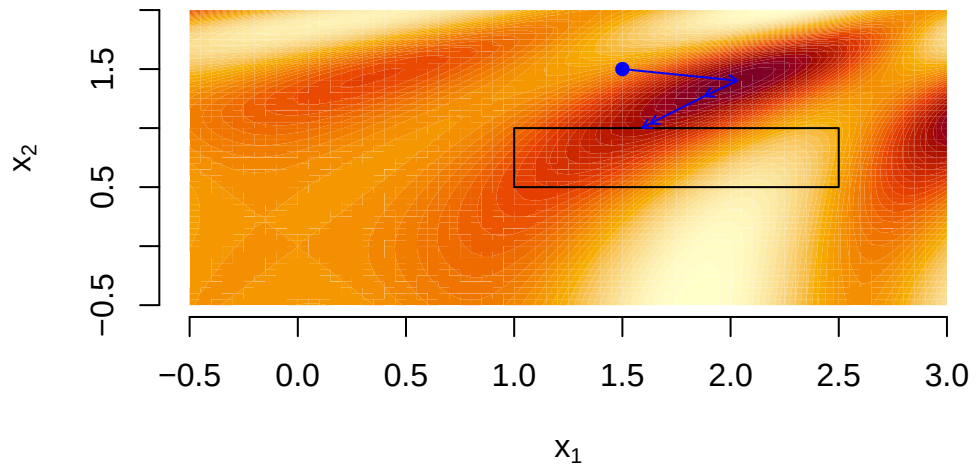
```




```

1 p(vX, va, vb)
2 #> [1] 0.25
3
4
5
6 #plotting the solutions for different gamma_h
7 {
8   plot(NA,xlim=range(vx1),
9         ylim=range(vx2),xlab=expression("x"[1]),ylab=expression("x"[2]),
10        frame=FALSE)
11   levels = pretty(range(mf), 50)
12   color.palette = function(n) hcl.colors(n, "YlOrRd", rev = TRUE)
13   .filled.contour(x=vx1, y=vx2, z=mf,
14                  levels=levels,
15                  col=color.palette(length(levels) - 1))
16   rect(xleft=va[1], ybottom=va[2], xright=vb[1], ytop=vb[2])
17 }
18
19
20 vX0<-c(1.5,1.5)
21 points(vX0[1],vX0[2], col="blue", pch=16)
22
23
24 #gamma_h=0, no penalization
25 vX_star<-ascent(f_p, grad.f_p, vX0, verbose=FALSE, dGamma=0, va=va, vb=vb)
26 arrows(x0=vX0[1], y0=vX0[2], x1=vX_star[1], y1=vX_star[2], length=0.05, col="blue")
27 vX_old<-vX_star
28
29 #gamma_h=1
30 vX_star<-ascent(f_p, grad.f_p, vX0=vX_old, verbose=FALSE, dGamma=1, va=va, vb=vb) #hot star
31 arrows(x0=vX_old[1], y0=vX_old[2], x1=vX_star[1], y1=vX_star[2], length=0.05, col="blue")
32 vX_old<-vX_star
33
34 #gamma_h=10
35 vX_star<-ascent(f_p, grad.f_p, vX0=vX_old, verbose=FALSE, dGamma=10, va=va, vb=vb)
36 arrows(x0=vX_old[1], y0=vX_old[2], x1=vX_star[1], y1=vX_star[2], length=0.05, col="blue")
37 vX_old<-vX_star
38
39 #gamma_h=100
40 vX_star<-ascent(f_p, grad.f_p, vX0, verbose=FALSE, dGamma=100, va=va, vb=vb)
41 arrows(x0=vX_old[1], y0=vX_old[2], x1=vX_star[1], y1=vX_star[2], length=0.05, col="blue")

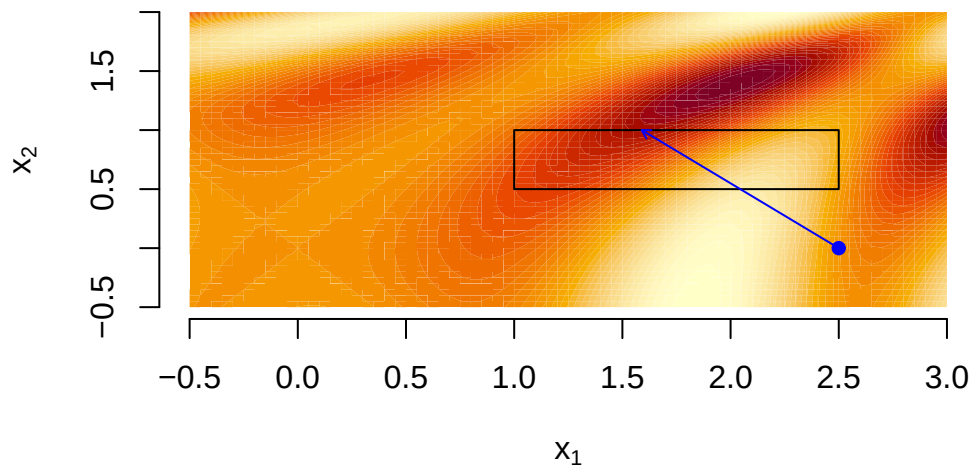
```



```

1
2
3
4 #making a function that iterates like above
5 penalized_ascent<-function(f_p, grad.f_p, vX0, epsilon_h = 1e-9, h.max = 100, verbose_ = TRUE)
6   #first iteration
7   vXh <- ascent(f_p, grad.f_p, vX0, verbose=FALSE, dGamma=0, ...)
8   vXh_old <- vX0
9   h <- 1
10  while( sum(abs(vXh - vXh_old)) > epsilon_h && h < h.max){
11    vXh_old <- vXh
12    vXh <- ascent(f_p, grad.f_p, vXh_old, verbose=FALSE, dGamma=10^(h-1), ...) #gamma_h gets
13    if(verbose_){
14      cat("at iteration", h, "the coordinates of x are", vXh, "\n")
15    }
16    h <- h + 1
17  }
18  return(vXh)
19 }
20
21 #test out how it works for different starting values!
22 {
23   plot(NA,xlim=range(vx1),
24        ylim=range(vx2),xlab=expression("x"[1]),ylab=expression("x"[2]),
25        frame=FALSE)
26   levels = pretty(range(mf), 50)
27   color.palette = function(n) hcl.colors(n, "YlOrRd", rev = TRUE)
28   .filled.contour(x=vx1, y=vx2, z=mf,
29                  levels=levels,
30                  col=color.palette(length(levels) - 1))
31   rect(xleft=va[1], ybottom=va[2], xright=vb[1], ytop=vb[2])
32 }
33
34
35 vX0<-c(2.5,0)
36 points(vX0[1],vX0[2], col="blue", pch=16)
37 vX<-penalized_ascent(f_p, grad.f_p, vX0, va=va, vb=vb)
38 #> at iteration 1 the coordinates of x are 3.024968 1.039979
39 #> at iteration 2 the coordinates of x are 1.629637 1.039826
40 #> at iteration 3 the coordinates of x are 1.593251 1.003943
41 #> at iteration 4 the coordinates of x are 1.589693 1.000394
42 #> at iteration 5 the coordinates of x are 1.589684 1.000039
43 #> at iteration 6 the coordinates of x are 1.589684 1.000004
44 #> at iteration 7 the coordinates of x are 1.589684 1
45 #> at iteration 8 the coordinates of x are 1.589684 1
46 #> at iteration 9 the coordinates of x are 1.589684 1
47 arrows(x0=vX0[1], y0=vX0[2], x1=vX[1], y1=vX[2], length=0.05, col="blue")

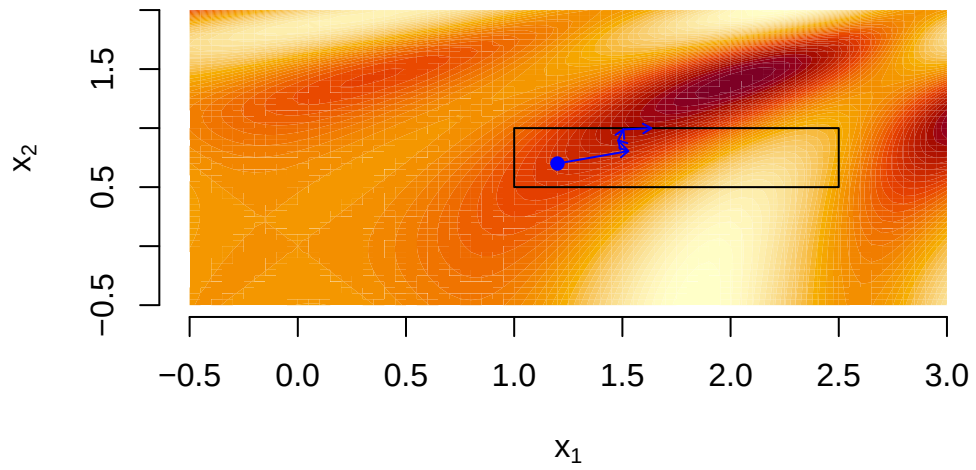
```



```

1
2 # constrained with barrier -----
3 #barrier function
4 b<-function(vX, va, vb){
5   gX<-c(vX-vb, va-vX)
6   if(all(gX<=0)){
7     return(-sum(log(-gX)))
8   }else{
9     return(Inf)
10  }
11 }
12 #new objective with a barrier
13 f_b<-function(vX, dGamma, ...){
14   return(f(vX)-dGamma*b(vX, ...)) #subtract the penalty since we're maximizing
15 }
16
17 grad.f_b<-function(vX, dGamma, ...){
18   return(grad(func=f_p, x=vX, dGamma=dGamma, ...)) # using numerical derivatives here because
19 }
20
21
22 #plotting the solutions for different gamma_h
23 {
24   plot(NA,xlim=range(vx1),
25        ylim=range(vx2),xlab=expression("x"[1]),ylab=expression("x"[2]),
26        frame=FALSE)
27   levels = pretty(range(mf), 50)
28   color.palette = function(n) hcl.colors(n, "YlOrRd", rev = TRUE)
29   .filled.contour(x=vx1, y=vx2, z=mf,
30                  levels=levels,
31                  col=color.palette(length(levels) - 1))
32   rect(xleft=va[1], ybottom=va[2], xright=vb[1], ytop=vb[2])
33 }
34
35
36 vX0<-c(1.2,0.7) #start inside the box
37 points(vX0[1],vX0[2], col="blue", pch=16)
38
39
40 #gamma_h=1
41 vX_star<-ascent(f_b, grad.f_b, vX0, verbose=FALSE, dGamma=1, va=va, vb=vb)
42 arrows(x0=vX0[1], y0=vX0[2], x1=vX_star[1], y1=vX_star[2], length=0.05, col="blue")
43 vX_old<-vX_star
44
45 #gamma_h=1/10
46 vX_star<-ascent(f_b, grad.f_b, vX0=vX_old, verbose=FALSE, dGamma=1/10, va=va, vb=vb) #hot start
47 arrows(x0=vX_old[1], y0=vX_old[2], x1=vX_star[1], y1=vX_star[2], length=0.05, col="blue")
48 vX_old<-vX_star
49
50 #gamma_h=1/100
51 vX_star<-ascent(f_b, grad.f_b, vX0=vX_old, verbose=FALSE, dGamma=1/100, va=va, vb=vb)
52 arrows(x0=vX_old[1], y0=vX_old[2], x1=vX_star[1], y1=vX_star[2], length=0.05, col="blue")
53 vX_old<-vX_star

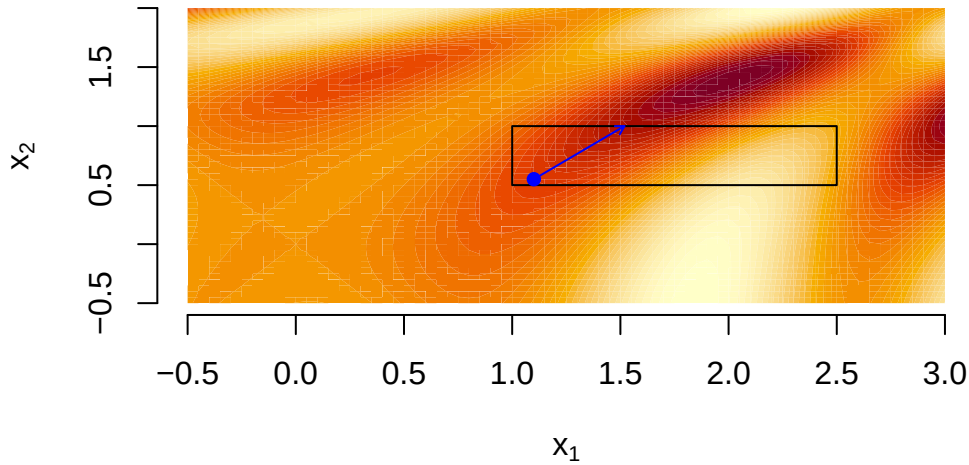
```



```

1
2 barrier_ascent<-function(f_b, grad.f_b, vX0, epsilon_h = 1e-9, h.max = 100, verbose_ = TRUE)
3   #first iteration
4   vXb <- ascent(f_b, grad.f_b, vX0, verbose=FALSE, dGamma=1, ...)
5   vXb_old <- vX0
6   h <- 1
7   while( sum(abs(vXb - vXb_old)) > epsilon_h && h < h.max){
8     vXb_old <- vXb
9     vXb <- ascent(f_b, grad.f_b, vXb_old, verbose=FALSE, dGamma=10^(-h), ...) #gamma_h gets
10    if(verbose_){
11      cat("at iteration", h, "the coordinates of x are", vXb, "\n")
12    }
13    h <- h + 1
14  }
15  return(vXb)
16 }
17
18 #test out how it works for different starting values!
19 {
20   plot(NA,xlim=range(vx1),
21        ylim=range(vx2),xlab=expression("x"[1]),ylab=expression("x"[2]),
22        frame=FALSE)
23   levels = pretty(range(mf), 50)
24   color.palette = function(n) hcl.colors(n, "YlOrRd", rev = TRUE)
25   .filled.contour(x=vx1, y=vx2, z=mf,
26                  levels=levels,
27                  col=color.palette(length(levels) - 1))
28   rect(xleft=va[1], ybottom=va[2], xright=vb[1], ytop=vb[2])
29 }
30
31
32 vX0<-c(1.1,0.55)
33 points(vX0[1],vX0[2], col="blue", pch=16)
34 vX<-barrier_ascent(f_b, grad.f_b, vX0, va=va, vb=vb)
35 #> at iteration 1 the coordinates of x are 1.490837 0.8984811
36 #> at iteration 2 the coordinates of x are 1.506099 0.9881654
37 #> at iteration 3 the coordinates of x are 1.517451 0.9987611
38 #> at iteration 4 the coordinates of x are 1.518635 0.9998752
39 #> at iteration 5 the coordinates of x are 1.518754 0.9999875
40 #> at iteration 6 the coordinates of x are 1.518766 0.9999988
41 #> at iteration 7 the coordinates of x are 1.518767 0.9999999
42 #> at iteration 8 the coordinates of x are 1.518767 1
43 #> at iteration 9 the coordinates of x are 1.518767 1
44 #> at iteration 10 the coordinates of x are 1.518767 1
45 #> at iteration 11 the coordinates of x are 1.518767 1
46 arrows(x0=vX0[1], y0=vX0[2], x1=vX[1], y1=vX[2], length=0.05, col="blue")

```



10.10 Pitfalls

There are some potential pitfalls with optimization routines in empirical practice. E.g, for local optima, different starting points may lead to different optima. A possible solution to this is simulated annealing which is a probabilistic technique for approximating the global optimum of a given function.

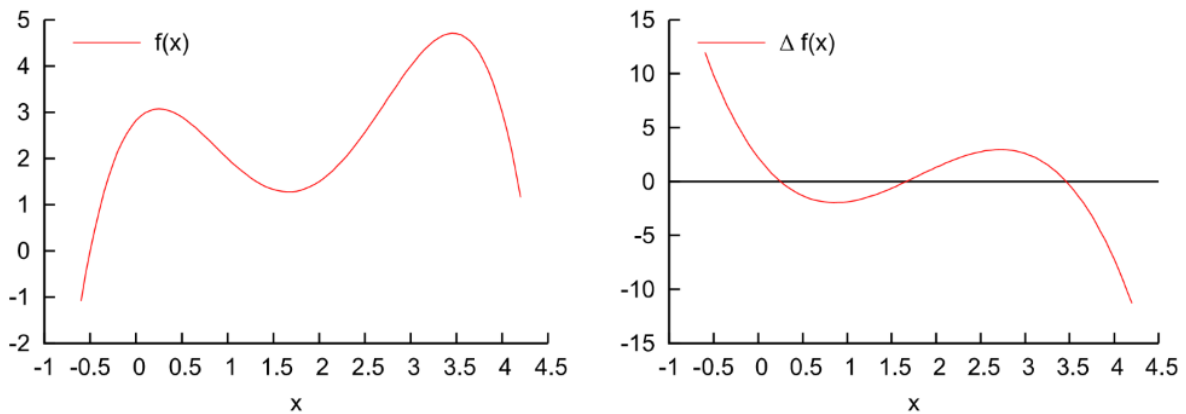


Figure 10.1: Pitfall 1

Another potential is related to flat surfaces: different starting points; transformation for parameter ($\theta = \log \sigma$ tends to optimise a lot better...)

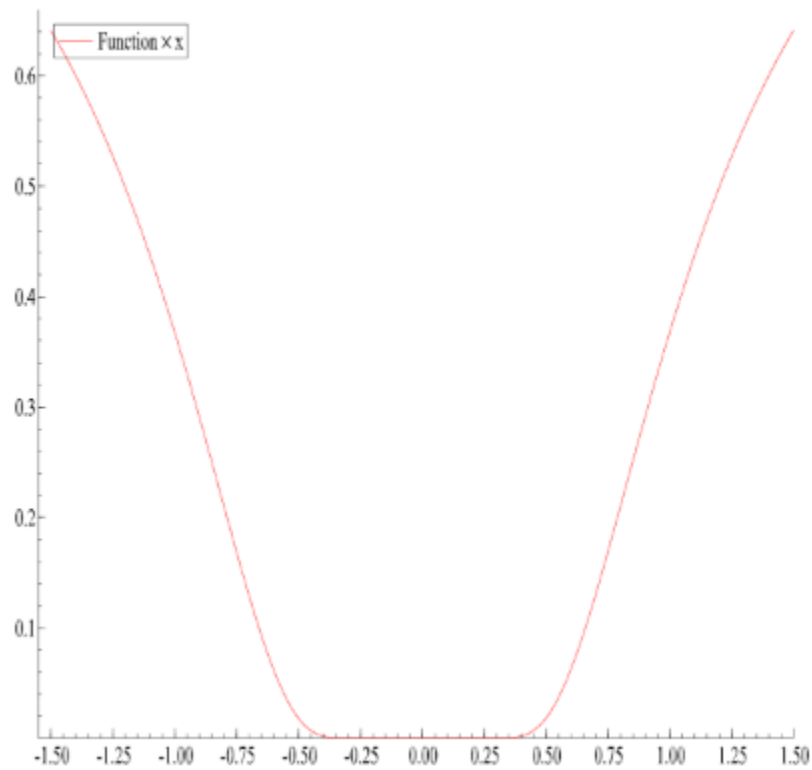


Figure 10.2: Pitfall 2

Noise on derivatives/function: Only smooth functions can be optimised using NR-approach. Use analytical derivatives. Robust programming of target function (optimise AVERAGE LOG likelihood, never likelihood itself).

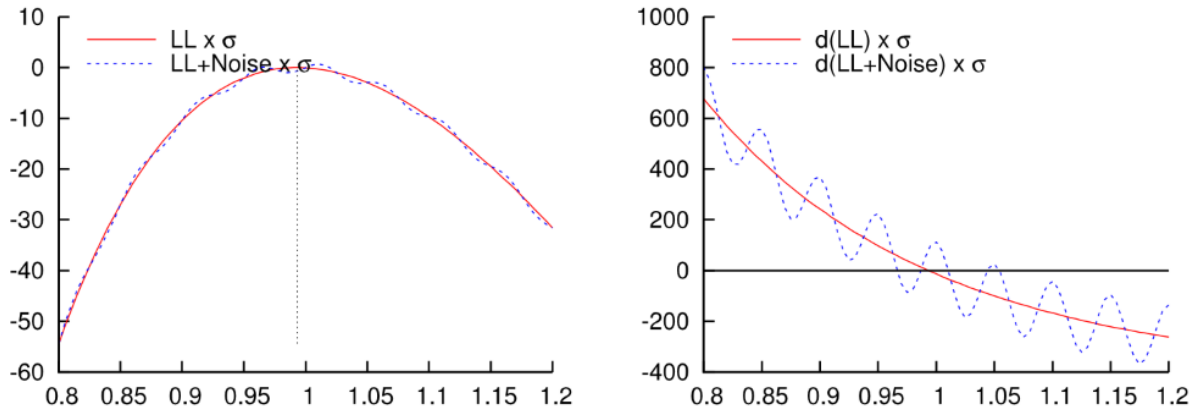


Figure 10.3: Pitfall 3

Non-uniqueness of optimum/indeterminacy in model specification: Optimize σ , both $\sigma = -1$, $\sigma = 1$ give same likelihood...

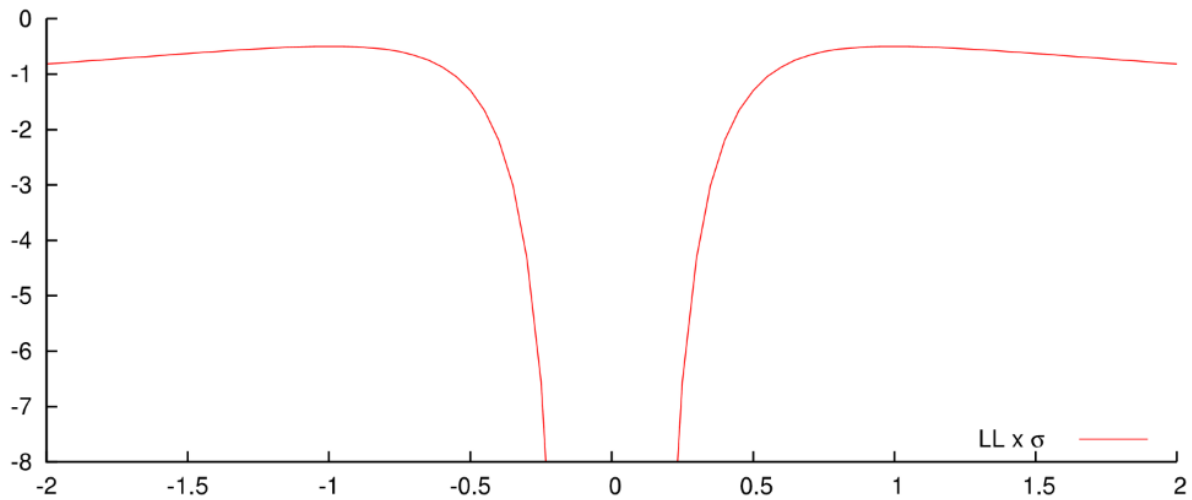


Figure 10.4: Pitfall 4

Bad Taylor-approximation: Sometimes transformation of parameters helps a bit, or it is just bad luck.

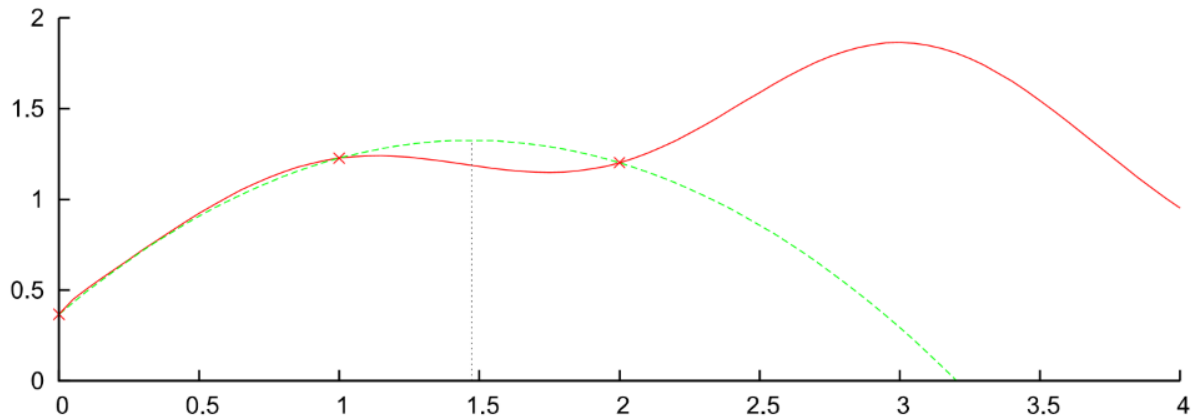


Figure 10.5: Pitfall 5

10.11 Problematic Hessian?

Algorithms based on NR need $\mathbf{H}(x_n)$.

A problem with NR is that the Hessian matrix is not guaranteed to be non-singular. If it is (near) singular at any step, the search routine could either diverge or break down altogether. Problematic:

- Taking derivatives is not stable (...)
- Needs many function evaluations
- $\mathbf{H}$ not guaranteed to be non-singular

Problem is in step

$$\mathbf{x}_{n+1} - \mathbf{x}_n = -\mathbf{H}^{-1}(\mathbf{x}_n) \nabla f(\mathbf{x}_n)$$

Replace $\mathbf{H}(\mathbf{x}_n)$ by some $\mathbf{M}_n$, invertible by construction?

10.12 Recap of Newton's method in higher dimensions

From last lecture we have the following quadratic approximation of the objective function f :

$$f(\mathbf{x}_{n+1}) \approx f(\mathbf{x}_n) + \mathbf{s}_n' \nabla f(\mathbf{x}_n) + \frac{1}{2} \mathbf{s}_n' \mathbf{H}(\mathbf{x}_n) \mathbf{s}_n,$$

where $\mathbf{s}_n = \mathbf{x}_{n+1} - \mathbf{x}_n$.

Taking first-order partial derivatives on both sides with respect to s_n , we get:

$$\nabla f(\mathbf{x}_{n+1}) \approx \nabla f(\mathbf{x}_n) + \mathbf{H}(\mathbf{x}_n)\mathbf{s}_n$$

To solve for the next step, set $\nabla f(\mathbf{x}_{n+1}) = 0$ and rearrange:

$$\mathbf{s}_n = -\mathbf{H}(\mathbf{x}_n)^{-1}\nabla f(\mathbf{x}_n)$$

Instead of $\mathbf{H}(\mathbf{x}_n)$, we could use some $\mathbf{M}_n$, which has nice properties:

$$\mathbf{s}_n = -\mathbf{M}_n^{-1}\nabla f(\mathbf{x}_n)$$

10.13 Broyden, Fletcher, Goldfarb and Shanno (BFGS)

Choose $\mathbf{M}_{n+1}$ such that it satisfies the “secant equation”

$$\mathbf{M}_{n+1}(\mathbf{x}_{n+1} - \mathbf{x}_n) = \nabla f(\mathbf{x}_{n+1}) - \nabla f(\mathbf{x}_n) \iff \mathbf{M}_{n+1}\mathbf{s}_n = \mathbf{y}_n,$$

where $\mathbf{y}_n = \nabla f(\mathbf{x}_{n+1}) - \nabla f(\mathbf{x}_n)$

When $\mathbf{x}_{n+1}$ is close to $\mathbf{x}_n$, then $\mathbf{H}(\mathbf{x}_{n+1}) \approx \mathbf{M}_{n+1}$

To determine $\mathbf{M}_{n+1}$ uniquely, then, we impose the additional condition that among all symmetric matrices satisfying the secant equation, $\mathbf{M}_{n+1}$ is, in some sense, closest to the current matrix $\mathbf{M}_n$

$$\mathbf{M}_{n+1} = \min_{\mathbf{M}} \|\mathbf{M} - \mathbf{M}_n\|$$

subject to $\mathbf{M} = \mathbf{M}'$, $\mathbf{M}\mathbf{s}_n = \mathbf{y}_n$

The solution:

$$\mathbf{M}_{n+1} = \mathbf{M}_n + \frac{\mathbf{y}_n\mathbf{y}_n'}{\mathbf{y}_n\mathbf{s}_n'} - \frac{\mathbf{M}_n\mathbf{s}_n\mathbf{s}_n'\mathbf{M}_n'}{\mathbf{s}_n'\mathbf{M}_n\mathbf{s}_n}$$

1. Start with $n = 0$ and positive definite $\mathbf{M}_0$, e.g. $\mathbf{M}_0 = I$
2. Calculate $\mathbf{s}_n = -\mathbf{M}_n^{-1}\nabla f(\mathbf{x}_n)$
3. Find new $\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{s}_n$ (or with line search: $\mathbf{x}_{n+1} = \mathbf{x}_n + \alpha_n\mathbf{s}_n$)

4. Calculate, with $\mathbf{y}_n = \nabla f(\mathbf{x}_{n+1}) - \nabla f(\mathbf{x}_n)$

$$\mathbf{M}_{n+1} = \mathbf{M}_n + \frac{\mathbf{y}_n \mathbf{y}_n'}{\mathbf{y}_n' \mathbf{s}_n} - \frac{\mathbf{M}_n \mathbf{s}_n \mathbf{s}_n' \mathbf{M}_n'}{\mathbf{s}_n' \mathbf{M}_n \mathbf{s}_n}$$

5. This produces a symmetric and positive definite matrix at each step if the initialization is a symmetric and positive definite matrix, like the identity matrix.

6. We can use the Sherman-Morrison formula to calculate $\mathbf{M}_{n+1}^{-1}$ efficiently.

Result: * No Hessian needed * Still fast convergence * No problems with singular H_n

10.14 Example: Regression Likelihood

$$y_i = X_i \beta + \epsilon_i \quad \epsilon_i \sim N(0, \sigma^2)$$

ML maximises likelihood (other options: Minimise sum-of-squares, optimise utility etc):

$$\begin{aligned} L(y; \theta) &= \prod_i \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - X_i\beta)^2}{2\sigma^2}\right) \\ &= (2\pi\sigma^2)^{-\frac{N}{2}} \exp\left(-\frac{1}{2\sigma^2}(y - X\beta)'(y - X\beta)\right) \end{aligned}$$

In this case, $\theta = (\beta, \sigma^2)$

And in order to write the likelihood:

To remember:

$$L(y; \theta) = (2\pi\sigma^2)^{-\frac{N}{2}} \exp\left(-\frac{1}{2\sigma^2}(y - X\beta)'(y - X\beta)\right)$$

$$\log L(y; \theta) = -\frac{1}{2} \left(N \log 2\pi + N \log \sigma^2 + \frac{e'e}{\sigma^2} \right)$$

In this case, $\theta = (\beta, \sigma)$ or $\theta = (\beta, \sigma^2)$.

- Extract your parameters from the vector, use sensible names
- Check if your parameters are valid, i.e. $\sigma^2 > 0$?
- And test...

10.15 `optim` continued

Function to maximize should have format

```
optim(par, fn, gr = NULL, ..., method = c("BFGS", "CG"), lower = -Inf, upper = Inf, control = list(), hessian = FALSE)
```

where

- **par** initial values for the parameters to be optimized over.
- **fn** a function to be minimized
- **gr** a function to return the gradient for the “BFGS” and “CG” methods. If it is `NULL`, a finite-difference approximation will be used.

The output of **fn** has to be the negative average log-likelihood value evaluated in **vPar**.

Why negative? Because by convention optimizers in R by default are minimizer. This option can be modified using arguments of the optimizer function.

The output from `optim` is a list with named elements:

- **par**: The estimated vector of parameters
- **value**: The value of **fn** corresponding to **par**.
- **counts**: A two-element integer vector giving the number of calls to **fn** and **gr** respectively.
- **convergence**: An integer code. 0 indicates successful completion, see `help(optim)` for other labels.
- **message**: A character string giving any additional information returned by the optimizer, or `NULL`.

10.16 Code for pitfalls and BFGS

 Click to view full code associated with the first part

11 Integration of C++ Code with Rcpp

11.1 Introduction

R is both a powerful interactive environment for data analysis, visualization, and modeling and an expressive programming language designed and built to support these tasks. However, sometimes R code just isn't fast enough. Generally, performance that relates to speed is improved by C++.

11.2 Rcpp

Rcpp is an R add-on package which makes it very simple to connect R with C++. While still relatively new (2008) as a project, Rcpp has already become widely deployed among users and developers in the R community. Rcpp continues to be under active development, and extensions are being added. It is now the most popular language extension for the R system and used by over 2700 CRAN Packages. This is about 14% of all R packages on CRAN, and 60% of all packages that use compiled code.

11.3 R versus C++

One of the key differences between R and C++ is that R is interpreted and C++ is compiled.

With a *compiled language*, code you enter is reduced or 'compiled' into the 'target language' (that is, the actual instructions in machine code). Once the translation is complete, the executable code is either run or set aside to run later.

With *interpreted languages*, the code is saved in the same format that you entered and is reduced/compiled to machine-specific instructions at runtime.

The interpreted languages thus allow for more flexibility because a program can be adaptively modified and there is no need for an additional 'compilation stage'.

The flexibility, however, comes at a cost; interpreted languages are significantly slower, since the compilation of the code happens every time it is run (think of loops).

Typical bottlenecks that C++ can address includes:

- Loops that can't easily be vectorized because subsequent iterations depend on previous ones.
- Recursive functions, or problems which include calling functions many times. The overhead (resources required) of calling a function in C++ is much lower than that in R.
- Problems that require advanced data structures and algorithms that R doesn't provide. C++ has efficient implementations of many advanced data structures that R does not provide, such as ordered maps or double-ended queues.

11.4 Basic C++ in R

In the following, we will consider how to write basic C++ code by converting simple R functions to their C++ equivalents.

All examples need version 0.10.1 or above of the Rcpp package. This version includes the functions `cppFunction()` and `sourceCpp()`, which make it very easy to connect C++ to R.

Install the latest version of Rcpp from CRAN with:

```
install.packages("Rcpp")
```

You'll also need a working compiler:

- On Windows, install 'Rtools'
- On Mac, install 'Xcode' from the app store and some other things need to be sorted.

11.5 C++ functions in R

`cppFunction()` allows you to write C++ functions in R:

```
suppressMessages(library(Rcpp))
#> Warning: pakke 'Rcpp' blev bygget under R version 4.3.3

cppFunction('
int add(int x, int y, int z) {
  int sum = x + y + z;
  return sum;
}')
```

When you run this code, Rcpp will compile the C++ code and construct an R function that connects to the compiled C++ function. You can then run the function in R:

```
add(1, 2, 3)
#> [1] 6
```

The following will teach you the basics of C++ by translating simple R functions to their C++ equivalents.

11.5.1 Example: No inputs, scalar output

We will start with a very simple function that takes no arguments and always returns the integer 1:

```
oneR <- function() 1L # R function

cppFunction('
int oneCpp() { # C++ function
  return 1,
}')'
```

The syntax to create a function in C++ looks like the syntax to call a function in R.

You must declare the type of output the function returns before the function name; this function returns an `int` (a scalar integer).

You must use an explicit `return` statement to return a value from a function (which is not necessary in R).

Every statement is terminated by a semicolon, `;`.

11.6 Variables declation

A main difference between R and C++ is the declaration of variables. In R, we that that we can write commands like:

```
iN <- 5
dK <- 0.156532
bJ <- TRUE
sT <- "Hello World"
```

Differently in C++, we need to specify the class of the variable before its definition:

```

int    iN = 5;           // integer
double dK = 0.156352;    // double
bool   bJ = true;        // boolean
string st = "Hello World"; // string

```

Also note that the logical value TRUE is indicated as `true` (same for FALSE).

Comments are indicated with `//` instead of `#`;

The arrows that we use to assign values to variables in R do not make sense in C++.

11.6.1 Example: scalar input, scalar output

```

signR <- function(iX) {
  if (iX > 0) {
    1
  } else if (iX == 0) {
    0
  } else {
    -1
  }
}

cppFunction('int SignC(int x) {
  if (x > 0) {
    return 1;
  } else if (x == 0) {
    return 0;
  } else {
    return -1;
  }
}')

```

From the example above, it can be noted that it is needed to declare the function in the same way as the output. The if-statement syntax in C++ is identical to that in R. C++ also has a while-loop that works in the same way as R's.

11.7 For loops in C++

One of the main reasons for including C++ code in our R program is to speed up the evaluation of loops. Indeed, there are situations where, unfortunately, we cannot avoid making a loop, e.g., when subsequent iterations depend on previous ones.

For loops in C++ have a different syntax than those in R:

```
for (int i = 0 (init); i < 10 (check); i++ (increment)) {  
  // some code here  
}
```

Note that we declare the class of the loop index `i` (this has to be an integer), we indicate the start (`i = 0`), the end (`i < 10`) and the increment at each iteration (`i++`).

The notation `i++` indicates that the variable `i` needs to be augmented by 1 at each iteration. The notation `i--` indicates that the value of `i` needs to be decreased by 1 at each iteration.

11.7.1 Example: vector input, scalar output

```
sumR <- function(vX) {  
  dTotal <- 0  
  for (i in 1:length(vX)) {  
    dTotal <- dTotal + vX[i]  
  }  
  return(dTotal)  
}  
  
cppFunction('  
double sumC(NumericVector x) {  
  int n = x.size();  
  double total = 0;  
  for(int i = 0; i < n; i++) {  
    total += x(i);  
  }  
  return total;  
}')
```

The classes for the most common types of R vectors are: `NumericVector`, `IntegerVector`, `CharacterVector`, and `LogicalVector`.

To find the length of the vector, we use the `.size()` method, which returns an integer. In general, C++ methods are called with `.`.

C++ is zero based! The first element of a vector is indexed at 0 and not at 1.

The call `total += x(i)` is equivalent to `total = total + x(i)`. Similar operators are `-=`, `*=`, `/=`.

Loops are not at all as slow compared to vectorized programs in C++ as in R, so it is fine to use loops in C++. In R, on the other hand, we always want to use vector operations when possible, because looping is just too slow in comparison.

11.7.2 Efficiency of loops: C++ vs. R

The above example is a situation where C++ is much more efficient than R:

```
suppressMessages(library(microbenchmark))
#> Warning: pakke 'microbenchmark' blev bygget under R version 4.3.3

vX <- runif(1e4)
microbenchmark(
  sum(vX),
  sumC(vX),
  sumR(vX)
)
#> Unit: microseconds
#>      expr    min     lq   mean median    uq   max neval
#>  sum(vX)   7.4    7.5  7.628   7.50   7.6    9.6   100
#> sumC(vX)  21.8   22.0 29.242  22.10  22.4  672.3   100
#> sumR(vX) 249.4  249.9 275.228 250.55 251.6 2545.2   100
```

The `sumC` function even come somewhat close to beating the built-in (and highly optimized) R function `sum()`, while `sumR()` is significantly slower.

11.7.3 Example: vector input, vector output

Next, we'll create a function that computes the Euclidean distance between a value and a vector of values:

```

pdistR <- function(dX, vY) {
  sqrt((dX - vY) ^ 2)
}

cppFunction('
NumericVector pdistC(double x, NumericVector y) {
  int n = y.size();
  NumericVector out(n);

  for (int i = 0; i < n; i++) {
    out(i) = sqrt(pow(y(i) - x, 2.0));
  }
  return out;
}')

```

We create a new numeric vector of length `n` with a constructor: `NumericVector out(n)`. Another useful way of making a vector is to copy an existing one: `NumericVector zs = clone(ys)`.

C++ uses the `pow()` and not `^` for exponentiation.

Even though an R function might be fully vectorized, a C++ function can still be faster. However, it also takes longer to write the C++ function.

Other useful math functions in C++: `min()`, `max()`, `abs()`, `sin()`, `cos()`, `tan()`, `exp()`, `log()`, `ceil()`, `floor()`, `round()`...

11.7.4 Example: matrix input, vector output

Consider the C++ function that reproduces `rowSums()`:

```

cppFunction('
NumericVector rowSumsC(NumericMatrix x) {
  int nrow = x.nrow(), ncol = x.ncol();
  NumericVector out(nrow);

  for (int i = 0; i < nrow; i++) {
    double total = 0;
    for (int j = 0; j < ncol; j++) {
      total += x(i, j);
    }
    out(i) = total;
  }
}

```

```

    }
    return out;
}')

```

Calling it, we get:

```

set.seed(1014)
mX <- matrix(sample(100), 10)
rowSums(mX)
#>  [1] 446 514 480 514 352 627 525 586 572 434
rowSumsC(mX)
#>  [1] 446 514 480 514 352 627 525 586 572 434

```

Each vector type has a matrix equivalent: `NumericMatrix`, `IntegerMatrix`, `CharacterMatrix`, and `LogicalMatrix`.

In C++, you subset a matrix with `()` and not `[]`.

Use `.nrow()` and `.ncol()` methods to get the dimensions of a matrix.

11.8 Using sourceCpp

So far, we have used inline C++ with `cppFunction`. This makes presentation simpler, but for real problems it is usually easier to use stand-alone C++ files and then source them into R using `sourceCpp()`.

Your stand-alone C++ files should have extension `.cpp` and needs to start with:

```

#include <Rcpp.h>

using namespace Rcpp;

```

And for each function that you want available within R, you need to prefix it with:

```

// [[Rcpp::export]]

```

You can embed R code in special C++ comment blocks, which is really convenient if you want to run some test code. The blocks look as follows:

```

/** R
# This is R code
*/

```


To compile the C++ code, use `sourceCpp("path/to/file.cpp")`. This will create the matching R functions and add them to your current session.

Note that these C++ functions cannot be saved in a .Rdata file and be reloaded in a later sessions; they must be recreated/recompiled each time you restart R.

As an example, running `sourceCpp()` on the following file implements `mean` in C++ and then compares it to the built-in R function `mean()`.

```
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
double meanC(NumericVector x) {
  int n = x.size();
  double total = 0;
  for (int i = 0; i < n; i++) {
    total += x(i);
  }
  return total / n;
}

/** R
suppressMessages(library(microbenchmark))
vX <- runif(1e5)
microbenchmark(mean(vX), meanC(vX))
*/
```

```
sourceCpp("CppSource.cpp")
#>
#> > suppressMessages(library(microbenchmark))
#>
#> > vX <- runif(1e+05)
#>
#> > microbenchmark(mean(vX), meanC(vX))
#> Unit: microseconds
#>      expr    min     lq   mean median     uq    max neval
#>  mean(vX) 151.9 152.2 155.875 153.05 154.55 222.1   100
#>  meanC(vX) 220.9 221.2 232.220 221.60 222.65 1017.7   100
suppressMessages(library(microbenchmark))
vX <- runif(1e5)
microbenchmark(
```

```

mean(vX),
meanC(vX)
)
#> Unit: microseconds
#>      expr    min      lq    mean median      uq    max neval
#>  mean(vX) 151.9 152.80 162.612   153.9 160.6 402.7   100
#> meanC(vX) 220.9 221.65 230.198   223.4 229.4 383.7   100

```

11.9 The Fibonacci sequence

Consider the problem of computing the Fibonacci sequence. The Fibonacci numbers F_n are defined as a recursive sum of the two preceding terms in the same sequence:

$$F_n = F_{n-1} + F_{n-2}$$

with initial conditions $F_0 = 0$ and $F_1 = 1$. For example, the first ten numbers of the sequence F_0 to F_9 are seen to be:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34

11.9.1 Fibonacci in R

Using recursive programming, we can compute the Fibonacci numbers in R:

```

fibR <- function(n) {
  if (n == 0) return(0)
  if (n == 1) return(1)
  return(fibR(n - 1) + fibR(n - 2))
}

```

This simple function has several key features:

- It is very short.
- It does not test for wrong input values less than zero.
- It is easy to comprehend.
- It is a very faithful execution of the relationship $F_n = F_{n-1} + F_{n-2}$.

However, it is very insufficient; it can be shown that the algorithm is exponential in n - or in other words, its run-time increases at an exponential rate relative to the argument n .

11.9.2 Fibonacci in C++

A simple solution to compute F_n much faster using the same simple and intuitive algorithm is to switch to C++. We can write a simple C++ script which contains a function for the Fibonacci number evaluation:

```
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
int fibC(int x) {
  if (x == 0) return(0);
  if (x == 1) return(1);
  return(fibC(x - 1) + fibC(x - 2));
}
```

11.9.3 Comparing fibR and fibC

To load the functions inside `fibonacci.cpp` which we want to export (those with `// [[Rcpp::export]]`), we simply run:

```
suppressMessages(library(Rcpp))
sourceCpp("fibonacci.cpp")
fibC(25)
#> [1] 75025
```

Comparing the two functions:

```
suppressMessages(library(microbenchmark))
microbenchmark(fibR(25), fibC(25))
#> Unit: microseconds
#>      expr      min       lq      mean  median      uq      max neval
#>  fibR(25) 93186.9 95343.2 98300.547 95888.7 97046.85 243238.2   100
#>  fibC(25)  119.1   120.2   140.635   132.6   133.80    912.3    100
```

11.10 Vector, Matrices and Linear Algebra in C++

C++ does not come with routines that implement linear algebra like in R. However, similar to R, we can include external libraries. One of the most used libraries for matrix calculus in

C++ is armadillo. In order to link R with C++ code which exploits armadillo, we can use the RcppArmadillo package for R.

Install the RcppArmadillo package from CRAN:

```
install.packages("RcppArmadillo")
```

Features of RcppArmadillo include:

- Better vector and matrix types; `arma::vec`, `arma::mat`.
- Very fast routines for linear algebra operations, e.g., matrix multiplication, matrix inversion, linear system solving etc.

<https://arma.sourceforge.net/docs.html>: documentation for all Armadillo functions. Great descriptions, examples, and links to related functions.

11.10.1 Example

The following code is saved in the file `armadilloexample.cpp`:

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
using namespace Rcpp;

// [[Rcpp:export]]
arma::mat matprodC(arma::mat m1, arma::mat m2) {
  if(m1.n_cols != m2.n_rows)
    stop("Incompatible matrix dimensions");

  return m1 * m2;
}
```

And then used in R:

```
suppressMessages(library(RcppArmadillo))
#> Warning: pakke 'RcppArmadillo' blev bygget under R version 4.3.3
sourceCpp("armadilloexample.cpp")
m1 <- matrix(1:9, 3, 3)
m2 <- matrix(1:9, 3, 3)

matprodC(m1, m2)
```

```

#>      [,1] [,2] [,3]
#> [1,]   30   66  102
#> [2,]   36   81  126
#> [3,]   42   96  150

m1 %*% m2
#>      [,1] [,2] [,3]
#> [1,]   30   66  102
#> [2,]   36   81  126
#> [3,]   42   96  150

```

11.11 Inverse of Matrix Product

Consider the R function:

```

FunR <- function(mX, mY) {
  mZ <- mX %*% mY
  mZInv <- solve(mZ)
  return(mZInv)
}

```

This function performs a matrix multiplication between `mX` and `mY`, and then returns the inverse of the product.

Consider now the C++ implementation exploiting the `RcppArmadillo` package:

```

// [[Rcpp:depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
using namespace arma;
using namespace Rcpp;

// [[Rcpp::export]]
mat FunC(mat mx, mat mY) {
  mat mZ = mx * mY;
  mat mZInv = mZ.i();
  return mZInv;
}

```

Comparison:

```

suppressMessages(library(RcppArmadillo))
sourceCpp("mycfuntion.cpp")
M <- 300
mX <- matrix(rnorm(M^2), M)
mY <- matrix(rnorm(M^2), M)

microbenchmark(FunR(mX, mY), FunC(mX, mY))
#> Unit: milliseconds
#>      expr      min       lq      mean  median      uq      max neval
#>  FunR(mX, mY) 35.401 36.2693 36.92739 36.5251 36.8137 42.2825   100
#>  FunC(mX, mY) 28.643 29.2406 29.76754 29.6378 30.0060 34.0209   100

```

11.12 Calling R Functions from C++

It is very easy to load C++ functions in R using `sourceCpp()`. Using Rcpp, it is possible to load R functions inside C++ functions. See: <https://gallery.rcpp.org/articles/r-function-from-c++/>.

Rcpp permits easy access to native R objects at the C++ level. Calling an R function in C++ can be very useful. For example to pick up parameter initializations, maybe to access a custom data summary that would be tedious to recode, or maybe even calling a plotting routine.

Define in R the following function which returns the sum of squares of a vector:

```

SumOfSquaresR <- function(VX) {
  dOut <- sum(vX^2)
  return(dOut)
}
vX <- rnorm(10)
SumOfSquaresR(vX)
#> [1] 15.98534

```

Consider now the following C++ code:

```

#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
SEXP callRFunction(NumericVector vX, Function f) {
  SEXP res = f(vX);

```

```
    return res;
}
```

Note the `SEXP` class for R objects in C++ and the `Function` class for general functions in C++. The code can be saved in a script `callRFunction.cpp` in the working directory and be loaded with the `sourceCpp` function as usual.

After running:

```
sourceCpp("callRFunction.cpp")
```

We have the `callRFunction` available in R. Run:

```
callRFunction(vX, SumOfSquaresR)
#> [1] 15.98534
SumOfSquaresR(vX)
#> [1] 15.98534
```

Define now the function sum of absolute values:

```
SumOfAbsoluteValuesR <- function(vX) {
  dOut <- sum(abs(vX))
  return(dOut)
}
```

We can use the same function `callRFunction` to execute `SumOfAbsoluteValuesR` in C++ as:

```
callRFunction(vX, SumOfAbsoluteValuesR)
#> [1] 10.25095
SumOfAbsoluteValuesR(vX)
#> [1] 10.25095
```

That is, `callRFunction(NumericVector vX, Function f)` is independent from the definition of `f`.

11.12.1 Warning

```

suppressMessages(library(microbenchmark))
vX <- rnorm(1e6)
microbenchmark(
  callRFunction(vX, SumOfSquaresR),
  SumOfSquaresR(vX)
)
#> Unit: milliseconds
#>
#>      expr      min       lq      mean  median      uq      max
#> callRFunction(vX, SumOfSquaresR) 2.097 2.24415 3.218276 2.35775 2.6713 6.7450
#>      SumOfSquaresR(vX) 2.111 2.25025 2.979293 2.43060 2.5300 6.6762
#> neval
#>    100
#>    100

```

Calling R functions in C++ is slower than calling R functions in R.

11.13 The Rcpp Family

Rcpp and RcppArmadillo are not the only two packages that belong to the Rcpp family. There also exist:

- The RcppGSL package provides an easy-to-use interface between data structures from the GNU Scientific Library (<https://www.gnu.org/software/gsl/>).
- The RcppEigen package provides an interface to the Eigen library with linear algebra: matrices, vectors, numerical solvers, and related algorithms.
- RcppNumerical: Rcpp integration for numerical computing libraries, including e.g., numerical integration, optimization.

And a bunch more...

11.14 A simple model for daily financial returns

Consider the following conditional model for financial log-returns (GARCH(1,1))

$$\begin{aligned}
 r_t &= \varepsilon_t, & \varepsilon_t &\sim N(0, \sigma_t^2) \\
 \sigma_t^2 &= \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2
 \end{aligned}$$

The conditional variance is σ_t^2 and it can be shown that the unconditional variance, given stationarity, is given by

$$\text{Var}(r_t) = \frac{\omega}{1 - \alpha - \beta}$$

Condition for stationarity is $\alpha + \beta < 1$. Conditions for positivity are $\omega > 0$, $\alpha > 0$ and $\beta > 0$.

11.15 Estimation of the GARCH(1,1) model

Given the stationarity and positivity conditions on the process σ_t^2 , the average Gaussian log-likelihood function is defined as

$$\ell(\theta, r_t) = \frac{1}{T} \sum_{t=1}^T \log(f(r_t, \theta))$$

where

$$\begin{aligned} \log(f(r_t, \theta)) &= \log \left(\frac{1}{\sqrt{2\pi\sigma_t^2}} \exp \left(-\frac{r_t^2}{2\sigma_t^2} \right) \right) \\ &= -\frac{1}{2} \log(2\pi\sigma_t^2) - \frac{r_t^2}{2\sigma_t^2} \\ &\propto -\log(2\pi) - \log(\sigma_t^2) - \frac{\varepsilon_t^2}{\sigma_t^2} \end{aligned}$$

so that

$$\hat{\theta} = \arg \max_{\theta} \ell(\theta, r_t)$$

11.15.1 GARCH practical issues

As evident from the likelihood formulation, the evaluation of the sequence of volatilities $\sigma_1, \dots, \sigma_T$ is required for model estimation. Unfortunately, a for loop is required for this task. Also for simulation from the GARCH(1,1) model, the computation of the aforementioned loop is required.

So let's do it in C++.

11.16 Simulation of GARCH(1,1) in C++

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>

using namespace arma;
using namespace Rcpp;
```

```
// [[Rcpp::export]]
Rcpp::List GarchSim(int iT, double dOmega, double dAlpha, double dBeta) {

  vec vY(iT); //observations
  vec vSigma2(iT); // conditional variances

  // initialize at the unconditional value
  vSigma2(0) = dOmega/(1.0 - dAlpha - dBeta);

  // sample the first observation
  vY(0) = pow(vSigma2(0), 0.5) * Rf_rnorm(0.0, 1.0);

  for (int t = 1; t < iT; t++) {
    vSigma2(t) = dOmega + dAlpha * pow(vY(t - 1), 2.0) + dBeta * vSigma2(t - 1);
    vY(t) = pow(vSigma2(t), 0.5) * Rf_rnorm(0.0, 1.0);
  }

  List lOut;
  lOut["vSigma2"] = vSigma2;
  lOut["vY"] = vY;

  return lOut;
}
```

Note:

- Vector declaration with `vec`.
- The use of `pow(double, double)` for powers.
- The function `Rf_rnorm(double, double)` for simulation from a Gaussian distribution.
- The list declaration.
- The list elements assignment.

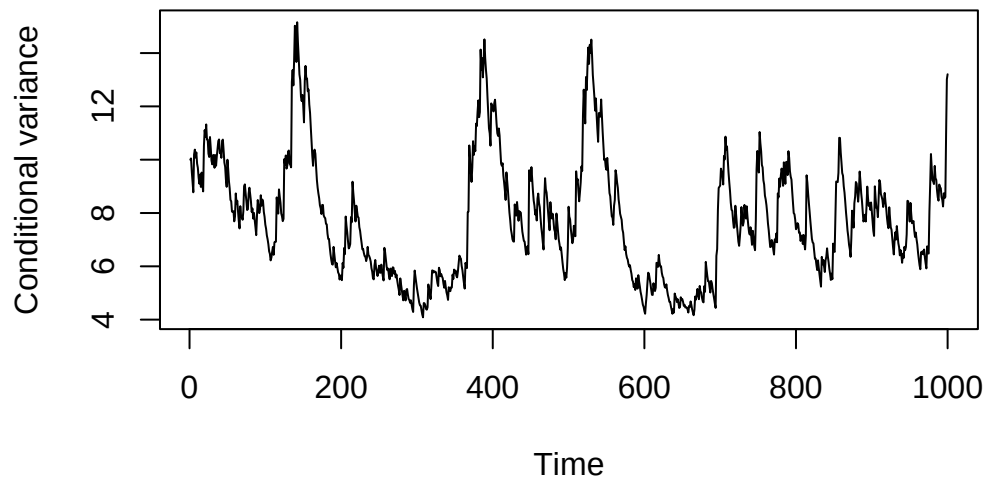
Assuming the code is saved in the file `SimGarch.cpp` inside the working directory, a GARCH(1,1) process can be simulated by running:

```
sourceCpp("SimGarch.cpp")

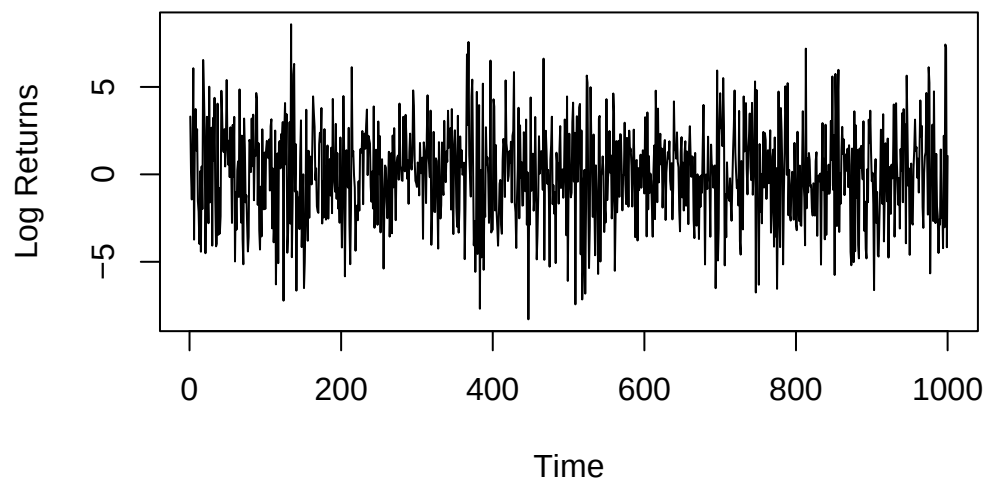
iT <- 1000
dOmega <- 0.1
dAlpha <- 0.05
dBeta <- 0.94
```

```
lSim <- GarchSim(iT, dOmega, dAlpha, dBeta)

plot(1:1000, lSim[["vSigma2"]], type = "l", lty = 1, xlab = "Time", ylab = "Conditional vari
```



```
plot(1:1000, lSim[["vY"]], type = "l", lty = 1, xlab = "Time", ylab = "Log Returns")
```



12 Creating R packages with RStudio

Book reference: <https://r-pkgs.had.co.nz>

12.1 Introduction

In R, the fundamental unit of shareable code is the package. A package bundles together code, data, documentation, and tests, and is easy to share with others.

As of now, there are over 2200 packages available on the Comprehensive R Archive Network, or CRAN, the public clearing house for R packages.

This huge variety of packages is one of the reasons that R is so successful: the chances are that someone has already solved a problem that you're working on, and you can benefit from their work by downloading their package.

12.2 Why a package?

Why write a package? One compelling reason is if you have code that you want to share with others. Bundling your code into a package makes it easy for other people to use it, because like you, they already know how to use packages. If your code is in a package, any R user can easily download it, install it, and learn how to use it.

Furthermore, organizing code in a package makes your life easier because packages come with conventions. These conventions are helpful because:

- They save you time - you don't need to think about the best way to organise a project, you can just follow a template.
- Standardised conventions lead to standardised tools - if you buy into R's package conventions, you get many tools for free.

12.3 Setting up

Install the following packages:

```
install.packages(c("Rcpp", "RcppArmadillo", "devtools"))
```

Before we start, we need to install `Rtools.exe` from <https://cran.r-project.org/bin/windows/Rtools/index.html>.

Rtools includes all the programs we require in order to compile our code.

Now, in RStudio, navigate to “File” -> “New Project...” -> “New Directory” -> “R Package using RcppArmadillo”.

12.4 Code in Packages

The first principle of using a package is that all R code goes in the `R` folder of the source package on your computer. Here is where we put all the `.R` files containing our functions organized in a sensible way. Assume to have the following function:

```
MySumFunction <- function(a, b) {  
  c <- a + b  
  return(c)  
}
```

In the file `Utilities.R` inside the `R` folder of your package. To load your package with your function you can press `Ctrl + Shift + L`.

```
MySumFunction(5, 7)  
#> [1] 12
```

C++ code has to be placed inside the `src` folder of your package.

A new C++ file contains this by default:

```
#include <Rcpp.h>  
using namespace Rcpp;  
  
// [[Rcpp::export]]  
NumericVector timesTwo(NumericVector x) {
```

```

    return x * 2;
}

// You can include R code blocks in C++ files processed with sourceCpp
// (useful for testing and development). The R code will be automatically
// run after the compilation.
//

/** R
timesTwo(42)
*/

```

Note that it includes Rcpp and uses the Rcpp namespace. If we want to use armadillo and all its functionalities we need to modify as follows.

```

#include <RcppArmadillo.h>

using namespace arma;
using namespace Rcpp;

```

Note that we have only included RcppArmadillo.h, this is because Rcpp.h is automatically loaded by #include <RcppArmadillo.h>. Indeed, we are still able to use the Rcpp namespace.

Let's define the following function after the namespace definition:

```

#include <RcppArmadillo.h>

using namespace arma;
using namespace Rcpp;

//[[Rcpp::export]]
arma::vec SumOfTwoVec_C(arma::vec vX, arma::vec vY) {
    arma::vec vZ = vX + vY;
    return vZ;
}

```

Now you can save the script as, say, `utilities_c.cpp` in the `src` folder of your package.

Running `Ctrl+Shift+L`, RStudio automatically compiles the C++ functions and export to R those that are signed with the `//[[Rcpp::export]]` flag.

Now the package is loaded and its functions can be used.

12.5 Object documentation

Documentation is one of the most important aspects of a good package. Without it, users will not know how to use your package.

- Learn about object documentation, as accessed by `? or help()`.
- Write `.Rd` files in the `man/` directory. These files use a custom syntax, loosely based on LaTeX, and are rendered to HTML, plain text and pdf for viewing.
- The `.Rd` files can be used for documentation of each function and the package itself.

12.5.1 Documentation for `-package`

The main `MyName-package.Rd` contains this code by default:

```
\name{packageexample-package}
\alias{packageexample-package}
\alias{packageexample}
\docType{package}
\title{
\packageTitle{packageexample}
}
\description{
\packageDescription{packageexample}
}
\details{

The DESCRIPTION file:
\packageDESCRIPTION{packageexample}
\packageIndices{packageexample}
~~ An overview of how to use the package, including the most important functions ~~
}
\author{
\packageAuthor{packageexample}

Maintainer: \packageMaintainer{packageexample}
}
\references{
~~ Literature or other references for background information ~~
}
~~ Optionally other standard keywords, one per line, from file KEYWORDS in the R documentation
\keyword{ package }
\seealso{
```



```

~~ Optional links to other man pages, e.g. ~~
~~ \code{\link[<pkg>:<pkg>-package]{<pkg>}} ~~
}
\examples{
~~ simple examples of the most important functions ~~
}

```

The function documentation contains this code by default:

```

\name{RcppArmadillo-Functions}
\alias{rcpparma_hello_world}
\alias{rcpparma_innerproduct}
\alias{rcpparma_outerproduct}
\alias{rcpparma_bothproducts}
\title{Set of functions in example RcppArmadillo package}
\description{
  These four functions are created when
  \code{RcppArmadillo.package.skeleton()} is invoked to create a
  skeleton packages.
}
\usage{
rcpparma_hello_world()
rcpparma_outerproduct(x)
rcpparma_innerproduct(x)
rcpparma_bothproducts(x)
}
\arguments{
  \item{x}{a numeric vector}
}
\value{
  \code{rcpparma_hello_world()} does not return a value, but displays a
  message to the console.

  \code{rcpparma_outerproduct()} returns a numeric matrix computed as the
  outer (vector) product of \code{x}.

  \code{rcpparma_innerproduct()} returns a double computer as the inner
  (vector) product of \code{x}.

  \code{rcpparma_bothproducts()} returns a list with both the outer and
  inner products.
}

```

```

}
\details{
  These are example functions which should be largely
  self-explanatory. Their main benefit is to demonstrate how to write a
  function using the Armadillo C++ classes, and to have to such a
  function accessible from R.
}
\references{
  See the documentation for Armadillo, and RcppArmadillo, for more details.
}
\examples{
  x <- sqrt(1:4)
  rcpparma_innerproduct(x)
  rcpparma_outerproduct(x)
}
\author{Dirk Eddelbuettel}

```

12.6 Description

The job of the description file is to store important metadata about your package. When you first start writing packages, you'll mostly use these metadata to record what packages are needed to run your package.

However, as time goes by and you start sharing your package with others, the metadata file becomes increasingly important because it specifies who can use it (the license) and whom to contact (you!) if there are any problems.

Dependencies: What does your package need? Packages that must be loaded along with your package need to be listed in the **Imports** section of **DESCRIPTION**.

```

Package: packageexample
Type: Package
Title: What the package does (short line)
Version: 1.0
Date: 2025-05-16
Author: Who wrote it
Maintainer: Who to complain to <yourfault@somewhere.net>
Description: More about what it does (maybe more than one line)
License: What license is it under?
Imports: Rcpp (>= 1.0.13-1)
LinkingTo: Rcpp, RcppArmadillo

```

12.7 Different types of R packages

Source packages: This is the development version of a package that lives on your computer. A source package is just a directory with components subfolders like `R`, and files like `DESCRIPTION`, and so on.

Bundled packages: This is a package that's been compressed into a single file. By convention (from Linux), package bundles in R use the extension `.tar.gz`. This means that multiple files have been reduced to a single file (`.tar`) and then compressed using (`.gz`). RStudio creates these files with `Build -> More -> Build Source Package`. (Yes, this is inconsistent with the previous point).

Binary packages: This is the type of package you want to distribute to users who don't have package development tools. Binary packages are platform specific: you can't install a Windows binary package on a Mac or vice versa. Also, while Mac binary packages end in `.tgz`, Windows binary packages end in `.zip`.

12.8 Beyond the basics

GitHub: <https://github.com/>

- Great place to share your packages (or other code) with others online.
- Multiple people working on a project.
- Version control.
- Directly connect RStudio to GitHub.
- Push/Pull versions between GitHub and your computer.
- <https://happygitwithr.com/>

CRAN: <https://cran.r-project.org/>

- As close as it gets to making an “official” R package.
- Team of volunteers in Vienna do quality control.
- Many checks you should do before submitting a package.
- Correct formatting, documentation, checking on many different OS's.
- `devtools` package helps a lot with this.
- Section 23 of Wickham's book: <https://r-pkgs.org/release.html>

12.9 Coding Style and Swirl

Style: You should get in the habit of writing R code with good and consistent style. Good style is important because while your code only has one author, it'll usually have multiple readers. Therefore, use:

- Wickham (<https://adv-r.had.co.nz/Style.html>) for a style guide.
- Hungarian notation.

Swirl: The `swirl` package is good for learning R. It asks you questions and gives you a prompt where you can type your answer.

```
devtools::load_all(".")
install.packages("swirl")
library(swirl)
swirl()
```

Part II

Problem sets

13 Exercise set 1

13.1 (1)

Do some simple computations of your choice. This could be for example defining two vectors and then performing addition, subtraction, multiplication of their elements. At each computation use `help` to find out which function you must use.

Solution:

```
vX <- 1:5 # Numbers 1, 2, 3, 4, 5
vY <- seq(2, 10, 2) # Numbers 2, 4, 6, 8, 10

# Addition
vX + 10
#> [1] 11 12 13 14 15
vY + 1
#> [1] 3 5 7 9 11

# Subtraction
vX - 10
#> [1] -9 -8 -7 -6 -5
vY - 1
#> [1] 1 3 5 7 9

# Multiplication
vX * 10
#> [1] 10 20 30 40 50
vY * -1
#> [1] -2 -4 -6 -8 -10

# Powers
vX ^ 2
#> [1] 1 4 9 16 25
vY ^ -1
#> [1] 0.5000000 0.2500000 0.1666667 0.1250000 0.1000000
```

```
# Other functions: division (/), exponentiation (^), square root (sqrt())...
```

13.2 (2)

Define `v1 = c(1, 2, 2, 1)` and `v2 = c(2, 3, 3, 2)`. Do element-wise addition, subtraction, multiplication etc. Perform concatenation between `v1` and `v2` into a new vector `v3`.

Solution:

```
v1 <- c(1, 2, 2, 1)
v2 <- c(2, 3, 3, 2)

# Element-wise addition
v1 + v2
#> [1] 3 5 5 3

# Element-wise subtraction
v1 - v2
#> [1] -1 -1 -1 -1

# Element-wise multiplication
v1 * v2
#> [1] 2 6 6 2

# Concatenation into new vector
v3 <- c(v1, v2)
v3
#> [1] 1 2 2 1 2 3 3 2
```

13.3 (3)

Define matrices of your choice. Use R and determine the rows or columns of each of these variables.

Solution:

```
mA <- matrix(1:6, nrow = 2, ncol = 3, byrow = TRUE)
mA
#>      [,1] [,2] [,3]
```

```

#> [1,] 1 2 3
#> [2,] 4 5 6

# Determining the dimensions (number of rows and columns)
dim(mA)
#> [1] 2 3

# Or retrieving either the number of rows or number of columns
nrow(mA) # Rows
#> [1] 2

ncol(mA) # Columns
#> [1] 3

```

13.4 (4)

Define a 3×4 matrix, `mA`, of your choice

```

mA <- matrix(seq(12, 1, -1), 3, 4, byrow = TRUE)
mA
#>      [,1] [,2] [,3] [,4]
#> [1,] 12  11  10   9
#> [2,]  8   7   6   5
#> [3,]  4   3   2   1

```

1. Print the minimum and the maximum of each row;

```

# Minimum and maximum of each row
apply(mA, 1, min) # `apply` applies the function `min` to each row (indicated by 1)
#> [1] 9 5 1

apply(mA, 1, max)
#> [1] 12 8 4

```

2. Compute the sum of each column and print it as a vector.

```

colSums(mA)
#> [1] 24 21 18 15

```

3. Generate a matrix `mB` with columns equal to the cumulative sum of the rows in `mA`;


```
mB <- apply(mA, 1, cumsum) # See notes for `apply` above. `cumsum` returns the cumulative sum
mB # Column 1 in mB is the cumulative sum of row 1 in mA, and so on....
#>      [,1] [,2] [,3]
#> [1,]   12    8    4
#> [2,]   23   15    7
#> [3,]   33   21    9
#> [4,]   42   26   10
```

4. Generate a matrix `mC` with columns equal to the cumulative product of the rows in `mA`.

```
mC <- apply(mA, 1, cumprod)
mC # Column 1 in mC is the cumulative product of row 1 in mA, and so on....
#>      [,1] [,2] [,3]
#> [1,]   12    8    4
#> [2,]  132   56   12
#> [3,] 1320  336   24
#> [4,]11880 1680   24
```

5. Sort in ascending order the elements in the first column of `mA`.

```
mA # Unsorted
#>      [,1] [,2] [,3] [,4]
#> [1,]   12   11   10    9
#> [2,]    8    7    6    5
#> [3,]    4    3    2    1

sort(mA[, 1]) # Sort and return only the first column
#> [1]  4  8 12

mA[order(mA[, 1]), ] # All rows sorted by the first column. `order` returns a permutation the
#>      [,1] [,2] [,3] [,4]
#> [1,]    4    3    2    1
#> [2,]    8    7    6    5
#> [3,]   12   11   10    9
```

13.5 (5)

Generate a vector of random numbers extracted from a Uniform distribution (use `runif`). The number samples is 250. Compute mean, variance, kurtosis and skewness parameters of the generated vector.

Solution:

```
set.seed(1) # Make the results reproducible by setting a seed
vU <- runif(n = 250, min = 0, max = 1) # The final two arguments are not necessary as they are
```

Install the library for simple summary statistics (if needed):

```
if (!require("e1071")) install.packages("e1071")
suppressMessages(library(e1071))
```

Calculate the summary statistics:

```
cat("The mean is: ", mean(vU), "\n")
#> The mean is: 0.5098254

cat("The variance is: ", var(vU), "\n")
#> The variance is: 0.07249006

cat("The skewness is: ", skewness(vU), "\n")
#> The skewness is: 0.04188663

cat("The kurtosis is: ", kurtosis(vU), "\n")
#> The kurtosis is: -1.121881
```

We could also have computed the kurtosis and skewness manually:

```
iN <- length(vU) # Number of observations
dStdU <- sqrt(var(vU)) # Standard deviation of U

dMoment3 <- sum((vU - mean(vU)) ^ 3) / iN # 3rd central moment
dSkewnews <- dMoment3 / dStdU ^ 3
dSkewnews
#> [1] 0.04188663

dMoment4 <- sum((vU - mean(vU)) ^ 4) / iN # 4th central moment
dKurtosis <- dMoment4 / dStdU ^ 4 - 3 # Excess kurtosis
dKurtosis
#> [1] -1.121881
```

13.6 (6)

Compute the following:

- *Define the string `sR = "RisGreat"`; Suppose that you want to replace the first "R" with an "S".*

Solution:

```
sR <- "RisGreat"
sub("R", "S", sR) # Note, `sub` only replaces the first occurrence. `gsub` replaces all.
#> [1] "SisGreat"
```

14 Exercise set 2

14.1 (1)

Give R assignment statements that set the variable z to.

(a). x^{a^b}

Solution:

```
x <- 2
a <- 3
b <- 2
z <- x ^ (a ^ b)
z
#> [1] 512
```

(b). $(x^a)^b$

Solution:

```
z <- (x ^ a) ^ b
z
#> [1] 64
```

(c). $3x^3 + 2x^2 + 6x + 1$

Solution:

```
z <- 3 * x ^ 3 + 2 * x ^ 2 + 6 * x + 1
z
#> [1] 45
```

(d). $z + 1$

Solution:

```
z <- z + 1
z
#> [1] 46
```

14.2 (2)

Give R expressions that return the following matrices and vectors:

(a). (1, 2, 3, 4, 5, 6, 7, 8, 7, 6, 5, 4, 3, 2, 1)

Solution:

```
c(1:8, 7:1) # Simple concatenation, but notice how 7:1 returns a decreasing vector of integers
#> [1] 1 2 3 4 5 6 7 8 7 6 5 4 3 2 1
```

(b). (1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5, 5)

Solution:

```
rep.int(x = 1:5, times = 1:5) # `rep.int` is faster than `rep` and only requires two arguments
#> [1] 1 2 2 3 3 3 4 4 4 4 5 5 5 5 5
```

(c).

$$\begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

Solution:

```
1 - diag(3) # We create a matrix of ones and subtract an identity matrix of size 3
#>      [,1] [,2] [,3]
#> [1,]    0    1    1
#> [2,]    1    0    1
#> [3,]    1    1    0
```

(d).

$$\begin{pmatrix} 0 & 2 & 3 \\ 0 & 4 & 0 \\ 7 & 0 & 0 \end{pmatrix}$$

Solution:

```
matrix(c(0, 0, 7, 2, 4, 0, 3, 0, 0), 3, 3) # Manually creating the matrix seems to be the eas
#>      [,1] [,2] [,3]
#> [1,]    0    2    3
#> [2,]    0    4    0
#> [3,]    7    0    0
```

14.3 (3)

Use R to produce vector containing all integers from 1 to 100 that are not divisible by 2, 3, or 7.

Solution:

```
vX <- 1:100
vX[(vX %% 2 != 0) & (vX %% 3 != 0) & (vX %% 7 != 0)] # We use the modulo operator for indexin
#> [1]  1  5 11 13 17 19 23 25 29 31 37 41 43 47 53 55 59 61 65 67 71 73 79 83 85
#> [26] 89 95 97
```

14.4 (4)

Build a 10 x 10 identity matrix. Then make all the non-zero elements 5. Do this latter step in at least two different ways.

Solution:

```
mI <- diag(10) # 10 x 10 identity matrix
mI[mI != 0] <- 5 # Making the non-zero elements 5 by using indexing
mI
#>      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
#> [1,]    5    0    0    0    0    0    0    0    0    0
#> [2,]    0    5    0    0    0    0    0    0    0    0
#> [3,]    0    0    5    0    0    0    0    0    0    0
#> [4,]    0    0    0    5    0    0    0    0    0    0
#> [5,]    0    0    0    0    5    0    0    0    0    0
#> [6,]    0    0    0    0    0    5    0    0    0    0
#> [7,]    0    0    0    0    0    0    5    0    0    0
#> [8,]    0    0    0    0    0    0    0    5    0    0
#> [9,]    0    0    0    0    0    0    0    0    5    0
```

```

#> [10,]    0    0    0    0    0    0    0    0    0    0    5

# Other solutions
mA <- diag(10)
mA <- ifelse(mA != 0, 5, mA) # Using if-else
all.equal(mI, mA)
#> [1] TRUE

mB <- diag(10)
diag(mB) <- 5 # Modify the diagonal directly
all.equal(mI, mB)
#> [1] TRUE

```

14.5 (5)

Consider the function $y = f(x)$ defined by:

$$f(x) = \begin{cases} -x^3, & \text{if } x \leq 0 \\ x^2, & \text{if } x \in (0, 1] \\ \sqrt{x}, & \text{if } x > 1 \end{cases}$$

Supposing that you are given x , write an R expression for y using `if` statements.

Solution:

```

f <- function(iX) {
  if (iX <= 0) {
    return(- iX ^ 3)
  } else if (iX <= 1) {
    return(iX ^ 2)
  } else {
    return(sqrt(iX))
  }
}

set.seed(1)
x <- runif(n = 1, min = -1, max = 2) # Random variable with equal probability of being in the
cat("x is: ", x, " and y is: ", f(x))
#> x is: -0.203474 and y is: 0.008424164

```

```
# Bonus: the function in one-line: f <- function(iX) ifelse(iX <= 0, -iX ^ 3, ifelse(iX <= 1
```

14.6 (6)

Let $h(x, n) = 1 + x + x^2 + x^3 + \dots + x^n = \sum_{i=0}^n x^i$. Write an R program to calculate $h(x, n)$ using a `for` loop.

Solution:

```
# Note, this is just a finite geometric series
iX <- 5 # Example values
iN <- 4
iGeoSum <- 0
for (iIte in 0:iN) {
  iGeoSum <- iGeoSum + iX ^ iIte
}
iGeoSum
#> [1] 781
```

14.7 (7)

First, write a program that achieves the same result as in Exercise 6 but using a `while` loop.

Then, write a program that does this using vector-operations (and no loops).

Solution:

```
iX <- 5 # Example values
iN <- 4
iGeoSum <- 0
iIte <- 0
while (iIte <= iN) {
  iGeoSum <- iGeoSum + iX ^ iIte
  iIte <- iIte + 1
}
iGeoSum
#> [1] 781

sum(iX ^ (0:iN)) # Vector-operations. The one-liner uses element-wise exponentiation against
#> [1] 781
```


14.8 (8)

How would you find the sum of every third element of a vector x ?

Solution:

```
vX <- 1:100
sum(vX[seq.int(3, length(vX), 3)]) # We use some smart indexing of our vector x. `seq.int` is
#> [1] 1683
```

14.9 (9)

Write a program that uses a loop to find the minimum of a vector x , without using any predefined functions like `min` or `sort`.

Solution:

```
set.seed(1)
vX <- runif(n = 250) # 250 random values
dMin <- vX[1]
for (iN in 1:length(vX)) {
  if (vX[iN] < dMin) dMin <- vX[iN] # We loop over the vector vX and assign the value at the
}
dMin # Print the minimum value
#> [1] 0.01307758
min(vX) # Verifying the result
#> [1] 0.01307758
```

14.10 (10)

A room contains 100 toggle switches, originally all turned off. 100 people enter the room in turn. The first toggles every switch, the second toggles every second switch, the third every third switch, and so on, to the last person who toggles the last switch only. At the end of this process, which switches are turned on?

Solution:

```

vX <- rep(0L, 100)

for (i in 1:100) {

  vX[seq_along(vX) %% i == 0] <- bitwXor(vX[seq_along(vX) %% i == 0], 1) # Using bitwise log

  # This also works
  # vX[seq_along(vX) %% i == 0] <- !vX[seq_along(vX) %% i == 0]

}

vX
#>   [1] 1 0 0 1 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0
#>  [38] 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
#>  [75] 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1

which(vX == 1) # Perfect squares! :-)
#>   [1]    1    4    9   16   25   36   49   64   81  100

```

14.11 (11)

Load the historical prices of MAERSK from the file `MAERSK-B.CO.csv` available in blackboard. Compute the percentage log-returns and plot the associated time-series. Add a red coloured horizontal line to the plot indicating the average value. Add one standard deviation confidence bands to the average value as dashed blue lines.

Solution:

```

mMaerskData <- read.csv("MAERSK-B.CO.csv", header = TRUE, sep = ",", na.strings = "null")

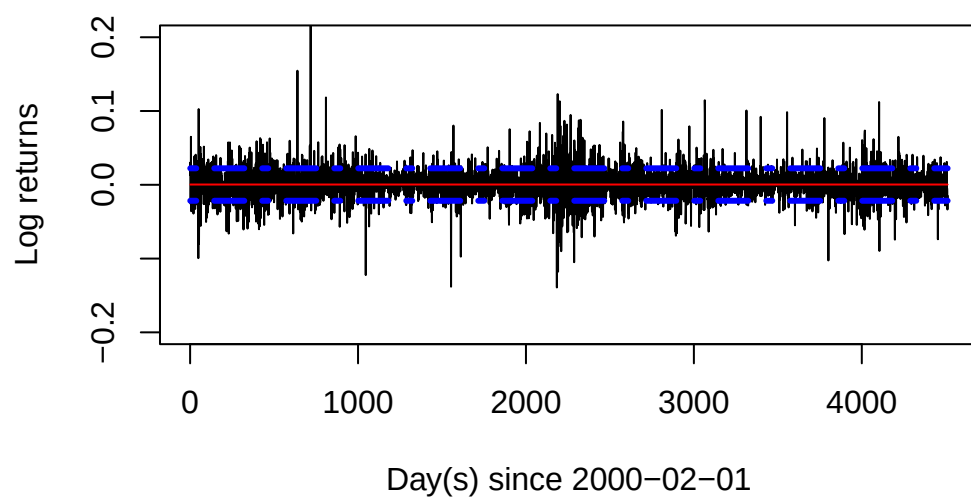
vY <- diff(log(na.omit(mMaerskData[, "Adj.Close"]))) # Log-returns

# Line plot
plot(1:length(vY), vY, type = "l", ylim = c(-0.2, 0.2), ylab = "Log returns", xlab = paste0(

# Mean
lines(1:length(vY), rep(mean(vY), length(vY)), col = "red", lwd = "1")

# Confidence bands
lines(1:length(vY), rep((mean(vY) + sqrt(var(vY))), length(vY)), col = "blue", lwd = "3", lty = 2)
lines(1:length(vY), rep((mean(vY) - sqrt(var(vY))), length(vY)), col = "blue", lwd = "3", lty = 2)

```



15 Exercise set 3

15.1 (1)

The (Euclidean) length of a vector $v = (a_0, \dots, a_k)$ is the square root of the sum of squares of its coordinates, that is $\sqrt{a_0^2 + \dots + a_k^2}$. Write a function that returns the Euclidean length of a vector.

Solution:

```
euclidean_length <- function(vInput) {  
  return(sqrt(sum(vInput^2)))  
}  
  
vX <- 1:10  
euclidean_length(vX)  
#> [1] 19.62142
```

15.2 (2)

The Game of Life is a cellular automaton and was devised by the mathematician J.H. Conway in 1970. It is played on a grid of cells, each of which is either alive or dead. The grid of cells evolves in time and each cell interacts with its eight neighbours, which are the cells directly adjacent horizontally, vertically, and diagonally.

At each time step cells change as follows:

- A live cell with fewer than two neighbours dies of loneliness.
- A live cell with more than three neighbours dies of overcrowding.
- A live cell with two or three neighbours lives on the the next generation.
- A dead cell with exactly three neighbours comes to life

The initial pattern constitutes the first generation of the system. The second generation is created by applying the above rules simultaneously to every cell in the first generation: births and deaths all happen simultaneously. The rules continue to be applied repeatedly to create further generations.

Theoretically, the Game of Life is played on an infinite grid, but in practice we use a finite grid arranged as a torus. That is, if you are in the left-most column of the grid then your left-hand neighbours are in the right-most column, and if you are in the top row, then your neighbours above are in the bottom row.

Write a program that replicates The Game of Life for $iT = 420$ periods, then stop. Organize the cells in a $n \times n$ matrix A . Start by defining the initial population with $A = \text{matrix}(\text{round}(\text{runif}(iN^2)), iN, iN)$, for $iN = 10$. At the beginning of the program, set the seed of the random number generator with `set.seed(999)` such that your results can be replicated.

After running the algorithm, you should get the following population:

```
#>      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
#> [1,]    0    0    0    0    0    0    0    0    0    0
#> [2,]    0    0    0    0    0    0    0    0    0    0
#> [3,]    0    0    0    0    0    0    0    0    0    0
#> [4,]    0    0    0    0    1    1    0    0    0    0
#> [5,]    0    0    0    1    0    0    1    0    0    0
#> [6,]    0    0    0    1    0    0    1    0    0    0
#> [7,]    0    0    0    0    1    1    0    0    0    0
#> [8,]    0    0    0    0    0    0    0    0    0    0
#> [9,]    0    0    0    0    0    0    0    0    0    0
#> [10,]   0    0    0    0    0    0    0    0    0    0
```

Notice that the distribution of the points is the same even if we let the program run for $iT = 1620$ iterations. Do you think we always observe this behaviour? Try to conjecture an answer.

Solution(s):

```
set.seed(999)
iT <- 420
iN <- 10
A <- matrix(round(runif(iN^2)), iN, iN)

## First solution (using a triple loop)
game_of_life_1 <- function(mInput, iT) {
  iR <- nrow(mInput)
  iC <- ncol(mInput)
```

```

mNew <- matrix(0, iR, iC)
for (t in 1:iT) {
  # Loop over rows and columns, i.e., each value in the matrix separately
  for (i in 1:iR) {
    for (j in 1:iC) {
      # Use the modulus operator for handling the borders
      # We need (i - 2) due to R indexing beginning at 1
      iNeighbourCount <- mInput[(i - 2) %% iR + 1, (j - 2) %% iC + 1] +
        mInput[(i - 2) %% iR + 1, (j - 1) %% iC + 1] +
        mInput[(i - 2) %% iR + 1, j %% iC + 1] +

        mInput[(i - 1) %% iR + 1, (j - 2) %% iC + 1] +
        mInput[(i - 1) %% iR + 1, j %% iC + 1] +

        mInput[i %% iR + 1, (j - 2) %% iC + 1] +
        mInput[i %% iR + 1, (j - 1) %% iC + 1] +
        mInput[i %% iR + 1, j %% iC + 1]

      if (mInput[i, j] == 1) { # If alive
        if (iNeighbourCount < 2 || iNeighbourCount > 3) {
          mNew[i, j] <- 0
        } else {
          mNew[i, j] <- 1
        }
      } else { # If dead
        if (iNeighbourCount == 3) {
          mNew[i, j] <- 1
        } else {
          mNew[i, j] <- 0
        }
      }
    }
  }
  mInput <- mNew
}
return(mInput)
}

print(game_of_life_1(A, 420))
#>      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
#> [1,]    0    0    0    0    0    0    0    0    0    0
#> [2,]    0    0    0    0    0    0    0    0    0    0

```

```

#> [3,] 0 0 0 0 0 0 0 0 0 0 0
#> [4,] 0 0 0 0 1 1 0 0 0 0 0
#> [5,] 0 0 0 1 0 0 1 0 0 0 0
#> [6,] 0 0 0 1 0 0 1 0 0 0 0
#> [7,] 0 0 0 0 1 1 0 0 0 0 0
#> [8,] 0 0 0 0 0 0 0 0 0 0 0
#> [9,] 0 0 0 0 0 0 0 0 0 0 0
#> [10,] 0 0 0 0 0 0 0 0 0 0 0

## Second solution (much faster)
game_of_life_2 <- function(mInput, iT) {
  iR <- nrow(mInput)
  iC <- ncol(mInput)
  for (t in 1:iT) {
    mPadded <- matrix(0, iR + 2, iC + 2) # Empty matrix with a border
    mPadded[2:(iR + 1), 2:(iC + 1)] <- mInput # Middle part

    mPadded[1, 2:(iC + 1)] <- mInput[iR, ] # Top row
    mPadded[iR + 2, 2:(iC + 1)] <- mInput[1, ] # Bottom row
    mPadded[2:(iR + 1), 1] <- mInput[, iC] # Left column
    mPadded[2:(iR + 1), iC + 2] <- mInput[, 1] # Right column

    mPadded[1, 1] <- mInput[iR, iC] # Top left value
    mPadded[1, iC + 2] <- mInput[iR, 1] # Top right value
    mPadded[iR + 2, 1] <- mInput[1, iC] # Bottom left value
    mPadded[iR + 2, iC + 2] <- mInput[1, 1] # Bottom right value

    # Now, we can create a neighbour count by shifting the padded matrix and adding the 1's
    mNeighbourCount <- mPadded[1:iR, 1:iC] +
      mPadded[2:(iR + 1), 1:iC] +
      mPadded[3:(iR + 2), 1:iC] +
      mPadded[1:iR, 2:(iC + 1)] +
      mPadded[3:(iR + 2), 2:(iC + 1)] +
      mPadded[1:iR, 3:(iC + 2)] +
      mPadded[2:(iR + 1), 3:(iC + 2)] +
      mPadded[3:(iR + 2), 3:(iC + 2)]

    # And we can use the neighbour count matrix for indexing
    mInput <- (mInput & (mNeighbourCount == 2 | mNeighbourCount == 3)) | (!mInput & mNeighbourCount == 3)
  }
}

```

```

    return(mInput)
}

# Use the plus-sign to return the matrix as integer booleans
print(+game_of_life_2(A, 420))
#>      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
#> [1,]    0    0    0    0    0    0    0    0    0    0
#> [2,]    0    0    0    0    0    0    0    0    0    0
#> [3,]    0    0    0    0    0    0    0    0    0    0
#> [4,]    0    0    0    0    1    1    0    0    0    0
#> [5,]    0    0    0    1    0    0    1    0    0    0
#> [6,]    0    0    0    1    0    0    1    0    0    0
#> [7,]    0    0    0    0    1    1    0    0    0    0
#> [8,]    0    0    0    0    0    0    0    0    0    0
#> [9,]    0    0    0    0    0    0    0    0    0    0
#> [10,]   0    0    0    0    0    0    0    0    0    0

## A quick speed comparison of the two solutions shows a huge increase in performance
suppressMessages(library(microbenchmark))
#> Warning: pakke 'microbenchmark' blev bygget under R version 4.3.3
microbenchmark(game_of_life_1(A, 420), +game_of_life_2(A, 420))
#> Unit: milliseconds
#>      expr      min       lq      mean  median       uq      max
#> game_of_life_1(A, 420) 96.7333 98.5862 100.553867 99.5174 100.84375 129.0533
#> +game_of_life_2(A, 420)  6.6478  6.9277   7.936645  7.3746   8.14575  35.9050
#> neval
#>    100
#>    100

```

Regarding the second part of the question, there are certain patterns in Conway's Game of Life that do not change between generations - implying that further iterations are pointless.

In our case, every “live” cell has two neighbours and thus survive. And dead cells either have 0, 2 or 4 neighbours, so they never come alive.

15.3 (3)

The number of ways you can choose r things from a set of n , ignoring the order in which they are chosen is:

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

Let x be the first element of the set of n things. We can partition the collection of possible size r subsets into those that contain x and those that don't: there must be $\binom{n-1}{r-1}$ subsets of the first type and $\binom{n-1}{r}$ subsets of the second type. Thus:

$$\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r}$$

Using this and the fact that $\binom{n}{n} = \binom{n}{0} = 1$, write a recursive function to calculate $\binom{n}{r}$.

Solution:

```
fbinom <- function(n, r) {
  if (n == r | r == 0) {
    return(1)
  } else {
    return(fbinom(n - 1, r - 1) + fbinom(n - 1, r))
  }
}
```

15.4 (4)

What will be the output of the following code? Try to answer this without typing it up.

```
fb <- function(n) {
  if (n == 1 || n == 2) {
    return(1)
  } else {
    return(fb(n - 1) + fb(n - 2))
  }
}
fb(8)
```

Solution:

The output will be the 8th number in the Fibonacci series (1, 1, 2, 3, 5, 8, 13, 21), i.e., 21.

15.5 (5)

Horner's algorithm for evaluating the polynomial $p(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n$ consists of re-expressing it as:

$$a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + xa_n)\cdots))$$

How many operations are required to evaluate $p(x)$ in each form?

Solution:

For the normal expression, we need $n - 1$ additions, $n - 1$ multiplications and $\sum_{i=0}^{n-1} = \frac{n^2-n}{2}$ multiplications. In total, this is $2*(n-1) + \frac{n^2}{2} - \frac{n}{2}$ elementary operations, which is approximately $O(n^2)$.

For Horner's algorithm: There are still $n - 1$ additive terms, but only $n - 1$ multiplications. Thus, there are $2 * (n - 1)$ elementary operations, which is approximately $O(n)$.

15.6 (6)

Create a 10 by 10 matrix $m1$ and let its elements be drawn from a Gaussian distribution with mean value 20 and variance 100 and then rounded. Using the `apply` function, return a new matrix whose entries are those of $m1$ modulo 3.

Solution:

```
m1 <- matrix(round(rnorm(100, mean = 20, sd = 10))), 10, 10)
m1
#>      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
#> [1,]   29   35   18    9   21   29   24   17   12   28
#> [2,]   18   24   23   11   27   37    2   32   13   13
#> [3,]   18   25    1   16   25   42    2   19   10   13
#> [4,]   11   29    1   31    5   29   20   19   28   15
#> [5,]   25   28   26   -2   28   10    0    8   33   32
#> [6,]   24   21    8   35    8   23   28   19   21   16
#> [7,]   10   13   23   35   21   29   11    7   33   -2
#> [8,]   12   24   13   10    5   12   13   25   35   11
#> [9,]   21   13   24   32    0   -6   11   15   26   16
#> [10,]    5   36   27   28   14   22   10   28   12    2
m2 <- apply(m1, c(1, 2), function(x) x %% 3)
m3 <- matrix(m1 %% 3, 10, 10) # Not the apply() function, but it also works
m2
```

#>		[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]	[,10]
#>	[1,]	2	2	0	0	0	2	0	2	0	1
#>	[2,]	0	0	2	2	0	1	2	2	1	1
#>	[3,]	0	1	1	1	1	0	2	1	1	1
#>	[4,]	2	2	1	1	2	2	2	1	1	0
#>	[5,]	1	1	2	1	1	1	0	2	0	2
#>	[6,]	0	0	2	2	2	2	1	1	0	1
#>	[7,]	1	1	2	2	0	2	2	1	0	1
#>	[8,]	0	0	1	1	2	0	1	1	2	2
#>	[9,]	0	1	0	2	0	0	2	0	2	1
#>	[10,]	2	0	0	1	2	1	1	1	0	2

16 Exercise set 4

16.1 (1)

Write an R function that finds the root of a scalar function using the secant method. The function should also have two starting value inputs, optional inputs for secondary arguments of the input function, and not run indefinitely. It should return a list with the following elements:

- (a) the root
- (b) the function value at the root
- (c) the number of iterations used
- (d) a short description which stopping criterion was used and whether convergence as achieved

You can validate your results using `uniroot()` with function:

$$f(x) = -2.6(x - 0.1) + 2.5(x - 0.1)^4$$

where the root is in the interval $[-1, 1]$. Use appropriate starting values.

Solution:

```
f <- function(dX) {  
  dOut <- -2.6 * (dX - 0.1) + 2.5 * (dX - 0.1)^4  
  return(dOut)  
}  
  
fSecant <- function(f, dX0, dX1, dTol = 1e-9, max.iter = 1000, ...) {  
  iter <- 0  
  dX2 <- dX1  
  while ((abs(f(dX2, ...)) > dTol) && (iter < max.iter)) {  
    dX2 <- dX1 - f(dX1, ...) * ((dX0 - dX1) / (f(dX0, ...) - f(dX1, ...)))  
    dX0 <- dX1  
    dX1 <- dX2  
    iter <- iter + 1  
    cat("At iteration ", iter, "value of x is: ", dX1, "\n")  
  }  
  if (abs(f(dX2, ...)) > dTol) {
```

```

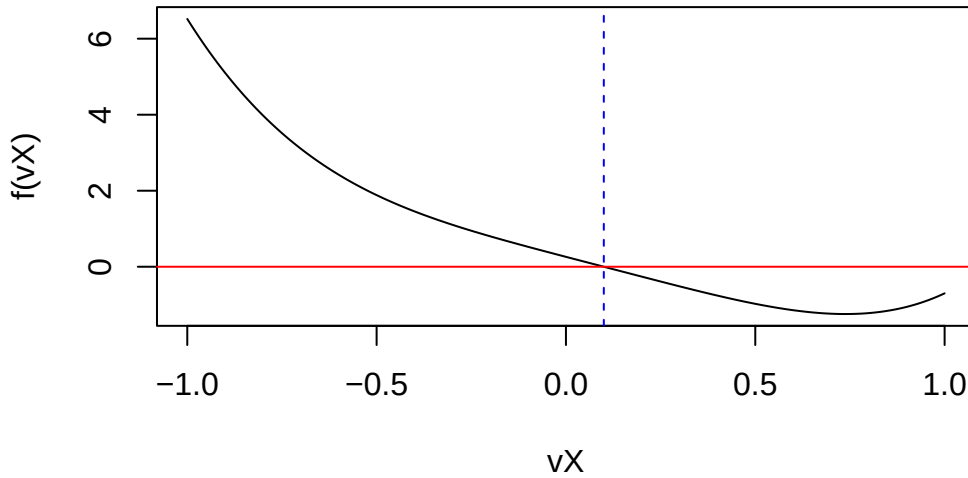
    return(list(root = NULL, f.root = NULL, iter = iter, "Algorithm failed to converge. Maxim
  } else {
    return(list(root = dX2, f.root = f(dX2), iterations = iter, "Convergence reached.))
  }
}

root <- fSecant(f, dX0 = -0.5, dX1 = 0.5)
#> At iteration 1 value of x is: 0.1587413
#> At iteration 2 value of x is: 0.09544817
#> At iteration 3 value of x is: 0.1000008
#> At iteration 4 value of x is: 0.1
root
#> $root
#> [1] 0.1
#>
#> $f.root
#> [1] -1.941947e-13
#>
#> $iterations
#> [1] 4
#>
#> [[4]]
#> [1] "Convergence reached."
uniroot(f, interval = c(-1,1))
#> $root
#> [1] 0.1000007
#>
#> $f.root
#> [1] -1.764589e-06
#>
#> $iter
#> [1] 6
#>
#> $init.it
#> [1] NA
#>
#> $estim.prec
#> [1] 6.103516e-05

vX <- seq(-1, 1, 0.01)
plot(vX, f(vX), type = "l")
abline(h = 0, col = "red")

```

```
abline(v = root["root"], col = "blue", lty = 2)
```



16.2 (2)

You would like to obtain information about the (in)equality of the income distribution in population of people living in Denmark. For that you would like to estimate the standard deviation σ of the distribution of yearly income y_i^* . You observe independent income data y_i for a sample $i = 1, \dots, n$. However, the observed data y_i has been modified to match data protection standards. In particular it has first been demeaned and then all units with income over $c = 1.250.000$ DKK have been removed. This gives us the truncation mechanism:

$$y_i = \begin{cases} y_i^* - E[y_i^*] & \text{if } y_i^* \leq c \\ - & \text{else} \end{cases}$$

To estimate the standard deviation, you assume that the true income y_i^* is normally distributed. This implies that the observed income y_i follows a truncated mean-zero normal distribution with density $f(y)$:

$$f(y; \sigma, c) = \frac{1}{\sigma} \frac{\phi(y/\sigma)}{\Phi(c/\sigma)}, \quad -\infty < y \leq c,$$

where

$$\phi(z) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}z^2\right)$$

$$\Phi(x) = \int_{-\infty}^x \phi(z)dz,$$

i.e. $\phi(\cdot)$ and $\Phi(\cdot)$ are the standard normal PDF and CDF respectively. To estimate σ , we can setup the log-likelihood function

$$\ln L(\sigma; \mathbf{y}, c) = \sum_{i=1}^n \ln(f(y_i; \sigma, c)),$$

which has first derivative (“Score”)

$$S(\sigma; \mathbf{y}, c) = \frac{-n}{\sigma} + \frac{1}{\sigma^3} \sum_{i=1}^n y_i^2 + \frac{cn}{\sigma} f(c; \sigma, c)$$

and second derivative (“Hessian”)

$$H(\sigma^2) = \frac{n}{\sigma^2} - \frac{3}{\sigma^4} \sum_{i=1}^n y_i^2 + \left(\frac{c^3 n}{\sigma^4} - \frac{2cn}{\sigma^2} \right) f(c; \sigma, c) + \frac{c^2 n}{\sigma^2} f(c; \sigma, c)^2.$$

We want to find the estimate $\hat{\sigma}$ which maximizes the log-likelihood function.

Write an *R* function `fTrunc()` that finds the maximum likelihood estimator for σ using a Gauss-Newton algorithm. It should take the following inputs:

- (a) A numerical vector Y of length n containing the dependent variable.
- (b) A vector of starting values for the maximization with default being the sample standard deviation of Y .
- (c) The number of maximum iterations for the optimization with default set to 200.
- (d) A tolerance level for the stopping criterion with default set to $1e-16$.

The overall function should return a list with the following components:

- (a) The estimated parameter $\hat{\sigma}$.
- (b) The log-likelihood, Score and Hessian at the optimum.
- (c) A character containing information about convergence and stopping condition of the optimization.

Load the data `QPE_income1.R` into R using `readRDS()` and estimate $\hat{\sigma}$. The solution should be 509699.8.

Try out a range of different starting values above and below the default. Plot the Score and Hessian along a grid of different σ values. What do you observe?

Solution:

```
f <- function(vY, dSigma, dC) {
  return((1 / dSigma) * ((dnorm(vY / dSigma)) / (pnorm(dC / dSigma))))
}

fScore <- function(vY, dSigma, f, dC) {
  return((( -1 * length(vY)) / dSigma) + (1 / dSigma^3) * (sum(vY^2)) + ((dC * length(vY)) /
}

fHessian <- function(vY, dSigma, f, dC) {
  return(length(vY) / dSigma^2 - (3 / dSigma^4) * sum(vY^2) + (dC^3 * length(vY) / dSigma^4
}

fTrunc <- function(f, fScore, fHessian, vY, start.val = sqrt(var(vY)), n.max = 200, dTol = 1e-06) {

  n <- 0
  dParam <- start.val

  # Keep updating until stopping criterion or max iterations reached
  while ((abs(fScore(vY, dParam, f, ...)) > dTol) && (n < n.max)) {
    # Newton-Raphson updating
    dParam <- dParam - fScore(vY, dParam, f, ...) / fHessian(vY, dParam, f, ...)
    n <- n + 1
    #cat("At iteration", n, "the value of the parameter is:", dParam, "\n")
  }

  if (n == n.max) {
    return(list(
      param = NULL,
      log.likelihood = NULL,
      score = NULL,
      hessian = NULL,
      iter = n,
      msg = "Algorithm failed to converge. Maximum iterations reached."
    ))
  } else {
```



```

    return(list(
      param = dParam,
      log.likelihood = sum(log(f(vY, dParam, ...))),
      score = fScore(vY, dParam, f, ...),
      hessian = fHessian(vY, dParam, f, ...),
      iter = n,
      msg = "Algorithm converged")
    )
  }
}

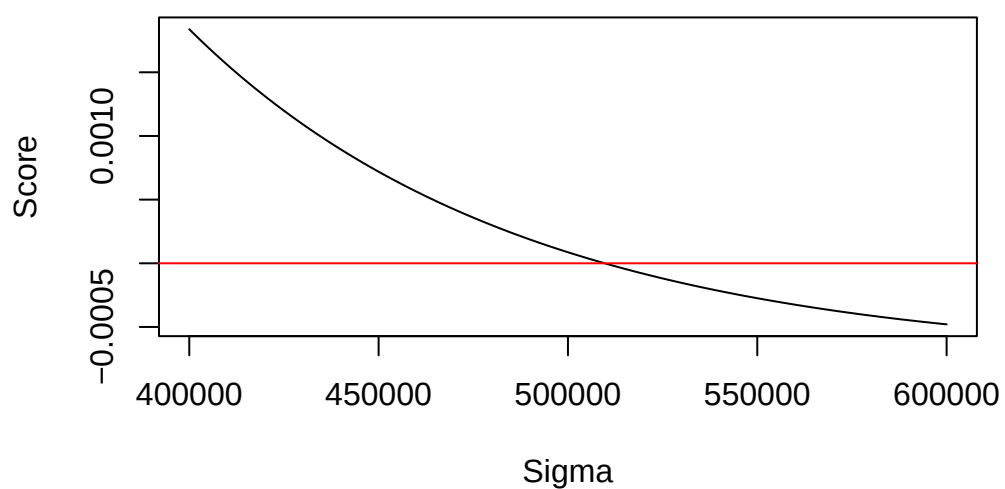
vX <- readRDS("QPE_income1.R")
results <- fTrunc(f, fScore, fHessian, vX, dC = 1250000)
results
#> $param
#> [1] 509699.8
#>
#> $log.likelihood
#> [1] -19280.04
#>
#> $score
#> [1] -5.692061e-19
#>
#> $hessian
#> [1] -8.458564e-09
#>
#> $iter
#> [1] 4
#>
#> $msg
#> [1] "Algorithm converged"

# Plot Score and Hessian
sigmaVals <- seq(400000, 600000, length.out = 200)
scoreVals <- sapply(sigmaVals, function(sigma) fScore(vX, sigma, f, dC = 1250000))
hessianVals <- sapply(sigmaVals, function(sigma) fHessian(vX, sigma, f, dC = 1250000))

# Plot Score
plot(sigmaVals, scoreVals, type = "l", main = "Score Function", xlab = "Sigma", ylab = "Score")
abline(h = 0, col = "red")

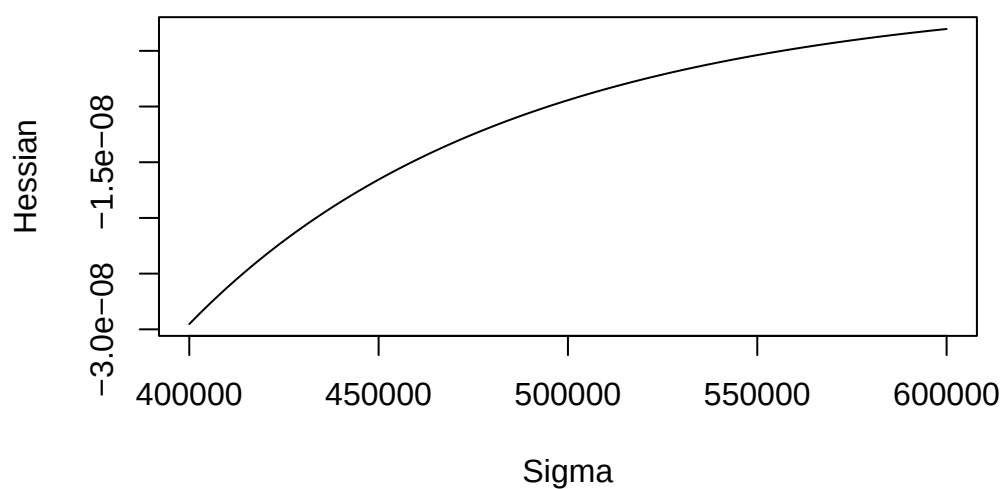
```

Score Function



```
# Plot Hessian  
plot(sigmaVals, hessianVals, type = "l", main = "Hessian Function", xlab = "Sigma", ylab = "Hessian")  
abline(h = 0, col = "red")
```

Hessian Function



17 Exercise set 5

Assume you observe independent data $(\mathbf{X}_i, Y_i)$ for $i = 1, \dots, n$ where $\mathbf{X}_i$ have realizations in $\mathbb{R}^p$ and Y_i is binary. You would like to model the relationship between $\mathbf{X}_i$ and Y_i . A logistic regression model is given by the following structure:

$$Y_i = \mathbb{I}(Y_i^* > 0)$$

$$Y_i^* = \mathbf{X}_i' + \varepsilon_i$$

$$P(\varepsilon_i \leq z) = p(z) = \frac{1}{1 + \exp(-z)}$$

Thus the average log-likelihood function is given by

$$\ln L() = \frac{1}{n} \sum_{i=1}^n [Y_i \ln(p(\mathbf{X}_i')) + (1 - Y_i) \ln(1 - p(\mathbf{X}_i'))]$$

with Score

$$S() = \frac{1}{n} \sum_{i=1}^n (Y_i - p(\mathbf{X}_i')) \mathbf{X}_i$$

and Hessian

$$H() = -\frac{1}{n} \sum_{i=1}^n p(\mathbf{X}_i')(1 - p(\mathbf{X}_i')) \mathbf{X}_i \mathbf{X}_i'$$

17.1 (1)

Write an R function `fLogit()` that finds the maximum likelihood estimator for using a Gauss-Newton algorithm. It should take the following inputs:

- (a) A numerical or logical input vector Y of length n containing the dependent variable.
- (b) A $[n \times p]$ dimensional matrix $\mathbf{X}$ containing the regressors.
- (c) An option whether a constant column should be added to $\mathbf{X}$ with default `TRUE`.
- (d) A vector of starting values for the maximization with default being a zero vector of dimension p .
- (e) The number of maximum iterations for the optimization with default set to 200.
- (f) A tolerance level for the stopping criterion with default set to `1e-16`.

Before optimization, if $\mathbf{X}$ does not contain a constant, your function should create a new $\mathbf{X}$ with an additional column of ones automatically. The overall function should return a list with the following components:

- (a) The estimated parameters $\hat{\beta}$.
- (b) The average log-likelihood, Score and Hessian at the optimum.
- (c) The log-likelihood function at $\beta = \mathbf{0}$.
- (d) The predicted probabilities $p(\mathbf{X}'_i)$.
- (e) A character containing information about convergence and stopping condition of the optimization.

Solution:

```
fLogit <- function(vY, mX, add.constant = TRUE, init.vals = NULL, max.iter = 200, dTol = 1e-10) {

  # probability function
  p <- function(vZ) {
    return(1 / (1 + exp(-vZ)))
  }

  # avg log-likelihood function
  f <- function(vY, mX, vB) {
    return(mean(vY * log(p(mX %*% vB)) + (1 - vY) * log(1 - p(mX %*% vB))))
  }

  fScore <- function(vY, mX, vB) {
    return(t(mX) %*% (vY - as.numeric(p(mX %*% vB))) / nrow(mX))
  }

  fHessian <- function(vY, mX, vB) {
    return(-(t(mX) %*% diag(as.numeric(p(mX %*% vB) * (1 - as.numeric(p(mX %*% vB))))) %*% mX))
  }

  if (add.constant == TRUE) {
    mX <- cbind(1, mX)
  }
  if (is.null(init.vals)) init.vals <- rep(0, ncol(mX))

  i <- 0
  vB <- init.vals

  # Keep updating until stopping criterion or max iterations reached
  while ((max(abs(fScore(vY, mX, vB))) > dTol) && (i < max.iter)) {
```

```

# Newton-Raphson updating
vB <- vB - solve(fHessian(vY, mX, vB), fScore(vY, mX, vB))
i <- i + 1
#cat("At iteration", n, "the value of the parameter is:", dParam, "\n")
}

if (i == max.iter) {
  return(list(
    beta_hat = NULL,
    log_likelihoood_opt = NULL,
    score_opt = NULL,
    hessian_opt = NULL,
    log_likelihoood_null = NULL,
    predicted_probabilities = NULL,
    iterations = i,
    msg = "Algorithm failed to converge. Maximum iterations reached.")
  )
} else {
  return(list(
    beta_hat = vB,
    log_likelihoood_opt = f(vY, mX, vB),
    score_opt = fScore(vY, mX, vB),
    hessian_opt = fHessian(vY, mX, vB),
    log_likelihoood_null = f(vY, mX, rep(0, ncol(mX))),
    predicted_probabilities = p(mX %*% vB),
    iterations = i,
    msg = "Algorithm converged")
  )
}
}

```

17.2 (2)

Simulate a process like the one above for $p = 10$ and $n = 2000$ using random variable generators and estimate it using your function. Validate your results using the built-in `glm()` function with option `family = binomial`.

```

set.seed(123)
n <- 2000
p <- 10
mX <- matrix(rnorm(n*p), n, p)

```

```

vB.actual <- matrix(1:p, p, 1)
vY.star <- mX %*% vB.actual + rnorm(2000, 0, 1)
vY.actual <- vY.star > 0

vResults <- fLogit(vY.actual, mX)
glm_fit <- glm(vY.actual ~ mX, family = binomial)
#> Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
glm_fit$coefficients
#> (Intercept)      mX1      mX2      mX3      mX4      mX5
#> -0.1927213  1.8925010  3.4933457  4.8288383  6.4561863  8.4108110
#>      mX6      mX7      mX8      mX9      mX10
#> 10.2085496 11.8431135 13.2432309 14.8829116 16.5606978
vResults$beta_hat
#>      [,1]
#> [1,] -0.1927213
#> [2,]  1.8925010
#> [3,]  3.4933457
#> [4,]  4.8288384
#> [5,]  6.4561864
#> [6,]  8.4108112
#> [7,] 10.2085499
#> [8,] 11.8431138
#> [9,] 13.2432312
#> [10,] 14.8829119
#> [11,] 16.5606982

```

18 Exercise set 6

In this problem, you are supposed to debug a C++ version of the function used in Exercise Set 5 for logistic regression. It is supposed to take the same inputs, but with a mandatory starting value input vector for the optimization and only have outputs (a) - (d).

- (1) Open the function `fLogitCpp()` that can be sourced via `sourceCpp()`
- (2) Make comments on where to debug the code. You may consider the following questions: - Were all objects declared? - Are all objects declared the correct type? - Are the correct functions used? - Are the input and outputs correct? - Is the algorithmic structure correct? - Hint: There are 8 bugs overall.
- (3) Debug the code and compare the solutions to the `fLogit()` function from Exercise Set 5.

Solution:

The original function including comments on where to debug has been added below:

```
//[[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
using namespace Rcpp;
using namespace arma;

//initialize auxiliary functions: prb, LL, score, hessian

// mX should be initialized with mat
vec fP(vec vBeta, mat mX){
  return(1 / (1 + exp( -mX * vBeta )));
}

// vP should be initialized with vec, and also it should return a double
vec fLnL(vec vBeta, vec vY, mat mX){
  vP = fP(vBeta,mX);
  vec vLnL = mean( vY % log(vP) + (ones<vec>(vY.n_elem) - vY) * log(ones<vec>(vP.n_elem) - vP) );
  return vLnL;
}

// vScore is not divided by the number of rows in mX
vec fScore(vec vBeta, vec vY, mat mX){
  vec vP = fP(vBeta,mX);
  vec vScore = trans(mX)*(vY - vP);
}
```

```

    return vScore;
}
//
mat fHessian(vec vBeta, vec vY, mat mX){
    int iN = vY.n_elem;
    vec vP = fP(vBeta,mX);
    mat mP = zeros<mat>(iN,iN);

    mP.diag(0) = vP % (ones<vec>(iN) - vP);
    mat mHessian = -trans(mX)*mP*mX / iN;

    return mHessian;
}

//[[Rcpp::export]]
List fLogitCpp(vec vY, mat mX, vec vBeta0, // vBeta should be initialized with default values
               bool constant = true,
               double dTol = 1e-9,
               bool imax = 200){ // this should be an int

    List lOut;
    int iN = vY.n_elem;

    if (constant == true){
        vec vOnes = zeros<vec>(iN); // this should be ones
        vBeta0 = join_cols(zeros<vec>(1),vBeta0);
        mX = join_rows(vOnes,mX);
        //Rcout << "Note: A constant has been added to data matrix mX" << std::endl;
    }

    vec vBeta = vBeta0;
    vec vScore = fScore(vBeta,vY,mX);
    mat mHessian = fHessian(vBeta,vY,mX);

    int i = 0;
    while ((max(abs(vScore)) < dTol) & (i < imax)) { // first sign should be flipped
        //Rcout << i << "\n";
        vBeta = vBeta - solve(fHessian(vBeta,vY,mX),fScore(vBeta,vY,mX));
        vScore = fScore(vBeta,vY,mX);
        mHessian = fHessian(vBeta0,vY,mX); // this should be vBeta

        i++;
    }

```



```

}
if (i == imax){
  Rcout << "newton failed to converge" << std::endl;
  return lOut;
} else {
  // Rcout << "Convergence achieved after " << i << " iterations" << std::endl
  //      << "due to max(abs(score)) <" << dTol << std::endl;
  lOut["coefficients"] = vBeta;
  lOut["logLik0"] = fLnL(zeros<vec>(vBeta.n_elem),vY,mX);
  lOut["logLik"] = fLnL(vBeta,vY,mX);
  lOut["score"] = vScore;
  lOut["hessian"] = mHessian;
  lOut["phat"] = fP(vBeta,mX);

  return lOut;
}
}

```

Comparing the solutions:

```

suppressMessages(library(Rcpp))
#> Warning: pakke 'Rcpp' blev bygget under R version 4.3.3
suppressMessages(library(RcppArmadillo))
#> Warning: pakke 'RcppArmadillo' blev bygget under R version 4.3.3
sourceCpp("fLogitCpp.cpp")

set.seed(123)
n <- 2000
p <- 10
mX <- matrix(rnorm(n*p), n, p)
vB.actual <- matrix(1:p, p, 1)
vY.star <- mX %*% vB.actual + rnorm(2000, 0, 1)
vY.actual <- vY.star > 0

vResults <- fLogitCpp(vY.actual, mX, vBeta = rep(0, ncol(mX)))
vResults$coefficients
#>           [,1]
#> [1,] -0.1927213
#> [2,]  1.8925004
#> [3,]  3.4933444
#> [4,]  4.8288365
#> [5,]  6.4561838

```

```
#> [6,] 8.4108079
#> [7,] 10.2085458
#> [8,] 11.8431091
#> [9,] 13.2432260
#> [10,] 14.8829059
#> [11,] 16.5606916
```

19 Ordinary exam 2020

19.1 Problem 1:

In this first part of the exam you are tasked with several independent problems. If you are unable to solve an exercise move on to the next. Set and use the seed 1337 for all exercises in **Problem 1**.

1. Create a data structure of the type `data.frame` consisting of two equal length columns. The first column should contain 100 draws from the Gaussian (normal) distribution with mean 10 and variance 100, i.e. $N(10, 100)$. The second column should contain 100 draws from the standard uniform distribution, i.e. $U(0, 1)$. Assign the two columns in the `data.frame` the names “*rvnorm*” and “*rvunif*”. Calculate and print the mean and standard deviation of each column of this `data.frame`.

Solution:

```
set.seed(1337)
dfData <- data.frame(
  rvnorm = rnorm(100, 10, sqrt(100)),
  rvunif = runif(100)
)

colMeans(dfData)
#>      rvnorm      rvunif
#> 12.3705575  0.4927623
apply(dfData, 2, sd)
#>      rvnorm      rvunif
#> 10.6541506  0.2865665
```

2. Count the number of strictly positive (> 0) numerically valued elements in

```
mA <- matrix(rnorm(10000), nrow = 100, ncol = 100)
```

Print your result to the console. Then define a new matrix equal to `mA` with every second row and every fifth column removed. Print to the console the second column of your new smaller matrix.

Solution:

```
set.seed(1337)
mA <- matrix(rnorm(10000), nrow = 100, ncol = 100)
sum(mA > 0)
#> [1] 4968

indices <- seq(1, nrow(mA))
mB <- mA[indices %% 2 != 0, indices %% 5 != 0]
mB[, 2]
#> [1] 0.20997680 1.37178507 -0.20485705 -0.80947218 0.06873847 -0.46916210
#> [7] -0.73977783 0.37774034 -0.90396001 -0.23914384 -0.31475060 -0.54221576
#> [13] -0.13401439 -0.44703293 -0.58097800 -0.64590441 -0.43772050 1.70627166
#> [19] 1.28239811 3.10367380 -0.25455142 0.11397658 0.28331947 -0.25257320
#> [25] -0.10787613 -1.68283890 0.59339153 1.42397245 -0.16762034 -0.28949467
#> [31] -0.38718739 -0.86600469 -0.16801229 -0.05423047 -0.15457321 1.20115726
#> [37] 0.89901990 -0.81578612 0.56388549 -0.52282158 0.55286246 -0.05125182
#> [43] -0.64952770 1.54842553 -1.13708264 -0.21124735 -1.45882620 -0.13385242
#> [49] 0.31359063 -0.94320880
```

3. Using 10^7 draws from the appropriate uniform distribution use Monte Carlo integration to approximate,

$$I = \int_3^{20} (x^3 - x^2 + 9)dx.$$

Print your approximation of I to the console.

Solution:

```
set.seed(1337)
MonteCarlo.Integration <- function(f, n, a, b) {
  U <- runif(n, min = a, max = b)
  return( (b-a)*mean(f(U)) )
}
MonteCarlo.Integration(function(x) (x^3 - x^2 + 9), 1e7, 3, 20)
#> [1] 37477.23
```

4. Consider the following function to calculate the diagonal of the sample covariance matrix between two matrices, `mX` and `mY`

```
diag.covar <- function(mX, mY) {
  varcov <- 0
  for(j in 1:10) {
    dsum <- 0
    for(i in 1:10) {
      dsum <- dsum + (mX[i,j] - mean(mX[,j]))*(mY[i,j] - mean(mY[,j]))
    }
    varcov[j] <- dsum/(dim(mX)[1]-1)
  }
  return(varcov)
}
```

(The above function is included in the `diagcovar.R` file in the exam hand-out.)

Create a new function, say `diag.covarVec`, that is fully vectorized (or as vectorized as possible). Compare the run time of the two functions using,

```
mX <- matrix(rnorm(100), nrow = 10, ncol = 10)
mY <- matrix(rnorm(100), nrow = 10, ncol = 10)
```

You may, but do not need to, use the functionalities of the `microbenchmark` package. How much faster is your new function? If you cannot implement the vectorization, explain potential causes for inefficiencies and poor programming practices in the `diag.covar` function via comments in your script. If your code is not faster, comment briefly in the script why that might be the case.

Solution:

```
set.seed(1337)
diag.covar <- function(mX, mY) {
  varcov <- 0
  for(j in 1:10) {
    dsum <- 0
    for(i in 1:10) {
      dsum <- dsum + (mX[i,j] - mean(mX[,j]))*(mY[i,j] - mean(mY[,j]))
    }
    varcov[j] <- dsum/(dim(mX)[1]-1)
  }
  return(varcov)
}

diag.covarVec <- function(mX, mY) {
```

```

mSum <- (t(mX) - colMeans(mX)) * (t(mY) - colMeans(mY))
varcov <- rowSums(mSum) / (dim(mX)[1]-1)
return(varcov)
}

mX <- matrix(rnorm(100), nrow = 10, ncol = 10)
mY <- matrix(rnorm(100), nrow = 10, ncol = 10)

diag.covar(mX, mY)
#> [1] -0.002280477  0.133113794  0.354419701 -0.881358047 -0.134682108
#> [6] -0.436054671 -0.425000809 -0.119402997 -0.309367075  0.287553344
diag.covarVec(mX, mY)
#> [1] -0.002280477  0.133113794  0.354419701 -0.881358047 -0.134682108
#> [6] -0.436054671 -0.425000809 -0.119402997 -0.309367075  0.287553344

suppressMessages(library(microbenchmark))
#> Warning: pakke 'microbenchmark' blev bygget under R version 4.3.3
microbenchmark(diag.covar(mX, mY), diag.covarVec(mX, mY))
#> Unit: microseconds
#>
#>      expr      min      lq     mean  median      uq     max neval
#> diag.covar(mX, mY) 1024.3 1080.20 1209.035 1154.80 1266.0 2946.8   100
#> diag.covarVec(mX, mY)   18.8   20.75   82.611   25.35   45.6 5063.3   100

```

5. Construct a function that uses the inversion method to generate random variables, $X \sim \text{Gumbel}(\mu, \beta)$, where $\mu \in \mathbb{R}$ is the location parameter and $\beta > 0$ the scale parameter of a Gumbel distribution. The Gumbel has the following cumulative distribution function,

$$F_X(x) = \exp\left(-\exp\left(-\frac{x-\mu}{\beta}\right)\right), \quad \beta > 0, \mu, x \in \mathbb{R}$$

with probability density function,

$$f_X(x) = \frac{1}{\beta} \exp\left(-\left(\frac{x-\mu}{\beta} + \exp\left(-\frac{x-\mu}{\beta}\right)\right)\right), \quad \beta > 0, \mu, x \in \mathbb{R}.$$

Use your function to generate a sequence of 10,000 random variables from $\text{Gumbel}(0.5, 2)$. Produce a histogram of your generated random variable sequence using the built-in function `hist()`. In the histogram set breaks equal to 141 and limit the range for the x-axis of the plot from the smallest to the largest value in your generated sequence of random Gumbels. In addition superimpose the theoretical probability density function for comparison. In order to superimpose the theoretical probability density unto the histogram use the built-in `lines()` function with appropriate inputs (one input could be the `seq()` function that generates a sequence from -5 to 20 by step increments of 0.05).

Solution:

```

set.seed(1337)
Gumbel <- function(mu, beta, size) {
  U <- runif(size)
  return(mu - beta * log(-log(U)))
}

vX <- Gumbel(0.5, 2, 10000)
hist(vX,
      freq = FALSE,
      breaks = 141,
      col = "cornflowerblue",
      xlab = "",
      ylab = "Density",
      main = "",
      xlim = c(min(vX), max(vX)))

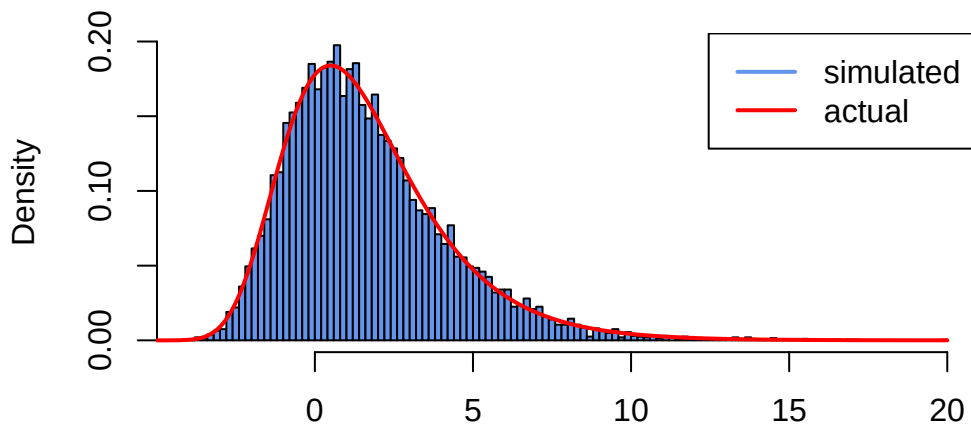
vInput <- seq(-5, 20, 0.05)

fGumbelPdf <- function(mu, beta, x) {
  return(1/beta * exp(-((x-mu)/beta + exp(-(x-mu)/beta))))
}

lines(vInput, fGumbelPdf(0.5, 2, vInput), type = "l", col = "red", lwd = 2)

legend("topright",
      legend = c("simulated", "actual"),
      col = c("cornflowerblue", "red"),
      lwd = 2)

```



19.2 Problem 2:

Consider the following function $f : \mathbb{R} \rightarrow \mathbb{R}$ and its derivatives:

$$f(x) = \sin(x + \pi - 1) \quad (1.1)$$

$$f'(x) = \cos(x + \pi - 1) \quad (1.2)$$

$$f''(x) = -\sin(x + \pi - 1) \quad (1.3)$$

1. Write three R functions with input x that return $f(x)$, $f'(x)$, and $f''(x)$. Provide a line plot of the function and its first derivative on the interval $[-1,1]$.

Solution:

```
f <- function(x) {
  return(sin(x + pi - 1))
}

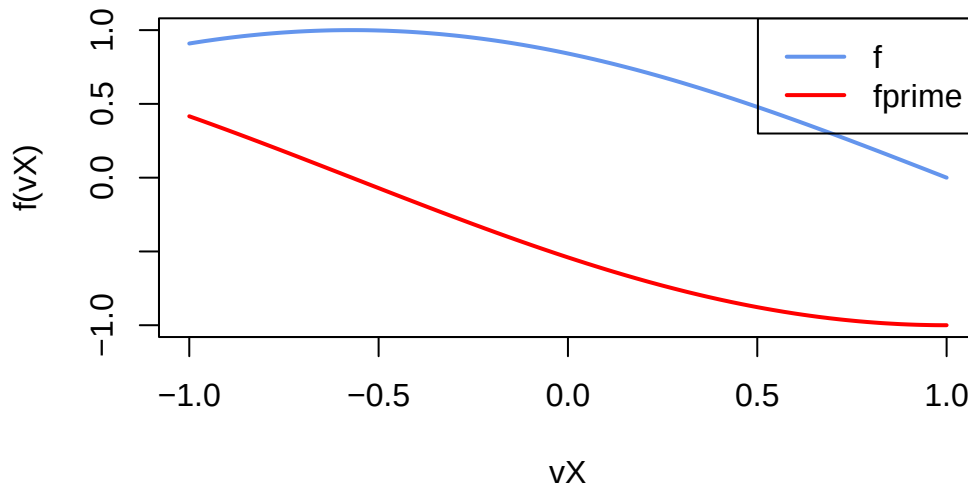
fPrime <- function(x) {
  return(cos(x + pi - 1))
}
```



```
fDoublePrime <- function(x) {
  return(-sin(x + pi - 1))
}

vX <- seq(-1, 1, 0.01)
plot(vX, f(vX), type = "l", col = "cornflowerblue", lwd = 2, ylim = c(-1, 1))
lines(vX, fPrime(vX), type = "l", col = "red", lwd = 2)

legend("topright",
      legend = c("f", "fprime"),
      col = c("cornflowerblue", "red"),
      lwd = 2)
```



2. Write an R function that maximizes R scalar functions using the simple Newton-Raphson method with analytical derivatives. The function should also have a starting value input, two reasonable options for the convergence criterion that can be specified by the user, and not run indefinitely. It should return a list with the following elements:
 - (a) the maximizer
 - (b) the function value at the optimum
 - (c) the number of iterations used
 - (d) a short description which stopping criterion was used and whether convergence was achieved

Solution:

```
NM <- function(f, f_prime, f_sec, dX0, dTol = 1e-9, n.max = 1000){
  dX <- dX0
  fx <- f(dX)
  fpx <- f_prime(dX)
  fsx <- f_sec(dX)
  n <- 0
  while ((abs(fpx) > dTol) && (n < n.max)) {
    dX <- dX - fpx/fsx
    fx <- f(dX)
    fpx <- f_prime(dX)
    fsx <- f_sec(dX)
    n <- n + 1
  }
  if (n == n.max) {
    return(list(
      maximizer = dX,
      function.val = f(dX),
      n.iter = n,
      msg = "Failed to converge. Max number of iterations reached."
    ))
  } else {
    return(list(
      maximizer = dX,
      function.val = f(dX),
      n.iter = n,
      msg = "Convergence reached."
    ))
  }
}
```

3. Write a C++ function using Rcpp and RcppArmadillo that can be loaded via `sourceCpp()`. The function should contain a bisection method to find the root of a scalar function. The function should be able to take any R scalar function as input. Set the default interval for the root between $[-1,1]$. It should return or print a warning if the starting conditions for the bisection method are not fulfilled. Else it should return a list with the following elements:
- (a) the root
 - (b) the function value at the root
 - (c) the number of iterations used *Remark: If you cannot solve it in C++ you can earn some points for doing it in R*

```

// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>

using namespace arma;
using namespace Rcpp;

// [[Rcpp::export]]
List bisection_cpp(Function f, double dXLeft = -1.0, double dXRight = 1.0, double dTol = 0.000001) {
  if (dXLeft >= dXRight) {
    stop("Starting conditions not met");
  }
  double fLeft = as<double>(f(dXLeft));
  double fRight = as<double>(f(dXRight));
  if (fLeft == 0) {
    List lOut;
    lOut["root"] = dXLeft;
    lOut["func_at_root"] = fLeft;
    lOut["num_ite"] = 0;
    return lOut;
  } else if (fRight == 0) {
    List lOut;
    lOut["root"] = dXRight;
    lOut["func_at_root"] = fRight;
    lOut["num_ite"] = 0;
    return lOut;
  } else if (fLeft * fRight > 0) {
    stop("error: f(x.l)*f(x.r) > 0");
  }

  int iter = 0;
  while ((dXRight - dXLeft) > dTol && (iter < maxIter)) {
    double dXMid = (dXLeft + dXRight)/2;
    double fMid = as<double>(f(dXMid));
    if (fMid == 0) {
      return(dXMid);
    } else if (fLeft * fMid < 0) {
      dXRight = dXMid;
      fRight = fMid;
    } else {
      dXLeft = dXMid;
      fLeft = fMid;
    }
  }
}

```

```

    iter = iter + 1;
  }

  List lOut;
  lOut["root"] = (dXLeft + dXRight)/2;
  lOut["func_at_root"] = as<double>(f((dXLeft + dXRight)/2));
  lOut["num_ite"] = iter;
  return lOut;
}

```

```

suppressMessages(library(Rcpp))
#> Warning: pakke 'Rcpp' blev bygget under R version 4.3.3
suppressMessages(library(RcppArmadillo))
#> Warning: pakke 'RcppArmadillo' blev bygget under R version 4.3.3
sourceCpp('exam2020cpp.cpp')

```

4. Use the functions from 2.) and 3.) and the build-in functions `optim()` and `uniroot()` to find the maximizer of $f(x)$ in the interval $[-1,1]$. Use appropriate starting values. Also provide a benchmark comparing their computation times. *Remark: If you cannot solve 2.) and/or 3.) you can still earn points for using the remaining functions only.*

Solution:

```

NM(f, fPrime, fDoublePrime, 0)$maximizer
#> [1] -0.5707963
bisection_cpp(fPrime, dXLeft = -1, dXRight = 1)$root
#> [1] -0.5707703
optim(0, f, method = "L-BFGS-B", lower = -1, upper = 1, control=list(fnscale=-1))$par
#> [1] -0.5707963
uniroot(fPrime, c(-1, 1))$root
#> [1] -0.5707835

suppressMessages(library(microbenchmark))
microbenchmark(
  NM = NM(f, fPrime, fDoublePrime, 0)$maximizer,
  Bisection = bisection_cpp(fPrime, dXLeft = -1, dXRight = 1)$root,
  Optim = optim(0, f, method = "L-BFGS-B", lower = -1, upper = 1, control=list(fnscale=-1))$par,
  UniRoot = uniroot(fPrime, c(-1, 1))$root
)
#> Unit: microseconds
#>      expr   min    lq   mean median    uq   max neval
#>      NM  8.0   9.40 10.253  10.10 10.80  16.8   100

```

```
#> Bisection 20.8 22.40 36.307 25.60 27.35 1087.1 100
#>      Optim 36.3 40.10 53.750 43.00 49.15 478.6 100
#>      UniRoot 33.6 37.25 44.603 40.15 45.55 148.6 100
```

5. Create an R package that contains the functions from 2.) and 3.) and edit the title description to “This is my exam package”. Export the package as a bundled development version. *Remark: If you cannot solve 2.) or 3.) you can still earn points. Create a package that contains an R and a C++ function with single scalar inputs that always return the number 5.*

20 Ordinary exam 2021

20.1 Problem 1:

Begin by setting the seed to 134.

1. Use the inversion method to simulate data from a Cauchy distribution with scale-parameter $\sigma = 2$. The CDF is given by

$$F(X) = \frac{1}{\pi} \tan^{-1} \left(\frac{X}{\sigma} \right) + \frac{1}{2}$$

and we wish to simulate $N = 5000$ samples of length $T = 1000$. Write a function, `fn_slowCauchy`, which uses a nested for-loop to loop over N and T and transforms one $U(0,1)$ variable at a time. The function must output a $T \times N$ matrix `mX`, and must be populated column-wise.

Solution:

```
set.seed(134)

fn_slowCauchy <- function(sigma, samples, length) {
  mX <- matrix(0, nrow = length, ncol = samples)
  U <- runif(samples * length)
  mU <- matrix(U, nrow = length, ncol = samples)
  for (i in 1:samples) {
    for (j in 1:length) {
      mX[j, i] <- tan(pi * (mU[j, i] - 1/2)) * sigma
    }
  }
  return(mX)
}

mX <- fn_slowCauchy(2, 5000, 1000)
```

2. You now realize that your function is very inefficient, and you decide to write a different one. Your new function, `fn_fastCauchy`, must be vectorized and generate a $T \times N$ matrix `mY` which is identical to `mX`. It is up to you to write it in whichever way you believe is

most efficient (while still using the inversion method). Verify that `mX` and `mY` are identical with the `all.equal` function. (Hint: remember to re-set the seed before calling each of your functions).

Solution:

```
set.seed(134)

fn_fastCauchy <- function(sigma, samples, length) {
  U <- runif(samples * length)
  vCauchy <- tan(pi * (U - 1/2)) * sigma
  mY <- matrix(vCauchy, length, samples, byrow = FALSE)
  return(mY)
}

mY <- fn_fastCauchy(2, 5000, 1000)

all.equal(mX, mY)
#> [1] TRUE
```

3. We can estimate the scale-parameter of our Cauchy random variables as

$$\hat{\sigma} = \text{median}(|X|)$$

Write a function, `fn_estimateScale`, which estimates $\hat{\sigma}$ for each column in `mX`. Save the resulting 5000 estimates in a vector `vSigma`. **However**, your function must NOT use the built-in functions `abs` and `median`. You must compute the median of the absolute value of each column without using those two functions.

Solution:

```
fn_estimateScale <- function(mX) {
  calc_scale <- function(vX) {
    vY <- ifelse(vX < 0, vX * -1, vX)
    vY <- sort(vY, decreasing = FALSE)
    return(vY[ceiling(length(vY) / 2)])
  }
  return(apply(mX, 2, calc_scale))
}

vSigma <- fn_estimateScale(mX)
```

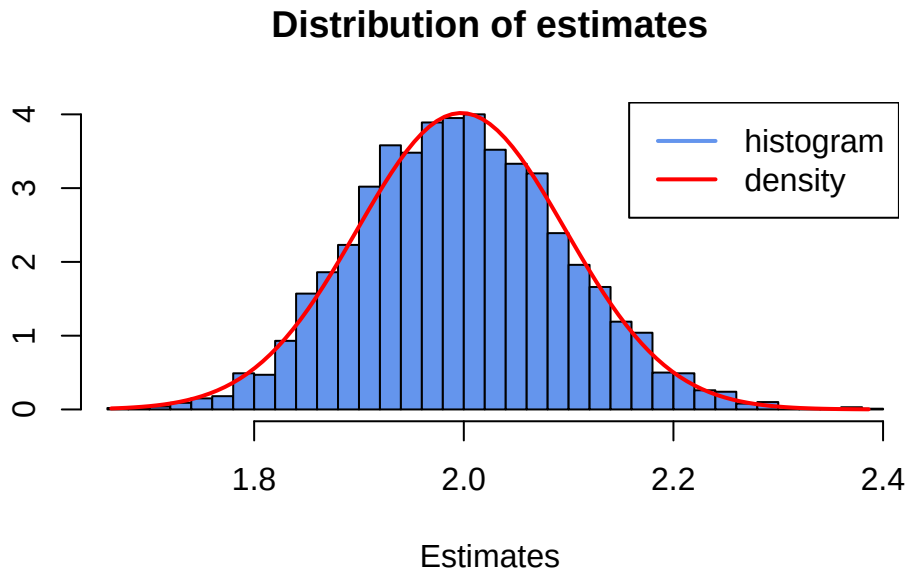
4. Create a histogram of the estimates $\hat{\sigma}$ contained in the vector `vSigma`. Set `breaks = 50` and `freq = F`. Let the title of the plot be “Distribution of estimates” and set the

label on the x-axis to “Estimates”. Add a density plot to the histogram you have just created. Assume $\hat{\sigma}$ is Gaussian and use the empirical mean and standard deviation as the parameters. Superimpose the density plot on the histogram with `lines()`.

```
hist(vSigma,
     freq = F,
     breaks = 50,
     col = "cornflowerblue",
     xlab = "Estimates",
     ylab = "",
     main = "Distribution of estimates",
     xlim = c(min(vSigma), max(vSigma)))

curve(dnorm(x, mean(vSigma), sd(vSigma)),
     from = min(vSigma),
     to = max(vSigma),
     col = "red",
     lwd = 2,
     add = TRUE)

legend("topright", legend = c("histogram", "density"), col = c("cornflowerblue", "red"), lwd
```



5. You wish to find the maximum of a 3rd order polynomial given by

$$g(x) = -0.2x^3 + 3x^2 + 5x - 3$$

The method you have chosen is an iterative scheme which proceeds by the following algorithm

- (a) Begin at $x_0 = 0$
- (b) Set $x_n = x_{n-1} + \frac{\text{sign}(g'(x_{n-1}))}{\sqrt{n}}$
- (c) Repeat until $|g'(x_n)| < \varepsilon$ with $\varepsilon = 0.00001$ where g' is the derivative of g wrt. x . Write a function, `fn_findMax`, that implements the algorithm and returns a List with the final value of x_n , the maximum value of $g(\cdot)$ in that point, as well as the number of iterations it took to get there.

```
g <- function(x) {
  return(-0.2 * x^3 + 3 * x^2 + 5 * x - 3)
}

gPrime <- function(x) {
  return(-0.6 * x^2 + 6 * x + 5)
}

fn_findMax <- function(f, gr, dX0 = 0, dTol = 0.00001, max.iter = 200) {
  dXn <- dX0 + sign(gr(dX0)) / sqrt(1)
  n <- 1
  while (abs(gr(dXn)) > dTol & n < max.iter) {
    dXn <- dXn + sign(gr(dXn)) / sqrt(n + 1)
    n <- n + 1
  }

  if (n == max.iter) {
    return(list(
      param = dXn,
      max.val = f(dXn),
      num.iter = n,
      msg = "Maximum number of iterations reached. Stopping..."
    ))
  } else {
    return(list(
      param = dXn,
      max.val = f(dXn),
      num.iter = n,
      msg = "Convergence reached."
    ))
  }
}
```

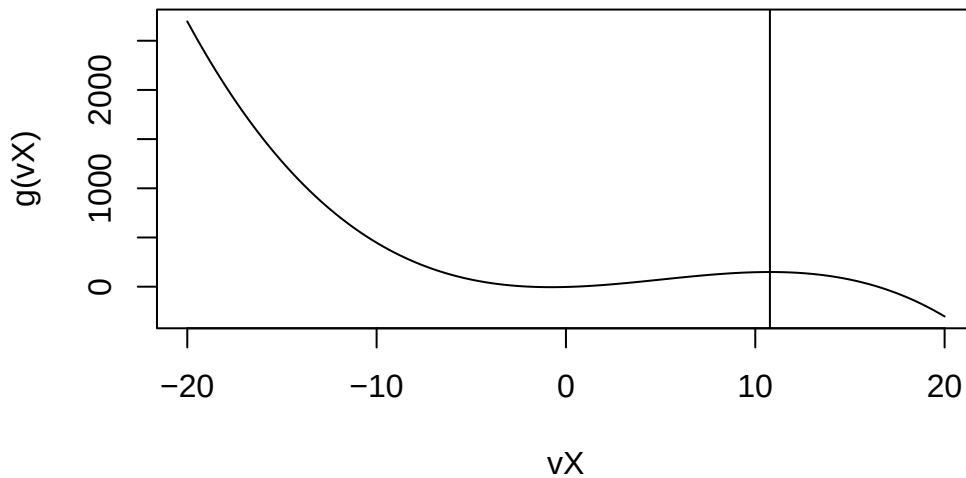
```

    ))
  }
}

lSolution <- fn_findMax(g, gPrime)
lSolution
#> $param
#> [1] 10.7735
#>
#> $max.val
#> [1] 148.98
#>
#> $num.iter
#> [1] 101
#>
#> $msg
#> [1] "Convergence reached."

vX <- seq(-20, 20, by = 0.05)
plot(vX, g(vX), type = "l")
abline(v = lSolution$param, col = "black")

```



20.2 Problem 2: (Constrained) Optimization, C++, and Packaging

Note: This problem is best solved in order from 1 - 7.

In this problem, you are supposed to work with the `mDATA.r` file provided with the exam. It contains the daily stock returns for the stocks of two Danish companies, Danske Bank and Ørsted, from 9th of June 2016 to 9th of March 2021 traded on the Copenhagen Stock Exchange. In the following, we denote $r_{A,t}$ as the return of Danske Bank and $r_{B,t}$ as the return of Ørsted at time period t .

1. Load the data file `mDATA.r` into your R workspace using `readRDS()`.

```
set.seed(123)
genData <- function() {
  # --- Generate mDATA.r ---
  set.seed(123) # for reproducibility

  # True parameters for data generation
  T_obs <- 1200 # Number of observations (approx 4.75 years of daily data)
  # GARCH parameters for stock A
  omega_A_true <- 0.00001
  alpha_A_true <- 0.08
  beta_A_true <- 0.90
  # GARCH parameters for stock B
  omega_B_true <- 0.000015
  alpha_B_true <- 0.07
  beta_B_true <- 0.91
  # Constant conditional correlation
  rho_true <- 0.6

  # Initialize vectors
  r_A <- numeric(T_obs)
  r_B <- numeric(T_obs)
  sigma2_A_t <- numeric(T_obs)
  sigma2_B_t <- numeric(T_obs)

  # Initial unconditional variances
  sigma2_A_t[1] <- omega_A_true / (1 - alpha_A_true - beta_A_true)
  sigma2_B_t[1] <- omega_B_true / (1 - alpha_B_true - beta_B_true)

  # Simulate GARCH processes and returns
  # Generate correlated normal innovations
  innovations <- matrix(rnorm(2 * T_obs), ncol = 2)
```

```

chol_R <- matrix(c(1, rho_true, rho_true, 1), nrow = 2)
chol_decomp <- chol(chol_R) # Cholesky decomposition of the correlation matrix
correlated_innovations <- innovations %*% chol_decomp

for (t in 1:T_obs) {
  # Generate returns
  r_A[t] <- sqrt(sigma2_A_t[t]) * correlated_innovations[t, 1]
  r_B[t] <- sqrt(sigma2_B_t[t]) * correlated_innovations[t, 2]

  # Update conditional variances for next period (if not the last observation)
  if (t < T_obs) {
    sigma2_A_t[t+1] <- omega_A_true + alpha_A_true * r_A[t]^2 + beta_A_true * sigma2_A_t[t]
    sigma2_B_t[t+1] <- omega_B_true + alpha_B_true * r_B[t]^2 + beta_B_true * sigma2_B_t[t]
  }
}

# Create the data matrix
simulated_mDATA <- cbind(r_A, r_B)
colnames(simulated_mDATA) <- c("DanskeBank", "Orsted")
return(simulated_mDATA)
}

mDATA <- genData()

```

You would like to model the joint risk of the two stocks. For that you choose to implement a multivariate constant conditional correlation GARCH (CCC-GARCH) model. In particular you assume that, for all periods $t = 1, \dots, T$, the two stock returns $(r_{A,t}, r_{B,t})$ follow a bivariate normal distribution conditional on the information from all previous periods (denoted as F_{t-1}), i.e

$$f(r_{A,t}, r_{B,t} | F_{t-1}) = \frac{1}{\sqrt{2\pi \det |\Sigma_t|}} \exp \left(-\frac{1}{2} (r_{A,t} \ r_{B,t}) \Sigma_t^{-1} (r_{A,t} \ r_{B,t})' \right)$$

$$\Sigma_t = \begin{pmatrix} \sigma_{A,t}^2 & \rho \sigma_{A,t} \sigma_{B,t} \\ \rho \sigma_{A,t} \sigma_{B,t} & \sigma_{B,t}^2 \end{pmatrix}$$

with conditional variances each following a GARCH process

$$\sigma_{A,t}^2 = \omega_A + \alpha_A r_{A,t-1}^2 + \beta_A \sigma_{A,t-1}^2$$

$$\sigma_{B,t}^2 = \omega_B + \alpha_B r_{B,t-1}^2 + \beta_B \sigma_{B,t-1}^2.$$

Their joint likelihood function is then given by

$$L(\omega_A, \alpha_A, \beta_A, \omega_B, \alpha_B, \beta_B, \rho) = \prod_{t=1}^T f(r_{A,t}, r_{B,t} | F_{t-1}).$$

- Write an R function that returns the average negative log-likelihood function for the model above for periods $t = 2, 3, \dots, T$. It should take a vector of parameters as first input and a matrix of dimension $[T \times 2]$ as second input. It should return a warning if the matrix has a column dimension different from 2. *Hint: Initialize both GARCH processes at $t = 1$ with $\sigma_{A,1}^2 = \omega_A / (1 - \alpha_A - \beta_A)$ and $\sigma_{B,1}^2 = \omega_B / (1 - \alpha_B - \beta_B)$. You can use `det()` and `solve()` to calculate the determinant and the inverse of a matrix in R.*

Solution:

```
fAvgNegLogLikGarch <- function(vParams, mInput) {
  if (dim(mInput)[2] != 2) {
    warning("Incorrect number of dimensions in input matrix")
    return(NULL)
  }

  dOmegaA <- vParams[1]
  dAlphaA <- vParams[2]
  dBetaA <- vParams[3]
  dOmegaB <- vParams[4]
  dAlphaB <- vParams[5]
  dBetaB <- vParams[6]
  dRho <- vParams[7]

  dT <- nrow(mInput)

  mSigma2 <- matrix(0, nrow(mInput), 2)
  mSigma2[1, 1] <- dOmegaA / (1 - dAlphaA - dBetaA)
  mSigma2[1, 2] <- dOmegaB / (1 - dAlphaB - dBetaB)

  dOut <- 0

  for (t in 2:dT) {
    mSigma2[t, 1] <- dOmegaA + dAlphaA * mInput[t - 1, 1]^2 + dBetaA * mSigma2[t - 1, 1]
    mSigma2[t, 2] <- dOmegaB + dAlphaB * mInput[t - 1, 2]^2 + dBetaB * mSigma2[t - 1, 2]
    mSigmaCov <- matrix(c(mSigma2[t, 1], rep(dRho * sqrt(mSigma2[t, 1]) * sqrt(mSigma2[t, 2]), 2)), 2, 2)
    dOut <- dOut - log(sqrt(2 * pi * det(mSigmaCov))) - 0.5 * as.numeric(t(mInput[t, ]) %*% mSigmaCov)
  }

  return(-dOut / (dT - 1))
}
```

- Use the BFGS algorithm with `optim()` to find the maximizers of the log-likelihood

function in 2). Use the following starting values:

$$\begin{aligned}\omega_A &= s_A^2 \times 0.05, & \alpha_A &= 0.05, & \beta_A &= 0.90, \\ \omega_B &= s_B^2 \times 0.05, & \alpha_B &= 0.05, & \beta_B &= 0.90, \\ \rho &= 0\end{aligned}$$

where s_A^2 and s_B^2 are the sample standard deviations of the return series A and B respectively over all time periods. Did your algorithm converge? If not, briefly describe a potential source of the problem.

```
vParams <- c(sd(mDATA[, 1]) * 0.05, 0.05, 0.9, sd(mDATA[, 2]) * 0.05, 0.05, 0.9, 0)

optim(vParams, fAvgNegLogLikGarch, mInput = mDATA, method = "BFGS")

# optim cannot converge if it decides to explore negative values, since we cannot take the s
```

4. Write an R function that returns the average negative log-likelihood function for a reparameterized CCC-GARCH with same input types as in 2). The reparameterization should impose the following constraints on the original parameters of the likelihood function:

$$\begin{aligned}\omega_A &> 0, & \alpha_A &\in (0, 1), & \beta_A &\in (0, 1), \\ \omega_B &> 0, & \alpha_B &\in (0, 1), & \beta_B &\in (0, 1), \\ \rho &\in (-1, 1)\end{aligned}$$

Solution:

```
fAvgNegLogLikGarchReparam <- function(vParams, mInput) {
  if (dim(mInput)[2] != 2) {
    warning("Incorrect number of dimensions in input matrix")
    return(NULL)
  }

  dOmegaA <- exp(vParams[1])
  dAlphaA <- exp(vParams[2]) / (1 + exp(vParams[2]) + exp(vParams[3]))
  dBetaA <- exp(vParams[3]) / (1 + exp(vParams[2]) + exp(vParams[3]))
  dOmegaB <- exp(vParams[4])
  dAlphaB <- exp(vParams[5]) / (1 + exp(vParams[5]) + exp(vParams[6]))
  dBetaB <- exp(vParams[6]) / (1 + exp(vParams[5]) + exp(vParams[6]))
  dRho <- -1 + 2 * (exp(vParams[7])) / (1 + exp(vParams[7]))

  dT <- nrow(mInput)
```

```

mSigma2 <- matrix(0, nrow(mInput), 2)
mSigma2[1, 1] <- dOmegaA / (1 - dAlphaA - dBetaA)
mSigma2[1, 2] <- dOmegaB / (1 - dAlphaB - dBetaB)

dOut <- 0

for (t in 2:dT) {
  mSigma2[t, 1] <- dOmegaA + dAlphaA * mInput[t - 1, 1]^2 + dBetaA * mSigma2[t - 1, 1]
  mSigma2[t, 2] <- dOmegaB + dAlphaB * mInput[t - 1, 2]^2 + dBetaB * mSigma2[t - 1, 2]
  mSigmaCov <- matrix(c(mSigma2[t, 1], rep(dRho * sqrt(mSigma2[t, 1]) * sqrt(mSigma2[t, 2]),
  dOut <- dOut - log(sqrt(2 * pi * det(mSigmaCov))) - 0.5 * as.numeric(t(mInput[t, ])%*%
}

return(-dOut / (dT - 1))
}

```

5. Use the BFGS algorithm with `optim()` to find the maximizers of the reparameterized log-likelihood function. Use starting values that correspond to the starting values of the original parameterization in 3). What are your estimates of the transformed parameters?

Solution:

```

vParams <- c(sd(mDATA[, 1]) * 0.05, 0.05, 0.9, sd(mDATA[, 2]) * 0.05, 0.05, 0.9, 0)
# transform
vParams[1] <- log(vParams[1])
vParams[2] <- log(vParams[2] / (1 - vParams[2] - vParams[3]))
vParams[3] <- log(vParams[3] / (1 - vParams[2] - vParams[3]))
vParams[4] <- log(vParams[4])
vParams[5] <- log(vParams[5] / (1 - vParams[5] - vParams[6]))
vParams[6] <- log(vParams[6] / (1 - vParams[5] - vParams[6]))
vParams[7] <- log(((vParams[7] + 1) / 2.0) / (1 - ((vParams[7] + 1) / 2.0)))

optim_results <- optim(vParams, fAvgNegLogLikGarchReparam, mInput = mData, method = "BFGS")
optim_results
#> $par
#> [1] -10.9045231  0.3523736  3.0242257 -11.6615536  1.7656300  4.3434653
#> [7]  1.3882186
#>
#> $value
#> [1] -5.925866
#>
#> $counts

```

```

#> function gradient
#>      100      98
#>
#> $convergence
#> [1] 0
#>
#> $message
#> NULL

vParams <- optim_results$par
dOmegaA <- exp(vParams[1])
dAlphaA <- exp(vParams[2]) / (1 + exp(vParams[2]) + exp(vParams[3]))
dBetaA <- exp(vParams[3]) / (1 + exp(vParams[2]) + exp(vParams[3]))
dOmegaB <- exp(vParams[4])
dAlphaB <- exp(vParams[5]) / (1 + exp(vParams[5]) + exp(vParams[6]))
dBetaB <- exp(vParams[6]) / (1 + exp(vParams[5]) + exp(vParams[6]))
dRho <- -1 + 2 * (exp(vParams[7])) / (1 + exp(vParams[7]))

print(paste0("Est. OmegaA: ", dOmegaA, " vs act.: ", 0.0001))
#> [1] "Est. OmegaA: 1.83749335729341e-05 vs act.: 1e-04"
print(paste0("Est. AlphaA: ", dAlphaA, " vs act.: ", 0.08))
#> [1] "Est. AlphaA: 0.0618438534243581 vs act.: 0.08"
print(paste0("Est. BetaA: ", dBetaA, " vs act.: ", 0.90))
#> [1] "Est. BetaA: 0.894678841062443 vs act.: 0.9"
print(paste0("Est. OmegaB: ", dOmegaB, " vs act.: ", 0.000015))
#> [1] "Est. OmegaB: 8.61889557714679e-06 vs act.: 1.5e-05"
print(paste0("Est. AlphaB: ", dAlphaB, " vs act.: ", 0.07))
#> [1] "Est. AlphaB: 0.0697365650771858 vs act.: 0.07"
print(paste0("Est. BetaB: ", dBetaB, " vs act.: ", 0.91))
#> [1] "Est. BetaB: 0.918332975052663 vs act.: 0.91"
print(paste0("Est. Rho: ", dRho, " vs act.: ", 0.6))
#> [1] "Est. Rho: 0.600615406946657 vs act.: 0.6"

```

6. Write a C++ function using Rcpp and RcppArmadillo (optional) that can be loaded with `sourceCpp()`. It should take a numerical vector input corresponding to the parameters of the reparameterized likelihood function input and return a numerical vector that corresponds to the parameters of the original likelihood function. Apply it to your results in 5). What is your estimate for ρ ?

Solution:


```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>

using namespace arma;
using namespace Rcpp;

// [[Rcpp::export]]
vec transToOrig_cpp(vec vInput) {
  vec vOut = zeros<vec>(vInput.size());
  vOut[0] = exp(vInput[0]);
  vOut[1] = exp(vInput[1]) / (1 + exp(vInput[1]) + exp(vInput[2]));
  vOut[2] = exp(vInput[2]) / (1 + exp(vInput[1]) + exp(vInput[2]));
  vOut[3] = exp(vInput[3]);
  vOut[4] = exp(vInput[4]) / (1 + exp(vInput[4]) + exp(vInput[5]));
  vOut[5] = exp(vInput[5]) / (1 + exp(vInput[4]) + exp(vInput[5]));
  vOut[6] = -1 + 2 * (exp(vInput[6])) / (1 + exp(vInput[6]));

  return vOut;
}
```

```
suppressMessages(library(Rcpp))
#> Warning: pakke 'Rcpp' blev bygget under R version 4.3.3
suppressMessages(library(RcppArmadillo))
#> Warning: pakke 'RcppArmadillo' blev bygget under R version 4.3.3
sourceCpp("exam2021cpp.cpp")

vNew <- transToOrig_cpp(vParams)
vNew[7]
#> [1] 0.6006154
```

7. Create an R package that contains the functions from 4.) and 6.) and edit the title description to “This is my exam package”. Export the package as a bundled development version. *Remark: If you cannot solve 4.) or 6.) create a package that contains an R and a C++ function with single scalar inputs that always return the number 5.*

21 Retake exam 2021

21.1 Problem 1:

Begin by setting the seed to 134.

1. Use the Acceptance-Rejection method with an $\text{Exp}(\lambda)$ envelope to simulate $T = 10,000$ draws from a $\text{Gamma}(k, \theta)$ distribution with parameters $(k, \theta) = (3, 3)$. You may set $\lambda = \frac{\theta}{k}$ and

$$c = \frac{k^k e^{-(k-1)}}{\Gamma(k)}$$

where Γ is the gamma function. The density function of the Gamma distribution is

$$f(x) = \frac{\theta^k}{\Gamma(k)} x^{k-1} e^{-\theta x}, \quad x > 0$$

while for the Exponential distribution it is

$$g(y; \lambda) = \lambda e^{-\lambda y}, \quad y > 0.$$

Use the inversion method to generate the Exponential random variables. The corresponding CDF is

$$G(y) = 1 - e^{-\lambda y}.$$

Store your Gamma random variables in a vector `vW`.

Solution:

```
set.seed(134)

### Auxiliary functions ###

gamma.pdf <- function(x, k, theta) {
  return(theta^k * x^(k-1) * exp(-theta*x)/gamma(k))
}

exponential.pdf <- function(x, lambda) {
  return(lambda * exp(-lambda*x))
}
```

```

}

Exponential.Simulate <- function(lambda, size = 1) {
  V <- runif(size)
  return(-1/lambda * log(V))
}

Gamma.Simulate <- function(k, theta, size = 1) {

  lambda <- theta/k
  c <- k^k * exp(-k+1) / gamma(k)

  U <- rep(NA, size)
  Y <- rep(NA, size)
  X <- rep(NA, size)
  Unaccepted <- rep(TRUE, size)

  while (any(Unaccepted)) {

    UnacceptedCount <- sum(Unaccepted)

    U <- runif(UnacceptedCount)
    Y <- Exponential.Simulate(lambda, UnacceptedCount)

    Accepted_ThisTime <- Unaccepted[Unaccepted] &
      ( U <= ( gamma.pdf(Y, k, theta) / exponential.pdf(Y, lambda)/c ) )

    X[Unaccepted][Accepted_ThisTime] <- Y[Accepted_ThisTime]
    Unaccepted[Unaccepted] <- !Accepted_ThisTime

  }

  return(X)
}

vW <- Gamma.Simulate(k = 3, theta = 3, size = 10000)

```

2. Use the Box-Muller algorithm to simulate a sample of length $T = 10,000$ from a Normal(0,1) distribution. Store your Normal random variables in a vector `vZ`.

Solution:

```
BoxMuller <- function(size = 1) {

  U <- runif(size)
  V <- runif(size)

  X <- sqrt(-2*log(U)) * cos(2*pi*V)
  Y <- sqrt(-2*log(U)) * sin(2*pi*V)

  return(c(X,Y))

}

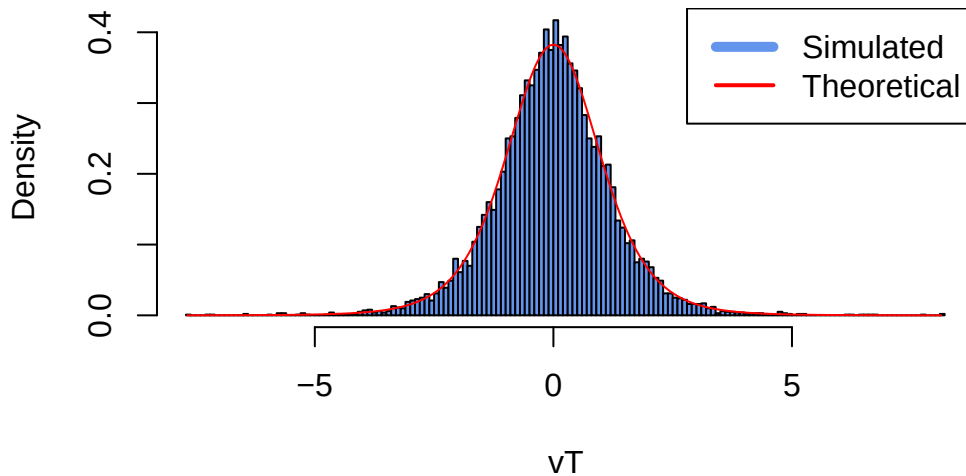
vZ <- BoxMuller(10000/2)
```

3. If $Z \sim \text{Normal}(0, 1)$ and $W \sim \text{Gamma}(\frac{\nu}{2}, \frac{1}{2}\nu)$ then $\frac{Z}{\sqrt{W}} \sim t(\nu)$. Use this result and the vectors `vZ` and `vW` generated in problems (1) and (2) to create 10,000 random variables from a Student's t distribution with ν degrees of freedom. Store these 10,000 draws in a vector `vT`. Then, create a histogram of `vT` and superimpose the theoretical density in red. Set `breaks = 200` in your call to `hist()`.

```
vT <- 1 / sqrt(vW) * vZ

hist(vT, breaks = 200, freq = FALSE,
     main = "",
     col = "cornflowerblue",
     xlim = c(min(vT), max(vT)))

xticks = seq(min(vT), max(vT), 0.1)
lines(xticks, dt(xticks, 6), col = "red")
legend("topright", legend = c("Simulated", "Theoretical"), lty = c(1, 1), lwd = c(5, 2), col
```



4. Write a function which takes three inputs: `iN`, `iA` and `iB`. The purpose of the function is to extract those integers smaller than or equal to `iN` that have `iA` *or* `iB` as factors. The function must return a `list` which contains two elements - a vector with the extracted numbers, and an integer which is the sum of this vector. Set `iN = 300`, `iA = 11` and `iB = 42` when you run your function. As an example, for `iN = 10`, `iA = 3` and `iB = 5` the extracted numbers are (3, 5, 6, 9, 10) and their sum is 33.

Solution:

```
f <- function(iN, iA, iB) {
  vX <- seq(1:iN)
  vOut <- vX[vX %% iA == 0 | vX %% iB == 0]
  return(list(
    numbers = vOut,
    sumofnumbers = sum(vOut)
  ))
}

f(iN = 300, iA = 11, iB = 42)
#> $numbers
#> [1] 11 22 33 42 44 55 66 77 84 88 99 110 121 126 132 143 154 165 168
#> [20] 176 187 198 209 210 220 231 242 252 253 264 275 286 294 297
#>
```

```
#> $sumofnumbers
#> [1] 5334
```

5. Write a function which returns the smallest natural number which is evenly divisible by all positive integers contained in a vector `vX`. Use this function to find the smallest number divisible by `vX = seq(1, 15)`. As an example, the smallest number which is evenly divisible by the vector `vX = (3, 5, 7)` is 105.

Solution:

```
f <- function(vX) {
  bFound <- FALSE
  dOut <- NULL
  i <- 1
  while (bFound == FALSE) {
    if (sum(i %% vX) == 0) {
      dOut <- i
      bFound <- TRUE
    }
    i <- i + 1
  }
  return(dOut)
}

f(c(3, 5, 7))
#> [1] 105
f(seq(1, 15))
#> [1] 360360
```

21.2 Problem 2: (Constrained) Optimization, C++, and Packaging

Note: This problem is best solved in order from 1 - 5.

In this problem, you are supposed to work with the `mDATA_retake.r` file provided with the exam. It contains the daily stock returns for the stocks of Danske Bank from 9th of June 2016 til 9th of March 2021 traded on the Copenhagen Stock Exchange. In the following, we denote r_t as the return of Danske Bank at time period t .

1. Load the data file `mDATA_retake.r` into your R workspace using `readRDS()`.

Solution:

```

set.seed(123)
genData <- function() {
  # --- Generate mData.r ---
  set.seed(123) # for reproducibility

  # True parameters for data generation
  T_obs <- 1200 # Number of observations (approx 4.75 years of daily data)
  # GARCH parameters for stock A
  omega_A_true <- 0.000001
  alpha_A_true <- 0.08
  beta_A_true <- 0.9
  # GARCH parameters for stock B
  omega_B_true <- 0.000001
  alpha_B_true <- 0.09
  beta_B_true <- 0.91
  # Constant conditional correlation
  rho_true <- 0.6

  # Initialize vectors
  r_A <- numeric(T_obs)
  r_B <- numeric(T_obs)
  sigma2_A_t <- numeric(T_obs)
  sigma2_B_t <- numeric(T_obs)

  # Initial unconditional variances
  sigma2_A_t[1] <- omega_A_true / (1 - alpha_A_true - beta_A_true)
  sigma2_B_t[1] <- omega_B_true / (1 - alpha_B_true - beta_B_true)

  # Simulate GARCH processes and returns
  # Generate correlated normal innovations
  innovations <- matrix(rnorm(2 * T_obs), ncol = 2)
  chol_R <- matrix(c(1, rho_true, rho_true, 1), nrow = 2)
  chol_decomp <- chol(chol_R) # Cholesky decomposition of the correlation matrix
  correlated_innovations <- innovations %*% chol_decomp

  for (t in 1:T_obs) {
    # Generate returns
    r_A[t] <- sqrt(sigma2_A_t[t]) * correlated_innovations[t, 1]
    r_B[t] <- sqrt(sigma2_B_t[t]) * correlated_innovations[t, 2]

    # Update conditional variances for next period (if not the last observation)
    if (t < T_obs) {

```

```

    sigma2_A_t[t+1] <- omega_A_true + alpha_A_true * r_A[t]^2 + beta_A_true * sigma2_A_t
    sigma2_B_t[t+1] <- omega_B_true + alpha_B_true * r_B[t]^2 + beta_B_true * sigma2_B_t
  }
}

# Create the data matrix
simulated_mDATA <- cbind(r_A, r_B)
colnames(simulated_mDATA) <- c("DanskeBank", "Orsted")
return(simulated_mDATA)
}

mDATA <- genData()

```

You want to model the volatility of the stock. You assume that, for all periods $t = 1, \dots, T$, the stock return r_t are generated by a location scale model

$$r_t = \mu + \varepsilon_t \sigma_t$$

with conditional variances each following a zero drift GARCH process

$$\sigma_t^2 = \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2$$

and ε_t following a t -distribution with ν degrees of freedom conditional on the information from all previous periods (denoted as F_{t-1}), i.e

$$f(\varepsilon_t | F_{t-1}) = \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\Gamma(\frac{\nu}{2})} \left(1 + \frac{\varepsilon_t^2}{\nu}\right)^{-\frac{\nu+1}{2}}.$$

The likelihood functions of the returns is then given by

$$L(\mu, \alpha, \beta, \nu) = \prod_{t=1}^T f(r_t | F_{t-1}).$$

2. Write an R function that returns the average **negative** log-likelihood function of a reparameterized version of the model above for periods $t = 2, 3, \dots, T$. It should
 - (a) take a vector of parameters as first input and a return series of length T as second input.
 - (b) always fulfill the following constraints $\alpha + \beta = 1$, $0 < \alpha < 1$, and $\nu > 2$. *Hint: Initialize the conditional variance processes at $t = 1$ with σ_1^2 equal to the unconditional variance of your returns. You can use `dt()` to calculate the density of a t -distributed random variable in R. Or you can use the formula above with `gamma()` for the Γ -function in R.*

Solution:


```
fAvgNegLogLik <- function(vParams, vInput) {
  dMu <- vParams[1]
  dAlpha <- exp(vParams[2]) / (1 + exp(vParams[2]))
  dBeta <- 1 - dAlpha
  dNu <- 2 + exp(vParams[4])

  dSum <- 0
  dT <- length(vInput)

  vSigma2 <- numeric(dT)
  vSigma2[1] <- var(vInput)

  for (t in 2:dT) {
    vSigma2[t] <- dAlpha * vInput[t - 1]^2 + dBeta * vSigma2[t-1]
    dSum <- dSum + log(dt((vInput[t] - dMu) / sqrt(vSigma2[t])), dNu)) - log(sqrt(vSigma2[t]))
  }

  return(-dSum / (dT - 1))
}
```

3. Use the BFGS algorithm with `optim()` to find the maximizers of the reparameterized log-likelihood function in 2). Use starting values that **correspond** to the original likelihood parameters

$$\mu = 0, \alpha = 0.90, \beta = 0.10, \nu = 5.$$

Retransform your results. What are the estimates for $(\mu, \alpha, \beta, \nu)$?

```
vParams <- c(0, log(0.9/(1-0.9)), 1 - log(0.9/(1-0.9)), log(5 - 2))
optim_results <- optim(vParams, fAvgNegLogLik, vInput = mData[, 1], method = "BFGS")
optim_results
#> $par
#> [1] 0.0001021925 -2.9909391286 -1.1972245773 6.1836752296
#>
#> $value
#> [1] -3.631868
#>
#> $counts
#> function gradient
#> 54 24
#>
#> $convergence
#> [1] 0
#>
```

```

#> $message
#> NULL
vParams <- optim_results$par

# retransformation
dMu_star <- vParams[1]
dAlpha_star <- exp(vParams[2]) / (1 + exp(vParams[2]))
dBeta_star <- 1 - dAlpha_star
dNu_star <- 2 + exp(vParams[4])

print(paste0("Est. mu: ", dMu_star))
#> [1] "Est. mu: 0.000102192473262923"
print(paste0("Est. alpha: ", dAlpha_star, " vs. true: ", 0.08))
#> [1] "Est. alpha: 0.0478368957544601 vs. true: 0.08"
print(paste0("Est. beta: ", dBeta_star, " vs. true: ", 0.90))
#> [1] "Est. beta: 0.95216310424554 vs. true: 0.9"
print(paste0("Est. nu: ", dNu_star))
#> [1] "Est. nu: 486.770328670692"

```

4. Write a C++ function using Rcpp and RcppArmadillo that can be loaded via `sourceCpp()`. It should simulate conditional variances and returns from a zero mean GARCH(1,1) for $t = 1, \dots, T$:

$$r_t = \varepsilon_t \sigma_t$$

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

with errors ε_t coming from an arbitrary distribution function specified by the *user*. The function should have the following inputs

- (a) The number of periods (default = 1)
- (b) A numerical vector of GARCH parameters (ω, α, β)
- (c) An R function that simulates random variables from a specific distribution (such as e.g. `rnorm()` or `rt()`)
- (d) An numerical vector of inputs for the optional parameters of input (c) (default = 0)

The function should return an `arma::List` object with two vectors containing i) conditional variances for all time periods and ii) simulated return series. Use your function to simulate 1000 draws from a GARCH(1,1) process with t-distributed errors with 5 degrees of freedom. Plot the volatilities. *Hint: If you would like to convert a one-dimensional SEXP object with numerical values in C++ to use it as a double for standard calculations, you can use `REAL()` in C++.* Example:

```

SEXP a = myFunction(input);
double x = *REAL(a);
double y = 2*x + pow(x,2);

// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>

using namespace arma;
using namespace Rcpp;

// Helper function to get a shock
inline double get_shock(Function f, const vec& vInputs) {
  SEXP s;
  int n_extra_args = vInputs.n_elem;

  if (n_extra_args == 0) { // e.g. f takes only n, like a pre-wrapped user function or rnorm
    s = f(1);
  } else if (n_extra_args == 1) { // e.g. rt(n, df) or rnorm(n, mean)
    s = f(1, vInputs[0]);
  } else if (n_extra_args == 2) { // e.g. rnorm(n, mean, sd)
    s = f(1, vInputs[0], vInputs[1]);
  } else {
    // Fallback or error for more arguments, or could extend this if/else
    Rcpp::stop("vInputs has too many elements for this simplified handler. Max 2 supported.");
  }
  if (TYPEOF(s) != REALSXP || Rf_length(s) != 1) {
    Rcpp::stop("The user-supplied function 'f' did not return a single numeric value.");
  }
  return Rcpp::as<double>(s);
}

// [[Rcpp::export]]
List GarchSim(int iT_in, vec vParams, Function f, vec vInputs) {
  int iT = iT_in;
  if (iT <= 0) { iT = 1; } // Simplified default handling

  double dOmega = vParams[0];
  double dAlpha = vParams[1];
  double dBeta = vParams[2];

  vec vY(iT, fill::zeros);

```

```

vec vSigma2(iT, fill::zeros);

if (dOmega <= 0 || dAlpha < 0 || dBeta < 0 || (dAlpha + dBeta) >= 1.0) {
  Rcpp::stop("Invalid GARCH parameters.");
}

double unconditional_variance = dOmega / (1.0 - dAlpha - dBeta);
vSigma2(0) = unconditional_variance;

double current_eps = get_shock(f, vInputs); // Call helper
vY(0) = sqrt(vSigma2(0)) * current_eps;

for (int t = 1; t < iT; t++) {
  vSigma2(t) = dOmega + dAlpha * pow(current_eps, 2) + dBeta * vSigma2(t-1);
  if (vSigma2(t) <= 0) vSigma2(t) = unconditional_variance;

  current_eps = get_shock(f, vInputs); // Call helper
  vY(t) = sqrt(vSigma2(t)) * current_eps;
}

return List::create(_["vSigma2"] = vSigma2, _["vY"] = vY);
}

```

```

suppressMessages(library(Rcpp))
#> Warning: pakke 'Rcpp' blev bygget under R version 4.3.3
suppressMessages(library(RcppArmadillo))
#> Warning: pakke 'RcppArmadillo' blev bygget under R version 4.3.3

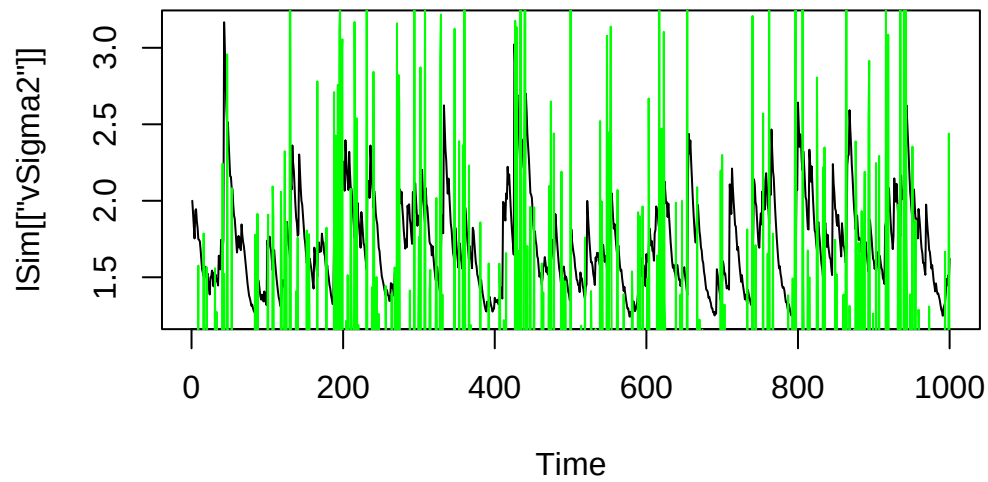
sourceCpp("exam2021recpp.cpp")

iT <- 1000
dOmega <- 0.1
dAlpha <- 0.05
dBeta <- 0.9
df_t <- 5

lSim <- GarchSim(iT, c(dOmega, dAlpha, dBeta), f = rt, vInputs = df_t)

plot(1:1000, lSim[["vSigma2"]], type = "l", lty = 1, xlab = "Time")
lines(1:1000, lSim[["vY"]], col="green")

```



5. Create an R package that contains the functions from 1.) and 4.) and edit the title description to “This is my exam package”. Export the package as a bundled development version. *Remark: If you cannot solve 1.) or 4.) create a package that contains an R and a C++ function with single scalar inputs that always return the number 5.*

22 Ordinary exam 2022

22.1 Problem 1:

Begin by setting the seed to 123.

1. Use the Acceptance-Rejection method with envelopes $\text{Exp}(\lambda_1)$ and $\text{Exp}(\lambda_2)$ to simulate data from two gamma distributions $\text{Gamma}(k_1, \theta)$ with parameters $(k_1, \theta) = (3, 3)$ and $\text{Gamma}(k_2, \theta)$ with parameters $(k_2, \theta) = (1, 3)$. You may set $\lambda_1 = \frac{\theta}{k_1}$, $\lambda_2 = \frac{\theta}{k_2}$ and

$$c_1 = \frac{k_1^{k_1} e^{-(k_1-1)}}{\Gamma(k_1)}$$

$$c_2 = \frac{k_2^{k_2} e^{-(k_2-1)}}{\Gamma(k_2)}$$

where Γ is the gamma function. We wish to simulate $N = 100$ samples each of length $T = 1000$. The density function of the Gamma distribution is

$$f(x) = \frac{\theta^{k_i}}{\Gamma(k_i)} x^{k_i-1} e^{-\theta x}, \quad x > 0, \quad i = 1, 2$$

while for the Exponential distribution it is

$$g(y; \lambda) = \lambda_i e^{-\lambda_i y}, \quad y > 0, \quad i = 1, 2.$$

Use the inversion method to generate the Exponential random variables. The corresponding CDF is

$$G(y) = 1 - e^{-\lambda_i y}, \quad i = 1, 2.$$

Store your Gamma random variables in vectors `vW1` and `vW2` of size $T \times N$ each.

Solution:

```
### Auxiliary functions ###  
  
gamma.pdf <- function(x, k, theta) {  
  return(theta^k * x^(k-1) * exp(-theta*x)/gamma(k))  
}
```

```

exponential.pdf <- function(x, lambda) {
  return(lambda * exp(-lambda*x))
}

Exponential.Simulate <- function(lambda, size = 1) {
  V <- runif(size)
  return(-1/lambda * log(V))
}

Gamma.Simulate <- function(k, theta, size = 1) {

  lambda <- theta/k
  c <- k^k * exp(-k+1) / gamma(k)

  U <- rep(NA, size)
  Y <- rep(NA, size)
  X <- rep(NA, size)
  Unaccepted <- rep(TRUE, size)

  while (any(Unaccepted)) {

    UnacceptedCount <- sum(Unaccepted)

    U <- runif(UnacceptedCount)
    Y <- Exponential.Simulate(lambda, UnacceptedCount)

    Accepted_ThisTime <- Unaccepted[Unaccepted] &
      ( U <= ( gamma.pdf(Y, k, theta) / exponential.pdf(Y, lambda)/c ) )

    X[Unaccepted][Accepted_ThisTime] <- Y[Accepted_ThisTime]
    Unaccepted[Unaccepted] <- !Accepted_ThisTime

  }

  return(X)
}

set.seed(123)

iN <- 100
iT <- 1000

```

```
vW1 <- matrix(Gamma.Simulate(3, 3, iN * iT), iT, iN)
vW2 <- matrix(Gamma.Simulate(1, 3, iN * iT), iT, iN)
```

2. Note that $B(k_1, k_2) = \frac{\text{Gamma}(k_1, \theta)}{\text{Gamma}(k_1, \theta) + \text{Gamma}(k_2, \theta)}$ follows Beta distribution. Use this result and the vectors **vW1** and **vW2** generated in the problem (1) to simulate $N = 100$ samples each of length $T = 1000$ of random variables from Beta distribution with the parameters k_1 and k_2 . Store these draws in a vector **vB** of size $T \times N$.

Solution:

```
vB <- matrix(vW1 / (vW1 + vW2), iT, iN)
```

3. The mean of the beta distribution is given as

$$\mu = \frac{k_1}{k_1 + k_2}$$

Write a function, **fn_estimateMean**, which provides estimates $\hat{\mu}$ for each column in **mB** by calculating its empirical mean and then subtract μ calculated with the formula above, i.e. calculate $\hat{\mu} - \mu$ for each column in **mB**. Save the resulting 100 estimates in a vector **vDiffMu**.

```
fn_estimateMean <- function(mInput) {
  mOut <- apply(mInput, 2, mean)
  return(mOut)
}

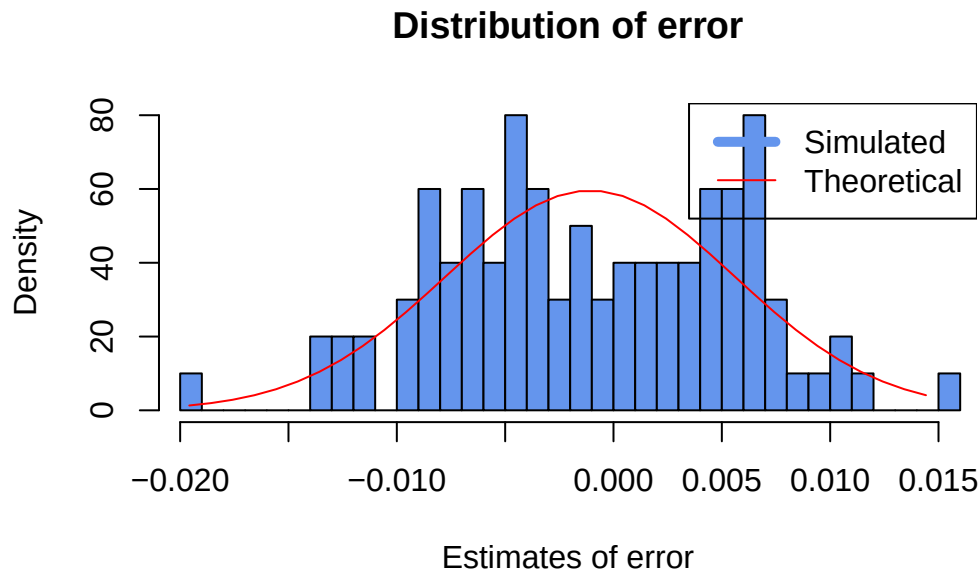
dU <- 3 / (1 + 3)
vDiffMu <- fn_estimateMean(vB) - dU
```

4. Create a histogram of the estimates contained in the vector **vDiffMu**. Let the title of the plot be “Distribution of error” and set the label on the x-axis to “Estimates of error”. Add a density plot to the histogram you have just created. Assume the error $\hat{\mu} - \mu$ is Gaussian and use it to calculate its empirical mean and standard deviation using the calculations contained in the vector **vDiffMu**. Superimpose the density plot on the histogram with **lines()** defined at these values of mean and standard deviation.

```
hist(vDiffMu, breaks = 25, freq = FALSE,
     main = "Distribution of error",
     col = "cornflowerblue",
     xlim = c(min(vDiffMu), max(vDiffMu)),
     xlab = "Estimates of error")
```



```
xticks = seq(min(vDiffMu), max(vDiffMu), 0.001)
lines(xticks, dnorm(xticks, mean(vDiffMu), sd(vDiffMu)), col = "red")
legend("topright", legend = c("Simulated", "Theoretical"), lty = c(1, 1), lwd = c(5, 1), col
```



22.2 Problem 2: (Constrained) Optimization, C++, and Packaging

Note: This problem is best solved in order from 1 - 5.

In this problem, you are supposed to work with the `vDATA.r` file provided with the exam. It contains the daily stock returns for the stocks of Danske Bank from 9 June 2016 to 9 March 2021 traded on the Copenhagen Stock Exchange. In the following, we denote r_t as the return of Danske Bank at time period t .

1. Load the data file `vDATA.r` into your R workspace using `readRDS()`.

```
# Ensure rugarch is installed and loaded
# install.packages("rugarch")
library(rugarch)
#> Warning: pakke 'rugarch' blev bygget under R version 4.3.3
#> Indlæser krævet pakke: parallel
#>
```

```

#> Vedhæfter pakke: 'rugarch'
#> Det følgende objekt er maskeret fra 'package:stats':
#>
#>      sigma

# --- True parameters for new data generation (MODIFIED FOR PERSISTENCE < 1) ---
set.seed(456) # Use a different seed
T_obs_new <- 1500 # Number of observations for the new series
mu_true_new <- 0.0002 # True mean
omega_true_new <- 0.000015 # True omega

# Modify alphas so their sum is slightly less than 1
alpha1_true_gen <- 0.65
alpha2_true_gen <- 0.349 # Sum = 0.65 + 0.349 = 0.999

cat("Generating data with persistence:", alpha1_true_gen + alpha2_true_gen, "\n")
#> Generating data with persistence: 0.999

# --- Simulate data using rugarch ---
spec_gen <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(2, 0)), # ARCH(2)
  mean.model = list(armaOrder = c(0, 0), include.mean = TRUE, archm = FALSE, arfima = FALSE)
  distribution.model = "norm",
  fixed.pars = list(mu = mu_true_new,
                    omega = omega_true_new,
                    alpha1 = alpha1_true_gen, # Use modified alpha
                    alpha2 = alpha2_true_gen) # Use modified alpha
)

# Simulate the path
# n.start is for burn-in; rugarchpath typically handles this well.
# For explicit burn-in to remove: n.sim = T_obs_new + 200, then trim first 200.
# Or use n.start within ugarchpath if it directly supports it for the path output.
# Simpler: simulate longer and trim.
path_sim_obj <- ugarchpath(spec_gen, n.sim = T_obs_new + 200, n.start = 0, m.sim = 1)
vR_new <- as.numeric(fitted(path_sim_obj)[- (1:200)]) # Get returns, remove first 200 as burn-in

cat("Generated vR_new with length:", length(vR_new), "\n")
#> Generated vR_new with length: 1500
# plot(vR_new, type="l", main="Simulated ARCH(2) Returns (Persistence < 1)")
# acf(vR_new^2, main="ACF of Squared Simulated Returns")

```

You want to model the volatility of the stock. You assume that, for all periods $t = 1, \dots, T$, the stock return r_t is generated by a location scale model

$$r_t = \mu + \varepsilon_t \sigma_t$$

with conditional variances following an ARCH(p) process

$$\sigma_t^2 = \omega + \sum_{j=1}^p \alpha_j r_{t-j}^2$$

Assume ε_t has a standard normal distribution conditional on the information from all previous periods. Thus, the likelihood function of the returns r_t is given by

$$L(\mu, \omega, \alpha_1, \alpha_2, \dots, \alpha_p) = \prod_{t=1}^T \left(\frac{1}{\sigma_t} \phi \left(\frac{r_t - \mu}{\sigma_t} \right) \right).$$

where $\phi(\cdot)$ denotes the density of the standard normal distribution.

2. Write an R function that returns the average **negative** log-likelihood function of a **reparameterized** version of the model above for periods $t = p + 1, p + 2, \dots, T$ for a flexible p . It should have the following inputs:

- (a) A vector of parameters $(\mu, \omega, \alpha_1, \alpha_2, \dots, \alpha_p)$,
- (b) a return series of length T ,
- (c) the order p of the ARCH process with default $p = 2$, and fulfill the following constraints:
 - i) $\omega > 0$
 - ii) $\sum_{j=1}^p \alpha_j = 1$ *Hint: You may initialize the conditional variance process at $t = 1, 2, \dots, p$ with σ_t^2 equal to the unconditional variance of your returns. You can use `dnorm()` to calculate the density of a normally distributed random variable in R.*

Solution:

```
fAvgNegLogLik <- function(vParams, vR, p = 2) {
  dMu <- vParams[1]
  dOmega <- exp(vParams[2])
  vParams[3:length(vParams)] <- vParams[3:length(vParams)] / sum(vParams[3:length(vParams)])

  iT <- length(vR)
  vSigma2 <- numeric(iT)
  for (j in 1:p) {
    vSigma2[j] <- dOmega / (1 - sum(vParams[3:length(vParams)]))
  }
}
```

```

dSum <- 0

for (t in (p + 1):iT) {
  vSigma2[t] <- d0omega

  for (j in 1:p) {
    vSigma2[t] <- vSigma2[t] + vParams[j + 2] * vR[t - j]^2
  }

  dSum <- dSum - log(sqrt(vSigma2[t])) + log(dnorm((vR[t] - dMu)/sqrt(vSigma2[t])))
}

return(-dSum / (iT - p))
}

```

3. Set $p = 2$. Use the BFGS algorithm with `optim()` to find the maximizers of the reparameterized log-likelihood function in 2). Use starting values that **correspond** to the original likelihood parameters

$$\mu = 0, \omega = \frac{1}{T} \sum_{t=1}^T r_t^2, \alpha_1 = 0.7, \alpha_2 = 0.3.$$

Retransform your results. What are the estimates for $(\mu, \omega, \alpha_1, \alpha_2)$?

```

optim_res <- optim(c(0, log(mean(vR_new^2)), 0.7, 0.3), fAvgNegLogLik, vR = vR_new, method =
optim_res
#> $par
#> [1] 0.000669589 -11.162179071 0.691584866 0.332063803
#>
#> $value
#> [1] -3.50962
#>
#> $counts
#> function gradient
#> 24 10
#>
#> $convergence
#> [1] 0
#>
#> $message
#> NULL

vParams <- optim_res$par

```

```

dMu_star <- vParams[1]
dOmega_star <- exp(vParams[2])
vAlphas_star <- vParams[3:length(vParams)] / sum(vParams[3:length(vParams)])

dMu_star
#> [1] 0.000669589
dOmega_star
#> [1] 1.420127e-05
vAlphas_star
#> [1] 0.6756076 0.3243924

spec_fit_rugarch <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(2, 0)), # ARCH(2)
  mean.model = list(armaOrder = c(0, 0), include.mean = TRUE), # Estimate mu
  distribution.model = "norm"
)

# Fit the model using rugarch
fit_rugarch <- ugarchfit(spec = spec_fit_rugarch, data = vR_new, solver = "nloptr") # or "nlminb"

print("--- rugarch Results ---")
#> [1] "--- rugarch Results ---"
print(fit_rugarch)
#>
#> *-----*
#> *          GARCH Model Fit          *
#> *-----*
#>
#> Conditional Variance Dynamics
#> -----
#> GARCH Model   : sGARCH(2,0)
#> Mean Model    : ARFIMA(0,0,0)
#> Distribution   : norm
#>
#> Optimal Parameters
#> -----
#>      Estimate Std. Error t value Pr(>|t|)
#> mu      0.000494   0.000017  28.8508      0
#> omega    0.000014   0.000000 132.2038      0
#> alpha1   0.670886   0.056970  11.7761      0
#> alpha2   0.323013   0.040834   7.9105      0
#>

```

```

#> Robust Standard Errors:
#>      Estimate Std. Error  t value Pr(>|t|)
#> mu      0.000494   0.000010  49.4934      0
#> omega    0.000014   0.000000 129.1425      0
#> alpha1   0.670886   0.058066  11.5538      0
#> alpha2   0.323013   0.037942   8.5133      0
#>
#> LogLikelihood : 5262.87
#>
#> Information Criteria
#> -----
#>
#> Akaike      -7.0118
#> Bayes      -6.9977
#> Shibata     -7.0118
#> Hannan-Quinn -7.0065
#>
#> Weighted Ljung-Box Test on Standardized Residuals
#> -----
#>
#>              statistic p-value
#> Lag[1]              0.5291  0.4670
#> Lag[2*(p+q)+(p+q)-1] [2]    0.5715  0.6610
#> Lag[4*(p+q)+(p+q)-1] [5]    2.5128  0.5032
#> d.o.f=0
#> H0 : No serial correlation
#>
#> Weighted Ljung-Box Test on Standardized Squared Residuals
#> -----
#>
#>              statistic p-value
#> Lag[1]              0.0004559  0.9830
#> Lag[2*(p+q)+(p+q)-1] [5] 0.7997152  0.9031
#> Lag[4*(p+q)+(p+q)-1] [9] 1.8692596  0.9200
#> d.o.f=2
#>
#> Weighted ARCH LM Tests
#> -----
#>
#>      Statistic Shape Scale P-Value
#> ARCH Lag[3]    0.08255 0.500 2.000  0.7739
#> ARCH Lag[5]    1.75598 1.440 1.667  0.5277
#> ARCH Lag[7]    2.36933 2.315 1.543  0.6395
#>
#> Nyblom stability test

```

```

#> -----
#> Joint Statistic:  0.4199
#> Individual Statistics:
#> mu      0.08636
#> omega  0.07282
#> alpha1 0.14335
#> alpha2 0.09463
#>
#> Asymptotic Critical Values (10% 5% 1%)
#> Joint Statistic:      1.07 1.24 1.6
#> Individual Statistic:  0.35 0.47 0.75
#>
#> Sign Bias Test
#> -----
#>
#>          t-value   prob sig
#> Sign Bias      0.7563 0.4496
#> Negative Sign Bias 0.8591 0.3904
#> Positive Sign Bias 0.3725 0.7096
#> Joint Effect      2.4834 0.4783
#>
#>
#> Adjusted Pearson Goodness-of-Fit Test:
#> -----
#>   group statistic p-value(g-1)
#> 1    20      11.97      0.8868
#> 2    30      14.68      0.9874
#> 3    40      32.64      0.7539
#> 4    50      39.73      0.8249
#>
#>
#> Elapsed time : 0.3109791

```

23 Retake exam 2022

23.1 Problem 1:

Begin by setting the seed to 134.

1. Use the inversion method to simulate data from a Laplace distribution with the location $\mu = 1$ and the scale $\sigma = 2$ parameters. The inverse CDF is given as

$$F^{-1}(U) = \mu - \sigma \text{sign}(U - 0.5) \ln(1 - 2|U - 0.5|)$$

and we wish to simulate $N = 5000$ samples of length $T = 1000$. **sign** is the sign function built in R. Write a function, **fn_Laplace**, which simulates and returns the $T \times N$ matrix **mX** as output for given values μ and σ .

Solution:

```
set.seed(134)

fn_Laplace <- function(dMu, dSigma, iSamples, iLength) {

  U <- runif(iSamples * iLength)
  vTemp <- dMu - dSigma * sign(U-0.5) * log(1 - 2 * abs(U - 0.5))
  return(matrix(vTemp, nrow = iLength, ncol = iSamples))

}

mX <- fn_Laplace(dMu = 1, dSigma = 2, iSamples = 5000, iLength = 1000)
```

2. The density function of the Laplace distribution is given as

$$f(X) = \frac{1}{2\sigma} \exp\left(-\frac{|X - \mu|}{\sigma}\right)$$

Pick the first column of the matrix **mX** and create a histogram and superimpose the theoretical density in red. Set **breaks = 200** in your call to **hist()**.

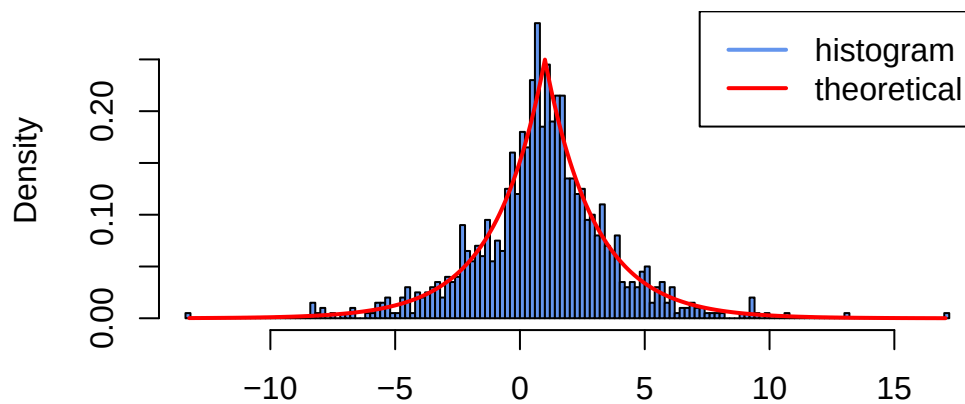
Solution:


```
fLapDen <- function(dMu, dSigma, x) {
  return(1 / (2 * dSigma) * exp(-abs(x - dMu) / dSigma))
}

hist(mX[, 1],
     freq = FALSE,
     breaks = 200,
     col = "cornflowerblue",
     xlab = "",
     ylab = "Density",
     main = "",
     xlim = c(min(mX[, 1]), max(mX[, 1])))

vX <- seq(min(mX[, 1]), max(mX[, 1]), by = 0.05)
lines(vX, fLapDen(dMu = 1, dSigma = 2, vX), type = "l", col = "red", lwd = 2)

legend("topright",
      legend = c("histogram", "theoretical"),
      col = c("cornflowerblue", "red"),
      lwd = 2)
```



3. The maximum likelihood estimates of the parameters are given as

$$\hat{\mu} = \text{median}(X)$$

$$\hat{\sigma} = \frac{1}{T} \sum_{t=1}^T |X_t - \hat{\mu}|$$

Write a function, `fn_estimateLaplace`, which estimates $\hat{\mu}$ and $\hat{\sigma}$ for each column in `mX`. Save the resulting 5000 estimates in vectors `vMu` and `vSigma`. **However**, your function must NOT use the built-in functions `abs` and `median`. You must compute the median of the absolute value of each column without using those two functions.

Solution:

```
fn_estimateLaplace <- function(mInput) {  
  fTempMed <- function(vX) {  
    return(sort(vX, decreasing = FALSE)[ceiling(length(vX) / 2)])  
  }  
  fSigmaCal <- function(vX) {  
    vOut <- ifelse(vX - fTempMed(vX) < 0, (vX - fTempMed(vX)) * -1, vX - fTempMed(vX))  
    return(mean(vOut))  
  }  
  
  return(list(  
    vMu = apply(mInput, 2, fTempMed),  
    vSigma = apply(mInput, 2, fSigmaCal)  
  ))  
}  
  
vMu <- fn_estimateLaplace(mX)$vMu  
vSigma <- fn_estimateLaplace(mX)$vSigma
```

4. Create two histograms of the estimates $\hat{\sigma}$ and $\hat{\mu}$ contained in the vectors `vSigma` and `vMu`. Set `breaks = 50` and `freq = F`. Let the title of the plot be “Distribution of estimates” and set the label on the x-axis to “Estimates”. Add a density plot to the histogram you have just created. Assume $\hat{\sigma}$ and $\hat{\mu}$ are Gaussian and use the empirical mean and standard deviation of the calculations in `vSigma` and `vMu` as the parameters. Superimpose the density plot on the histogram with `lines()`.

Solution:

```
hist(vMu,  
     freq = FALSE,  
     breaks = 50,
```

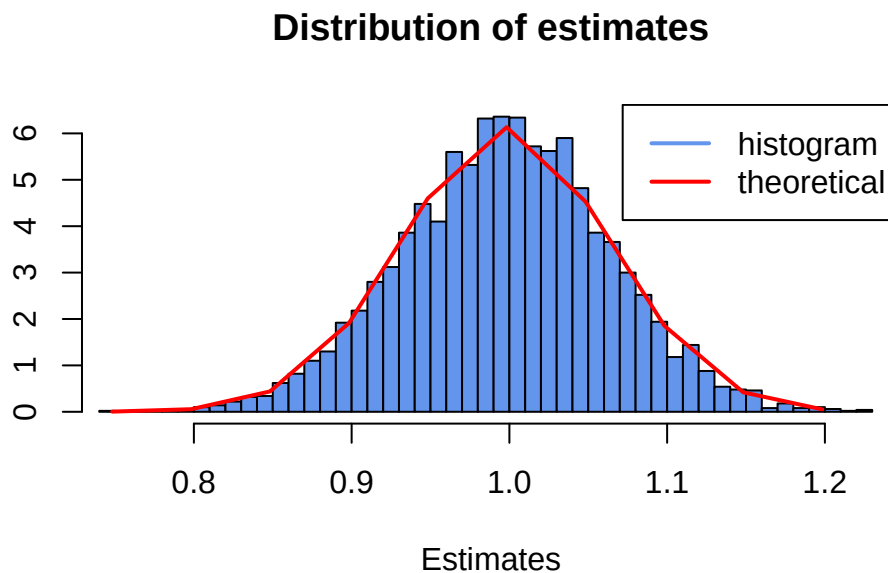
```

col = "cornflowerblue",
xlab = "Estimates",
ylab = "",
main = "Distribution of estimates",
xlim = c(min(vMu), max(vMu)))

vX <- seq(min(vMu), max(vMu), by = 0.05)
lines(vX, dnorm(vX, mean = mean(vMu), sd = sd(vMu)), type = "l", col = "red", lwd = 2)

legend("topright",
      legend = c("histogram", "theoretical"),
      col = c("cornflowerblue", "red"),
      lwd = 2)

```



```

hist(vSigma,
     freq = FALSE,
     breaks = 50,
     col = "cornflowerblue",
     xlab = "Estimates",
     ylab = "",
     main = "Distribution of estimates",
     xlim = c(min(vSigma), max(vSigma)))

```

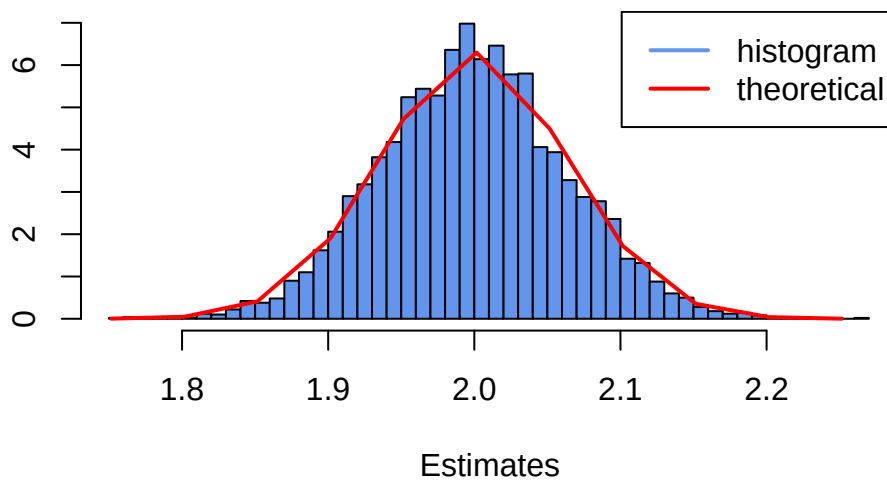
```

vX <- seq(min(vSigma), max(vSigma), by = 0.05)
lines(vX, dnorm(vX, mean = mean(vSigma), sd = sd(vSigma)), type = "l", col = "red", lwd = 2)

legend("topright",
      legend = c("histogram", "theoretical"),
      col = c("cornflowerblue", "red"),
      lwd = 2)

```

Distribution of estimates



5. The variance of the Laplace distribution is given as

$$\text{var}(X) = 2\sigma^2$$

Write a function, `fn_estimateVariance`, which defines $\text{var}(X)$ for each column `mX` by calculating its empirical variance. Define $\text{var}(X) = 2\sigma^2$ and calculate its mean and standard deviation.

Solution:

```

fn_estimateVariance <- function(mInput) {
  return(apply(mInput, 2, var))
}

dSigma <- 2

```

```
vTemp <- fn_estimateVariance(mX) - 2 * dSigma^2
mean(vTemp)
#> [1] 0.01253421
sd(vTemp)
#> [1] 0.5653084
```

23.2 Problem 2: (Constrained) Optimization, C++, and Packaging

Note: This problem is best solved in order from 1 - 6.

In this problem, you are supposed to work with the `vDATA.r` file provided with the exam. It contains the daily stock returns for the stocks of Danske Bank from 9 June 2016 to 9 March 2021 traded on the Copenhagen Stock Exchange. In the following, we denote r_t as the return of Danske Bank at time period t .

1. Load the data file `vDATA.r` into your R workspace using `readRDS()`. You want to model the volatility of the stock. You assume that, for all periods $t = 1, \dots, T$, the stock return r_t is generated by a simple scale model

$$r_t = \varepsilon_t \sigma_t$$

with conditional variances each following a zero drift GARCH(2,1) process

$$\sigma_t^2 = \alpha_1 r_{t-1}^2 + \alpha_2 r_{t-2}^2 + \beta \sigma_{t-1}^2$$

and independent ε_t following some distribution conditional on the information from all previous periods (denoted as F_{t-1}) according to density function $f(\cdot|F_{t-1};\theta)$. θ denotes the parameters of the distribution (to be specified later). The likelihood function of the returns is then given by

$$L(\mu, \alpha, \beta, \theta) = \prod_{t=1}^T \frac{1}{\sigma_t} f(r_t/\sigma_t|F_{t-1};\theta)$$

Solution:

```
generate_sample_garch_data <- function(num_observations = 1200,
                                         burn_in_period = 200,
                                         alpha1_sim = 0.10,
                                         alpha2_sim = 0.10,
                                         beta_sim = 0.80,
                                         nu_sim = 5,
                                         initial_sigma2 = (0.01)^2,
                                         seed = 42) {
```

```

if (!is.null(seed)) {
  set.seed(seed)
}

total_length = num_observations + burn_in_period

# --- Parameter Validation ---
if (alpha1_sim < 0 || alpha2_sim < 0 || beta_sim < 0) {
  stop("GARCH parameters (alpha1, alpha2, beta) must be non-negative.")
}
# The problem later constrains alpha1+alpha2+beta=1.
# For simulation, stationarity usually requires alpha1+alpha2+beta < 1 if an intercept omega is present.
# Here, omega=0. If alpha1+alpha2+beta >= 1, it's an IGARCH or explosive process.
# The chosen defaults sum to 1 (IGARCH), which is common for financial data.

if (nu_sim <= 2) {
  stop("Degrees of freedom 'nu_sim' for t-distribution must be > 2 for finite variance.")
}
if (initial_sigma2 <= 0) {
  stop("Initial variance 'initial_sigma2' must be positive.")
}

# --- Initialize Vectors ---
r_t <- numeric(total_length)      # Vector for returns
sigma2_t <- numeric(total_length) # Vector for conditional variances

# --- Generate Standardized t-distributed Innovations (mean 0, variance 1) ---
# A standard t-distribution with nu degrees of freedom has variance nu / (nu - 2).
innovations <- rt(total_length, df = nu_sim) * sqrt((nu_sim - 2) / nu_sim)

# --- Set Initial Values ---
# For GARCH(2,1): sigma_t^2 = alpha1*r_{t-1}^2 + alpha2*r_{t-2}^2 + beta*sigma_{t-1}^2
# We need to bootstrap the process for the first few periods.

# Period 1:
sigma2_t[1] <- initial_sigma2
r_t[1] <- innovations[1] * sqrt(sigma2_t[1])

# Period 2:
# sigma_2^2 depends on r_1^2, r_0^2, sigma_1^2.
# Assume r_0 = 0 (or its squared value is negligible or absorbed into an effective initial variance)
# So, sigma_2^2 = alpha1*r_1^2 + beta*sigma_1^2 (alpha2*r_0^2 term is zero)

```

```

sigma2_t[2] <- alpha1_sim * r_t[1]^2 + beta_sim * sigma2_t[1]
if (sigma2_t[2] <= 1e-9) sigma2_t[2] <- 1e-9 # Floor variance
r_t[2] <- innovations[2] * sqrt(sigma2_t[2])

# --- Simulate GARCH(2,1) Process ---
for (t in 3:total_length) {
  sigma2_t[t] <- alpha1_sim * r_t[t-1]^2 +
    alpha2_sim * r_t[t-2]^2 +
    beta_sim * sigma2_t[t-1]^2

  # Ensure variance remains positive (numerical stability)
  if (sigma2_t[t] <= 1e-9) {
    sigma2_t[t] <- 1e-9 # Floor variance to prevent issues like sqrt(negative)
  }

  r_t[t] <- innovations[t] * sqrt(sigma2_t[t])
}

# --- Discard Burn-in Period ---
sample_returns <- r_t[(burn_in_period + 1):total_length]

# Final check for non-finite values (e.g. if sigma2_t became 0 or negative despite floor)
if(any(!is.finite(sample_returns))){
  warning("Non-finite values were generated in sample_returns. This might indicate issues")
  sample_returns[!is.finite(sample_returns)] = 0
}

return(sample_returns)
}

vDATA <- generate_sample_garch_data(
  num_observations = 1197,
  alpha1_sim = 0.10,
  alpha2_sim = 0.10,
  beta_sim = 0.80, # Note: alpha1+alpha2+beta = 1.0 (IGARCH process)
  nu_sim = 5,
  seed = 123 # For reproducibility
)

```

2. Write an R function that returns the average **negative** log-likelihood function of the model above for periods $t = 2, 3, \dots, T$ for an **arbitrary, user-specified density function** $f(\cdot)$. The R function should have the following inputs:

- (a) A vector of parameters collecting all the GARCH parameters $(\alpha_1, \alpha_2, \beta)$ and the conditional density parameters θ ,
- (b) a time-series of length T ,
- (c) a flexible function object for the density $f(\cdot)$ with default `rnorm`. *Hint: Initialize the conditional variance process at $t = 1$ with σ_1^2 equal to the unconditional variance of your returns.*

Solution:

```
fAvgNegLogLik <- function(vParams, vR, f = rnorm) {
  dAlpha1 <- vParams[1]
  dAlpha2 <- vParams[2]
  dBeta <- vParams[3]
  vDensParams <- vParams[4:length(vParams)]

  dT <- length(vR)
  dSum <- 0
  vSigma2 <- numeric(dT)

  vSigma2[1] <- var(vR)
  vSigma2[2] <- dAlpha1 * vR[1]^2 + dBeta * vSigma2[1]

  for (t in 3:dT) {
    vSigma2[t] <- dAlpha1 * vR[t-1]^2 + dAlpha2 * vR[t-2]^2 + dBeta * vSigma2[t-1]
    dSum <- dSum - log(sqrt(vSigma2[t])) + log(do.call(f, as.list(c(vR[t]/sqrt(vSigma2[t]), v
  }

  return(-dSum / (dT - 1))
}
```

3. Write another R function that uses the function in (2.) to return the average **negative** log-likelihood function of a **reparameterized** version of the model above for periods $t = 2, 3, \dots, T$ assuming a t -distribution with ν degrees of freedom, i.e

$$f(\varepsilon_t | F_{t-1}) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi}\Gamma\left(\frac{\nu}{2}\right)} \left(1 + \frac{\varepsilon_t^2}{\nu}\right)^{-\frac{\nu+1}{2}}$$

It should have the following inputs:

- (a) A vector of parameters collecting all the GARCH parameters and the conditional density parameters,
- (b) a time-series of length T , and fulfill the following constraints
 - i) $0 < \alpha_1 < 1$

- ii) $0 < \alpha_2 < 1$
- iii) $0 < \beta < 1$
- iv) $\alpha_1 + \alpha_2 + \beta = 1$,
- v) $\nu > 2$. *Note: You do not need to write a new likelihood function. Use 2. Hint: You can use `dt()` to calculate the density of a t -distributed random variable in R. Or you can use the formula above with `gamma()` for the Γ -function in R.*

Solution:

```
fAvgNegLogLikRep <- function(vParams, vR) {
  dAlpha1 <- exp(vParams[1]) / (1 + exp(vParams[1]) + exp(vParams[2]))
  dAlpha2 <- exp(vParams[2]) / (1 + exp(vParams[1]) + exp(vParams[2]))
  dBeta <- 1 - dAlpha1 - dAlpha2
  dNu <- 2 + exp(vParams[4])

  vParams <- c(dAlpha1, dAlpha2, dBeta, dNu)
  return(fAvgNegLogLik(vParams, vR, dt))
}
```

4. Use the BFGS algorithm with `optim()` to find the maximizers of the reparameterized log-likelihood function in 3). Use starting values that **correspond** to the original likelihood parameters

$$\alpha_1 = 0.10, \alpha_2 = 0.10, \beta = 0.80, \nu = 5.$$

Retransform your results. What are the estimates for $(\alpha_1, \alpha_2, \beta, \nu)$?

Solution:

```
vParams <- c(0.1, 0.1, 0.8, 5)
# transform
dAlpha1 <- log(vParams[1] / (1 - vParams[1] - vParams[2]))
dAlpha2 <- log(vParams[2] / (1 - vParams[1] - vParams[2]))
dBeta <- vParams[3] # constraint ensured by functions
dNu <- log(vParams[4] - 2)
vParams <- c(dAlpha1, dAlpha2, dBeta, dNu)

optim_results <- optim(vParams, fAvgNegLogLikRep, vR = vDATA, method = "BFGS")
optim_results
#> $par
#> [1] -9.187285 -9.375245  0.800000  2.441869
#>
#> $value
#> [1] -8.964638
#>
```

```

#> $counts
#> function gradient
#>      38      33
#>
#> $convergence
#> [1] 0
#>
#> $message
#> NULL

vParams <- optim_results$par
dAlpha1 <- exp(vParams[1]) / (1 + exp(vParams[1]) + exp(vParams[2]))
dAlpha2 <- exp(vParams[2]) / (1 + exp(vParams[1]) + exp(vParams[2]))
dBeta <- 1 - dAlpha1 - dAlpha2
dNu <- 2 + exp(vParams[4])

print(paste0("Est. Alpha1: ", dAlpha1, " vs act.: ", 0.1))
#> [1] "Est. Alpha1: 0.00010231320414325 vs act.: 0.1"
print(paste0("Est. Alpha2: ", dAlpha2, " vs act.: ", 0.1))
#> [1] "Est. Alpha2: 8.47815778203704e-05 vs act.: 0.1"
print(paste0("Est. Beta: ", dBeta, " vs act.: ", 0.8))
#> [1] "Est. Beta: 0.999812905218036 vs act.: 0.8"
print(paste0("Est. Nu: ", dNu, " vs act.: ", 5))
#> [1] "Est. Nu: 13.4945084264308 vs act.: 5"

```

Sanity check:

```

suppressMessages(library(rugarch))
#> Warning: pakke 'rugarch' blev bygget under R version 4.3.3

spec_fit_rugarch <- ugarchspec(
  variance.model = list(model = "sGARCH", garchOrder = c(2, 1)), # ARCH(2)
  mean.model = list(armaOrder = c(0, 0), include.mean = FALSE), # Estimate mu
  distribution.model = "std"
)

# Fit the model using rugarch
fit_rugarch <- ugarchfit(spec = spec_fit_rugarch, data = vDATA, solver = "nloptr") # or "nlo"

print("--- rugarch Results ---")
#> [1] "--- rugarch Results ---"
print(fit_rugarch)

```

```

#>
#> *-----*
#> *          GARCH Model Fit          *
#> *-----*
#>
#> Conditional Variance Dynamics
#> -----
#> GARCH Model   : sGARCH(2,1)
#> Mean Model    : ARFIMA(0,0,0)
#> Distribution   : std
#>
#> Optimal Parameters
#> -----
#>      Estimate  Std. Error    t value Pr(>|t|)
#> omega    0.000000    0.000000    0.048587  0.96125
#> alpha1   0.000071    0.022045    0.003227  0.99743
#> alpha2   0.000256    0.022012    0.011636  0.99072
#> beta1    0.969460    0.002236  433.632645  0.00000
#> shape    4.974784    0.730757    6.807711  0.00000
#>
#> Robust Standard Errors:
#>      Estimate  Std. Error    t value Pr(>|t|)
#> omega    0.000000    0.000000  0.000812  0.999352
#> alpha1   0.000071    0.051016  0.001394  0.998887
#> alpha2   0.000256    0.294820  0.000869  0.999307
#> beta1    0.969460    0.307556  3.152144  0.001621
#> shape    4.974784    8.785449  0.566253  0.571222
#>
#> LogLikelihood : 10763.72
#>
#> Information Criteria
#> -----
#>
#> Akaike          -17.976
#> Bayes           -17.955
#> Shibata         -17.976
#> Hannan-Quinn   -17.968
#>
#> Weighted Ljung-Box Test on Standardized Residuals
#> -----
#>
#>                                statistic p-value
#> Lag[1]                                0.1481  0.7004

```

```

#> Lag[2*(p+q)+(p+q)-1][2]    0.9820  0.5041
#> Lag[4*(p+q)+(p+q)-1][5]    2.1393  0.5859
#> d.o.f=0
#> H0 : No serial correlation
#>
#> Weighted Ljung-Box Test on Standardized Squared Residuals
#> -----
#>
#>                statistic p-value
#> Lag[1]                0.005858  0.9390
#> Lag[2*(p+q)+(p+q)-1][8]    0.966063  0.9763
#> Lag[4*(p+q)+(p+q)-1][14]  3.722254  0.9052
#> d.o.f=3
#>
#> Weighted ARCH LM Tests
#> -----
#>
#>                Statistic Shape Scale P-Value
#> ARCH Lag[4]        0.5723 0.500 2.000  0.4494
#> ARCH Lag[6]        0.7583 1.461 1.711  0.8178
#> ARCH Lag[8]        0.8049 2.368 1.583  0.9508
#>
#> Nyblom stability test
#> -----
#> Joint Statistic:  280.0863
#> Individual Statistics:
#> omega  77.26025
#> alpha1  0.27309
#> alpha2  0.22811
#> beta1   0.16494
#> shape   0.03994
#>
#> Asymptotic Critical Values (10% 5% 1%)
#> Joint Statistic:      1.28 1.47 1.88
#> Individual Statistic:  0.35 0.47 0.75
#>
#> Sign Bias Test
#> -----
#>
#>                t-value   prob sig
#> Sign Bias        0.3901 0.6965
#> Negative Sign Bias 0.1214 0.9034
#> Positive Sign Bias 0.2282 0.8195
#> Joint Effect      0.5120 0.9163
#>

```

```

#>
#> Adjusted Pearson Goodness-of-Fit Test:
#> -----
#>   group statistic p-value(g-1)
#> 1    20      12.41      0.8675
#> 2    30      24.38      0.7101
#> 3    40      28.03      0.9041
#> 4    50      39.97      0.8178
#>
#>
#> Elapsed time : 0.980319

```

5. Write a C++ function named “CppGfunc” with two inputs i) a numerical vector x and an arbitrary R function $g(\cdot)$ with no default. It should return $g(x) + 2$.

Solution:

```

// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>

using namespace arma;
using namespace Rcpp;

// [[Rcpp::export]]
arma::vec CppGfunc(Function g, arma::vec x) {

  arma::vec vOut = zeros<vec>(x.size());

  for (arma::uword i = 0; i < x.size(); i++) {
    SEXP s;
    s = g(x[i]);
    vOut[i] = as<double>(s) + 2.0;
  }

  return(vOut);
}

```

```

suppressMessages(library(Rcpp))
#> Warning: pakke 'Rcpp' blev bygget under R version 4.3.3
suppressMessages(library(RcppArmadillo))
#> Warning: pakke 'RcppArmadillo' blev bygget under R version 4.3.3

sourceCpp("exam2022recpp.cpp")

```

```
CppGfunc(function(x) x^2, c(1,2,3))
#>      [,1]
#> [1,]    3
#> [2,]    6
#> [3,]   11
```

6. Create an R package that contains the functions from 2.) and 5.) and edit the title description to “This is my exam package”. Export the package as a bundled development version. *Remark: If you cannot solve 2.) or 5.) create a package that contains an R and a C++ function with single scalar inputs that always return the number 17.*

24 Ordinary exam 2023

24.1 Problem 1:

Begin by setting the seed to 123.

1. Use the inversion method to simulate data from two gamma distributions $\text{Gamma}(k_1, \theta_1)$ and $\text{Gamma}(k_2, \theta_2)$ with the parameters $(k_1, \theta_1) = (2, 1)$, and $(k_2, \theta_2) = (1, 2)$, where k_1 and k_2 are shape parameters and θ_1 and θ_2 are scale parameters. (Hint: You may use R's built-in inverse cumulative distribution function (quantile function) `qgamma()`.) We wish to simulate $N = 100$ samples each of length $T = 1000$. Store your gamma random variables in the matrices `mG1` and `mG2` each of size $T \times N$.

Solution:

```
set.seed(123)

Gamma.Simulate <- function(iK, dTheta, iSamples, iLength) {

  U <- runif(iSamples * iLength)
  vOut <- qgamma(U, shape = iK, scale = dTheta)
  return(matrix(vOut, nrow = iLength, ncol = iSamples))

}

mG1 <- Gamma.Simulate(iK = 2, dTheta = 1, iSamples = 100, iLength = 1000)
mG2 <- Gamma.Simulate(iK = 1, dTheta = 2, iSamples = 100, iLength = 1000)
```

2. Define a new random variable $L(k_1, \theta_1, k_2, \theta_2) = \text{Gamma}(k_1, \theta_1) - \text{Gamma}(k_2, \theta_2)$. Use this representation and the matrices `mG1` and `mG2` generated in question 1 to generate $N = 100$ samples each of length $T = 1000$ of random variables $L(k_1, \theta_1, k_2, \theta_2)$. Store these random variables in a vector `mL` of size $T \times N$.

Solution:

```
mL <- mG1 - mG2
```

3. The skewness of the random variable $L(k_1, \theta_1, k_2, \theta_2)$ is given by

$$\text{sk} = \frac{2k_1\theta_1^3 - 2k_2\theta_2^3}{(k_1\theta_1^2 + k_2\theta_2^2)^{3/2}}$$

and the empirical skewness of a vector $\mathbf{X}$ is given by

$$\hat{\text{sk}} = \frac{\frac{1}{T} \sum_{i=1}^T (X_i - \bar{X})^3}{\left[\frac{1}{T} \sum_{i=1}^T (X_i - \bar{X})^2 \right]^{3/2}}, \quad \bar{X} = \frac{1}{T} \sum_{i=1}^T X_i.$$

Write a function, `fn_estimateSkewness`, which provides estimates $\hat{\text{sk}}$ for each column in `mL` by calculating its empirical skewness and then subtract `sk` calculated with the formulas above, i.e., calculate $\hat{\text{sk}} - \text{sk}$ for each column in `mL`. Save the resulting 100 estimates in a vector `vDiffSk`.

Solution:

```
fn_estimateSkewness <- function(mInput) {
  fn_CalcSkew <- function(vX) {
    dNum <- mean((vX - mean(vX))^3)
    dDenom <- mean((vX - mean(vX))^2)^(3/2)
    return(dNum / dDenom)
  }
  return(apply(mInput, 2, fn_CalcSkew))
}

iK1 <- 2
iK2 <- 1
dTheta1 <- 1
dTheta2 <- 2

vSkHat <- fn_estimateSkewness(mL)
vSkAct <- (2 * iK1 * dTheta1^3 - 2 * iK2 * dTheta2^3) / (iK1 * dTheta1^2 + iK2 * dTheta2^2)

vDiffSk <- vSkHat - vSkAct
```

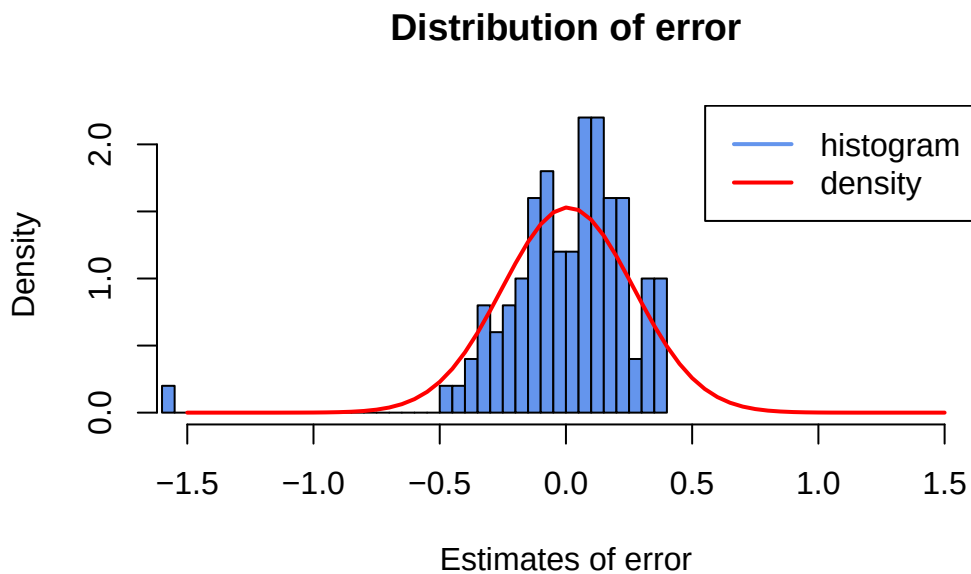
4. Create a histogram of the estimates contained in the vector `vDiffSk`. Let the title of the plot be “Distribution of error” and set the label on the x-axis to “Estimates of error”. Add a density plot to the histogram you have just created. Assume the error $\hat{\text{sk}} - \text{sk}$ is Gaussian and use the empirical mean and standard deviation of the vector `vDiffSk` as parameters of the density. Superimpose the density plot on the histogram with `lines()`.

Solution:


```
hist(vDiffSk,
     freq = FALSE,
     breaks = 41,
     col = "cornflowerblue",
     xlab = "Estimates of error",
     main = "Distribution of error",
     xlim = c(-1.5, 1.5))

vX <- seq(-1.5, 1.5, 0.05)
lines(vX, dnorm(vX, mean = mean(vDiffSk), sd = sd(vDiffSk)), type = "l", col = "red", lwd = 2)

legend("topright",
     legend = c("histogram", "density"),
     col = c("cornflowerblue", "red"),
     lwd = 2)
```



24.2 Problem 2: (Constrained) Optimization, C++, and Packaging

Note: This problem is best solved in order from 1 - 6.

In this problem, you will work with the `Y.R` and `X.R` files provided with the exam. They contain data which you will use to estimate a Poisson generalized linear model by maximum likelihood.

The model is as follows:

$$y_i \sim \text{Pois}(\lambda_i), \quad \lambda_i = \exp(\mathbf{x}_i' \theta), \quad i = 1, \dots, 500.$$

The file `Y.R` contains the dependent variables y_i stacked in an integer vector, and `X.R` contains the regressors stacked in a 500×5 matrix.

1. Load the data files `Y.R` and `X.R` into your R workspace using `readRDS()`.

Solution:

```
set.seed(123)
genData <- function() {
  n <- 500 # Number of observations
  p <- 5   # Number of regressors (including intercept)

  # Define the true parameter vector theta.
  # We set theta_1 > 0 to make the hypothesis test H0: theta_1 <= 0 meaningful.
  theta_true <- c(0.8, -1.2, 0.5, 0.1, 2.0)

  # Create the regressor matrix X
  # First column is the intercept
  X <- matrix(rnorm(n * (p - 1)), nrow = n, ncol = p - 1)
  X <- cbind(1, X)
  colnames(X) <- paste0("X", 1:p)

  # Calculate the linear predictor and the mean lambda
  # eta = X * theta
  eta <- X %*% theta_true
  # lambda = exp(eta)
  lambda <- exp(eta)

  # Generate the dependent variable Y from the Poisson distribution
  Y <- rpois(n, lambda)
  return(list(X = X, Y = Y))
}

X <- genData()$X
Y <- genData()$Y
```

You want to find the maximum likelihood estimator of θ , using the average log-likelihood and average score functions:

$$\ell(\theta|\mathbf{y}, \mathbf{X}) = \frac{1}{n} \sum_{i=1}^n y_i(\mathbf{x}_i' \theta) - \exp(\mathbf{x}_i' \theta) - \ln(y_i!),$$

$$s(\theta|\mathbf{y}, \mathbf{X}) = \frac{1}{n} \sum_{i=1}^n (y_i - \exp(\mathbf{x}_i' \theta)) \mathbf{x}_i.$$

2. Create a C++ script, and write a C++ function called `factorial_cpp` which computes the factorial of an integer. It should accept a single integer `iN` as an input, and return `iN!`. The function should check whether the input is negative; if it is, it should generate an informative error message regarding this issue.

Solution:

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>

using namespace arma;
using namespace Rcpp;

// [[Rcpp::export]]
double factorial_cpp(int iN) {
  if (iN < 0) {
    Rcpp::stop("Input to factorial must be a non-negative integer.");
  }
  if (iN == 0) {
    return 1.0;
  }
  double result = 1.0;
  for (int i = 1; i <= iN; ++i) {
    result *= i;
  }
  return result;
}
```

```
suppressMessages(library(Rcpp))
#> Warning: pakke 'Rcpp' blev bygget under R version 4.3.3
suppressMessages(library(RcppArmadillo))
#> Warning: pakke 'RcppArmadillo' blev bygget under R version 4.3.3
sourceCpp("exam2023cpp.cpp")
```

3. Write two C++ functions `fLnL_cpp` and `fScore_cpp` which accept as arguments the vector θ , integer vector $\mathbf{y}$, and matrix $\mathbf{X}$. They should return $\ell(\theta|\mathbf{y}, \mathbf{X})$ and $s(\theta|\mathbf{y}, \mathbf{X})$ respectively. In your R script, include a `sourceCpp()` command which compiles the C++ script and adds these functions to your environment. *Remark: If you cannot solve question 2, you may skip the $-\ln(y_i!)$ term in the `fLnL_cpp` function.*

Solution:

```
// [[Rcpp::export]]
double fLnL_cpp(vec vParams, vec vY, mat mX) {
  double dSum = 0.0;
  int iN = mX.n_rows;

  for (int i = 0; i < iN; i++) {
    //dSum = dSum + vY[i] * as_scalar(mX.row(i) * vParams) - exp(as_scalar(mX.row(i) * vParams))
    dSum = dSum + vY[i] * as_scalar(mX.row(i) * vParams) - exp(as_scalar(mX.row(i) * vParams))
  }

  return(dSum / iN);
}

// [[Rcpp::export]]
vec fScore_2_cpp(vec vParams, vec vY, mat mX){
  vec vP = exp(mX*vParams);
  vec vScore = trans(mX)*(vY - vP) / vY.n_elem;

  return vScore;
}

// [[Rcpp::export]]
vec fScore_cpp(vec vParams, vec vY, mat mX){

  int n_obs = mX.n_rows; // Number of observations
  int p = mX.n_cols;      // Number of parameters

  vec vOut = zeros<vec>(p);

  for (int i = 0; i < n_obs; i++) {
    vOut += (vY[i] - exp(as_scalar(mX.row(i) * vParams))) * mX.row(i).t();
  }

  return vOut / n_obs;
}
```

```
suppressMessages(library(Rcpp))
suppressMessages(library(RcppArmadillo))
sourceCpp("exam2023cpp.cpp")
```

4. Maximize the average log-likelihood with the BFGS algorithm, using the `optim()` function.

You should provide your `fScore_cpp` function as an input to the `gr` argument, and you may use a vector of zeroes as the initial guess. Report the maximum likelihood estimator of θ and the average log-likelihood evaluated at that point. *Remark: If you cannot fully solve question 3, in this question and all subsequent questions, you can earn partial points by implementing `fLnL_cpp` in R instead of C++, and using `gr=NULL` in the `optim()` function instead of providing the score directly.*

Solution:

```
optim_res <- optim(c(0,0,0,0,0), fLnL_cpp, fScore_2_cpp, vY = Y, mX = X, method = "BFGS", con
optim_res
#> $par
#> [1] 3.630563467 0.126546522 -0.008899956 0.282246449 0.236120415
#>
#> $value
#> [1] -114.4235
#>
#> $counts
#> function gradient
#>      55      19
#>
#> $convergence
#> [1] 0
#>
#> $message
#> NULL
```

You want to test the following hypothesis about the first element of θ :

$$H_0 : \theta_1 \leq 0 \quad \text{vs.} \quad H_1 : \theta_1 > 0.$$

One way to perform this test is with a likelihood ratio test, using the test statistic

$$LR = -2 \left[\max_{\theta \in (-\infty, 0] \times \mathbb{R}^4} \{\ell(\theta|\mathbf{y}, \mathbf{X})\} - \max_{\theta \in \mathbb{R}^5} \{\ell(\theta|\mathbf{y}, \mathbf{X})\} \right].$$

The first term is obtained by maximizing the log-likelihood subject to the constraint that $\theta_1 \leq 0$, and the second term is the unconstrained maximum which you found in question 4.

5. Write an R function `LR_stat` which computes the likelihood ratio test statistic above, and does the following:
 - (a) The arguments should be $\mathbf{y}$, and $\mathbf{X}$, and vectors of initial guesses for the constrained and unconstrained maximization, with vectors of 0 as default.

- (b) It should check whether $\mathbf{y}$ is an integer vector. If not, it should stop and generate an informative error message regarding this issue.
- (c) The constrained maximization should be solved using the `optim()` function with argument `method="L-BFGS-B"` and appropriately chosen arguments `lower` and `upper`. You should provide your `fScore_cpp` function as an input to the `gr` argument.
- (d) The unconstrained maximization should be solved as in question 4.
- (e) The output should be a list containing the likelihood ratio test statistic, the maximum likelihood estimator from the unconstrained maximization, and a message (or two) giving information about the convergence of both maximization problems. Run this function and report its outputs.

Solution:

```
LR_stat <- function(vY, mX, vInitGuessesCon = rep(0, ncol(mX)), vInitGuessesUncon = rep(0, ncol(mX))) {
  if (all(vY == floor(vY)) == FALSE) {
    stop("y is not an integer vector. Stopping...")
  }

  # unconstrained max
  fitUnconMax <- optim(vInitGuessesUncon, fLnL_cpp, fScore_2_cpp, vY = Y, mX = X, method = "L-BFGS-B")

  # constrained max
  fitConMax <- optim(vInitGuessesCon, fLnL_cpp, fScore_2_cpp, vY = Y, mX = X, method = "L-BFGS-B")

  return(list(
    LR_test_stat = -2 * (fitConMax$value - fitUnconMax$value),
    MLE_estimator_uncon = fitUnconMax$par,
    convergence_messages = c(
      unconstrained = paste0("Convergence code:", fitUnconMax$convergence, "-", fitUnconMax$message),
      constrained = paste0("Convergence code:", fitConMax$convergence, "-", fitConMax$message)
    )
  ))

LR_stat(vY = Y, mX = X)
#> $LR_test_stat
#> [1] 172.3262
#>
#> $MLE_estimator_uncon
#> [1] 3.630563467 0.126546522 -0.008899956 0.282246449 0.236120415
#>
#> $convergence_messages
#>
#> unconstrained
#> "Convergence code:0-"
#> constrained
```

```
#> "Convergence code:0-CONVERGENCE: REL_REDUCTION_OF_F <= FACTR*EPSMCH"
```

6. Create an R package that contains the functions from questions 2, 3, and 5. Edit the title description to “This is my exam package”. Export the package as a bundled development version (with file extension `.tar.gz`), and include it as part of your exam submission. *Remark: If you cannot solve 2, 3, or 5, create a package that contains an R and a C++ function with single scalar inputs that always return the number 2023.*

25 Retake exam 2023

25.1 Problem 1:

Begin by setting the seed to 134.

1. The density function of the Pareto distribution is given as

$$f(X) = \begin{cases} \frac{\alpha \gamma^\alpha}{X^{\alpha+1}}, & \text{if } X \geq \gamma, \\ 0, & \text{if } X < \gamma, \end{cases}$$

where $\gamma > 0$ and $\alpha > 0$ are the parameters. Write a function `fn_denPareto` that returns the value $f(X)$ for any X given the parameters γ and α .

Solution:

```
set.seed(134)

fn_denPareto <- function(x, gamma, alpha) {
  vOut <- ifelse(x < gamma, 0, alpha * gamma^alpha / x^(alpha + 1))
  return(vOut)
}
```

2. Use the inversion method to simulate data with Pareto distribution with the parameters values $\gamma = 1$ and $\alpha = 3$. The inverse CDF is given as

$$F^{-1}(U) = \gamma(1 - U)^{-\frac{1}{\alpha}},$$

where U is a standard uniform random variable. We wish to simulate $N = 100$ samples of length $T = 500$ each.

- (a) Write a function `fn_ParetoSimul`, which simulates and returns these samples in the $T \times N$ matrix `mX` as output.
- (b) Simulate `mX` and create a histogram of its first column. Superimpose the theoretical density defined in question 1. in red using `lines()`.

Solution:


```

fn_ParetoSimul <- function(gamma, alpha, samples, length) {

  U <- runif(samples * length)
  vSim <- gamma * (1 - U)^(-1/alpha)
  return(matrix(vSim, nrow = length, ncol= samples))

}

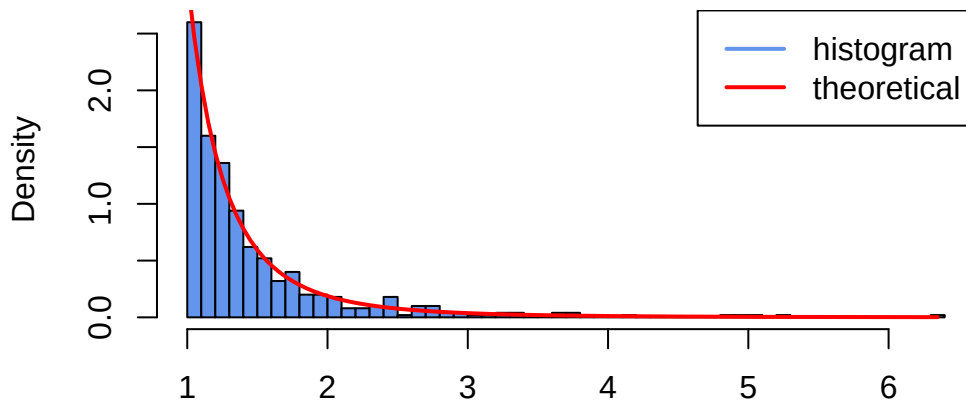
mX <- fn_ParetoSimul(gamma = 1, alpha = 3, samples = 100, length = 500)

hist(mX[, 1],
     freq = FALSE,
     breaks = 41,
     col = "cornflowerblue",
     xlab = "",
     ylab = "Density",
     main = "",
     xlim = c(min(mX[, 1]), max(mX[, 1])))

vX <- seq(min(mX[, 1]), max(mX[, 1]), by = 0.05)
lines(vX, fn_denPareto(vX, 1, 3), type = "l", col = "red", lwd = 2)

legend("topright",
      legend = c("histogram", "theoretical"),
      col = c("cornflowerblue", "red"),
      lwd = 2)

```



3. The maximum likelihood estimates of the parameters based on a sample $X_1, \dots, X_T$ are given by

$$\hat{\gamma} = \min(X_t)$$

$$\hat{\alpha} = \frac{T}{\sum_{t=1}^T \ln(X_t / \hat{\gamma})}$$

Write a function `fn_estimateParams`, which estimates $\hat{\gamma}$ and $\hat{\alpha}$ for each column in `mX`. Save the resulting 100 estimates of $\hat{\alpha}$ in a vector `vAlpha`.

Solution:

```
fn_estimateParams <- function(mInput) {
  fAlphas <- function(vX) {
    vOut <- length(vX) / sum(log(vX / min(vX)))
    return(vOut)
  }

  vGammas <- apply(mInput, 2, min)
  vAlphas <- apply(mInput, 2, fAlphas)

  return(list(
    alphas = vAlphas,
    gammas = vGammas
  ))
}
```

```

))
}

vAlpha <- fn_estimateParams(mX)$alphas

```

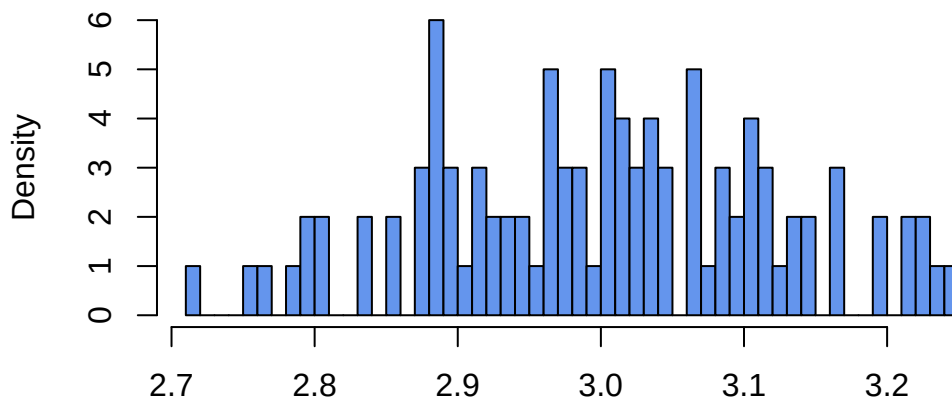
4. Create a histogram of the estimates $\hat{\alpha}$ contained in the vector `vAlpha`. Let the title of the plot be “Distribution of estimates” and set the label on the x-axis to “Estimates”.

Solution:

```

hist(vAlpha,
     freq = FALSE,
     breaks = 41,
     col = "cornflowerblue",
     xlab = "",
     ylab = "Density",
     main = "",
     xlim = c(min(vAlpha), max(vAlpha)))

```



25.2 Problem 2: (Constrained) Optimization, C++, and Packaging

Note: This problem is best solved in order from 1 - 6.

In this problem, you will work with the Y.R file provided with the exam. It contains data which you will use to estimate an autoregressive model by maximum likelihood. The model is as follows:

$$y_t = \mu + \phi y_{t-1} + \epsilon_t, \quad \epsilon_t \stackrel{iid}{\sim} N(0, \sigma^2), \quad t = 2, \dots, T, \quad (2.1)$$

with initial value $y_1 \sim N\left(\frac{\mu}{1-\phi}, \frac{\sigma^2}{1-\phi^2}\right)$ and sample size $T = 500$. Naturally $\sigma^2 > 0$, and since we're only interested in stationary models $-1 < \phi < 1$. For convenience, we stack the parameters into the vector $\theta = (\mu, \phi, \sigma^2)'$. Under this model, the joint pdf of the data is

$$f_{Y_1, \dots, Y_T}(\mathbf{y}|\theta) = f_{Y_1}(y_1|\theta) \prod_{t=2}^T f_{Y_t|Y_{t-1}}(y_t|y_{t-1}, \theta).$$

where

$$f_{Y_1}(y_1|\theta) = \frac{1}{\sqrt{2\pi\sigma^2/(1-\phi^2)}} \exp\left(-\frac{1}{2} \frac{[y_1 - \mu/(1-\phi)]^2}{\sigma^2/(1-\phi^2)}\right),$$

$$f_{Y_t|Y_{t-1}}(y_t|y_{t-1}, \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2} \frac{[y_t - \mu - \phi y_{t-1}]^2}{\sigma^2}\right).$$

1. Load the data file Y.R into your R workspace using `readRDS()`. It contains the variables y_t stacked in a vector.

Solution:

```
set.seed(123)

genData <- function() {
  mu_true <- 0.5
  phi_true <- 0.7
  sigma2_true <- 1.5
  T_val <- 500

  # Ensure parameters meet conditions
  if (abs(phi_true) >= 1) {
    stop("phi must be between -1 and 1 for stationarity.")
  }
  if (sigma2_true <= 0) {
    stop("sigma2 must be positive.")
  }

  # Initialize the time series vector
  y <- numeric(T_val)

  # 1. Generate y_1 from its stationary distribution
  # y_1 ~ N(mu / (1 - phi), sigma^2 / (1 - phi^2))
```

```

mean_y1 <- mu_true / (1 - phi_true)
var_y1 <- sigma2_true / (1 - phi_true^2)
sd_y1 <- sqrt(var_y1)

y[1] <- rnorm(1, mean = mean_y1, sd = sd_y1)

# 2. Generate y_t for t = 2, ..., T
# y_t = mu + phi * y_{t-1} + epsilon_t, epsilon_t ~ N(0, sigma^2)
sd_epsilon <- sqrt(sigma2_true)
for (t in 2:T_val) {
  epsilon_t <- rnorm(1, mean = 0, sd = sd_epsilon)
  y[t] <- mu_true + phi_true * y[t-1] + epsilon_t
}

return(y)
}

y <- genData()

```

2. Create a C++ script, and write two C++ functions called `f_Y1` and `f_cond` which compute $f_{Y_1}(y_1|\theta)$ and $f_{Y_t|Y_{t-1}}(y_t|y_{t-1}, \theta)$ respectively. In your R script, include a `sourceCpp()` command which compiles the C++ script and adds these functions to your environment.

You want to find the maximum likelihood estimator of θ , using the average log-likelihood function:

$$\ell(\theta|\mathbf{y}) = \frac{1}{T} \left(\ln f_{Y_1}(y_1|\theta) + \sum_{t=2}^T \ln f_{Y_t|Y_{t-1}}(y_t|y_{t-1}, \theta) \right)$$

Solution:

```

// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>

using namespace arma;
using namespace Rcpp;

// [[Rcpp::export]]
double f_Y1(double y1, vec vParams) {
  double dPi = atan(1)*4;
  double dMu = vParams[0];
  double dPhi = vParams[1];
  double dSigma2 = vParams[2];

```

```

double dOut = 1 / pow(2 * dPi * dSigma2 / (1 - pow(dPhi, 2)), 0.5) * exp(-0.5 * pow((y1 - dMu - dPhi * yt_1), 2) / (1 - pow(dPhi, 2)))

return dOut;
}

// [[Rcpp::export]]
double f_cond(double y1, double yt_1, vec vParams) {
  double dPi = atan(1)*4;
  double dMu = vParams[0];
  double dPhi = vParams[1];
  double dSigma2 = vParams[2];

  double dOut = 1 / pow(2 * dPi * dSigma2, 0.5) * exp(-0.5 * pow((y1 - dMu - dPhi * yt_1), 2) / (1 - pow(dPhi, 2)))

  return dOut;
}

```

```

suppressMessages(library(Rcpp))
#> Warning: pakke 'Rcpp' blev bygget under R version 4.3.3
suppressMessages(library(RcppArmadillo))
#> Warning: pakke 'RcppArmadillo' blev bygget under R version 4.3.3
sourceCpp("exam2023recpp.cpp")

```

3. Write an R function called `fLnL_rep` which computes the **reparametrized** average log-likelihood, such that the restrictions $\sigma^2 > 0$ and $-1 < \phi < 1$ are satisfied. The function should accept a vector θ and the data $\mathbf{y}$. It should also appropriately transform θ into $\tilde{\theta}$ which respects the restrictions. It should also use your functions `f_Y1` and `f_cond` to compute $\ell(\theta|\mathbf{y})$. *Remark: If you cannot fully solve question 2., you can earn partial points by implementing the functions `f_Y1` and `f_cond` in R.*

Solution:

```

fLnL_rep <- function(vParams, vY) {
  dMu <- vParams[1]
  dPhi <- -1 + 2 * (exp(vParams[2]) / (1 + exp(vParams[2])))
  dSigma2 <- exp(vParams[3])

  dT <- length(vY)
  dSum <- log(f_Y1(vY[1], c(dMu, dPhi, dSigma2)))

  for (t in 2:dT) {
    dSum <- dSum + log(f_cond(vY[t], vY[t - 1], c(dMu, dPhi, dSigma2)))
  }
}

```

```

}

return(dSum / dT)
}

```

4. Maximize the **reparametrized** average log-likelihood with the BFGS algorithm, using the `optim()` function. You may use $\theta_0 = (0, 0, 1)$ as the initial guess. Report the maximum likelihood estimator of θ and the average log-likelihood evaluated at that point. *Hint: Due to the reparameterization, you may need to transform the solution again to obtain θ .*

Solution:

```

vParams <- c(0, 0, 1)
vParams[2] <- log(((vParams[2] + 1) / 2) / (1 - ((vParams[2] + 1) / 2.0)))
vParams[3] <- log(vParams[3])
optim_res <- optim(vParams, fLnL_rep, vY = y, method = "BFGS", control=list(fnscale=-1))
vParams <- optim_res$par

dMu <- vParams[1]
dPhi <- -1 + 2 * (exp(vParams[2]) / (1 + exp(vParams[2])))
dSigma2 <- exp(vParams[3])

print(paste0("Est. Mu: ", dMu, " vs act.: ", 0.5))
#> [1] "Est. Mu: 0.67406977684722 vs act.: 0.5"
print(paste0("Est. Phi: ", dPhi, " vs act.: ", 0.7))
#> [1] "Est. Phi: 0.62648860056871 vs act.: 0.7"
print(paste0("Est. Sigma2: ", dSigma2, " vs act.: ", 1.5))
#> [1] "Est. Sigma2: 1.40416059123645 vs act.: 1.5"
print(paste0("Maximized average log-likelihood: ", optim_res$value))
#> [1] "Maximized average log-likelihood: -1.58915508954246"

```

5. The file `simulateAR1_buggy.cpp` contains a C++ function using Rcpp and RcppArmadillo. The code contains multiple errors. Debug the function. Add a short comment before each line where you have made changes. The function should simulate an AR(1) process as in Equation (2.1), with user-provided arguments for μ, ϕ, σ^2 and T . It should check whether these arguments satisfy the restrictions $\sigma^2 > 0$ and $-1 < \phi < 1$, and provide an informative error message if not. It should return the simulated time series as a vector.

Solution:

```
arma::vec simulateAR1_buggy(double mu_true = 0.5, double phi_true = 0.7, double sigma2_true = 1.0) {
  if (abs(phi_true) >= 1) {
    stop("phi must be between -1 and 1 for stationarity.");
  }
  if (sigma2_true <= 0) {
    stop("sigma2 must be positive.");
  }

  arma::vec y = zeros<vec>(T_val);

  double mean_y1 = mu_true / (1 - phi_true);
  double var_y1 = sigma2_true / (1 - pow(phi_true, 2));
  double sd_y1 = sqrt(var_y1);

  y[0] = Rf_rnorm(mean_y1, sd_y1);

  double sd_epsilon = sqrt(sigma2_true);
  double epsilon_t = 0.0;
  for (int t = 1; t < T_val; t++) {
    epsilon_t = Rf_rnorm(0, sd_epsilon);
    y[t] = mu_true + phi_true * y[t-1] + epsilon_t;
  }

  return(y);
}
```

```
suppressMessages(library(Rcpp))
suppressMessages(library(RcppArmadillo))
sourceCpp("exam2023recpp.cpp")

vY <- simulateAR1_buggy(0.5, 0.7, 1.5, 500)
```

And just for fun:

```
vParams <- c(0, 0, 1)
vParams[2] <- log(((vParams[2] + 1) / 2) / (1 - ((vParams[2] + 1) / 2.0)))
vParams[3] <- log(vParams[3])
optim_res <- optim(vParams, fLnL_rep, vY = vY, method = "BFGS", control=list(fnscale=-1))
vParams <- optim_res$par

dMu <- vParams[1]
```



```

dPhi <- -1 + 2 * (exp(vParams[2]) / (1 + exp(vParams[2])))
dSigma2 <- exp(vParams[3])

print(paste0("Est. Mu: ", dMu, " vs act.: ", 0.5))
#> [1] "Est. Mu: 0.50608151243852 vs act.: 0.5"
print(paste0("Est. Phi: ", dPhi, " vs act.: ", 0.7))
#> [1] "Est. Phi: 0.694273961540576 vs act.: 0.7"
print(paste0("Est. Sigma2: ", dSigma2, " vs act.: ", 1.5))
#> [1] "Est. Sigma2: 1.52925097542527 vs act.: 1.5"

```

6. Create an R package that contains the functions from questions 2., 3. and 5., and edit the title description to “This is my exam package”. Export the package as a bundled development version (with file extension `.tar.gz`), and include it as part of your exam submission. *Remark: If you cannot solve 2., 3. or 5., create a package that contains an R and a C++ function with single scalar inputs that always return the number 42.*

26 Ordinary exam 2024

26.1 Problem 1

Begin by setting the seed to 123.

26.1.1 Problem 1.1

1. Write a function `rndRayleigh` to simulate a random variable using the inversion method.

- First, use the inversion method to simulate data y from the Exponential distribution $y \sim \text{Exp}(\lambda)$ with the parameter $\lambda = 2$.
- The cumulative distribution function (CDF) of the exponential distribution is given as:

$$G(y) = 1 - e^{-\lambda y}.$$

- Define a new random variable $x = \sqrt{y}$.
- Store simulated random variables x in the matrix `mR` of size $T \times N$, where $T = 1000$ and $N = 100$.

Solution:

```
set.seed(123)

rndRayleigh <- function(lambda, size) {

  # lambda - intensity of the exponential distribution
  # size - number of i.i.d. random variables to be drawn

  U <- runif(size)
  return(-1/lambda * log(U))

}

set.seed(123) # set seed

# matrix size and default variables
iT <- 1000
```

```

iN <- 100
lambda <- 2

# simulate data, y
y <- rndRayleigh(lambda, iT * iN)

x <- sqrt(y)

mR <- matrix(x, iT, iN)

```

26.1.2 Problem 1.2

2. Calculate empirical moments using a vector-based command (Lecture on Vector-based programming using functions).
 - Define a constant $\gamma = \frac{1}{\sqrt{2\lambda}}$, and parameters $\mu = \gamma\sqrt{\frac{\pi}{2}}$ and $\sigma^2 = \frac{4-\pi}{2}\gamma^2$.
 - Using a vector-based command, calculate the mean $\bar{\mu}$ and variance $\bar{\sigma}^2$ of each column of the matrix `mR`, and subtract the values of μ and σ^2 , storing them separately in the vectors `vmuError` and `vsigmaError`. Each of these vectors must be of dimension $N \times 1$.
 - Calculate the mean and standard deviation of the elements of the vectors `vmuError` and `vsigmaError`.

Solution:

```

# defining the constants
gamma <- 1 / (sqrt(2 * lambda))
mu <- gamma * sqrt(2 / pi)
sigma2 <- (4 - pi) / 2 * gamma^2

# mean and variance of each column
mu_hat <- colMeans(mR)
sigma2_hat <- apply(mR, 2, var)

# store the errors
vmuError <- mu_hat - mu
vsigmaError <- sigma2_hat - sigma2

# calculate mean and std of these
mean(vmuError)
#> [1] 0.228063
mean(vsigmaError)

```

```
#> [1] -0.0006383356
sqrt(var(vmuError))
#> [1] 0.009235813
sqrt(var(vsigmaError))
#> [1] 0.004542095
```

26.1.3 Problem 1.3

3. Produce histogram and density plots.

- Create a histogram of the data contained in any of the columns of the matrix `mR`.
- Calculate the density function based on the parameter values γ as follows:

$$f(x) = \frac{x}{\gamma^2} \exp\left(-\frac{x^2}{2\gamma^2}\right).$$

- Add a plot of this density function to the histogram.

Solution:

```
# histogram
hist(mR,
     freq = FALSE,
     breaks = 30,
     col = "cornflowerblue",
     xlab = "x",
     ylab = "Density",
     main = "Histogram",
     xlim = c(0, 3))

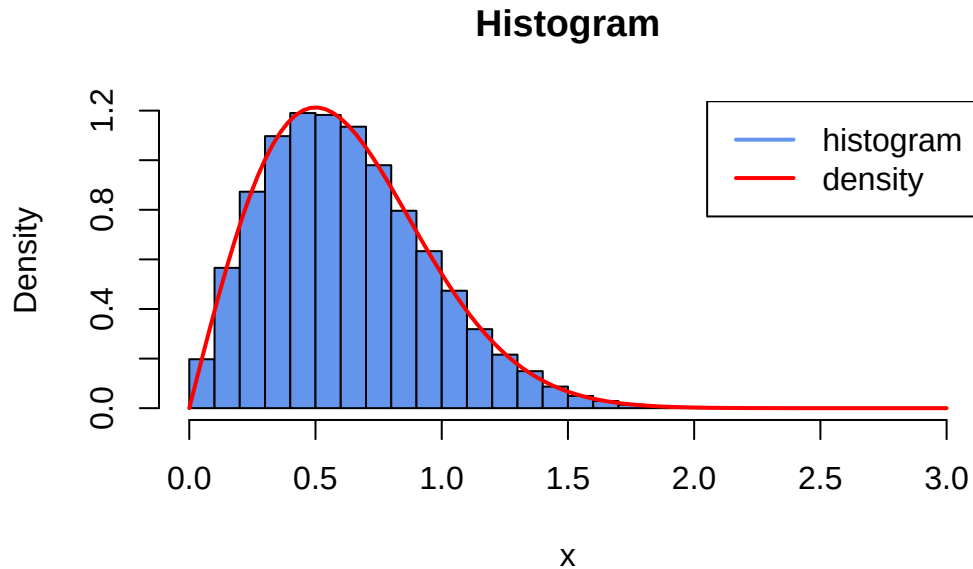
# superimpose the density function
fDensity <- function(x, gamma) {
  return((x / gamma^2) * exp(-(x^2 / (2*gamma^2))))
}

curve(fDensity(x, gamma),
     from = 0,
     to = 3,
     col = "red",
     lwd = 2,
     add = TRUE)
```

```

legend("topright",
      legend = c("histogram", "density"),
      col = c("cornflowerblue", "red"),
      lwd = 2)

```



26.1.4 Problem 1.4

4. Produce histogram and density plot using the Box-Muller algorithm.
 - Simulate two standard random normal variables y_1 and y_2 using the Box-Muller Algorithm, each of size 1000.
 - Define a new random variable $z = \sqrt{y_1^2 + y_2^2}$ and store it in a vector $\mathbf{vz}$. The vector is of size 1000×1 .
 - Plot a histogram of the data contained in the vector $\mathbf{vz}$.
 - Add a density plot to the histogram, where the density is given by

$$f(z) = z \exp\left(-\frac{z^2}{2}\right).$$

Solution:

```

BoxMuller <- function(size = 1) {

  # size: number of standard bivariate normal random variables to simulate

  U <- runif(size)
  V <- runif(size)

  X <- sqrt(-2*log(U)) * cos(2*pi*V)
  Y <- sqrt(-2*log(U)) * sin(2*pi*V)

  return(c(X,Y))
}

y_1 <- BoxMuller(1000)
y_2 <- BoxMuller(1000)

vZ <- sqrt(y_1^2 + y_2^2)

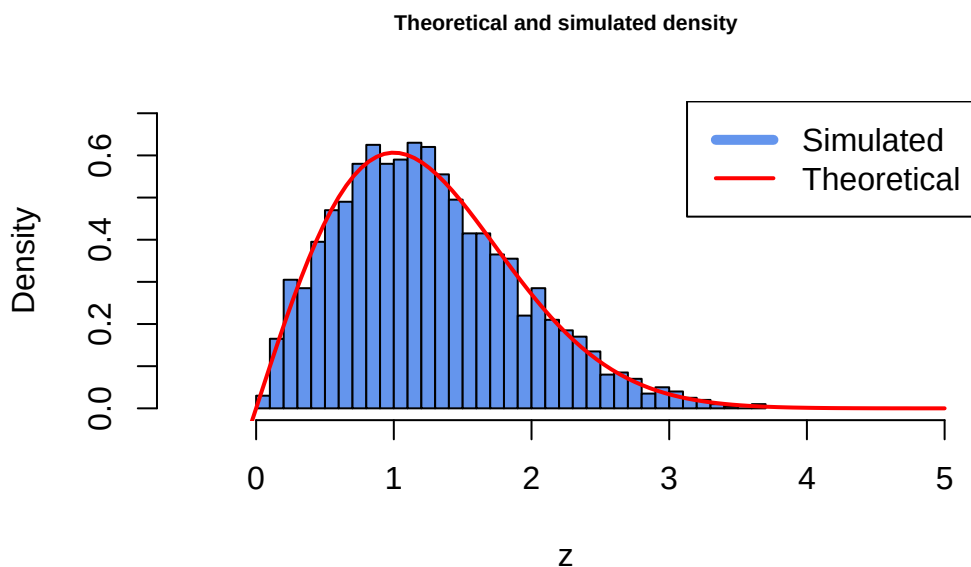
# histogram
hist(vZ, breaks = 35, freq = FALSE,
     main = "Theoretical and simulated density",
     col = "cornflowerblue",
     xlim = c(-0.5, 5), ylim = c(0, 0.7),
     xlab = "z",
     cex.main = 0.7)

fDensityNew <- function(z) {
  return(z * exp(- z^2/2))
}

x <- vZ
curve(fDensityNew(x),
     from = -0.5,
     to = 5,
     col = "red",
     lwd = 2,
     add = TRUE)

legend("topright", legend = c("Simulated", "Theoretical"), lty = c(1, 1), lwd = c(5, 2), col

```



26.2 Problem 2: (Constrained) Optimization, C++, and Packaging

Note: This problem is best solved in order from 1 - 6.

In this problem, you will work with the `d.R` file provided with the exam. It contains a sample of $T = 250$ draws from a binomial distribution with number of trials $n = 10$ and unknown probability p . The probability density function is

$$f(k, n, p) = P(d_t = k) = \binom{n}{k} p^k (1-p)^{n-k},$$

where $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ denotes the binomial coefficient.

26.2.1 Problem 2.1

1. Load the data file `d.R` into your R workspace using `readRDS()`.

Solution:

```
vD <- readRDS("d.R")
```

You will find the maximum likelihood estimator of p by numerically maximizing the average log-likelihood using Newton's Method (recall Lecture 8). For this purpose, you will need the average log-likelihood:

$$\ell(p|n, d) = \frac{1}{T} \sum_{t=1}^T \left[\log \binom{n}{d_t} + d_t \log(p) + (n - d_t) \log(1 - p) \right], \quad (1)$$

the average score:

$$s(p|n, d) = \frac{1}{T} \sum_{t=1}^T \left[\frac{d_t}{p} - \frac{n - d_t}{1 - p} \right], \quad (2)$$

and the average second derivative:

$$sd(p|n, d) = \frac{1}{T} \sum_{t=1}^T \left[-\frac{d_t}{p^2} - \frac{n - d_t}{(1 - p)^2} \right]. \quad (3)$$

(Remark: By $\log$, we always mean the natural logarithm.)

26.2.2 Problem 2.2

2. (a) Write an R function called `log_n_choose_k`, which computes the first term of in equation (1) above. It should accept two integer arguments n and k , and return the log binomial coefficient:

$$\log \binom{n}{k} = \begin{cases} 0 & \text{if } k = 0 \\ \sum_{i=n-k+1}^n \log(i) - \sum_{i=1}^k \log(i) & \text{otherwise} \end{cases}$$

Solution:

```
log_n_choose_k <- function(n, k) {
  if (k == 0) {
    return(0)
  } else {
    dSumPart1 <- 0
    dSumPart2 <- 0
    for (i in (n - k + 1):n) {
      dSumPart1 <- dSumPart1 + log(i)
    }
    for (i in 1:k) {
      dSumPart2 <- dSumPart2 + log(i)
    }
    return(dSumPart1 - dSumPart2)
  }
}
```


- (b) Create a C++ script and write a C++ function called `log_n_choose_k_cpp` which does the same as your R function in part (a).

Solution:

```
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
double log_n_choose_k_cpp(int n, int k) {

    if (k == 0) {
        return(0);
    } else {
        double dSumPart1 = 0;
        double dSumPart2 = 0;
        for (int i = (n - k + 1); i <= n; i++) {
            dSumPart1 = dSumPart1 + log(i);
        }
        for (int i = 1; i <= k; i++) {
            dSumPart2 = dSumPart2 + log(i);
        }
        return(dSumPart1 - dSumPart2);
    }
}
```

```
suppressMessages(library(Rcpp))
#> Warning: pakke 'Rcpp' blev bygget under R version 4.3.3
sourceCpp("cpp_functions.cpp")

# naive check that the functions are equal
log_n_choose_k(10, 3)
#> [1] 4.787492
log_n_choose_k_cpp(10, 3)
#> [1] 4.787492
```

26.2.3 Problem 2.3

3. (a) Write three R functions `fLnL`, `fScore`, and `fSD` which implement equations (1)-(3) respectively. They should accept a decimal number p , an integer n , and an integer vector d as arguments.

Solution:

```
fLnL <- function(p, n, d) {
  result <- numeric(length(d))

  for (i in 1:length(d)) {
    result[i] <- log_n_choose_k(n, d[i]) + d[i] * log(p) + (n - d[i]) * log(1 - p)
  }

  return(mean(result))
}

fScore <- function(p, n, d) {
  return(mean(d / p - (n - d) / (1 - p)))
}

fSD <- function(p, n, d) {
  return(mean(-(d / p^2) - (n - d) / ((1 - p)^2)))
}
```

- (b) Write three C++ functions `fLnL_cpp`, `fScore_cpp`, and `fSD_cpp`, which do the same as your R functions in part (a). In your R script, include a `sourceCpp()` command from the Rcpp package which compiles the C++ script and adds all C++ functions from questions 2 and 3 to your environment. *Remark: If you cannot solve question 2, you may skip the $\log\binom{n}{k}$ term in the `fLnL` and `fLnL_cpp` functions.*

```
// [[Rcpp::export]]
double fLnL_cpp(double p, int n, vec d) {
  int len = d.n_elem;
  vec result(len);

  for (int i = 0; i < len; i++) {
    result[i] = log_n_choose_k_cpp(n, d[i]) + d[i] * log(p) + (n - d[i]) * log(1 - p);
  }
  return(mean(result));
}

// [[Rcpp::export]]
double fScore_cpp(double p, int n, vec d) {
  return(mean(d / p - (n - d) / (1 - p)));
}

// [[Rcpp::export]]
```

```
double fSD_cpp(double p, int n, vec d) {
  return(mean(-(d / pow(p, 2)) - (n - d) / pow((1 - p), 2)));
}
```

26.2.4 Problem 2.4

4. Write an R function called `Newton` which maximizes the average log likelihood in equation (1) using Newton's Method. This function should have the following arguments: a function to be maximized, its first derivative, its second derivative, a starting value, a tolerance threshold, maximum number of iterations, and other arguments which need to be passed the function you're maximizing and its derivatives (i.e. $n = 10$ and d). It should return a list containing the value which maximizes the function, the maximum itself, and it should give an indication whether convergence was achieved or not.

Run this function using `fLnL_cpp`, `fScore_cpp`, and `fSD_cpp` as the inputs, and use starting value $p = 0.5$. Compare the optimal value of p you find with the analytical maximum $p^* = (\frac{1}{T} \sum_{t=1}^T d_t) / n$. Remark: You may adapt the example code from Lecture 8 if you wish. If you cannot implement the C++ functions in questions 2 or 3, you may use the corresponding R functions to do this optimization instead, for partial credit.

Solution:

```
Newton <- function(f, f_prime, f_sec, X0, Tol = 1e-9, max.iter = 1000, ...){
  dX <- X0
  fx <- f(dX, ...)
  fpx <- f_prime(dX, ...)
  fsx <- f_sec(dX, ...)
  i <- 0
  while ((abs(fpx) > Tol) && (i < max.iter)) {
    dX <- dX - fpx/fsx
    fx <- f(dX, ...)
    fpx <- f_prime(dX, ...)
    fsx <- f_sec(dX, ...)
    i <- i + 1
    #cat("At iteration", n, "the value of x is:", dX, "\n")
  }
  if (i == max.iter) {
    return(
      list(
        maximizing.val = NULL,
        maximum = NULL,
        "Algorithm failed to converge. Maximum iterations reached.")
    )
  }
}
```

```

    )
  } else {
    return(
      list(
        maximizing.val = dX,
        maximum = f(dX, ...),
        "Algorithm converged.")
      )
  }
}

# Newton max
Newton(fLnL_cpp, fScore_cpp, fSD_cpp, 0.5, n = 10, d = vD)
#> $maximizing.val
#> [1] 0.2032
#>
#> $maximum
#> [1] -1.543041
#>
#> [[3]]
#> [1] "Algorithm converged."

# Analytical max
p.star <- mean(vD) / 10
p.star
#> [1] 0.2032

```

Suppose you obtain additional information that the p which maximizes the likelihood lies somewhere between 0.1 and 0.9. You will use reparameterization to enforce this constraint. Consider the transformation $p = \lambda(\tilde{p}) = 0.1 + 0.8 \frac{\exp(\tilde{p})}{1 + \exp(\tilde{p})}$, such that p is between 0.1 and 0.9 for any value of $\tilde{p}$. This transformation has a corresponding transformed average log-likelihood

$$\tilde{\ell}(\tilde{p}|n, d) = \ell(\lambda(\tilde{p})|n, d) = \frac{1}{T} \sum_{t=1}^T \left[\log \binom{n}{d_t} + d_t \log(\lambda(\tilde{p})) + (n - d_t) \log(1 - \lambda(\tilde{p})) \right]. \quad (4)$$

26.2.5 Problem 2.5

5. (a) Write a new R function `fLnL_t`, which implements equation (4).

Solution:

```
fLnL_t <- function(p, n, d) {
  new_p <- 0.1 + 0.8 * (exp(p)) / (1 + exp(p))
  result <- numeric(length(d))

  for (i in 1:length(d)) {
    result[i] <- log_n_choose_k(n, d[i]) + d[i] * log(new_p) + (n - d[i]) * log(1 - new_p)
  }

  return(mean(result))
}
```

- (b) Maximize the transformed average log-likelihood using the function `optim` (or `optimize` (your choice)), paying close attention to the initial values/bounds that you use. Report the maximum and maximizer you find, making sure that those results are transformed back into the original scale, such that they are comparable with the results you found in question 4.

Solution:

```
dOptimalP <- optimize(fLnL_t, lower = -1000, upper = 1000, n = 10, d = vD, maximum = TRUE)$o
dOptimalP.orig.scale <- 0.1 + 0.8 * (exp(dOptimalP)) / (1 + exp(dOptimalP))

dOptimalP.orig.scale
#> [1] 0.1942476
```

26.2.6 Problem 2.6

6. Create an R package that contains the all functions you wrote for questions 2-5, and edit the Title of your package in the description file to “This is my exam package”. Export the package as a bundled development version (with file extension `.tar.gz`), and include it as part of your exam submission. *Remark: If you cannot solve questions 2-5, create a package that contains an R and a C++ function with single scalar input that always return the number 2024.*

27 Retake exam 2024

27.1 Problem 1

27.1.1 Problem 1.1

1. Monte Carlo Integration

- Consider a function

$$f(u) = -\frac{\ln(1-u)}{\lambda},$$

where $\lambda > 0$ and $u \in (0, 1)$. * Consider the following integral

$$\gamma = \frac{1}{1-\alpha} \int_{\alpha}^1 f(u) du$$

- Evaluate the above integral for $\alpha = 0.95$ and $\lambda = 2$ using the Monte Carlo integration technique.
- Compare the result of your calculations with the following expression

$$\gamma = \frac{-\ln(1-\alpha) + 1}{\lambda}.$$

Solution:

```
MonteCarlo.Integration <- function(f, n, a, b) {  
  
  U <- runif(n, min = a, max = b)  
  return( (b-a)*mean(f(U)) )  
  
}  
  
set.seed(123)  
  
dAlpha <- 0.95
```

```

dLambda <- 2

# Monte Carlo Integration
dGammaMI <- MonteCarlo.Integration(function(x) (-log(1-x)) / dLambda), 100, dAlpha, 1)
dGammaMI
#> [1] 0.09927233

# Numerical evaluation
dGammaNE <- (-log(1-dAlpha) + 1) / dLambda
dGammaNE
#> [1] 1.997866

# Thus, we should have this relationship
# dGammaNE * (1 - dAlpha) approx dGammaMI
dGammaNE * (1 - dAlpha)
#> [1] 0.09989331

```

27.1.2 Problem 1.2

```

mU <- matrix(runif(40000), 200, 200)
mU <- ifelse(mU < 0.5, -1, 1)

sum(mU == 1)
#> [1] 19715
sum(mU == -1)
#> [1] 20285

```

2. Vectorized if/else statement

- Generate a 200×200 matrix where each entry is a standard Uniform random variable.
- Assign the value -1 to those elements of the matrix that are less than 0.5 and 1 to those that are larger than or equal to 0.5.
- Count how many entries are -1 and 1, and report the counts.

Solution:

27.1.3 Problem 1.3

3. Recursive functions

- Consider the following power function that calculates z^k written as

$$\text{Power}(z, k) = \begin{cases} 1 & \text{if } k = 0 \\ z \times \text{Power}(z, k - 1) & \text{if } k > 0 \end{cases}$$

- Write the recursive function.

Solution:

```
fPower <- function(z, k) {
  if (k == 0) {
    return(1)
  } else if (k > 0) {
    return(z * fPower(z, k - 1))
  } else {
    return(NULL)
  }
}
# Test
fPower(2, 4)
#> [1] 16
```

27.1.4 Problem 1.4

4. Simulation by Application of the Inversion Method

- Begin by setting the seed to 134.
- Suppose $\mu = 0.5$ is a location parameter and $\gamma = 2$.
- Use the inversion method to simulate 1000 Cauchy-distributed random variables using the following inverse cumulative distribution function:

$$g(u) = \mu + \gamma \tan \left(\pi \left(u - \frac{1}{2} \right) \right),$$

where $\tan()$ stands for the trigonometric tangent function.

- Plot a histogram using the variables you have just generated.

Solution:

```
set.seed(134)
dMu <- 0.5
dGamma <- 2

Cauchy.Simulate <- function(mu, lambda, size) {
```

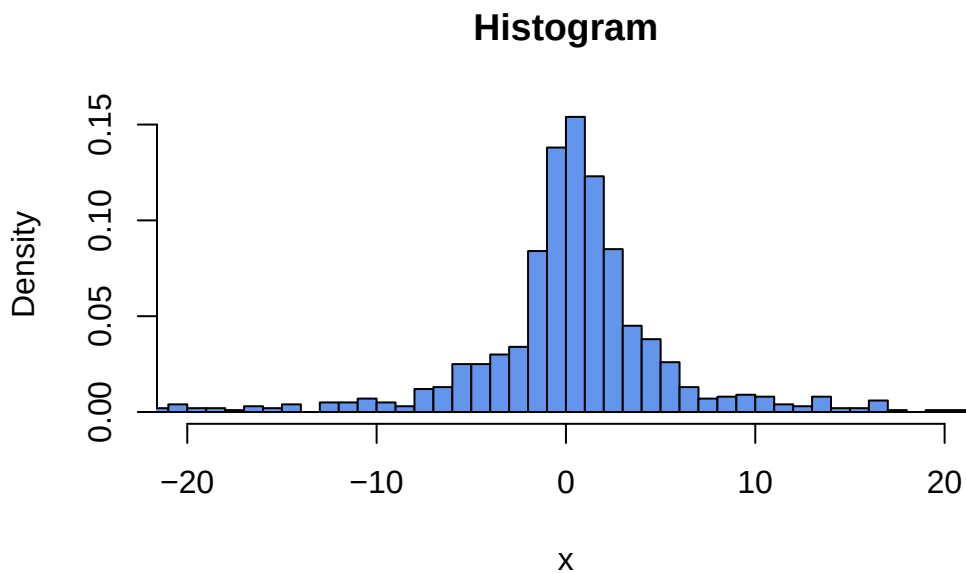


```

U <- runif(size)
return(mu + lambda * tan(pi * (U - 1/2)))
}

vX <- Cauchy.Simulate(dMu, dGamma, 1000)
hist(vX,
     freq = FALSE,
     breaks = 5000,
     col = "cornflowerblue",
     xlab = "x",
     ylab = "Density",
     main = "Histogram",
     xlim = c(-20, 20))

```



27.2 Problem 2: (Constrained) Optimization, C++, and Packaging

Note: This problem is best solved in order from 1 - 5.

In this problem, you will numerically maximize the negative Rosenbrock function

$$f(x, y) = -(\pi - x)^2 - 100(y - x^2)^2, \quad (1)$$

where π is the mathematical constant 3.141593... You will use the Steepest Ascent Method (recall Lecture 9). For this purpose, you will need the gradient of f :

$$\nabla f(x, y) = \begin{pmatrix} 2\pi - 2x - 400x^3 + 400xy \\ 200(x^2 - y) \end{pmatrix}. \quad (2)$$

27.2.1 Problem 2.1

1. (a) Write R functions called `fRosen` and `fRosen_grad` which implement equations (1) and (2).

Solution:

```
fRosen <- function(vX) {
  return(-(pi - vX[1])^2 - 100 * (vX[2] - vX[1]^2)^2)
}

fRosen_grad <- function(vX) {
  vOutput <- numeric(2)
  vOutput[1] <- 2 * pi - 2 * vX[1] - 400 * vX[1]^3 + 400 * vX[1] * vX[2]
  vOutput[2] <- 200 * (vX[1]^2 - vX[2])
  return(vOutput)
}
```

- (b) Create a C++ script and write C++ functions called `fRosen_cpp` and `fRosen_grad_cpp` which do the same as your R functions in part (a). In your R script, include a `sourceCpp()` command from the `Rcpp` package which compiles the C++ script and adds all C++ functions from questions 2 and 3 to your environment.

Solution:

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
using namespace Rcpp;
using namespace arma;

// [[Rcpp::export]]
double fRosen_cpp(vec vX) {
  double pi = atan(1)*4; //pi
  return(-pow((pi - vX[0]), 2) - 100 * pow((vX[1] - pow(vX[0], 2)), 2));
}
```

```
// [[Rcpp::export]]
vec fRosen_grad_cpp(vec vX) {
  vec vOutput(2);
  double pi = atan(1)*4; //pi
  vOutput[0] = 2 * pi - 2 * vX[0] - 400 * pow(vX[0], 3) + 400 * vX[0] * vX[1];
  vOutput[1] = 200 * (pow(vX[0], 2) - vX[1]);
  return(vOutput);
}
```

```
suppressMessages(library(Rcpp))
#> Warning: pakke 'Rcpp' blev bygget under R version 4.3.3
suppressMessages(library(RcppArmadillo))
#> Warning: pakke 'RcppArmadillo' blev bygget under R version 4.3.3
sourceCpp("cpp_functions_retake_2024.cpp")

# sanity checks
fRosen(c(0.5, 2))
#> [1] -313.228
fRosen_cpp(c(0.5, 2))
#> [1] -313.228
fRosen_grad(c(0.5, 2))
#> [1] 355.2832 -350.0000
fRosen_grad_cpp(c(0.5, 2))
#> [1,] 355.2832
#> [2,] -350.0000
```

27.2.2 Problem 2.2

2. (a) Write an R function called **ascent** which implemented the Steepest Ascent Method with the Golden Section Method used to compute the step size. The function should accept as arguments the function to be maximized, its gradient, a starting value, a tolerance level, and a maximum number of iterations. It should output a list containing the function maximum and the vector at which the maximum is attained. In case the algorithm does not converge, the list should contain **-Inf** as the function maximum and **NA** as the vector.

Solution:

```
# golden section algorithm from last week
gsection <- function(f, dX.l, dX.r, dX.m, dTol = 1e-9) {
```

```

# golden ratio plus one
dGR1 <- 1 + (1 + sqrt(5))/2

# successively refine x.l, x.r, and x.m
f.l <- f(dX.l)
f.r <- f(dX.r)
f.m <- f(dX.m)
while ((dX.r - dX.l) > dTol) {
  if ((dX.r - dX.m) > (dX.m - dX.l)) { # if the right segment is wider than the left
    dY <- dX.m + (dX.r - dX.m)/dGR1 # put Y into the right segment according to the golden
    f.y <- f(dY)
    if (f.y >= f.m) {
      dX.l <- dX.m
      f.l <- f.m
      dX.m <- dY
      f.m <- f.y
    } else {
      dX.r <- dY
      f.r <- f.y
    }
  } else { #if the left segment is wider than the right
    dY <- dX.m - (dX.m - dX.l)/dGR1 # put Y into the left segment according to the golden
    f.y <- f(dY)
    if (f.y >= f.m) {
      dX.r <- dX.m
      f.r <- f.m
      dX.m <- dY
      f.m <- f.y
    } else {
      dX.l <- dY
      f.l <- f.y
    }
  }
}
return(dX.m)
}

line.search <- function(f, vX, vG, dTol = 1e-9, dA.max = 2^5) {
  # f is a real function that takes a vector of length d
  # x and y are vectors of length d
  # line.search uses gsection to find a >= 0 such that
  # g(a) = f(x + a*y) has a local maximum at a,

```

```

# within a tolerance of tol
# if no local max is found then we use 0 or a.max for a
# the value returned is x + a*y
if (sum(abs(vG)) == 0){
  return(vX) # +0*vG
} # g(a) constant
g <- function(dA){
  return(f(vX + dA*vG))
}
# find a triple  $a.l < a.m < a.r$  such that
#  $g(a.l) \leq g(a.m)$  and  $g(a.m) \geq g(a.r)$ 

# choose a.l
dA.l <- 0
g.l <- g(dA.l)
# find a.m
dA.m <- 1
g.m <- g(dA.m)
while ((g.m < g.l) & (dA.m > dTol)) {
  dA.m <- dA.m/2
  g.m <- g(dA.m)
}
# if a suitable a.m was not found then use 0 for a, so just return vX as the next step
if ((dA.m <= dTol) & (g.m < g.l)){
  return(vX)
}
# find a.r
dA.r <- 2*dA.m
g.r <- g(dA.r)
while ((g.m < g.r) & (dA.r < dA.max)) {
  dA.m <- dA.r
  g.m <- g.r
  dA.r <- 2*dA.m
  g.r <- g(dA.r)
}
# if a suitable a.r was not found then use a.max for a
if ((dA.r >= dA.max) & (g.m < g.r)){
  return(vX + dA.max*vG)
}
# apply golden-section algorithm to g to find a
dA <- gsection(g, dA.l, dA.r, dA.m)
return(vX + dA*vG)

```

```

}

ascent <- function(f, grad.f, vX0, dTol = 1e-9, n.max = 100) {
  vX.old <- vX0
  vX <- line.search(f, vX0, grad.f(vX0))
  n <- 1
  while ((f(vX) - f(vX.old) > dTol) & (n < n.max)) {
    vX.old <- vX
    vX <- line.search(f, vX, grad.f(vX))
    #cat("at iteration", n, "the coordinates of x are", vX, "\n")
    n <- n + 1
  }
  if (n == n.max) {
    return(
      list(
        maximizing.val = NA,
        maximum = -Inf,
        paste0("Algorithm failed to converge. Maximum iterations reached. Last values:", vX)
      )
    )
  } else {
    return(
      list(
        maximizing.val = vX,
        maximum = f(vX),
        "Algorithm converged."
      )
    )
  }
}

```

- (b) Use this function to attempt to maximize your `fRosen_cpp` function, using `fRosen_grad_cpp` as its gradient, and starting value (1,1). This function is difficult to optimize, so it will likely fail to converge.

Remark: You may adapt the example code from Lecture 9 if you wish, including the functions `gsection` and `line.search`. If you did not solve question 1(b), you can use the R functions instead of the C++ ones for partial credit.

Solution:

```

ascent(fRosen_cpp, fRosen_grad_cpp, vX0 = c(1, 1))
#> $maximizing.val
#> [1] NA
#>

```

```
#> $maximum
#> [1] -Inf
#>
#> [[3]]
#> [1] "Algorithm failed to converge. Maximum iterations reached. Last values:1.2071304159174
#> [2] "Algorithm failed to converge. Maximum iterations reached. Last values:1.4571638511778"
```

27.2.3 Problem 2.3

You will now write a function which systematically applies the ascent function over a grid of starting values. To define the grid, let vector $(x_1, x_2, \dots, x_K)$ and $(y_1, y_2, \dots, y_K)$ be the grid coordinates. These are typically in ascending order and evenly spaced. The grid consists of all K^2 points (x_i, y_j) , $i, j \in \{1, \dots, K\}$.

3. (a) Write a new R function `grid_ascent` which has inputs similar to the `ascent` function, but instead of a single initial value, it accepts two coordinate vectors. The function should evaluate the `ascent` function starting at each grid point. It should return a $K \times K$ matrix containing the corresponding function maxima found using these starting points.

Solution:

```
grid_ascent <- function(f, grad.f, vX0, vY0, dTol = 1e-9, n.max = 100) {
  mOutput <- matrix(NA, length(vX0), length(vY0))

  for (x in 1:length(vX0)) {
    for (y in 1:length(vY0)) {
      vXY0 <- c(vX0[x], vY0[y])
      vXY.old <- vXY0
      vXY <- line.search(f, vXY0, grad.f(vXY0))
      n <- 1
      while ((f(vXY) - f(vXY.old) > dTol) & (n < n.max)) {
        vXY.old <- vXY
        vXY <- line.search(f, vXY, grad.f(vXY))
        n <- n + 1
      }
      if (n == n.max) {
        mOutput[x, y] <- -Inf
      } else {
        mOutput[x, y] <- f(vXY)
      }
    }
  }
}
```

```

}
return(mOutput)
}

```

- (b) Choose an appropriate grid, and use the output of the `grid_ascent` function to find a starting point from which the `ascent` function converges to a maximum. To give you an idea where to start, try a grid with x values around 3 and y values around 9. To begin with, try a relatively small K (e.g. $K = 5$) because the run time can be significant for large K . If your grid cannot find any valid starting point, i.e. `grid_ascent` returns a matrix of all `-Inf`, here are some things you can try: make the grid larger by making the min and max of the coordinates further apart, make the grid finer by increasing K , choose a smaller tolerance, or increase the maximum number of iterations. Do these changes gradually, as most of them will make the function run slower.

Solution:

```

grid_ascent(fRosen_cpp, fRosen_grad_cpp, vX0 = c(2.5, 2.75, 3, 3.25, 3.5), vY0 = c(8.5, 8.75)
#>      [,1]      [,2] [,3]      [,4] [,5]
#> [1,] -Inf      -Inf -Inf      -Inf -Inf
#> [2,] -Inf      -Inf -Inf      -Inf -Inf
#> [3,] -Inf -2.871553e-10 -Inf      -Inf -Inf
#> [4,] -Inf      -Inf -Inf -1.443747e-09 -Inf
#> [5,] -Inf      -Inf -Inf      -Inf -Inf

```

- (c) When you find a grid point from which `ascent` converges, evaluate `ascent` at this starting point and report the function maximum and the point at which this maximum is achieved.

Solution:

```

ascent(fRosen_cpp, fRosen_grad_cpp, vX0 = c(3, 8.75))
#> $maximizing.val
#>      [,1]
#> [1,] 3.141604
#> [2,] 9.869677
#>
#> $maximum
#> [1] -2.871553e-10
#>
#> [[3]]
#> [1] "Algorithm converged."
ascent(fRosen_cpp, fRosen_grad_cpp, vX0 = c(3.25, 9.25))
#> $maximizing.val
#>      [,1]

```



```
#> [1,] 3.141555
#> [2,] 9.869366
#>
#> $maximum
#> [1] -1.443747e-09
#>
#> [[3]]
#> [1] "Algorithm converged."
```

27.2.4 Problem 2.4

4. Maximize your `fRosen_cpp` function with the `optim` function in R, using starting point $(1, 1)$, `fRosen_cpp_cpp` for the `gr` argument, and `method = "BFGS"`. Compare the result with your answer in question 3.

Solution:

```
optim(par = c(1, 1), fn = fRosen_cpp, gr = fRosen_grad_cpp, method = "BFGS", control = list(
#> $par
#> [1] 3.141593 9.869604
#>
#> $value
#> [1] -7.901082e-19
#>
#> $counts
#> function gradient
#>      154      62
#>
#> $convergence
#> [1] 0
#>
#> $message
#> NULL
```

27.2.5 Problem 2.5

5. Create an R package that contains all the functions you wrote for questions 1-3, and edit the Title of your package in the description file to “This is my exam package”. Export the package as a bundled development version (with file extension `.tar.gz`), and include it as part of your exam submission.

Remark: If you cannot solve questions 1-3, create a package that contains an R and a C++ function with single scalar input that always return the number 2024.2.

28 Extra exercises

29 Exercise Set 1: R Basics, Data Structures, and Control Flow

29.1 (1)

- a) Create a vector `vNumbers` containing integers from 1 to 50.
- b) Calculate the sum of all even numbers in `vNumbers`.
- c) Create a new vector `vLog` containing the natural logarithm of each number in `vNumbers`. Handle any potential warnings or errors if numbers were 0 or negative (though not the case here).
- d) What is the data type of `vLog`?

```
vNumbers <- 1:50
sum(vNumbers[vNumbers %% 2 == 0])
#> [1] 650
vLog <- log(vNumbers)
typeof(vLog)
#> [1] "double"
```

29.2 (2)

- a) Create a 5x5 matrix `mRandom` with elements drawn from a uniform distribution between -10 and 10. Set the seed to 42.
- b) Replace all negative values in `mRandom` with NA.
- c) Calculate the sum of each row, ignoring NA values.
- d) Calculate the mean of each column, ignoring NA values.

```
set.seed(42)
mRandom <- matrix(runif(25, -10, 10), 5, 5)
mRandom[mRandom < 0] <- NA
rowSums(mRandom, na.rm = TRUE)
#> [1] 25.558958 27.420048 18.471280 18.682163 8.142861
colMeans(mRandom, na.rm = TRUE)
#> [1] 6.620373 3.088707 6.537845 6.523825 8.930609
```

29.3 (3)

- Write an R script that does the following:
 - Initializes a variable `dTotalSum` to 0.
 - Uses a `for` loop to iterate from 1 to 100.
 - In each iteration, if the current number `i` is a multiple of 3 or a multiple of 5, add `i` to `dTotalSum`.
 - Print the final `dTotalSum`.

```
dTotalSum <- 0
for (i in 1:100) {
  if (i %% 3 == 0 | i %% 5 == 0) {
    dTotalSum <- dTotalSum + i
  }
}
dTotalSum
#> [1] 2418
```

29.4 (4)

- Create a list `lMyList` with the following named elements:
 - `sName`: Your name (as a string).
 - `vLuckyNumbers`: A vector of your 3 favorite numbers.
 - `mMatrixA`: A 2x2 identity matrix.
 - `dfInnerData`: A data frame with two columns: `Month` (Jan, Feb, Mar) and `Rainfall_mm` (50, 30, 65).
- Access and print the `Rainfall_mm` for February.
- Add a new element to the list called `sCourse` with the value “PQE”.

```
lMyList <- list(
  sName = "MyName",
  vLuckyNumbers = c(1, 3, 5),
  mMatrixA = identity(2),
  dfInnerData = data.frame(
    Month = c("Jan", "Feb", "Mar"),
    Rainfall_mm = c(50, 30, 65)
  )
)
lMyList$dfInnerData$Rainfall_mm[lMyList$dfInnerData$Month == "Feb"]
```

```
#> [1] 30
lMyList$sCourse <- "PQE"
```

29.5 (5)

- a) Create a data frame `dfStudents` with 5 students and 3 variables: `StudentID` (101 to 105), `Grade` (A, B, C, A, B), and `Age` (20, 21, 20, 22, 21).
- b) Convert the `Grade` column to an ordered factor with levels `C < B < A`.
- c) Add a new column `Pass` which is `TRUE` if `Grade` is A or B, and `FALSE` otherwise.
- d) Create a subset of `dfStudents` containing only students older than 20.

```
dfStudents <- data.frame(
  StudentID = 101:105,
  Grade = c("A", "B", "C", "A", "B"),
  Age = c(20, 21, 20, 22, 21)
)
dfStudents$Grade <- factor(dfStudents$Grade, levels = c("C", "B", "A"), ordered = TRUE)
dfStudents$Pass <- ifelse(dfStudents$Grade == "A" | dfStudents$Grade == "B", TRUE, FALSE)
dfStudents[dfStudents$Age > 20, ]
#>   StudentID Grade Age Pass
#> 2         102    B  21 TRUE
#> 4         104    A  22 TRUE
#> 5         105    B  21 TRUE
```

29.6 (6)

- Using `ifelse()`, create a vector `vSign` of length 20. For each element `i` from -9 to 10, `vSign[i+10]` should be -1 if `i` is negative, 0 if `i` is zero, and 1 if `i` is positive. (Adjust indexing as R vectors are 1-indexed).

```
vI <- -9:10
vSign <- ifelse(vI < 0, -1, ifelse(vI == 0, 0, 1))
vSign
#> [1] -1 -1 -1 -1 -1 -1 -1 -1 -1 0 1 1 1 1 1 1 1 1 1
```

29.7 (7)

- Write a `while` loop that simulates flipping a fair coin (0 for Tails, 1 for Heads) until 3 consecutive Heads are observed. Count and print the total number of flips. Set seed to 101.

```
set.seed(101)
iFlipCount <- 0
iConsecHeads <- 0
while (iConsecHeads < 3) {
  iFlip <- sample(0:1, 1)
  iFlipCount <- iFlipCount + 1
  if (iFlip == 1) {
    iConsecHeads <- iConsecHeads + 1
  } else {
    iConsecHeads <- 0
  }
}
iFlipCount
#> [1] 36
```

29.8 (8)

- a) Create a character vector `vsFruits` with elements: “apple”, “banana”, “cherry”, NA, “date”.
- b) How many missing values are in `vsFruits`?
- c) Create a new vector `vsFruitsClean` by removing NA values from `vsFruits`.
- d) For each fruit in `vsFruitsClean`, print the fruit name and its number of characters using `nchar()`.

```
vsFruits <- c("apple", "banana", "cherry", NA, "date")
sum(is.na(vsFruits))
#> [1] 1
vsFruitsClean <- vsFruits[which(!is.na(vsFruits))]
for (fruit in vsFruitsClean) {
  print(fruit)
  print(nchar(fruit))
}
#> [1] "apple"
#> [1] 5
#> [1] "banana"
```

```
#> [1] 6
#> [1] "cherry"
#> [1] 6
#> [1] "date"
#> [1] 4
```

29.9 (9)

- Explain the difference between `&` and `&&`, and between `|` and `||` in R. Provide a small example where `&&` or `||` would be preferred over `&` or `|` due to short-circuiting, especially when dealing with potentially undefined objects or expensive computations.

```
#`&` and `|` are element-wise. `&&` and `||` are short-circuiting, evaluating only the first
x <- NULL
# No error
if (FALSE && x[1] > 5) {
  print("Not evaluated")
} else {
  print("OK")
}
# Error if x[1] is problematic
if (FALSE & x[1] > 5) {
  print("Error here")
}
```

29.10 (10)

- Consider the matrix `mA <- matrix(1:12, nrow=4, byrow=TRUE)`.
- Extract the second row.
- Extract the third column.
- Extract the element in the 1st row and 2nd column.
- Create a new matrix `mB` by selecting rows 1 and 3, and columns 2 and 3 from `mA`.

```
mA <- matrix(1:12, nrow = 4, byrow = TRUE)
mA[2, ]
#> [1] 4 5 6
mA[, 3]
#> [1] 3 6 9 12
```



```
mA[1, 2]
#> [1] 2
mB <- mA[c(1, 3), c(2, 3)]
mB
#>      [,1] [,2]
#> [1,]    2    3
#> [2,]    8    9
```

30 Exercise Set 2: Functions, Scope, and Efficiency

30.1 (1)

- Write an R function `fCalculateStats` that takes a numeric vector as input.
- The function should return a list containing:
 - `dMean`: The mean of the vector.
 - `dMedian`: The median of the vector.
 - `dSD`: The standard deviation of the vector.
 - `iNAs`: The number of NA values in the vector.
- The function should handle NA values gracefully by removing them before calculating mean, median, and SD.
- Test your function with `vTest <- c(1, 5, NA, 7, 3, NA, 8)`.

```
fCalculateStats <- function(vInput) {  
  outputList <- list(  
    dMean = mean(vInput, na.rm = TRUE),  
    dMedian = median(vInput, na.rm = TRUE),  
    dSD = sqrt(var(vInput, na.rm = TRUE)),  
    iNAs = sum(is.na(vInput))  
  )  
  return(outputList)  
}  
vTest <- c(1, 5, NA, 7, 3, NA, 8)  
fCalculateStats(vTest)  
#> $dMean  
#> [1] 4.8  
#>  
#> $dMedian  
#> [1] 5  
#>  
#> $dSD  
#> [1] 2.863564
```

```
#>
#> $iNAs
#> [1] 2
```

30.2 (2)

- Consider the following R code:

```
x <- 10
my_func <- function(y) {
  x <- 5
  return(x + y)
}
z <- my_func(3)
```

- What are the values of `x` and `z` in the global environment after this code is executed? Explain R's scoping rules in this context.

The value of `z` is 8, and the value of `x` is 10. `x` is local within the function.

30.3 (3)

- Write a function `fSumSquaresUpToN` that calculates the sum of squares of integers from 1 up to `n` (i.e., $1^2 + 2^2 + \dots + n^2$).
- Implement this first using a `for` loop.
- Implement this second using a vectorized approach (without an explicit loop).
- Use `system.time()` or `microbenchmark()` to compare the execution time of both versions for `n = 10000`.

```
suppressMessages(library(microbenchmark))
#> Warning: pakke 'microbenchmark' blev bygget under R version 4.3.3

fSumSquaresUpToN <- function(iN) {
  iSumSquares <- 0
  for (i in 1:iN) {
    iSumSquares <- iSumSquares + i^2
  }
}
```

```
fSumSquaresVector <- function(iN) {
  return(sum((1:iN)^2))
}

microbenchmark(fSumSquaresVector(10000), fSumSquaresUpToN(10000))
#> Unit: microseconds
#>          expr      min       lq      mean  median      uq      max neval
#> fSumSquaresVector(10000) 22.9   33.55   64.238    58.4   61.20 1534.3   100
#> fSumSquaresUpToN(10000) 222.6  223.30  246.753   223.7  224.45 2383.1   100
```

30.4 (4)

- Write a recursive R function `fRecursiveFactorial` to calculate the factorial of a non-negative integer `n`. Include basic error checking for negative inputs.

```
fRecursiveFactorial <- function(n) {
  if (n < 0) stop("Input must be non-negative")
  if (n == 0) return(1)
  else return(n * fRecursiveFactorial(n - 1))
}
fRecursiveFactorial(5)
#> [1] 120
```

30.5 (5)

- Write a function `fApplyToMatrix` that takes a matrix and a function `FUN` as arguments.
- The function should apply `FUN` to each column of the matrix using `sapply` or `apply`.
- Test it by creating a 5x3 matrix of random numbers and applying the `mean` function to its columns.

```
fApplyToMatrix <- function(mInput, FUN) {
  return(apply(mInput, 2, FUN))
}

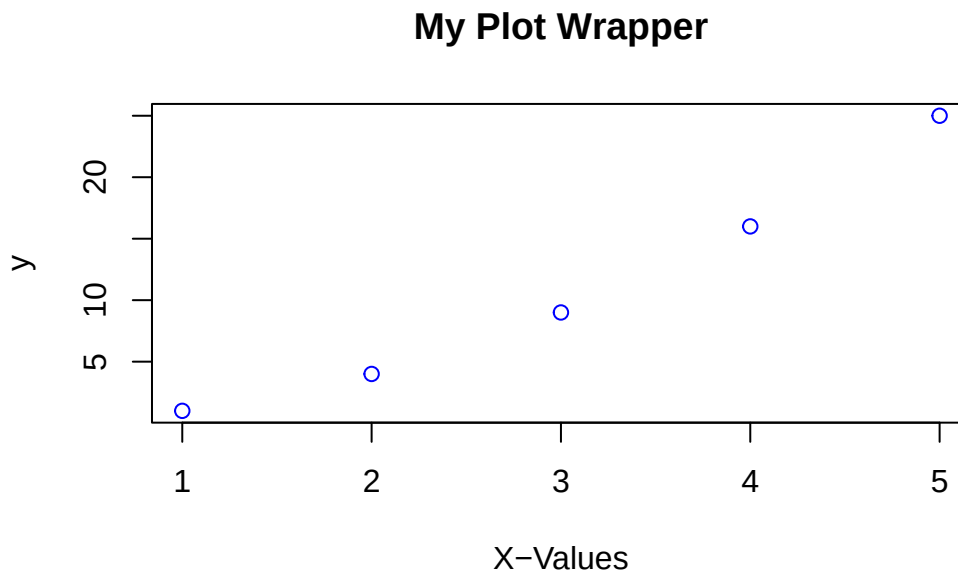
mA <- matrix(rnorm(15), 5, 3)
fApplyToMatrix(mA, mean)
#> [1] 0.360807854 -0.005839041 -0.312895340
```

30.6 (6)

- Explain the purpose of the ellipsis (...) argument in R functions. Provide an example of a simple function you write that uses ... to pass arguments to an internal R function (e.g., plot or mean).

It passes additional arguments.

```
fCalculateMean <- function(vInput, ...) {  
  return(mean(vInput, ...))  
}  
vX <- c(1, 2, NA, 5)  
fCalculateMean(vX, na.rm = TRUE)  
#> [1] 2.666667  
  
myPlotWrapper <- function(x, y, ...) {  
  plot(x, y, main = "My Plot Wrapper", ...)  
}  
myPlotWrapper(1:5, (1:5)^2, xlab = "X-Values", col = "blue")
```



30.7 (7)

- Create two numeric vectors, `vA = 1:1000000` and `vB = rnorm(1000000)`.
- Compare the time it takes to calculate their element-wise product using:
 - A `for` loop.
 - The vectorized `*` operator.
- Use `microbenchmark` for the comparison.

```
vA <- 1:1000000
vB <- rnorm(1000000)

fLoopApproach <- function(vA, vB) {
  iProduct <- numeric(length(vA))
  for (i in 1:length(vA)) {
    iProduct[i] <- vA[i] * vB[i]
  }
}

fVectorApproach <- function(vA, vB) {
  return(vA * vB)
}

# microbenchmark(fLoopApproach(vA, vB), fVectorApproach(vA, vB))
```

30.8 (8)

- You are given a list `lData` where each element is a numeric vector of varying lengths, possibly containing NAs.

```
set.seed(123)
lData <- list(rnorm(10), c(NA, rnorm(5)), rnorm(15, mean=5))
```

- Write code using `lapply` to return a new list where each element is the mean of the corresponding vector in `lData`, after removing NAs.

```
lDataClean <- lapply(lData, mean, na.rm = TRUE)
lDataClean
#> [[1]]
#> [1] 0.07462564
```

```
#>
#> [[2]]
#> [1] 0.3079017
#>
#> [[3]]
#> [1] 4.753408
```

30.9 (9)

- Write a function `fMatrixRedimension` that first initializes an empty vector `vResult <- c()`.
- Then, in a loop that runs 10,000 times, it appends the loop index `i` to `vResult` using `vResult <- c(vResult, i)`.
- Write a second function `fMatrixPreallocate` that first initializes `vResult <- numeric(10000)`.
- Then, in a loop that runs 10,000 times, it assigns `vResult[i] <- i`.
- Compare the execution time of these two functions. Explain the difference.

```
fMatrixRedimension <- function(iN) {
  vResult <- c()
  for (i in 1:iN) {
    vResult <- c(vResult, i)
  }
}

fMatrixPreallocate <- function(iN) {
  vResult <- numeric(10000)
  for (i in 1:iN) {
    vResult[i] <- i
  }
}

# microbenchmark(fMatrixRedimension(10000), fMatrixPreallocate(10000))
```

30.10 (10)

- When might you prefer using `ifelse()` over a standard `if-else` structure within a loop? Provide a small example.

Use `ifelse()` for vectorized conditions. Loop `if-else` for scalar/complex logic per iteration.

```
x <- -2:2
ifelse(x > 0, "positive", "negative")
#> [1] "negative" "negative" "negative" "positive" "positive"
```

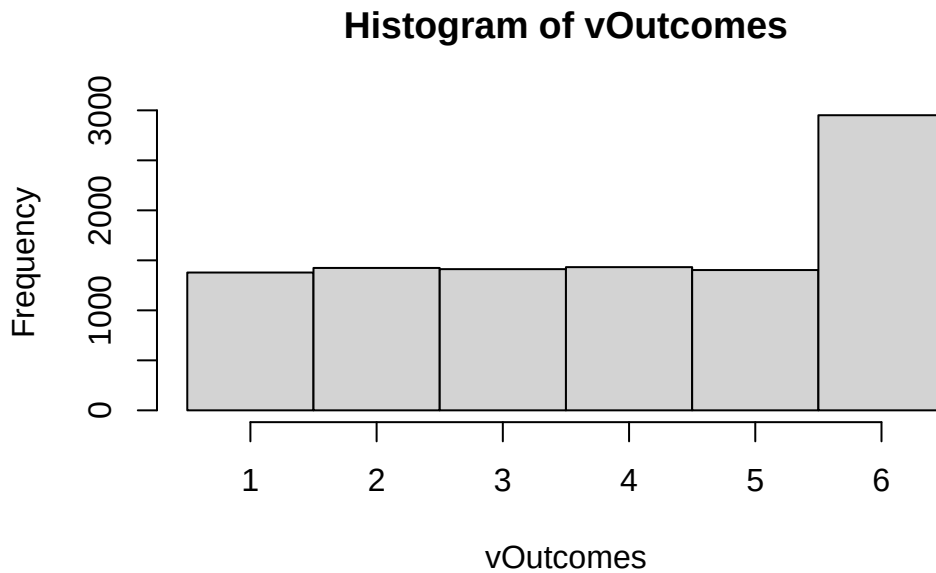

31 Exercise Set 3: Simulation

31.1 (1)

- **Simulating a Biased Die Roll:**

- Write an R function `fSimulateBiasedDie` to simulate rolling a 6-sided die where the probability of rolling a '6' is 0.3, and the probabilities of rolling '1' through '5' are equal (each 0.14).
- The function should take an integer `n_rolls` as input and return a vector of `n_rolls` simulated outcomes.
- Use the discrete random variable simulation method (using `runif` and comparing to CDF).
- Generate 10,000 rolls and plot a histogram of the results.

```
fSimulateBiasedDie <- function(n_rolls) {  
  probs <- c(rep(0.14, 5), 0.3)  
  outcomes <- 1:6  
  CDF <- cumsum(probs)  
  
  vRolls <- numeric(n_rolls)  
  
  for (i in 1:n_rolls) {  
    u <- runif(1)  
    vRolls[i] <- outcomes[min(which(u <= CDF))]  
  }  
  return(vRolls)  
}  
  
vOutcomes <- fSimulateBiasedDie(10000)  
  
hist(vOutcomes, breaks = 0.5:6.5)
```



31.2 (2)

- **Inverse Transform Sampling for a Custom Distribution:**

- A random variable X has a probability density function (PDF) $f(x) = 3x^2$ for $0 \leq x \leq 1$, and $f(x) = 0$ otherwise.
- a) Find the cumulative distribution function (CDF), $F(x)$.
- b) Find the inverse CDF, $F^{-1}(u)$.
- c) Write an R function `fSimulateCustom` to generate `n` random samples from this distribution using the inverse transform method.
- d) Generate 10,000 samples and plot a histogram. Superimpose the theoretical PDF on the histogram.

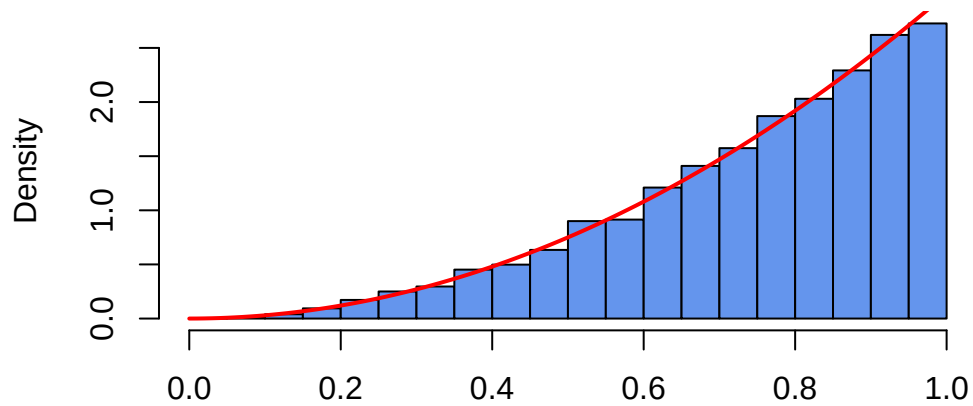
The CDF is $F(x) = x^3$. So the inverse CDF is $F^{-1}(u) = u^{1/3}$.

```
fSimulateCustom <- function(size) {
  U <- runif(size)
  return(U^(1/3))
}

set.seed(123)
vX <- fSimulateCustom(10000)
```

```
hist(vX,
     freq = FALSE,
     breaks = 15,
     col = "cornflowerblue",
     xlab = "",
     ylab = "Density",
     main = "",
     xlim = c(0, 1))

curve(3 * x^2,
      from = 0,
      to = 1,
      col = "red",
      lwd = 2,
      add = TRUE)
```



31.3 (3)

- Acceptance-Rejection Method for a Truncated Normal:

- You want to simulate from a standard normal distribution truncated to the interval $[0, 2]$. The PDF is $f(x) = \frac{\phi(x)}{\Phi(2) - \Phi(0)}$ for $0 \leq x \leq 2$, where ϕ is the standard normal PDF and Φ is the standard normal CDF.
- Use a uniform distribution on $[0, 2]$ as the proposal distribution $g(x) = 1/2$ for $0 \leq x \leq 2$.
- a) Determine the constant c such that $f(x) \leq c \cdot g(x)$ for all $x \in [0, 2]$.
- b) Write an R function `fSimulateTruncNormAR` using the acceptance-rejection method to generate `n` samples from this truncated normal distribution.
- c) Generate 10,000 samples. Plot a histogram and superimpose the theoretical PDF.
- d) What is the acceptance rate of your sampler?

a)

$$c = \frac{\phi(0)}{\Phi(2) - \Phi(0)} / \frac{1}{2 - 0} \approx 1.6716$$

```
fTruncNormSim <- function(size) {

  c <- (dnorm(0) / (pnorm(2) - pnorm(0))) / 0.5

  U <- rep(NA, size)
  Y <- rep(NA, size)
  X <- rep(NA, size)
  Unaccepted <- rep(TRUE, size)

  while (any(Unaccepted)) {

    UnacceptedCount <- sum(Unaccepted)

    U <- runif(UnacceptedCount, 0, 1)
    Y <- runif(UnacceptedCount, 0, 2)

    Accepted_ThisTime <- Unaccepted[Unaccepted] & (U <= ((dnorm(Y) / (pnorm(2) - pnorm(0))) / c))

    X[Unaccepted][Accepted_ThisTime] <- Y[Accepted_ThisTime]
    Unaccepted[Unaccepted] <- !Accepted_ThisTime

  }

  return(X)
}
```

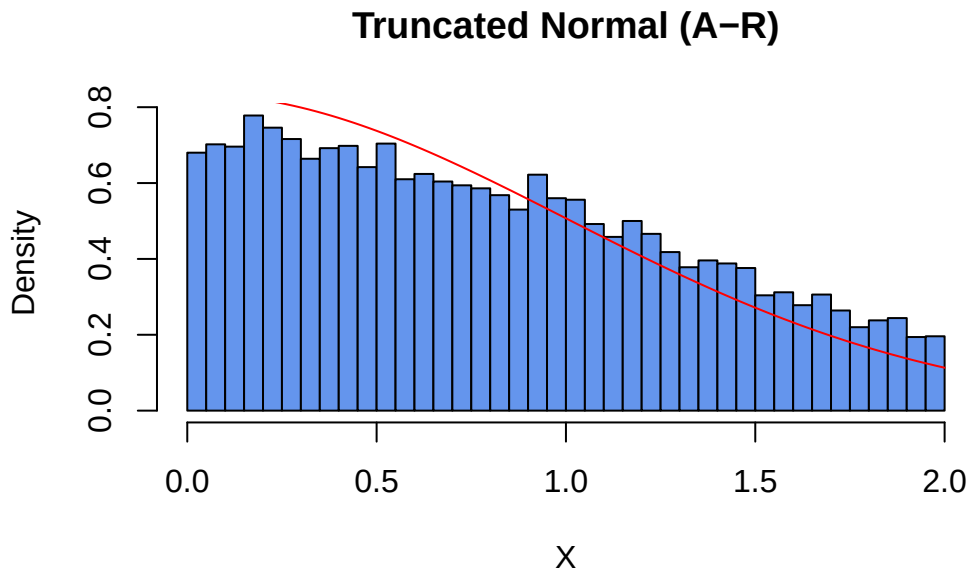
```
}
```

```
set.seed(1)
```

```
X <- fTruncNormSim(10000)
```

```
hist(X, freq = FALSE, col = "cornflowerblue", xlim = c(0, 2), breaks = 50, main = "Truncated
```

```
curve(dnorm(x) / (pnorm(2) - pnorm(0)), add = TRUE, col = "red", from = 0, to = 2)
```



31.4 (4)

- **Box-Muller Transform:**

- a) Write an R function `fMyBoxMuller` that implements the Box-Muller algorithm to generate `n_pairs` of independent standard normal random variables. The function should return a list or a matrix with two columns/elements.
- b) Generate 10,000 standard normal random variables (i.e., call your function to generate 5,000 pairs). Combine them into a single vector.
- c) Plot a histogram of these 10,000 variables and superimpose the standard normal PDF.
- d) Create a scatter plot of the pairs of normal variables generated. What pattern do you expect?

```

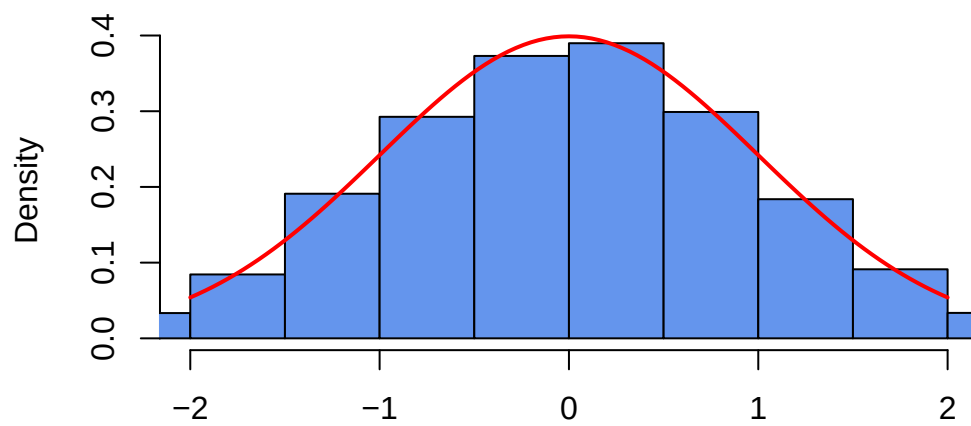
fMyBoxMuller <- function(size = 1) {
  U <- runif(size)
  V <- runif(size)
  X <- sqrt(-2*log(U)) * cos(2*pi*V)
  Y <- sqrt(-2*log(U)) * sin(2*pi*V)
  return(matrix(c(X,Y), ncol = 2))
}

mX <- fMyBoxMuller(5000)
vX <- as.numeric(mX)

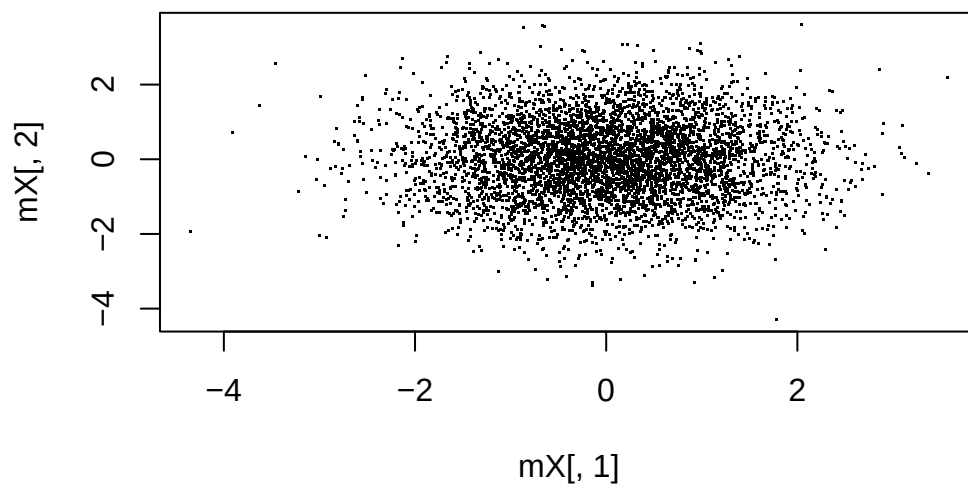
hist(vX,
     freq = FALSE,
     breaks = 15,
     col = "cornflowerblue",
     xlab = "",
     ylab = "Density",
     main = "",
     xlim = c(-2, 2))

curve(dnorm(x),
     from = -2,
     to = 2,
     col = "red",
     lwd = 2,
     add = TRUE)

```



```
# Expected: circular pattern centered at (0,0)
plot(mX[, 1], mX[, 2], pch = ".")
```



31.5 (5)

- **Monte Carlo Integration:**

- a) Estimate the value of $\int_0^1 e^{x^2} dx$ using Monte Carlo integration with 10,000 uniform random samples from $[0, 1]$.
- b) The `integrate()` function in R can compute this numerically. Compare your Monte Carlo estimate to the result from `integrate(function(x) exp(x^2), lower = 0, upper = 1)`.
- c) How could you improve the accuracy of your Monte Carlo estimate?

```
MonteCarlo.Integration <- function(f, n, a, b) {  
  U <- runif(n, min = a, max = b)  
  return( (b-a)*mean(f(U)) )  
}  
set.seed(10086)  
MonteCarlo.Integration(function(x) exp(x^2), 10000, 0, 1)  
#> [1] 1.45782  
  
integrate(function(x) exp(x^2), lower = 0, upper = 1)  
#> 1.462652 with absolute error < 1.6e-14  
  
# Improve by increasing the number of samples.
```

31.6 (6)

- **Simulating a Poisson Distribution:**

- The CDF of a Poisson distribution with rate λ is $F(k; \lambda) = \sum_{i=0}^k \frac{e^{-\lambda} \lambda^i}{i!}$.
- Write an R function `fSimulatePoissonDiscrete` using the method for simulating discrete random variables (based on `runif` and the CDF) to generate `n` samples from a Poisson distribution with $\lambda = 3$.
- (Hint: You can use `ppois` to get the CDF values, or implement the sum yourself).
- Generate 1,000 samples and compare its histogram to the one generated by `rpois(1000, lambda = 3)`.

```
fSimulatePoissonDiscrete <- function(F, size, ...) {  
  m <- 0  
  U <- runif(size)  
  X <- rep(NA, size)  
  X[F(0, ...) >= U] <- 0
```



```

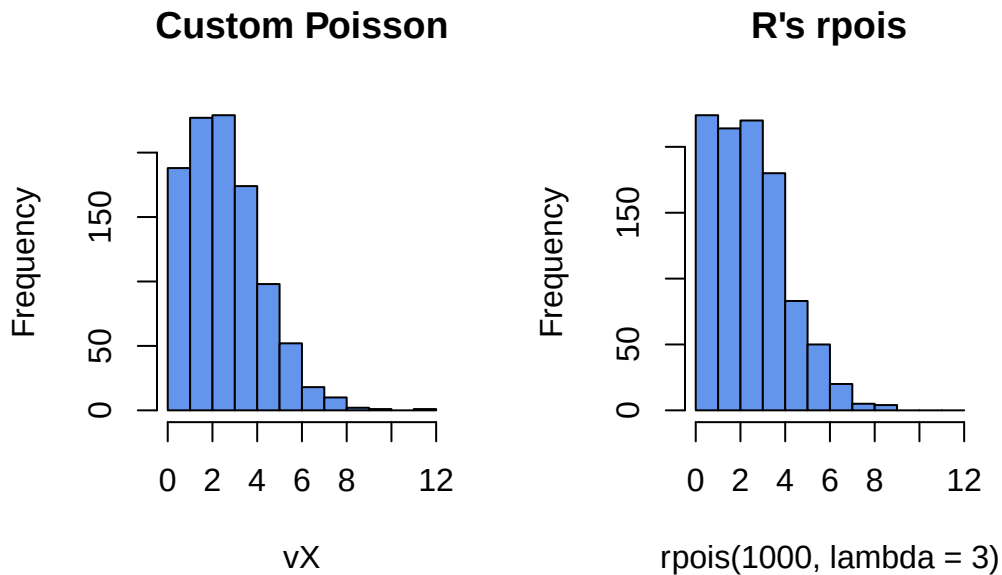
while (any(F(m, ...) < U)) {
  m <- m + 1
  X[(F(m, ...) >= U) & (F(m - 1, ...) < U)] <- m
}
return(X)
}

fPoissonCDF <- function(size, lambda) {
  dSum <- 0
  for (i in 0:size) {
    dSum <- dSum + (exp(-lambda) * lambda^i) / factorial(i)
  }
  return(dSum)
}

set.seed(10086)
vX <- fSimulatePoissonDiscrete(fPoissonCDF, size = 1000, lambda = 3)

par(mfrow=c(1,2))
hist(vX,
     breaks = 0:max(vX),
     col = "cornflowerblue",
     main = "Custom Poisson")
hist(rpois(1000, lambda = 3),
     breaks = 0:max(vX),
     col = "cornflowerblue",
     main = "R's rpois")

```



```
par(mfrow=c(1,1))
```

31.7 (7)

- **Seeding and Reproducibility:**

- a) Generate a vector of 5 random numbers from a standard normal distribution.
- b) Generate another vector of 5 random numbers from a standard normal distribution. Are they the same as in (a)?
- c) Set the seed to 12345. Generate a vector of 5 random numbers from a standard normal distribution.
- d) Set the seed to 12345 again. Generate another vector of 5 random numbers from a standard normal distribution. Are they the same as in (c)? Explain why.
- e) How can you save and restore the RNG state in R? Demonstrate with an example.

```
vRand1 <- rnorm(5)
vRand2 <- rnorm(5)
vRand1 - vRand2
#> [1] 0.02333262 -0.90672903 -2.01327089 1.66283561 0.14078765
set.seed(12345)
```

```
vRand1 <- rnorm(5)
set.seed(12345)
RNG.state <- .Random.seed
vRand2 <- rnorm(5)
vRand1 - vRand2
#> [1] 0 0 0 0 0
.Random.seed <- RNG.state
vRand3 <- rnorm(5)
vRand1 - vRand3
#> [1] 0 0 0 0 0
```

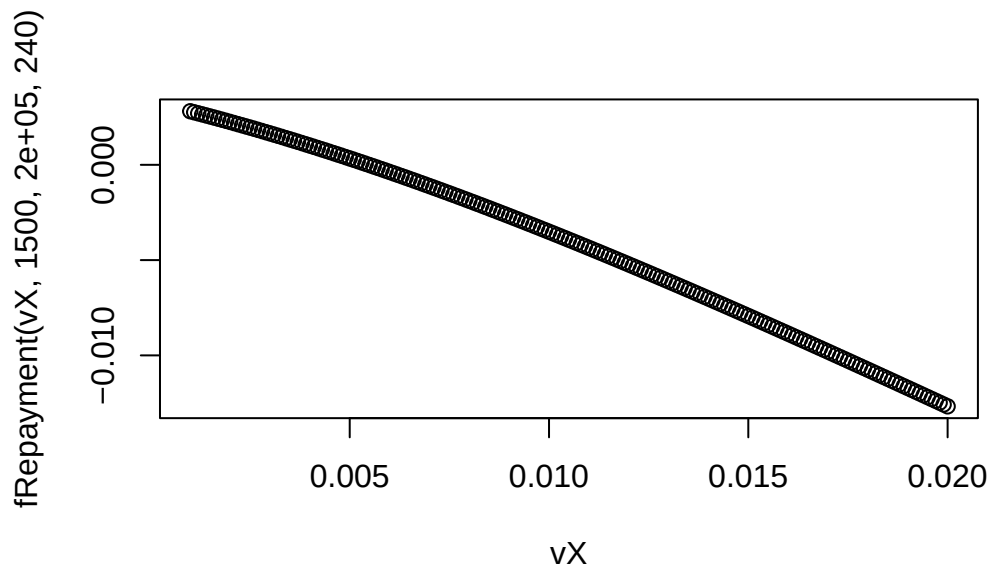
32 Exercise Set 4: Root-Finding

32.1 (1)

- **Loan Repayment Interest Rate:**

- Recall the loan repayment problem where $f(r) = \frac{A}{P} - \frac{r(1+r)^N}{(1+r)^N - 1} = 0$.
- Given a loan amount $P = 200000$, $N = 240$ months (20 years), and a monthly repayment $A = 1500$.
- a) Write an R function for $f(r)$ based on these parameters.
- b) Plot $f(r)$ for $r \in [0.001, 0.02]$ to visually inspect for a root.
- c) Implement the Bisection method to find the monthly interest rate r . Use an initial interval $[0.001, 0.01]$ and a tolerance of 10^{-7} . Print the root and the number of iterations.
- d) (Optional) If you have the `numDeriv` package, calculate the derivative $f'(r)$ numerically.

```
fRepayment <- function(r, A, P, N) {  
  return(A/P - (r * (1+r)^N) / ((1+r)^N - 1))  
}  
vX <- seq(0.001, 0.02, by = 0.0001)  
plot(vX, fRepayment(vX, 1500, 200000, 240))
```



```

bisection <- function(f, dX.l, dX.r, dTol = 10e-7, max.iter = 1000, ...) {

  #check inputs
  if (dX.l >= dX.r) {
    cat("error: x.l >= x.r \n")
    return(NULL)
  }
  f.l <- f(dX.l, ...)
  f.r <- f(dX.r, ...)
  if (f.l == 0) {
    return(dX.l)
  } else if (f.r == 0) {
    return(dX.r)
  } else if (f.l*f.r > 0) {
    cat("error: f(x.l)*f(x.r) > 0 \n")
    return(NULL)
  }

  # successively refine x.l and x.r
  iter <- 0
  while ((dX.r - dX.l) > dTol && (iter < max.iter)) {
    dX.m <- (dX.l + dX.r)/2
  }
}

```

```

    f.m <- f(dX.m, ...)
    if (f.m == 0) {
      return(dX.m)
    } else if (f.l*f.m < 0) {
      dX.r <- dX.m
      f.r <- f.m
    } else {
      dX.l <- dX.m
      f.l <- f.m
    }
    iter <- iter + 1
  }
  cat("at iteration", iter, "the root lies between", dX.l, "and", dX.r, "\n")
  # return approximate root
  return((dX.l + dX.r)/2)
}

bisection(fRepayment, dX.l = 0.001 , dX.r = 0.01, dTol = 10e-7, A = 1500, P = 200000, N = 24)
#> at iteration 14 the root lies between 0.005479126 and 0.005479675
#> [1] 0.005479401

```

32.2 (2)

- **Newton-Raphson Method:**

- Consider the function $f(x) = x^3 - 2x - 5$.
- a) Analytically find the first derivative $f'(x)$.
- b) Implement the Newton-Raphson method in R to find a root of $f(x)$. The function should take f, f' , an initial guess x_0 , a tolerance, and max iterations as input.
- c) Use your function to find a root starting with $x_0 = 2$. Set tolerance to 10^{-6} and max iterations to 100.
- d) What happens if you start with $x_0 = 0$?

Analytical derivative: $f'(x) = 3x^2 - 2$

```

f <- function(x) {
  dOut = x^3 - 2 * x - 5
  return(dOut)
}

```

```

f_prime <- function(x) {
  dOut = 3 * x^2 - 2
  return(dOut)
}

NR <- function(f, f_prime, dX0, dTol = 1e-9, max.iter = 1000, ...) {
  dX <- dX0
  fx <- f(dX, ...)
  iter <- 0
  while ((abs(fx) > dTol) && (iter < max.iter)) {
    dX <- dX - f(dX, ...)/f_prime(dX, ...)
    fx <- f(dX, ...)
    iter <- iter + 1
  }
  if (abs(fx) > dTol) {
    cat("Algorithm failed to converge\n")
    return(NULL)
  } else {
    cat("Algorithm converged\n")
    cat("At iteration ", iter, "value of x is: ", dX, "\n")
    return(dX)
  }
}

NR(f, f_prime, dTol = 10e-6, dX0 = 2, max.iter = 100)
#> Algorithm converged
#> At iteration 3 value of x is: 2.094551
#> [1] 2.094551
NR(f, f_prime, dTol = 10e-6, dX0 = 0, max.iter = 100) # Slower to converge. Could find a dif
#> Algorithm converged
#> At iteration 18 value of x is: 2.094551
#> [1] 2.094551

```

32.3 (3)

- **Secant Method:**

- Consider the same function $f(x) = x^3 - 2x - 5$.
- a) Implement the Secant method in R to find a root of $f(x)$. The function should take f , two initial guesses x_0, x_1 , a tolerance, and max iterations.

- b) Use your function to find a root starting with $x_0 = 2, x_1 = 2.5$. Set tolerance to 10^{-6} and max iterations to 100.
- c) Compare the number of iterations taken by Newton-Raphson and Secant method for this problem.

```
f <- function(x) {
  dOut = x^3 - 2 * x - 5
  return(dOut)
}
fSecant <- function(f, dX0, dX1, dTol = 1e-9, max.iter = 1000, ...) {
  iter <- 0
  dX2 <- dX1
  while ((abs(f(dX2, ...)) > dTol) && (iter < max.iter)) {
    dX2 <- dX1 - f(dX1, ...) * ((dX0 - dX1) / (f(dX0, ...) - f(dX1, ...)))
    dX0 <- dX1
    dX1 <- dX2
    iter <- iter + 1
  }
  if (abs(f(dX2, ...)) > dTol) {
    cat("At iteration ", iter, "value of x is: ", dX1, "\n")
    return(list(root = NULL, f.root = NULL, iter = iter, "Algorithm failed to converge. Maximum iterations reached."))
  } else {
    cat("At iteration ", iter, "value of x is: ", dX1, "\n")
    return(list(root = dX2, f.root = f(dX2), iterations = iter, "Convergence reached."))
  }
}

root <- fSecant(f, dX0 = 2, dX1 = 2.5, dTol = 10e-6, max.iter = 100)
#> At iteration 4 value of x is: 2.094551
root # 1 more step than Newton-Raphson
#> $root
#> [1] 2.094551
#>
#> $f.root
#> [1] -9.612876e-07
#>
#> $iterations
#> [1] 4
#>
#> [[4]]
#> [1] "Convergence reached."

uniroot(f, interval = c(-3,3))
```

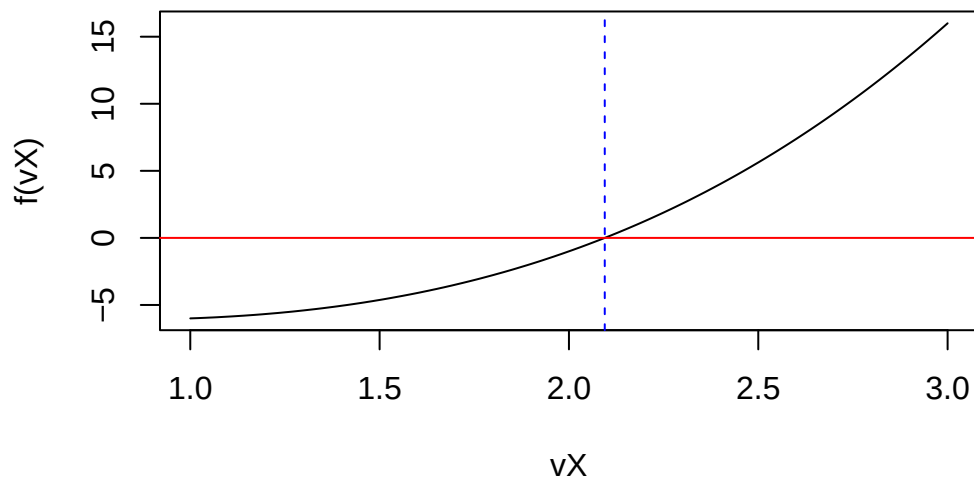


```

#> $root
#> [1] 2.094555
#>
#> $f.root
#> [1] 3.690185e-05
#>
#> $iter
#> [1] 7
#>
#> $init.it
#> [1] NA
#>
#> $estim.prec
#> [1] 6.103516e-05

vX <- seq(1, 3, 0.01)
plot(vX, f(vX), type = "l")
abline(h = 0, col = "red")
abline(v = root["root"], col = "blue", lty = 2)

```

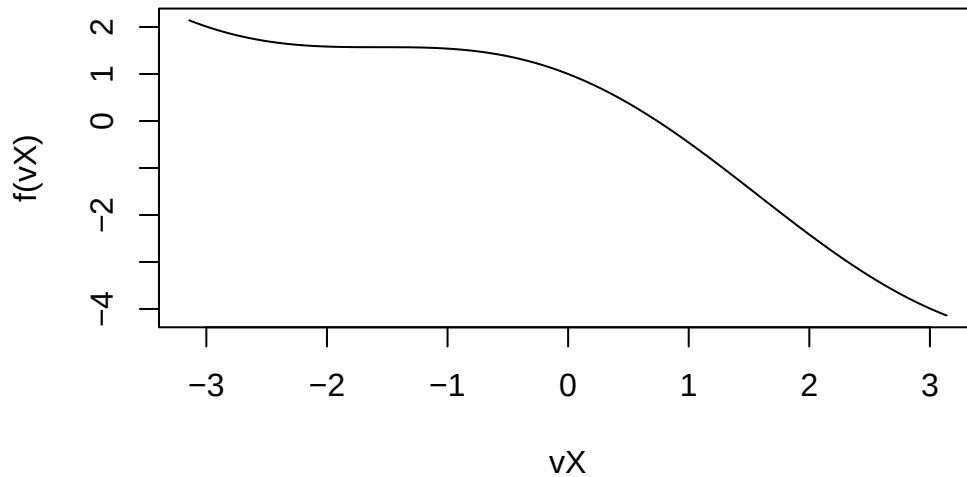


32.4 (4)

- Using `uniroot`:

- For the function $f(x) = \cos(x) - x$:
- a) Plot the function over an interval (e.g., $[-\pi, \pi]$) to identify a bracketing interval for a root.
- b) Use the `uniroot` function to find the root within your chosen interval.
- c) Write your own simple Bisection method function and compare its result and number of iterations to `uniroot` for the same interval and a similar tolerance.

```
f <- function(x) {  
  return(cos(x) - x)  
}  
  
vX <- seq(-pi, pi, 0.01)  
plot(vX, f(vX), type = "l")
```



```
uniroot(f, interval = c(-1,2))  
#> $root  
#> [1] 0.7390929  
#>
```

```

#> $f.root
#> [1] -1.292031e-05
#>
#> $iter
#> [1] 6
#>
#> $init.it
#> [1] NA
#>
#> $estim.prec
#> [1] 6.103516e-05

# See exercise just before this one
root <- fSecant(f, dX0 = -1, dX1 = 2, dTol = 10e-6, max.iter = 100)
#> At iteration 5 value of x is: 0.7390831
root
#> $root
#> [1] 0.7390831
#>
#> $f.root
#> [1] 3.363451e-06
#>
#> $iterations
#> [1] 5
#>
#> [[4]]
#> [1] "Convergence reached."

```

32.5 (5)

- **Numerical Derivatives with numDeriv:**

- Consider the function $g(x) = e^{\sin(x)} \cdot \ln(x^2 + 1)$ for $x > 0$.
- a) If the **numDeriv** package is installed, use the **grad** function to numerically compute the derivative of $g(x)$ at $x = 1$, $x = 2$, and $x = 3$.
- b) (Challenge) Try to find the analytical derivative of $g(x)$ and compare its value at $x = 2$ with the numerical estimate.

```

g <- function(x) {
  return(exp(sin(x)) * log(x^2 + 1))
}

```

```

suppressMessages(library(numDeriv))
grad(g, 1)
#> [1] 3.188554
grad(g, 2)
#> [1] 0.3233247
grad(g, 3)
#> [1] -1.934098

# Analytically
gGrad <- function(x) {
  return((exp(sin(x))*cos(x)*log(x^2+1) + exp(sin(x))*(2*x/(x^2+1))))
}
gGrad(2)
#> [1] 0.3233247

```

32.6 (6)

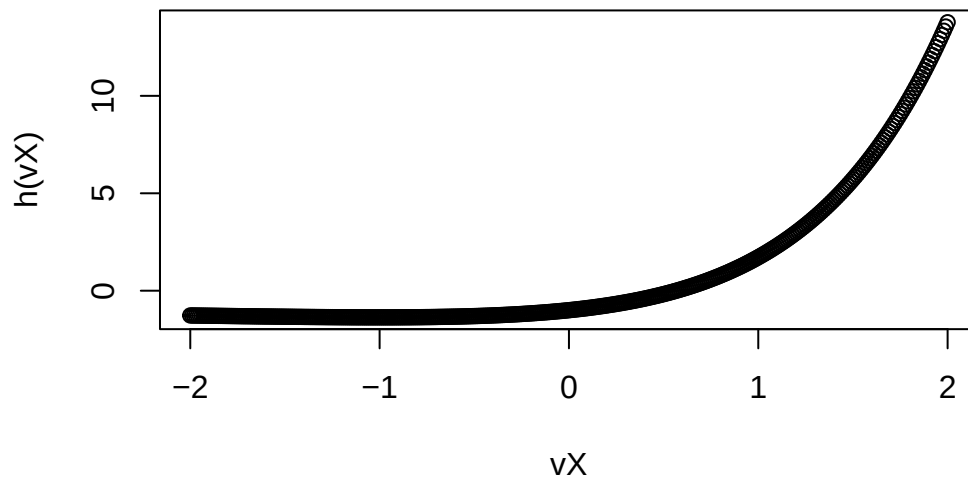
- **Comparing Root-Finding Methods:**

- Find a root for $h(x) = x \cdot e^x - 1 = 0$.
- a) Plot the function to find a suitable starting interval/point.
- b) Solve using your Bisection method implementation.
- c) Solve using your Newton-Raphson method implementation (you'll need $h'(x)$).
- d) Solve using your Secant method implementation.
- e) Solve using `uniroot`.
- f) Briefly compare the methods in terms of requirements (e.g., derivative, interval), convergence speed (number of iterations), and robustness for this specific problem.

```

h <- function(x) {
  return(x * exp(x) - 1)
}
vX <- seq(-2, 2, 0.01)
plot(vX, h(vX))

```



```

bisection(h, 0, 2)
#> at iteration 21 the root lies between 0.5671425 and 0.5671434
#> [1] 0.567143

hGrad <- function(x) {
  return(x * exp(x) + exp(x))
}

NR(h, hGrad, 1.5)
#> Algorithm converged
#> At iteration 6 value of x is: 0.5671433
#> [1] 0.5671433
fSecant(h, 0, 2)
#> At iteration 8 value of x is: 0.5671433
#> $root
#> [1] 0.5671433
#>
#> $f.root
#> [1] -3.581359e-10
#>
#> $iterations
#> [1] 8
#>

```

```
#> [[4]]
#> [1] "Convergence reached."
uniroot(h, c(0, 2))
#> $root
#> [1] 0.5671321
#>
#> $f.root
#> [1] -3.093327e-05
#>
#> $iter
#> [1] 8
#>
#> $init.it
#> [1] NA
#>
#> $estim.prec
#> [1] 6.103516e-05
```

33 Exercise Set 5: Numerical Optimization (Univariate and Multivariate)

33.1 (1)

- **Univariate Optimization with Newton's Method:**

- Consider the function $f(x) = x^4 - 14x^3 + 60x^2 - 70x$.
- a) Find the first $f'(x)$ and second $f''(x)$ derivatives analytically.
- b) Implement Newton's method for optimization to find a local minimum/maximum of $f(x)$. Your function should search for x where $f'(x) = 0$.
- c) Try to find a local extremum starting from $x_0 = 0$. What do you find?
- d) Try to find a local extremum starting from $x_0 = 6$. What do you find?
- e) Use $f''(x)$ to classify the extrema found.

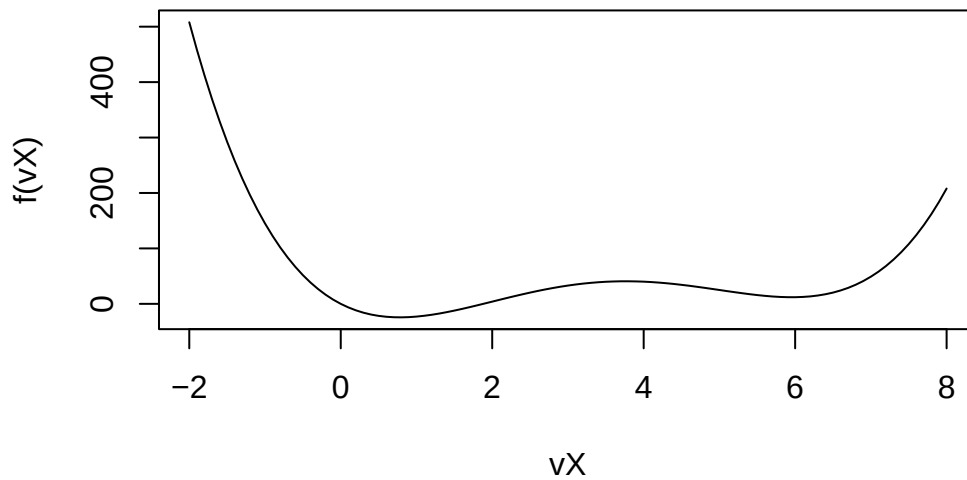
Analytical derivatives: $f'(x) = 4x^3 - 42x^2 + 120x - 70$ and $f''(x) = 12x^2 - 84x + 120$.

```
f <- function(x) {  
  return(x^4 - 14 * x^3 + 60 * x^2 - 70 * x)  
}  
  
fprime <- function(x) {  
  return(4 * x^3 - 42 * x^2 + 120 * x - 70)  
}  
  
fsecond <- function(x) {  
  return(12 * x^2 - 84 * x + 120)  
}  
  
NM <- function(f, f_prime, f_sec, dX0, dTol = 1e-9, n.max = 1000){  
  dX <- dX0  
  fx <- f(dX)  
  fpx <- f_prime(dX)  
  fsx <- f_sec(dX)  
  n <- 0  
  while ((abs(fpx) > dTol) && (n < n.max)) {
```

```

    dX <- dX - fpx/fsx
    fx <- f(dX)
    fpx <- f_prime(dX)
    fsx <- f_sec(dX)
    n <- n + 1
  }
  if (n == n.max) {
    cat('newton failed to converge\n')
  } else {
    cat("At iteration", n, "the value of x is:", dX, "\n")
    return(dX)
  }
}
x1 <- NM(f, fprime, fsecond, dX0 = 0)
#> At iteration 5 the value of x is: 0.7808841
x2 <- NM(f, fprime, fsecond, dX0 = 3)
#> At iteration 4 the value of x is: 3.761921
x3 <- NM(f, fprime, fsecond, dX0 = 6)
#> At iteration 3 the value of x is: 5.957195
fsecond(x1) # local minimum
#> [1] 61.7231
fsecond(x2) # local maximum
#> [1] -26.17677
fsecond(x3) # local minimum
#> [1] 45.45367
vX <- seq(-2, 8, 0.1)
plot(vX, f(vX), type = "l")

```

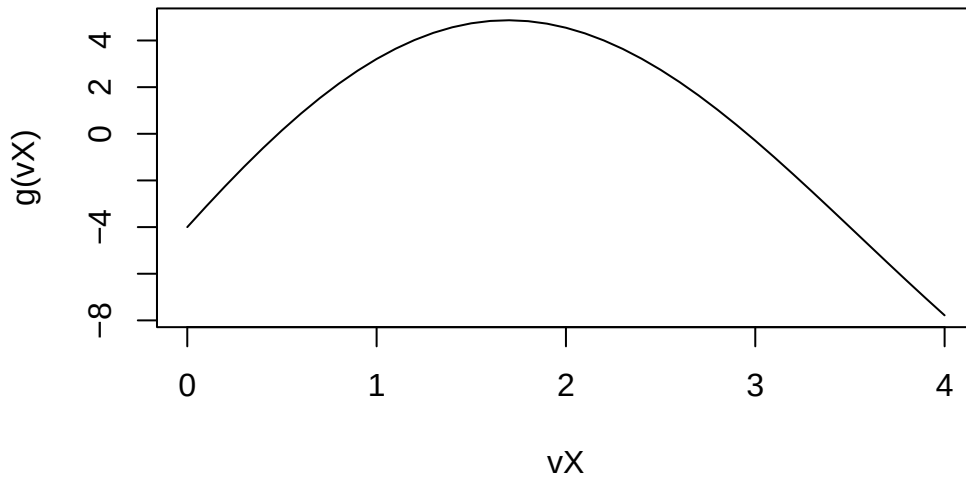
33.2 (2)

- **Golden Section Search:**

- Consider the function $g(x) = -(x - 2)^2 + 5 \sin(x)$ on the interval $[0, 4]$.
- a) Plot the function to visualize its behavior.
- b) Implement the Golden Section Search algorithm to find the maximum of $g(x)$ within this interval. Set a tolerance of 10^{-5} .
- c) Compare your result with R's `optimize` function (remember to set `maximum = TRUE`).

```
g <- function(x) {
  return(-(x - 2)^2 + 5 * sin(x))
}

vX <- seq(0, 4, 0.1)
plot(vX, g(vX), type = "l")
```



```
gsection <- function(f, dX.l, dX.r, dX.m, dTol = 1e-9) {

  # golden ratio plus one
  dGR1 <- 1 + (1 + sqrt(5))/2

  # successively refine x.l, x.r, and x.m
  f.l <- f(dX.l)
  f.r <- f(dX.r)
  f.m <- f(dX.m)
  while ((dX.r - dX.l) > dTol) {
    if ((dX.r - dX.m) > (dX.m - dX.l)) { # if the right segment is wider than the left
      dY <- dX.m + (dX.r - dX.m)/dGR1 # put Y into the right segment according to the golden
      f.y <- f(dY)
      if (f.y >= f.m) {
        dX.l <- dX.m
        f.l <- f.m
        dX.m <- dY
        f.m <- f.y
      } else {
        dX.r <- dY
        f.r <- f.y
      }
    }
  }
}
```

```

    } else { #if the left segment is wider than the right
      dY <- dX.m - (dX.m - dX.l)/dGR1 # put Y into the left segment according to the golden r
      f.y <- f(dY)
      if (f.y >= f.m) {
        dX.r <- dX.m
        f.r <- f.m
        dX.m <- dY
        f.m <- f.y
      } else {
        dX.l <- dY
        f.l <- f.y
      }
    }
  }
}
return(dX.m)
}
gsection(g, dX.l = 0, dX.r = 4, dX.m = 2, dTol = 10e-5)
#> [1] 1.693623
optimize(g, interval = c(0, 4), maximum = TRUE)
#> $maximum
#> [1] 1.693664
#>
#> $objective
#> [1] 4.868465

```

33.3 (3)

- **Multivariate Optimization: Steepest Ascent:**

- Consider the function $h(x, y) = -(x - 1)^2 - 2(y - 2)^2 + xy$.
- a) Analytically compute the gradient $\nabla h(x, y)$.
- b) Implement the Steepest Ascent method. For line search, you can use a fixed small step size initially or implement a simple line search (e.g., by testing a few step sizes or adapting the Golden Section search for the 1D problem along the gradient direction).
- c) Find the maximum starting from $(x_0, y_0) = (0, 0)$. Set tolerance for gradient norm to 10^{-4} .

```

h <- function(vX) {
  return(-(vX[1] - 1)^2 - 2 * (vX[2] - 2)^2 + vX[1] * vX[2])
}

```

```

hgrad <- function(vX) {
  h1 <- -2 * (vX[1] - 1) + vX[2]
  h2 <- -4 * (vX[2] - 2) + vX[1]
  return(c(h1, h2))
}

# golden section algorithm from last week
gsection <- function(f, dX.l, dX.r, dX.m, dTol = 1e-9) {

  # golden ratio plus one
  dGR1 <- 1 + (1 + sqrt(5))/2

  # successively refine x.l, x.r, and x.m
  f.l <- f(dX.l)
  f.r <- f(dX.r)
  f.m <- f(dX.m)
  while ((dX.r - dX.l) > dTol) {
    if ((dX.r - dX.m) > (dX.m - dX.l)) { # if the right segment is wider than the left
      dY <- dX.m + (dX.r - dX.m)/dGR1 # put Y into the right segment according to the golden
      f.y <- f(dY)
      if (f.y >= f.m) {
        dX.l <- dX.m
        f.l <- f.m
        dX.m <- dY
        f.m <- f.y
      } else {
        dX.r <- dY
        f.r <- f.y
      }
    } else { #if the left segment is wider than the right
      dY <- dX.m - (dX.m - dX.l)/dGR1 # put Y into the left segment according to the golden
      f.y <- f(dY)
      if (f.y >= f.m) {
        dX.r <- dX.m
        f.r <- f.m
        dX.m <- dY
        f.m <- f.y
      } else {
        dX.l <- dY
        f.l <- f.y
      }
    }
  }
}

```

```

}
return(dX.m)
}

line.search <- function(f, vX, vG, dTol = 1e-9, dA.max = 2^5) {
  # f is a real function that takes a vector of length d
  # x and y are vectors of length d
  # line.search uses gsection to find a >= 0 such that
  # g(a) = f(x + a*y) has a local maximum at a,
  # within a tolerance of tol
  # if no local max is found then we use 0 or a.max for a
  # the value returned is x + a*y
  if (sum(abs(vG)) == 0){
    return(vX) # +0*vG
  } # g(a) constant
  g <- function(dA){
    return(f(vX + dA*vG))
  }
  # find a triple a.l < a.m < a.r such that
  # g(a.l) <= g(a.m) and g(a.m) >= g(a.r)

  # choose a.l
  dA.l <- 0
  g.l <- g(dA.l)
  # find a.m
  dA.m <- 1
  g.m <- g(dA.m)
  while ((g.m < g.l) & (dA.m > dTol)) {
    dA.m <- dA.m/2
    g.m <- g(dA.m)
  }
  # if a suitable a.m was not found then use 0 for a, so just return vX as the next step
  if ((dA.m <= dTol) & (g.m < g.l)){
    return(vX)
  }
  # find a.r
  dA.r <- 2*dA.m
  g.r <- g(dA.r)
  while ((g.m < g.r) & (dA.r < dA.max)) {
    dA.m <- dA.r
    g.m <- g.r
    dA.r <- 2*dA.m
  }
}

```

```

    g.r <- g(dA.r)
  }
  # if a suitable a.r was not found then use a.max for a
  if ((dA.r >= dA.max) & (g.m < g.r)){
    return(vX + dA.max*vG)
  }
  # apply golden-section algorithm to g to find a
  dA <- gsection(g, dA.l, dA.r, dA.m)
  return(vX + dA*vG)
}

ascent <- function(f, grad.f, vX0, dTol = 1e-9, n.max = 100) {
  vX.old <- vX0
  vX <- line.search(f, vX0, grad.f(vX0))
  n <- 1
  while ((f(vX) - f(vX.old) > dTol) & (n < n.max)) {
    vX.old <- vX
    vX <- line.search(f, vX, grad.f(vX))
    n <- n + 1
  }
  cat("at iteration", n, "the coordinates of x are", vX, "\n")
  return(vX)
}

ascent(h, hgrad, vX0 = c(0, 0))
#> at iteration 16 the coordinates of x are 2.285706 2.57142
#> [1] 2.285706 2.571420

```

33.4 (4)

- **Multivariate Optimization: Newton's Method in Higher Dimensions:**

- For the same function $h(x, y) = -(x - 1)^2 - 2(y - 2)^2 + xy$ from Q3:
- a) Analytically compute the Hessian matrix $H(x, y)$.
- b) Implement Newton's method for multivariate optimization.
- c) Find the maximum starting from $(x_0, y_0) = (0, 0)$. Set tolerance for gradient norm to 10^{-4} .
- d) Compare the number of iterations with the Steepest Ascent method.

```

hcomb <- function(vX) {
  h <- -(vX[1] - 1)^2 - 2 * (vX[2] - 2)^2 + vX[1] * vX[2]

```

```

h1 <- -2 * (vX[1] - 1) + vX[2]
h2 <- -4 * (vX[2] - 2) + vX[1]
h11 <- -2
h12 <- 1
h22 <- -4
return(list(h, c(h1, h2), matrix(c(h11, h12, h12, h22), 2, 2)))
}

newton <- function(f3, vX0, dTol = 1e-9, n.max = 100) {
  # Newton's method for optimisation, starting at x0
  # f3 is a function that given x returns the list
  # {f(x), grad f(x), Hessian f(x)}, for some f
  vX <- vX0
  f3.x <- f3(vX)
  n <- 0
  while ((max(abs(f3.x[[2]])) > dTol) & (n < n.max)) {
    vX <- vX - solve(f3.x[[3]], f3.x[[2]])
    #vX <- vX - solve(f3.x[[3]])%*%f3.x[[2]]
    f3.x <- f3(vX)
    cat("At iteration", n, "the coordinates of x are", vX, "\n")
    n <- n + 1
  }
  if (n == n.max) {
    cat('newton failed to converge\n')
  } else {
    return(vX)
  }
}

n1 <- newton(hcomb, vX0 = c(0, 0))
#> At iteration 0 the coordinates of x are 2.285714 2.571429
n1
#> [1] 2.285714 2.571429
eigen(hcomb(n1)[[3]]) ## maximum
#> eigen() decomposition
#> $values
#> [1] -1.585786 -4.414214
#>
#> $vectors
#>           [,1]      [,2]
#> [1,] -0.9238795 -0.3826834
#> [2,] -0.3826834  0.9238795

```

33.5 (5)

- Using `optim` in R:

- Consider the Rosenbrock function (often used for testing optimization algorithms):
 $f(x, y) = (a - x)^2 + b(y - x^2)^2$. Let $a = 1$ and $b = 100$. We want to find its minimum.
- a) Write an R function for the Rosenbrock function.
- b) Use the `optim` function to find the minimum. Try the following methods:
 - * “Nelder-Mead”
 - * “BFGS”
 - * “L-BFGS-B” (try with and without bounds, e.g., $x \in [-2, 2], y \in [-1, 3]$)
- Start from an initial guess of $(-1.2, 1)$.
- c) For the “BFGS” method, if you have the `numDeriv` package, provide the gradient numerically using `grad`. Does it converge faster or to a better point?

```
suppressMessages(library(numDeriv))
fRosenbrock <- function(vX) {
  a <- 1
  b <- 100
  return((a-vX[1])^2 + b * (vX[2]-vX[1]^2)^2)
}

optim(c(-1.2, -1), fRosenbrock, method = "Nelder-Mead")$par
#> [1] 1.003505 1.006996
optim(c(-1.2, -1), fRosenbrock, method = "BFGS")$par
#> [1] 0.9998000 0.9996001
optim(c(-1.2, -1), fRosenbrock, method = "L-BFGS-B")$par
#> [1] 0.9998032 0.9996078
optim(c(-1.2, -1), fRosenbrock, method = "L-BFGS-B", lower=c(-2,-1), upper=c(2,3))$par
#> [1] 0.9998006 0.9996012
grad_rosen <- function(par) numDeriv::grad(fRosenbrock, par)
optim(c(-1.2, -1), fRosenbrock, gr = grad_rosen, method = "BFGS")$par
#> [1] 1 1
```

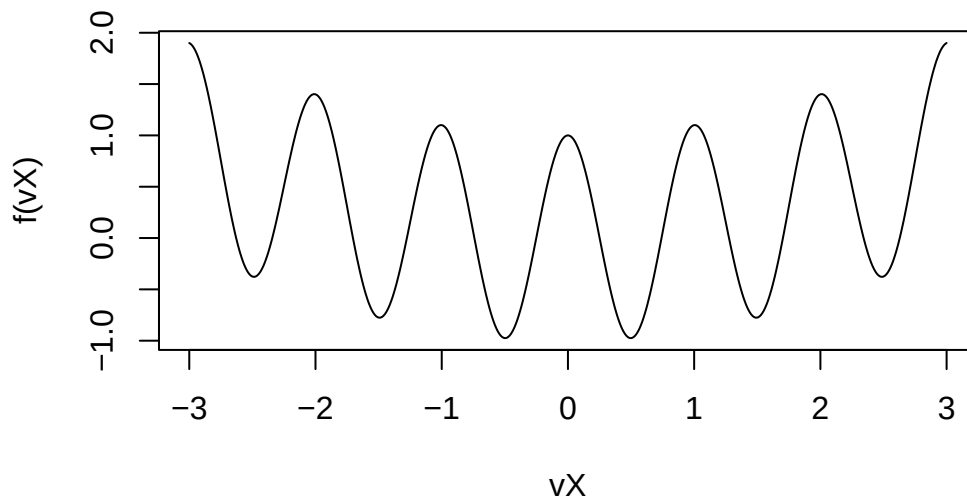
33.6 (6)

- Optimization Pitfalls:

- Create a simple R function $f(x) = \cos(2\pi x) + 0.1x^2$.
- a) Plot this function over $x \in [-3, 3]$.

- b) Use `optimize()` to find the global minimum in this interval.
- c) Now, use `optimize()` to find a local minimum in the interval $[-0.5, 0.5]$ and another in $[0.5, 1.5]$. Discuss how starting points or intervals affect finding local vs. global optima.

```
f <- function(x) {  
  return(cos(2 * pi * x) + 0.1 * x^2)  
}  
  
vX <- seq(-3, 3, 0.01)  
plot(vX, f(vX), type = "l")
```



```
optimize(f, interval = c(-3, 3))  
#> $minimum  
#> [1] 1.492439  
#>  
#> $objective  
#> [1] -0.7761343  
optimize(f, interval = c(-0.5, 0.5))  
#> $minimum  
#> [1] -0.4974641  
#>
```

```
#> $objective
#> [1] -0.975126
optimize(f, interval = c(0.5, 1.5))
#> $minimum
#> [1] 0.5000661
#>
#> $objective
#> [1] -0.9749933
```

33.7 (7)

- **Constrained Optimization via Reparameterization:**

- You want to maximize $f(p) = p^2(1-p)^3$ subject to $0 < p < 1$.
- a) Use the transformation $p = \frac{e^{\tilde{p}}}{1+e^{\tilde{p}}}$ to convert this into an unconstrained problem for $\tilde{p}$. Write the new function $g(\tilde{p}) = f(\lambda(\tilde{p}))$.
- b) Use `optim` with “BFGS” to find the maximum of $g(\tilde{p})$.
- c) Transform the optimal $\tilde{p}^*$ back to p^* and report the maximum value $f(p^*)$.
- d) (Optional) Analytically find the maximum and compare.

```
f <- function(p) {
  return(p^2 * (1 - p)^3)
}

g <- function(p_tilde) {
  p <- exp(p_tilde) / (1 + exp(p_tilde))
  return(f(p))
}

optim_res <- optim(0, g, method = "BFGS", , control=list(fnscale=-1))
p_tilde <- optim_res$par
p_optim <- exp(p_tilde) / (1 + exp(p_tilde))
p_optim
#> [1] 0.4
f(p_optim)
#> [1] 0.03456
```

33.8 (8)

- **Likelihood Maximization:**

- Suppose you have $n = 10$ observations $y = (1, 0, 1, 1, 0, 1, 0, 0, 1, 1)$ from a Bernoulli distribution with unknown probability p . The log-likelihood function is $L(p|y) = \sum y_i \log(p) + \sum (1 - y_i) \log(1 - p)$.
- a) Write an R function for the negative log-likelihood (since `optim` minimizes).
- b) Use `optim` with method “L-BFGS-B” and appropriate bounds for p (e.g., $[0.001, 0.999]$) to find the Maximum Likelihood Estimate (MLE) of p .
- c) Compare this to the analytical MLE, which is $\bar{y}$.

```
neg_log <- function(p, y) {
  dSum1 <- 0
  dSum2 <- 0
  for (i in 1:length(y)) {
    dSum1 <- dSum1 - y[i] * log(p)
    dSum2 <- dSum2 - (1 - y[i]) * log(1 - p)
  }
  return(dSum1 + dSum2)
}

neg_log_lik_bern <- function(p, data) {
  if (p <= 0 || p >= 1) return(Inf) # Bounds for log
  -sum(data * log(p) + (1 - data) * log(1 - p))
}

vY <- c(1, 0, 1, 1, 0, 1, 0, 0, 1, 1)
optim(0.5, neg_log, y = vY, method = "L-BFGS-B", lower = 0.001, upper = 0.999)$par
#> [1] 0.5999997
optim(0.5, neg_log_lik_bern, data = vY, method = "L-BFGS-B", lower = 0.001, upper = 0.999)$par
#> [1] 0.5999997
# Analytical MLE
mean(vY)
#> [1] 0.6
```

34 Exercise Set 6: C++ with Rcpp and RcppArmadillo

34.1 (1)

- **Basic Rcpp Function:**

- Write a C++ function using Rcpp (via `cppFunction` or `sourceCpp`) named `sum_greater_than_C` that takes a numeric vector `x` and a scalar `threshold` as input.
- The function should return the sum of all elements in `x` that are strictly greater than `threshold`.
- Test your C++ function in R and compare its output with an equivalent R implementation for `x = rnorm(1000)` and `threshold = 0`.

```
suppressMessages(library(Rcpp))
#> Warning: pakke 'Rcpp' blev bygget under R version 4.3.3

cppFunction('
double sum_greater_than_C(NumericVector x, double threshold) {
  double total = 0;
  for (int i = 0; i < x.size(); i++) {
    if (x[i] > threshold) {
      total += x[i];
    }
  }
  return total;
}')

vX <- rnorm(1000)
threshold <- 0
sum_greater_than_C(vX, threshold)
#> [1] 420.1322
sum(vX[vX>0])
#> [1] 420.1322
```

34.2 (2)

- **Fibonacci Sequence in C++:**

- a) Write a recursive C++ function `fib_cpp(n)` to compute the n -th Fibonacci number.
- b) Write an iterative (loop-based) C++ function `fib_iter_cpp(n)` to compute the n -th Fibonacci number.
- c) Compare the execution speed of `fib_cpp(25)`, `fib_iter_cpp(25)`, and an R recursive Fibonacci function for $n = 25$ using `microbenchmark`.

```
suppressMessages(library(Rcpp))
suppressMessages(library(microbenchmark))

cppFunction('
int fib_cpp(int n) {
  if (n <= 1) return(0);
  return fib_cpp(n - 1) + fib_cpp(n - 2);
}')

cppFunction('
int fib_iter_cpp(int n) {
  if (n <= 1) return(0);
  int a = 0, b = 1, temp;
  for (int i = 2; i <= n; i++) {
    temp = a + b;
    a = b;
    b = temp;
  }
  return b;
}')

fib_r <- function(n) {
  if (n <= 1) {
    return(0)
  } else {
    return(fib_r(n - 1) + fib_r(n - 2))
  }
}

microbenchmark(fib_cpp(25), fib_iter_cpp(25), fib_r(25))
#> Unit: nanoseconds
#>           expr      min       lq      mean    median      uq     max neval
```

```
#>      fib_cpp(25)  149600  150600  167253  157250  164800  945000  100
#>  fib_iter_cpp(25)    700    800   13033   1300   10200  833800  100
#>      fib_r(25) 85986900 87456800 88982486 88352950 89253150 116590200 100
```

34.3 (3)

- **RcppArmadillo: Matrix Operations:**

- a) Create an R script that defines a C++ function `arma_matrix_ops` using `RcppArmadillo`.
- This function should take two matrices, `mA` and `mB`, as input (use `arma::mat`).
- It should return a list containing:
 - * `mProd`: The standard matrix product $mA \times mB$.
 - * `mInvA`: The inverse of `mA` (if possible, handle potential errors if not invertible, e.g. by trying and catching or checking determinant if simple).
 - * `vEigenvalsA`: The eigenvalues of `mA` (use `arma::eig_sym` if symmetric, or `arma::eig_gen` if general; assume real eigenvalues for simplicity or return complex as string if needed).
- b) In R, create two 3x3 matrices, test your function, and verify the product with R's `%*%`.

```
suppressMessages(library(Rcpp))
suppressMessages(library(RcppArmadillo))
#> Warning: pakke 'RcppArmadillo' blev bygget under R version 4.3.3

sourceCpp("extra_cpp.cpp")
#>
#> > vX <- 1:10
#>
#> > dS <- 5
#>
#> > res <- cpp_vector_scalar_mult(vX, dS)
#>
#> > print(res)
#>      [,1]
#> [1,]    5
#> [2,]   10
#> [3,]   15
#> [4,]   20
#> [5,]   25
```

```

#> [6,] 30
#> [7,] 35
#> [8,] 40
#> [9,] 45
#> [10,] 50

mA <- matrix(1:9, 3, 3)
mB <- matrix(9:1, 3, 3)
arma_matrix_ops(mA, mB)
#> Matrix mA is singular.
#> $mProd
#>      [,1] [,2] [,3]
#> [1,]  90  54  18
#> [2,] 114  69  24
#> [3,] 138  84  30
#>
#> $vEigenvalsA
#>      [,1]
#> [1,] 1.611684e+01+0i
#> [2,] -1.116844e+00+0i
#> [3,] -5.700691e-16+0i
mA %*% mB
#>      [,1] [,2] [,3]
#> [1,]  90  54  18
#> [2,] 114  69  24
#> [3,] 138  84  30

```

34.4 (4)

- **Simulating GARCH(1,1) process in C++:**

- The GARCH(1,1) process is defined by: $r_t = \sigma_t \epsilon_t$, where $\epsilon_t \sim N(0,1)$ $\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2$
- Write a C++ function using RcppArmadillo named `simulate_garch_cpp` that takes parameters ω, α, β , number of time steps T, and an initial σ_0^2 as input.
- The function should simulate and return a list containing two vectors: `vR` (the returns r_t) and `vSigma2` (the variances σ_t^2).
- (Hint: Use `R::rnorm(0,1)` for ϵ_t . Initialize r_0 using σ_0).
- Test by simulating 1000 steps with $\omega = 0.1, \alpha = 0.05, \beta = 0.9, \sigma_0^2 = 1$. Plot the simulated returns and variances.

```

suppressMessages(library(Rcpp))
suppressMessages(library(RcppArmadillo))

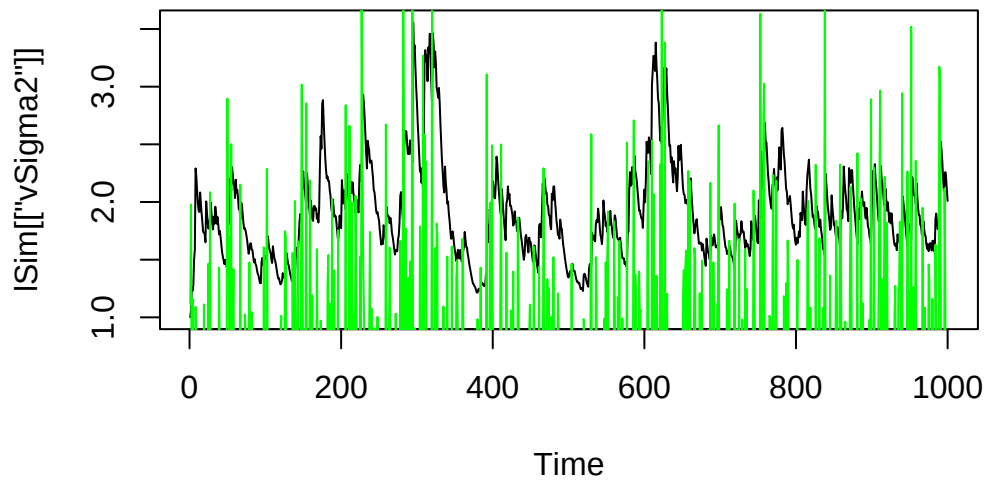
sourceCpp("extra_cpp.cpp")
#>
#> > vX <- 1:10
#>
#> > dS <- 5
#>
#> > res <- cpp_vector_scalar_mult(vX, dS)
#>
#> > print(res)
#>      [,1]
#> [1,]    5
#> [2,]   10
#> [3,]   15
#> [4,]   20
#> [5,]   25
#> [6,]   30
#> [7,]   35
#> [8,]   40
#> [9,]   45
#> [10,]  50

iT <- 1000
dOmega <- 0.1
dAlpha <- 0.05
dBeta <- 0.9
dSigmaInit <- 1.0

lSim <- simulate_garch_cpp(iT, dOmega, dAlpha, dBeta, dSigmaInit)

plot(1:1000, lSim[["vSigma2"]], type = "l", lty = 1, xlab = "Time")
lines(1:1000, lSim[["vR"]], col="green")

```

34.5 (5)

- Calling R functions from C++:

- a) Write a simple R function `my_r_summary <- function(x) list(mean=mean(x), sd=sd(x))`.
- b) Write a C++ function `call_r_from_cpp` that takes a `NumericVector x` and an R Function `r_func` as input.
- Inside the C++ function, call `r_func` with `x` and return the result.
- c) Test this by calling `call_r_from_cpp(rnorm(100), my_r_summary)` from R.

```
my_r_summary <- function(x) list(mean=mean(x), sd=sd(x))

cppFunction('
SEXP call_r_from_cpp(NumericVector x, Function r_func) {
    return r_func(x);
}')

call_r_from_cpp(rnorm(100), my_r_summary)
#> $mean
#> [1] -0.10438
#>
```

```
#> $sd  
#> [1] 1.056556
```

34.6 (6)

- **SourceCpp and Embedded R code:**

- Create a `.cpp` file.
- Inside it, define a simple C++ function `cpp_vector_scalar_mult` that takes an `arma::vec v` and a `double s` and returns their element-wise product `v * s`. Make sure to include `RcppArmadillo.h` and necessary `using namespace` directives, and the `[[Rcpp::export]]` tag.
- In the same `.cpp` file, use the `/** R ... */` block to write R code that:
 - * Defines a sample vector and a scalar in R.
 - * Calls your `cpp_vector_scalar_mult` function.
 - * Prints the result.
- Use `sourceCpp()` in an R script to compile and run the C++ function and the embedded R code.

```
// [[Rcpp::depends(RcppArmadillo)]]  
#include <RcppArmadillo.h>  
using namespace Rcpp;  
using namespace arma;  
  
//[[Rcpp::export]]  
vec cpp_vector_scalar_mult(vec v, double s) {  
  return v * s;  
}  
  
/** R  
vX <- 1:10  
dS <- 5  
res <- cpp_vector_scalar_mult(vX, dS)  
print(res)  
*/
```

35 Exercise Set 7: Creating R Packages

35.1 (1)

- **Simple R Package:**

- Create a new R package using RStudio named “MyUtils”.
- a) Inside the `R/` directory, create an R script file (e.g., `arithmetic.R`). Add a simple R function `fAddTwoNumbers(a, b)` that returns their sum.
- b) Edit the `DESCRIPTION` file:
 - * Give it a **Title**: “My First Utility Package”.
 - * Set yourself as the **Author** and **Maintainer**.
 - * Add a brief **Description**.
- c) Document your `fAddTwoNumbers` function. Create an `.Rd` file in the `man/` directory (e.g., `fAddTwoNumbers.Rd`). Include sections for `\name`, `\alias`, `\title`, `\description`, `\usage`, `\arguments`, `\value`, and a simple `\examples` block.
- d) Install and restart your package (Ctrl+Shift+L or Build -> Install and Restart).
- e) Test your function: `MyUtils::fAddTwoNumbers(5, 3)` and check its help page `?fAddTwoNumbers`.

```
MyUtils::fAddTwoNumbers(5, 2)
#> [1] 7
```

35.2 (2)

- **Package with C++ Function (Rcpp):**

- Extend the “MyUtils” package (or create a new one “MyCppUtils”).
- a) In the `src/` directory, create a C++ file (e.g., `cpp_helpers.cpp`).
- b) In this file, write a C++ function `fGeometricSumCpp(a, r, n)` that calculates the sum of the first `n` terms of a geometric series: $a + ar + ar^2 + \dots + ar^{n-1}$. Use a loop. Ensure it’s exported to R using `[[Rcpp::export]]` and includes `<Rcpp.h>`.

- c) Modify the DESCRIPTION file:
 - * Add Rcpp to Imports.
 - * Add Rcpp to LinkingTo.
- d) Add an R wrapper function in an R/ script (e.g., wrappers.R) called `fGeometricSum(a, r, n)` that simply calls your C++ function `fGeometricSumCpp(a, r, n)`.
- e) Document the R wrapper function `fGeometricSum` in the `man/` directory.
- f) Install and restart. Test `MyCppUtils::fGeometricSum(1, 0.5, 10)`.

35.3 (3)

- **Package with RcppArmadillo Function:**

- Extend one of your previous packages (or create “MyArmaUtils”).
- a) In the `src/` directory, create/modify a C++ file.
- b) Write a C++ function `fSolveSystemArma(mA, vB)` that takes an `arma::mat mA` and `arma::vec vB` and solves the linear system $mA \cdot x = vB$ for x using `arma::solve()`. Return x . Export it. Make sure to include `<RcppArmadillo.h>` and the `[[Rcpp::depends(RcppArmadillo)]]` attribute.
- c) Modify DESCRIPTION:
 - * Ensure Rcpp, RcppArmadillo are in Imports.
 - * Ensure Rcpp, RcppArmadillo are in LinkingTo.
- d) Write an R wrapper for `fSolveSystemArma`.
- e) Document the R wrapper.
- f) Install, restart, and test with a sample 3x3 matrix and vector.

35.4 (4)

- **Package Structure and NAMESPACE:**

- a) For any of the packages you’ve started, after installing and restarting (which RStudio often handles via `devtools::load_all()`), inspect the `NAMESPACE` file that RStudio/roxygen2 (if used) might generate.
- b) What is the purpose of the `NAMESPACE` file in an R package?
- c) If you were not using `roxygen2` (which RStudio uses by default for new packages), how would you manually ensure your R functions and C++ functions (via their R wrappers) are exported for users of your package? (Hint: `export()` in `NAMESPACE`).

35.5 (5)

- **Building and Sharing (Conceptual):**
 - a) For your “MyUtils” package, use RStudio’s “Build” pane to “Build Source Package”. What is the file extension of the output?
 - b) If you wanted to share this package with someone who also has R development tools, which file would you send them?
 - c) If you wanted to submit this package to CRAN (hypothetically), what are some of the key checks and requirements CRAN would have? (Refer to lecture notes/Hadley Wickham’s book concepts).

36 Exercise Set 8: Parallel Computing

36.1 (1)

- **Basic Parallel lapply:**

- a) Create a list of 10 numeric vectors, each containing 1,000,000 random numbers from a $N(0, 1)$ distribution.
- b) Write a function `fSlowFunction(v)` that calculates `mean(log(abs(v) + 1))` and then pauses for 0.1 seconds using `Sys.sleep(0.1)`.
- c) Use the standard `lapply` to apply `fSlowFunction` to your list of vectors. Time this operation using `system.time()`.
- d) Now, use the `parallel` package. Detect the number of cores on your machine. Create a cluster using `max(1, detectCores() - 1)` cores.
- e) Use `parLapply` from the `parallel` package to apply `fSlowFunction` to your list. Time this operation. Remember to export necessary objects/functions to the cluster if they are not in the base environment or loaded packages on workers.
- f) Compare the execution times. Stop the cluster.

```
listV <- list(  
  v1 = rnorm(1000000),  
  v2 = rnorm(1000000),  
  v3 = rnorm(1000000),  
  v4 = rnorm(1000000),  
  v5 = rnorm(1000000),  
  v6 = rnorm(1000000),  
  v7 = rnorm(1000000),  
  v8 = rnorm(1000000),  
  v9 = rnorm(1000000),  
  v10 = rnorm(1000000)  
)  
  
fSlowFunction <- function(v) {  
  Sys.sleep(0.1)  
  return(mean(log(abs(v) + 1)))  
}
```

```

}

system.time({lapply(listV, fSlowFunction)})
#>   bruger   system forløbet
#>   0.25    0.03    1.39

suppressMessages(library(parallel))
cluster <- makeCluster(max(1, detectCores() - 1))

system.time(parLapply(cluster, listV, fSlowFunction))
#>   bruger   system forløbet
#>   0.07    0.02    0.39
stopCluster(cluster)

```

36.2 (2)

- **Parallel Simulation and Aggregation:**

- You want to run 1000 simulations. In each simulation, you:
 1. Generate 500 random numbers from an Exponential distribution with rate = 0.5.
 2. Calculate the mean of these 500 numbers.
- a) Write an R function `fOneSim()` that performs one such simulation and returns the mean.
- b) Use `lapply` or a `for` loop to run 1000 simulations and collect the means. Time it.
- c) Use `parLapply` (or `mclapply` on non-Windows) to run these 1000 simulations in parallel. Time it and compare.
- d) Plot a histogram of the 1000 means. What does the Central Limit Theorem suggest about the shape of this distribution?

```

Exponential.Simulate <- function(lambda, size) {
  U <- runif(size)
  return(mean(-1/lambda * log(U)))
}

set.seed(10086)

X <- Exponential.Simulate(0.5, 500)

```

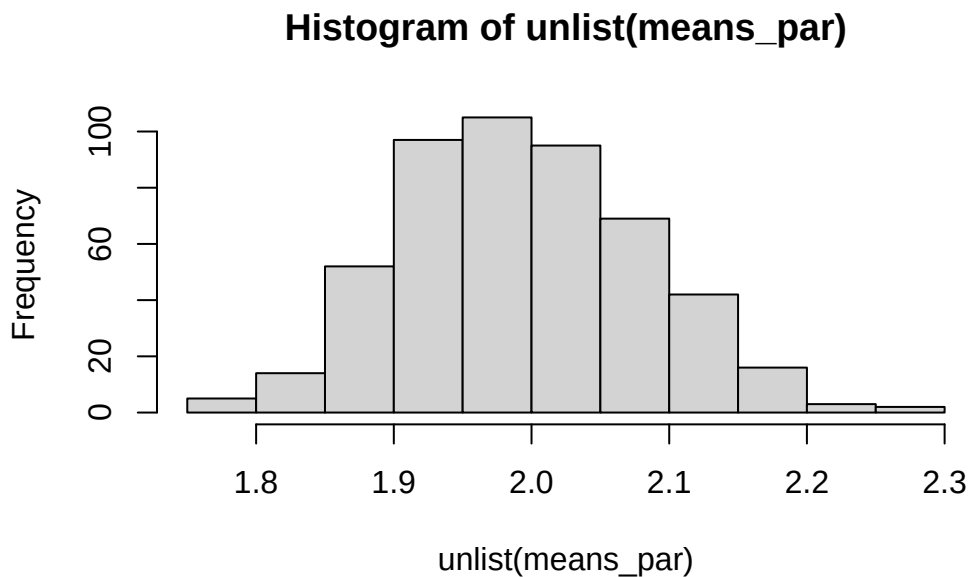
```

system.time(lResult <- lapply(1:500, function(x) Exponential.Simulate(0.5, 500)))
#>   bruger   system forløbet
#>   0.01    0.00    0.02

suppressMessages(library(parallel))
cluster <- makeCluster(max(1, detectCores() - 1))
clusterExport(cluster, "Exponential.Simulate")
system.time(means_par <- parLapply(cluster, 1:500, function(x) Exponential.Simulate(0.5, 500)))
#>   bruger   system forløbet
#>   0.00    0.00    0.01
stopCluster(cluster)

# CLT suggests distribution of sample means will be approximately normal
hist(unlist(means_par))

```



36.3 (3)

- **parSapply vs sapply:**
 - Consider the task of calculating `sqrt(i)` for `i` from 1 to 1,000,000, but make each calculation artificially slow by adding `Sys.sleep(0.00001)`.

- a) Implement this using `sapply`. Time it.
- b) Implement this using `parSapply`. Time it and compare. Remember cluster setup and shutdown.

```
system.time(sapply(1:1000000, function(x) {sqrt(x); Sys.sleep(0.00001)}}))
#>   bruger   system forløbet
#>    0.81    0.02    0.83

suppressMessages(library(parallel))
cluster <- makeCluster(max(1, detectCores() - 1))
system.time(parLapply(cluster, 1:1000000, function(x) {sqrt(x); Sys.sleep(0.00001)}}))
#>   bruger   system forløbet
#>    0.07    0.00    0.28
stopCluster(cluster)
```

36.4 (4)

• Considerations for Parallelization:

- a) When is parallelization most beneficial? (Think about task granularity and communication overhead).
- b) Give an example of a task that is “embarrassingly parallel” and one that is difficult or impossible to parallelize effectively.
- c) What are some potential pitfalls or complexities when writing parallel code (e.g., random number generation in parallel, race conditions - though less of an issue with `parLapply`’s model)?

```
# A: Task granularity high, communication overhead low.
# B: Embarrassingly parallel: Monte Carlo sims.
#   Difficult: Highly sequential algorithms like some recursive functions.
# C) RNG state per worker, data transfer, debugging
```

37 Advanced exercises

38 Exercise Set 9: Advanced R, Data Manipulation, and Debugging

38.1 (1)

- **Data Frame Manipulation with dplyr (if allowed, otherwise base R):**
 - Load the built-in `iris` dataset.
 - a) Filter the dataset to include only `Species == "setosa"` and `Sepal.Length > 5.0`.
 - b) Create a new column `Petal.Area` calculated as `Petal.Length * Petal.Width`.
 - c) For each `Species`, calculate the mean `Sepal.Length` and mean `Petal.Area`.
 - d) Arrange the results from (c) in descending order of mean `Sepal.Length`.
 - (If `dplyr` is not allowed, use base R functions like `subset()`, `transform()`, `aggregate()`, `order()`).

```
df_subset <- subset(iris, Species == "setosa" & Sepal.Length > 5.0)
df <- iris
df <- transform(df, Petal.Area = Petal.Length * Petal.Width)
agg_df <- aggregate(cbind(Sepal.Length, Petal.Area) ~ Species, data = df, FUN = mean)
agg_df[order(agg_df$Sepal.Length, decreasing = TRUE), ]
#>      Species Sepal.Length Petal.Area
#> 3  virginica      6.588      11.2962
#> 2 versicolor      5.936       5.7204
#> 1   setosa       5.006       0.3656
```

38.2 (2)

- **Working with Dates and Times (lubridate if allowed, otherwise base R):**
 - a) Create a character vector of dates: `vsDateStrings <- c("2023-01-15", "2023/03/22", "07-Apr-2023")`.
 - b) Convert `vsDateStrings` into R Date objects.
 - c) For each date, find the day of the week.

- d) Calculate the number of days between the first and last date in your converted date objects.

```
vsDateStrings <- c("2023-01-15", "2023/03/22", "07-Apr-2023")
date_objects <- as.Date(vsDateStrings, tryFormats = c("%Y-%m-%d", "%Y/%m/%d", "%d-%b-%Y"))
weekdays(date_objects)
#> [1] "søndag" NA      NA
diff(range(date_objects))
#> Time difference of NA days
```

38.3 (3)

- **Error Handling and Debugging:**

- Consider the following (buggy) R function:

```
fBuggyMean <- function(vData, bRemoveNA) {
  if (bRemoveNA = TRUE) { # Intentional bug: assignment instead of comparison
    vData <- vData[!is.na(vData)]
  }
  total_sum <- 0
  for (val in vData) {
    total_sum <- total_sum + val
  }
  return(total_sum / length(vData)) # Potential bug: division by zero if vData is empty
}
```

- a) Test `fBuggyMean(c(1, 2, NA, 4), bRemoveNA = TRUE)`. What is the output and why is it incorrect?
- b) Test `fBuggyMean(c(NA, NA), bRemoveNA = TRUE)`. What happens?
- c) Use R's debugging tools (`browser()`, `debug()`, or RStudio's debugger) to step through the function with the input from (a) and identify the logical error in the if condition.
- d) Correct the function. Also, add error handling using `tryCatch()` or if conditions to prevent division by zero if the vector (after NA removal) is empty, returning `NaN` or a message in such cases.

```
fBuggyMean_corrected <- function(vData, bRemoveNA) {
  if (isTRUE(bRemoveNA)) { # Corrected comparison
    vData <- vData[!is.na(vData)]
  }
}
```

```

if (length(vData) == 0) { # Handle empty vector
  return(NaN)
}
total_sum <- 0
for (val in vData) { # Loop is fine, sum() is better
  total_sum <- total_sum + val
}
return(total_sum / length(vData))
}
fBuggyMean_corrected(c(1,2,NA,4), bRemoveNA=TRUE)
#> [1] 2.333333
fBuggyMean_corrected(c(NA,NA), bRemoveNA=TRUE)
#> [1] NaN

```

38.4 (4)

- **String Manipulation (stringr if allowed, otherwise base R):**
 - You have a vector of product codes: `vsCodes <- c("PROD-A-123-X", "PROD-B-45-Y", "ITEM-C-6789-Z")`.
 - a) Extract the middle part (A, B, C) from each code.
 - b) Extract the numeric part (123, 45, 6789) from each code and convert it to a numeric vector.
 - c) Replace “PROD” with “PRODUCT” in all codes.

```

vsCodes <- c("PROD-A-123-X", "PROD-B-45-Y", "ITEM-C-6789-Z")
sapply(strsplit(vsCodes, "-"), function(x) x[2])
#> [1] "A" "B" "C"
as.numeric(sapply(strsplit(vsCodes, "-"), function(x) x[3]))
#> [1] 123 45 6789
gsub("PROD", "PRODUCT", vsCodes)
#> [1] "PRODUCT-A-123-X" "PRODUCT-B-45-Y" "ITEM-C-6789-Z"

```

38.5 (5)

- **Writing Efficient R Code - Revisiting Loops:**
 - You need to calculate a cumulative product of a vector `vX = 1:1000`.
 - a) Implement this using a `for` loop.
 - b) Implement this using R’s built-in `cumprod()` function.

- c) Compare their performance using `microbenchmark`. Explain why the built-in function is significantly faster.

```
vX_cumprod <- 1:100 # Smaller for quick demo
fCumprodLoop <- function(v) {
  res <- numeric(length(v))
  if (length(v) > 0) res[1] <- v[1]
  if (length(v) > 1) {
    for (i in 2:length(v)) res[i] <- res[i - 1] * v[i]
  }
  return(res)
}
microbenchmark::microbenchmark(fCumprodLoop(vX_cumprod), cumprod(vX_cumprod))
#> Unit: nanoseconds
#>           expr   min    lq  mean median    uq      max neval
#> fCumprodLoop(vX_cumprod) 8400 8600 62406   8700 8900 5317300   100
#>      cumprod(vX_cumprod)   500   600   886    600   700    9100   100
```

39 Exercise Set 10: Advanced C++ and Package Development

39.1 (1)

- **RcppArmadillo: More Linear Algebra:**

- Write a C++ function `arma_advanced_linalg` using RcppArmadillo that takes a square `arma::mat M`.
- The function should return a list containing:
 - * `dDeterminant`: The determinant of `M` (`arma::det(M)`).
 - * `dTrace`: The trace of `M` (`arma::trace(M)`).
 - * `mCholesky`: The Cholesky decomposition of `M`, if `M` is positive definite. (Use `arma::chol(M)`. You might want to wrap this in a try-catch block in C++ or check `arma::is_sympd(M)` first, returning an empty matrix or NA if not applicable).
- Test with a 3x3 symmetric positive definite matrix and a non-symmetric matrix.

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
// [[Rcpp::export]]
Rcpp::List arma_advanced_linalg(arma::mat M) {
    double dDet = arma::det(M);
    double dTrace = arma::trace(M);
    arma::mat mChol;
    bool chol_success = false;
    if(M.is_sympd()) { // Check if symmetric positive definite for Cholesky
        chol_success = arma::chol(mChol, M);
    }

    return Rcpp::List::create(
        Rcpp::Named("dDeterminant") = dDet,
        Rcpp::Named("dTrace") = dTrace,
        Rcpp::Named("mCholesky") = (chol_success ? Rcpp::wrap(mChol) : Rcpp::wrap(arma::mat()))
    );
}
```

39.2 (2)

- **Structuring C++ code in a Package:**

- Imagine you are building a package “StatTools” that includes several statistical computations.
- a) Create two C++ functions:
 - * `weighted_mean_cpp(arma::vec x, arma::vec weights)`: calculates the weighted mean.
 - * `variance_cpp(arma::vec x)`: calculates the sample variance.
- b) Place these in a single `.cpp` file within the `src/` directory of your package.
- c) Write R wrappers for both in an `R/` script.
- d) Ensure both are exported and documented.
- e) Update `DESCRIPTION` if necessary (e.g., if you used `RcppArmadillo`). Install and test.

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>

// [[Rcpp::export]]
double weighted_mean_cpp(arma::vec x, arma::vec weights) {
  return arma::sum(x % weights) / arma::sum(weights); // % is element-wise product
}

// [[Rcpp::export]]
double variance_cpp(arma::vec x) {
  return arma::var(x, 0); // 0 for sample variance (N-1 denominator)
}
```

39.3 (3)

- **Using C++ Standard Library Features within Rcpp:**

- Write a C++ function `cpp_unique_sorted(Rcpp::NumericVector x)` that returns a sorted vector containing only the unique elements of `x`.
- Hint: You can use `std::sort` and `std::unique` from the C++ `<algorithm>` header, then resize the Rcpp vector. Remember to include `<algorithm>`.
- Compare its output and speed to `sort(unique(x_r_vector))` in R.


```

#include <Rcpp.h>
#include <algorithm> // For std::sort, std::unique
#include <vector>     // For std::vector

// [[Rcpp::export]]
Rcpp::NumericVector cpp_unique_sorted(Rcpp::NumericVector x) {
  if (x.size() == 0) {
    return Rcpp::NumericVector(0);
  }
  // Convert to std::vector for std algorithms
  std::vector<double> std_x = Rcpp::as<std::vector<double>>(x);

  std::sort(std_x.begin(), std_x.end());
  // std::unique moves unique elements to the front and returns an iterator
  // to the end of the unique range.
  std_x.erase(std::unique(std_x.begin(), std_x.end()), std_x.end());

  return Rcpp::wrap(std_x); // Convert back to Rcpp::NumericVector
}

```

39.4 (4)

- **Package Vignettes (Conceptual/Simple Implementation):**

- a) What is the purpose of a vignette in an R package?
- b) In one of your previously created packages (e.g., “MyUtils”), create a simple vignette.
 - * Use RStudio: File > New File > R Markdown... > From Template > Package Vignette (devtools).
 - * Write a short introduction to your package and demonstrate the use of one of its functions.
 - * Build the vignette (RStudio’s Build pane -> More -> Build Vignettes).
 - * How would a user typically access this vignette after installing your package? (Hint: `vignette("vignette_name", package="PackageName")`).

a: Vignettes: long-form documentation, tutorials. e: `browseVignettes("MyUtils")` or `vignette("my-vignette-name", package="MyUtils")`

40 Exercise Set 11: Advanced Simulation and Optimization

40.1 (1)

- **Optimization with Noisy Objective Function:**

- Suppose you want to minimize $g(x) = (x - 2)^2$, but you can only observe $h(x) = g(x) + \epsilon$, where $\epsilon \sim N(0, 0.1^2)$ (observation noise).
- a) Write an R function for $h(x)$.
- b) Can you directly use `optimize` on $h(x)$ to reliably find the minimum of $g(x)$? Try it a few times. What happens?
- c) A common strategy is to average multiple evaluations of $h(x)$ to reduce noise. Create a new function `h_avg(x, n_reps)` that computes `mean(replicate(n_reps, h(x)))`.
- d) Use `optimize` on `h_avg(x, n_reps=100)`. Is the result more stable/accurate for finding the minimum of $g(x)$?

```
h <- function(x) {  
  return((x - 2)^2 + rnorm(1, 0, 0.1))  
}  
  
optimize(h, interval = c(-4, 4))$minimum  
#> [1] 2.058824  
optimize(h, interval = c(-4, 4))$minimum  
#> [1] 2.15631  
optimize(h, interval = c(-4, 4))$minimum  
#> [1] 2.111456  
optimize(h, interval = c(-4, 4))$minimum  
#> [1] 1.966731  
  
h_avg <- function(x, n_reps) {  
  return(mean(replicate(n_reps, h(x))))  
}  
  
optimize(h_avg, interval = c(-4,4), n_reps = 100)
```

```
#> $minimum
#> [1] 1.927585
#>
#> $objective
#> [1] -0.00223891
```

40.2 (2)

• Simulated Annealing (Conceptual/Simple Implementation):

- Goal: Find the global minimum of a “wiggly” function, e.g., $f(x) = (x^2 - 1)^2 + \cos(5\pi x)$ over $x \in [-2, 2]$.
- Simulated Annealing idea:
 1. Start at a random point x_c .
 2. Propose a new point x_n (e.g., $x_n = x_c + N(0, \text{stepsize})$).
 3. If $f(x_n) < f(x_c)$, move to x_n .
 4. If $f(x_n) \geq f(x_c)$, move to x_n with probability $P = \exp(-\frac{f(x_n) - f(x_c)}{T})$, where T is the “temperature”.
 5. Decrease T slowly (e.g., $T_{\text{new}} = T \times \text{coolingrate}$). Repeat.
- a) Plot $f(x)$.
- b) Implement a basic simulated annealing function. Choose a starting T (e.g., 10), a cooling rate (e.g., 0.99), a step size for proposals, and a number of iterations.
- c) Run it. Does it find a good minimum? Experiment with parameters. (This is more about the concept than a perfect implementation).

```
f <- function(x) {
  return((x^2 - 1)^2 + cos(5 * pi * x))
}

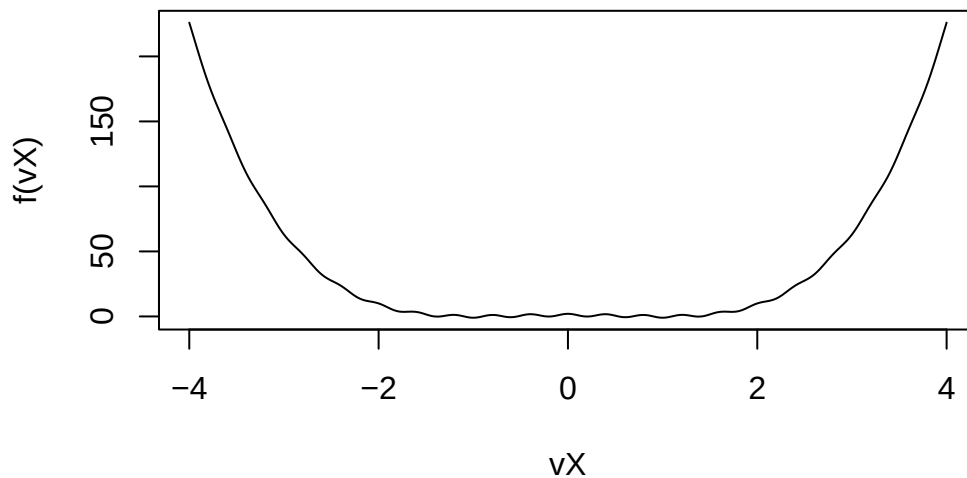
fSimAnn <- function(f, dLower, dUpper, dStart = (dUpper - dLower)/2, dCool, dStepSize, n.iter) {
  # start at a random point x_c
  x_c <- dStart
  T_new <- dT
  for (i in 1:n.iter) {
    x_n <- x_c + rnorm(1, 0, dStepSize)
    if (f(x_n) < f(x_c)) {
      x_c <- x_n
    } else if (runif(1) < exp(-(f(x_n) - f(x_c)) / T_new)) {
      x_c <- x_n
    }
  }
}
```

```

    T_new <- T_new * dCool
  }
  return(x_c)
}

vX <- seq(-4, 4, 0.01)
plot(vX, f(vX), type = "l")

```



```

fSimAnn(f, -2, 2, dCool = 0.99, dT = 10, dStepSize = 0.1, n.iter = 5000)
#> [1] 1.000058

```

40.3 (3)

- **Bootstrap Standard Errors for a Custom Statistic:**

- Suppose you have data `vData <- rexp(50, rate = 0.2)`. You are interested in the statistic $\theta = 1/\text{mean}(vData)$.
- a) Calculate $\hat{\theta}$ from `vData`.
- b) Write a bootstrap procedure:
 1. Set a number of bootstrap replications, B (e.g., 1000).

2. For each replication b from 1 to B :
 - * Resample $vData$ with replacement to get $vData_b$.
 - * Calculate $\hat{\theta}_b = 1/\text{mean}(vData_b)$.
 3. The bootstrap standard error of $\hat{\theta}$ is the standard deviation of the $\hat{\theta}_b$ values.
- c) Implement this and report the bootstrap standard error for $\hat{\theta}$.
 - d) (Optional) Can this be parallelized? How?

```
vData <- rexp(50, rate = 0.2)
# Theta-hat
1/mean(vData)
#> [1] 0.2036015

# bootstrap
B <- 1000
vB <- numeric(B)
for (b in 1:B) {
  vB[b] <- 1/mean(sample(vData, length(vData), replace = TRUE))
}

mean(vB)
#> [1] 0.2061833
sqrt(var(vB))
#> [1] 0.03275956
```

41 Exercise Set 12: Exam-Style Problems and Combinations

41.1 (1)

- **Problem: Estimating Parameters of a Mixture Distribution via Likelihood Maximization**

- You observe data $y_1, \dots, y_n$ which you believe comes from a mixture of two normal distributions: $f(y|\pi, \mu_1, \sigma_1, \mu_2, \sigma_2) = \pi \cdot N(y|\mu_1, \sigma_1^2) + (1 - \pi) \cdot N(y|\mu_2, \sigma_2^2)$ where π is the mixing proportion ($0 < \pi < 1$), and $N(y|\mu, \sigma^2)$ is the PDF of a normal distribution.
- Assume $\sigma_1 = 1$ and $\sigma_2 = 1$ are known. You need to estimate $\theta = (\pi, \mu_1, \mu_2)$.
- Given data: `set.seed(42); y_data <- c(rnorm(30, -2, 1), rnorm(70, 2, 1))`
- a) Write an R function for the log-likelihood of this mixture model, given `y_data` and parameters `pi`, `mu1`, `mu2`. (Remember to sum the logs of the individual likelihoods).
- b) Reparameterize π using the logit transform: $\pi = \frac{e^{\text{logit-pi}}}{1 + e^{\text{logit-pi}}}$ to make it unconstrained. Modify your log-likelihood function to take `logit_pi`, `mu1`, `mu2`.
- c) Use `optim` with the “BFGS” method to find the MLEs for `logit_pi`, `mu1`, `mu2`. Choose reasonable starting values.
- d) Transform `logit_pi_hat` back to $\hat{\pi}$. Report $\hat{\pi}, \hat{\mu}_1, \hat{\mu}_2$.
- e) (Optional C++) Write the log-likelihood calculation (for one data point) in C++ and try to use it within your R optimization (this is advanced, involving passing C++ pointers or using Rcpp sugar for calling R functions that wrap C++). Or, simply write the full log-likelihood sum in C++ and call it from R.

```
set.seed(42)
y_data <- c(rnorm(30, -2, 1), rnorm(70, 2, 1))

log_likelihood <- function(data, params, sigma_1, sigma_2) {
  logit_pi <- params[1]
  mu1 <- params[2]
  mu2 <- params[3]
```

```

pi <- exp(logit_pi) / (1 + exp(logit_pi))

return(-sum(log(pi * dnorm(data, mu1, sigma_1) + (1 - pi) * dnorm(data, mu2, sigma_2))))
}

logit_pi_hat <- optim(c(0.5, 0, 0), log_likelihood, data = y_data, sigma_1 = 1, sigma_2 = 1,
pi_hat <- exp(logit_pi_hat$par[1]) / (1 + exp(logit_pi_hat$par[1]))
pi_hat
#> [1] 0.7133243
logit_pi_hat$par[2]
#> [1] 1.988721
logit_pi_hat$par[3]
#> [1] -2.044431

```

41.2 (2)

- **Problem: Simulating a Queueing System and Root-Finding for Stability**

- Consider a simple M/M/1 queue. Customers arrive according to a Poisson process with rate λ . Service times are exponential with rate μ . The system is stable if $\rho = \lambda/\mu < 1$.

- **Simulation Part:**

- * a) Write an R function `simulate_queue(lambda, mu, n_customers)` that simulates the arrival and service of `n_customers`.
- * It should track arrival times and service completion times. For simplicity, assume the first customer arrives at time 0.
- * Return the average waiting time in the queue (time from arrival until service starts) for the customers.
- * (Hint: $Arrival_{i+1} = Arrival_i + Exp(\lambda)$. $ServiceStart_i = \max(Arrival_i, Completion_{i-1})$. $Completion_i = ServiceStart_i + Exp(\mu)$).
- * b) Run this simulation for $\lambda = 0.8, \mu = 1$ for 1000 customers. Report average waiting time. Repeat a few times. Does it vary?

- **Root-Finding Part:**

- * The theoretical average waiting time in an M/M/1 queue is $W_q = \frac{\lambda}{\mu(\mu-\lambda)}$ for $\rho < 1$.
- * c) Suppose $\mu = 1$ and you want to find the arrival rate λ that results in an average waiting time $W_q = 5$.
- * Define a function $g(\lambda) = \frac{\lambda}{1(1-\lambda)} - 5$.
- * Use `uniroot` to find $\lambda \in [0, 0.99]$ such that $g(\lambda) = 0$.

- d) (Package Creation) Create a small R package “QueueSim” containing your `simulate_queue` function. Document it and build the package.

```
simulate_queue <- function(lambda, mu, n_customers) {  
  if (n_customers == 0) return(0)  
  arrival_times <- numeric(n_customers)  
  service_start_times <- numeric(n_customers)  
  service_end_times <- numeric(n_customers)  
  service_times <- rexp(n_customers, lambda)  
  
  # First customer  
  arrival_times[1] <- 0  
  service_start_times[1] <- arrival_times[1]  
  service_end_times[1] <- service_start_times[1] + service_times[1]  
  
  if (n_customers > 1) {  
    inter_arrival_times <- rexp(n_customers - 1, rate = lambda)  
    for (i in 2:n_customers) {  
      arrival_times[i] <- arrival_times[i - 1] + inter_arrival_times[i - 1]  
      service_start_times[i] <- max(arrival_times[i], service_end_times[i - 1])  
      service_end_times[i] <- service_start_times[i] + service_times[i]  
    }  
  }  
  waiting_times <- service_start_times - arrival_times  
  return(mean(waiting_times))  
}  
  
simulate_queue(0.8, 1, 1000)  
#> [1] 61.33853  
simulate_queue(0.8, 1, 1000)  
#> [1] 12.78504  
simulate_queue(0.8, 1, 1000)  
#> [1] 71.37086  
simulate_queue(0.8, 1, 1000)  
#> [1] 35.86602  
  
g_lambda <- function(lambda) lambda / (1 * (1 - lambda)) - 5  
uniroot(g_lambda, interval = c(1e-6, 0.999))$root  
#> [1] 0.8333285
```


41.3 (3)

- **Problem: Path Dependent Option Pricing using Monte Carlo and C++ Speedup**

- An Asian call option's payoff depends on the average price of the underlying asset over a period. Payoff = $\max(0, \text{AvgPrice} - K)$, where K is the strike price.
- Asset price S_t follows a geometric Brownian motion: $dS_t = rS_t dt + \sigma S_t dW_t$, which discretizes to $S_{t+\Delta t} = S_t \exp((r - 0.5\sigma^2)\Delta t + \sigma\sqrt{\Delta t}Z_t)$, where $Z_t \sim N(0, 1)$.
- Parameters: $S_0 = 100, K = 100, r = 0.05, \sigma = 0.2, T = 1$ (1 year). Discretize into $M = 250$ time steps ($\Delta t = T/M$).

- **R Implementation:**

- * a) Write an R function `simulate_one_path_R(S0, r, sigma, T, M)` that simulates one price path and returns the average price over the M steps.
- * b) Write an R function `price_asian_option_R(S0, K, r, sigma, T, M, n_sims)` that:
 - Runs `n_sims` simulations of the average price using `simulate_one_path_R`.
 - Calculates the payoff for each simulation.
 - Returns the discounted average payoff: $e^{-rT} \cdot \text{mean}(\text{payoffs})$.
- * c) Price the option with `n_sims = 10000`. Time it.

- **C++ Implementation:**

- * d) Write a C++ function `simulate_one_path_Cpp(S0, r, sigma, T, M)` (using Rcpp/RcppArmadillo for random normals like `R::rnorm(0,1)` or `arma::randn`).
- * e) Write a C++ function `price_asian_option_Cpp(S0, K, r, sigma, T, M, n_sims)` that does the same as its R counterpart, calling `simulate_one_path_Cpp`.
- * f) Price the option using the C++ version with `n_sims = 10000`. Time it and compare with R.
- g) (Parallel) Use `parallel::parLapply` to run the R simulations (from part b) in parallel. Compare the time. (Export necessary functions/variables).

```
simulate_one_path_R <- function(S0, r, sigma, T_val, M) {  
  dt <- T_val / M  
  St_path <- numeric(M + 1)  
  St_path[1] <- S0  
  for (i in 1:M) {  
    Zt <- rnorm(1)  
    St_path[i+1] <- St_path[i] * exp((r - 0.5*sigma^2)*dt + sigma*sqrt(dt)*Zt)  
  }  
  return(mean(St_path[-1]))  
}
```

```

}

price_asian_option <- function(S0, K, r, sigma, T_val, M, n_sims) {
  payoffs <- numeric(n_sims)
  for (i in 1:n_sims) {
    payoffs[i] <- max(0, simulate_one_path_R(S0, r, sigma, T_val, M) - K)
  }
  # alternative
  # avg_prices <- replicate(n_sims, simulate_one_path_R(S0, r, sigma, T_val, M))
  # payoffs <- pmax(0, avg_prices - K)
  #
  return(exp(-r * T_val) * mean(payoffs))
}

price_asian_option(100, 100, 0.05, 0.2, 1, 250, 1000)
#> [1] 5.762387

```

42 Extra extra exercises

43 Exercise Set 13: R Fundamentals and Data Structures (Review)

43.1 (1)

- a) Create a sequence of numbers from 10 down to -10, decreasing by 0.5, and store it in a vector `vSeq`.
- b) How many elements are in `vSeq`?
- c) Create a logical vector `bIsPositive` which is `TRUE` for elements of `vSeq` greater than 0, and `FALSE` otherwise.
- d) Calculate the product of all elements in `vSeq` that are strictly between 1 and 5 (exclusive of 1 and 5).

```
vSeq <- seq(10, -10, -0.5)
length(vSeq)
#> [1] 41
bIsPositive <- vSeq > 0
prod(vSeq[vSeq > 1 & vSeq < 5])
#> [1] 1417.5
```

43.2 (2)

- a) Construct a 4x4 matrix `mIdentity` which is an identity matrix.
- b) Construct a 4x4 matrix `mValues` where elements are integers from 1 to 16, filled row-wise.
- c) Compute the matrix product `mC = mIdentity %*% mValues`. What is `mC`?
- d) Replace the diagonal elements of `mValues` with 0.

```
mIdentity <- diag(4)
mValues <- matrix(1:16, 4, 4, byrow = TRUE)
mC <- mIdentity %*% mValues
mC
#>      [,1] [,2] [,3] [,4]
```

```
#> [1,] 1 2 3 4
#> [2,] 5 6 7 8
#> [3,] 9 10 11 12
#> [4,] 13 14 15 16
diag(mValues) <- 0
mValues
#>      [,1] [,2] [,3] [,4]
#> [1,] 0 2 3 4
#> [2,] 5 0 7 8
#> [3,] 9 10 0 12
#> [4,] 13 14 15 0
```

43.3 (3)

- Write an R script that:
 - Initializes a counter `iCount` to 0 and a sum `dRunningSum` to 0.
 - Uses a **repeat** loop. Inside the loop:
 - * Generate a random integer between 1 and 20 (inclusive).
 - * Add this integer to `dRunningSum`.
 - * Increment `iCount`.
 - * If `dRunningSum` exceeds 100 or `iCount` reaches 15, **break** the loop.
 - Print the final `dRunningSum` and `iCount`. Set seed to 777.

```
set.seed(777)

iCount <- 0
dRunningSum <- 0

repeat {
  dRunningSum <- dRunningSum + sample(1:20, 1)
  iCount <- iCount + 1

  if (dRunningSum > 100 || iCount > 15) {
    break
  }
}

print(paste("Sum:", dRunningSum, "Count:", iCount))
#> [1] "Sum: 106 Count: 11"
```

43.4 (4)

- Create a list `lExperiment` with the following structure:
 - `sExpID`: “EXP007”
 - `dParams`: A named numeric vector with `alpha = 0.05`, `beta = 1.2`.
 - `lResults`: An inner list containing:
 - * `vObservations`: `c(10.2, 11.1, NA, 9.8, 10.5, NA, 10.9)`
 - * `sStatus`: “Preliminary”
- a) Access the `beta` parameter.
- b) Calculate the mean of `vObservations`, ignoring NAs.
- c) Change `sStatus` to “Completed”.

```
lExperiment <- list(  
  sExpID = "EXP007",  
  dParams = c(alpha = 0.05, beta = 1.2),  
  lResults = list(  
    vObservations = c(10.2, 11.1, NA, 9.8, 10.5, NA, 10.9),  
    sStatus = "Preliminary"  
  )  
)  
  
lExperiment[["dParams"]][["beta"]]  
#> [1] 1.2  
mean(lExperiment[["lResults"]][["vObservations"]], na.rm = TRUE)  
#> [1] 10.5  
lExperiment[["lResults"]][["sStatus"]] <- "Completed"  
lExperiment[["lResults"]][["sStatus"]]  
#> [1] "Completed"
```

43.5 (5)

- a) Create a data frame `dfSales` with columns `Month` (character: “Jan”, “Feb”, “Mar”, “Apr”), `ProductA_Units` (numeric: 100, 120, 90, 110), `ProductB_Units` (numeric: 80, 85, 95, 70).
- b) Add a new column `Total_Units` which is the sum of `ProductA_Units` and `ProductB_Units`.
- c) Create a new column `ProductA_Share` which is `ProductA_Units / Total_Units`.
- d) Find the month(s) where `ProductB_Units` were greater than 90.

```
dfSales <- data.frame(
  Month = c("Jan", "Feb", "Mar", "Apr"),
  ProductA_Units = c(100, 120, 90, 110),
  ProductB_Units = c(80, 85, 95, 70)
)
dfSales <- transform(dfSales, ProductA_Share = ProductA_Units / (ProductA_Units + ProductB_Units))
dfSales$Month[dfSales$ProductB_Units > 90]
#> [1] "Mar"
```

43.6 (6)

- Using `ifelse()`, create a character vector `vCategory` based on a numeric vector `vScores` = `c(45, 88, 62, 95, 70, 55)`.
- `vCategory` should be “Fail” if score < 50, “Pass” if 50 ≤ score < 70, “Merit” if 70 ≤ score < 90, and “Distinction” if score ≥ 90. (Hint: you might need nested `ifelse` statements).

```
vScores <- c(45, 88, 62, 95, 70, 55)
vCategory <- ifelse(vScores < 50, "Fail", ifelse(vScores < 70, "Pass", ifelse(vScores < 90, "Merit", "Distinction")))
vCategory
#> [1] "Fail"          "Merit"         "Pass"          "Distinction" "Merit"
#> [6] "Pass"
```

43.7 (7)

- Write a `for` loop that iterates 10 times. In each iteration `k`:
 - If `k` is even, print `k` is even.
 - If `k` is odd and also a multiple of 3, print `k` is odd and a multiple of 3.
 - Otherwise (if `k` is odd and not a multiple of 3), print `k` is odd.

```
for (k in 1:10) {
  if (k %% 2 == 0) {
    print(paste0(k, " is even."))
  } else if (k %% 3 == 0) {
    print(paste0(k, " is odd and a multiple of 3."))
  } else {
    print(paste0(k, " is odd."))
  }
}
```

```

}
#> [1] "1 is odd."
#> [1] "2 is even."
#> [1] "3 is odd and a multiple of 3."
#> [1] "4 is even."
#> [1] "5 is odd."
#> [1] "6 is even."
#> [1] "7 is odd."
#> [1] "8 is even."
#> [1] "9 is odd and a multiple of 3."
#> [1] "10 is even."

```

43.8 (8)

- a) Create a character vector `vsSentences = c("R is fun", "Data analysis with R", "Missing data NA", "Another sentence")`.
- b) Use `grep()` or `grepl()` to find which sentences contain the word “R” (case sensitive).
- c) Use `sub()` or `gsub()` to replace “R” with “R Language” in all sentences.
- d) Split the second sentence (“Data analysis with R”) into individual words.

```

vsSentences <- c("R is fun", "Data analysis with R", "Missing data NA", "Another sentence")
grep("R", vsSentences)
#> [1] 1 2
sub("R", "R Language", vsSentences)
#> [1] "R Language is fun"          "Data analysis with R Language"
#> [3] "Missing data NA"             "Another sentence"
strsplit(vsSentences[2], " ")
#> [[1]]
#> [1] "Data"      "analysis" "with"     "R"

```


44 Exercise Set 14: Functions, Scope, and Vectorization Focus

44.1 (1)

- Write an R function `fNormalizeVector` that takes a numeric vector `vX` as input.
- The function should return a “normalized” vector where each element x_i is transformed to $(x_i - \text{mean}(vX)) / \text{sd}(vX)$.
- The function should handle cases where `sd(vX)` is 0 (e.g., if all elements are the same) by returning a vector of zeros or NAs, with a warning.
- Test with `c(1,2,3,4,5)` and `c(5,5,5,5)`.

```
fNormalizeVector <- function(vX) {  
  if (sd(vX) == 0) {  
    warning("Standard deviation is 0 or all NA. Returning NAs or zeros.")  
    return(rep(NA, length(vX))) # Or zeros: numeric(length(vX))  
  } else {  
    return((vX - mean(vX)) / sd(vX))  
  }  
}  
  
fNormalizeVector(c(1,2,3,4,5))  
#> [1] -1.2649111 -0.6324555 0.0000000 0.6324555 1.2649111  
fNormalizeVector(c(5,5,5,5))  
#> Warning in fNormalizeVector(c(5, 5, 5, 5)): Standard deviation is 0 or all NA.  
#> Returning NAs or zeros.  
#> [1] NA NA NA NA
```

44.2 (2)

- What is lexical scoping in R? Provide a simple example with a function defined inside another function, where the inner function uses a variable from the outer function’s environment.

```
fOuter <- function(a) {
  b <- a * 2
  fInner <- function(c) {
    return(b + c) # 'b' is from fOuter's environment
  }
  return(fInner)
}
myInnerFunc <- fOuter(10)
result <- myInnerFunc(5)
result
#> [1] 25
```

Explain what `result` will be and why.

Result will be 25. `fInner` “closes over” the environment of `fOuter` where `b` (which is 20) was defined.

44.3 (3)

- Write a function `fCountValuesInRanges` that takes a numeric vector `vData` and a vector of `vBreakpoints` (sorted) as input.
- The function should count how many values in `vData` fall into each interval defined by `vBreakpoints`. For example, if `vBreakpoints = c(0, 10, 20, 30)`, the intervals are $(-\infty, 0]$, $(0, 10]$, $(10, 20]$, $(20, 30]$, $(30, \infty)$.
- Implement this using:
 - a) A loop structure.
 - b) R’s `cut()` or `findInterval()` function.
- Compare their outputs for `vData = rnorm(1000, mean=15, sd=10)` and `vBreakpoints = c(0, 10, 20, 30)`.

```
# a
fCountValuesInRanges <- function(vData, vBreakpoints) {
  full_breaks <- c(-Inf, sort(unique(vBreakpoints)), Inf)
  counts <- numeric(length(full_breaks) - 1)
  for (val in vData) {
    for (j in 1:(length(full_breaks) - 1)) {
      if (val > full_breaks[j] && val <= full_breaks[j + 1]) {
        counts[j] <- counts[j] + 1
        break
      }
    }
  }
}
```

```

    }
  }
  names(counts) <- cut((full_breaks[-1] + full_breaks[-length(full_breaks)])/2 , breaks=full_
  return(counts)
}

fCountValuesInRangesCut <- function(vData, vBreakpoints) {
  full_breaks <- c(-Inf, sort(unique(vBreakpoints)), Inf)
  return(table(cut(vData, breaks = full_breaks, include.lowest = TRUE, right = TRUE)))
}

vData <- rnorm(1000, mean = 15, sd = 10)
vBreakpoints <- c(0, 10, 20, 30)
fCountValuesInRanges(vData, vBreakpoints)
#>  [-Inf,0]    (0,10]    (10,20]    (20,30]    (30, Inf]
#>      81      258      394      209      58
fCountValuesInRangesCut(vData, vBreakpoints)
#>
#>  [-Inf,0]    (0,10]    (10,20]    (20,30]    (30, Inf]
#>      81      258      394      209      58

```

44.4 (4)

- Write a recursive R function `fRecursiveSum` that computes the sum of elements in a numeric vector `vX` without using the built-in `sum()` function or any loops. (Hint: sum of `vX` is `vX[1] + sum of vX[-1]`). Define the base case.

```

fRecursiveSum <- function(vX) {
  if (length(vX) == 0) return(0)
  return(vX[1] + fRecursiveSum(vX[-1]))
}
fRecursiveSum(1:5)
#> [1] 15

```

44.5 (5)

- Create a list of matrices, where each matrix has 3 columns but a random number of rows (between 5 and 10) and contains random integers.

```
set.seed(1)
lMatrices <- lapply(1:5, function(i) matrix(sample(1:50, sample(5:10,1)*3, replace=TRUE), nc
```

- Use `lapply` (or `sapply` if appropriate) to return a vector where each element is the sum of the diagonal elements (trace) of the corresponding matrix in `lMatrices`.

```
sapply(lMatrices, function(x) sum(diag(x)))
#> [1] 64 67 92 68 125
```

44.6 (6)

- Explain the difference between `apply()`, `lapply()`, `sapply()`, and `tapply()`. For each, provide a very brief example of a situation where it would be the most appropriate choice.

`apply()`: applies function over margins of an array/matrix. Ex: `apply(myMatrix, 1, sum)` for row sums. `lapply()`: applies function to each element of a list, returns a list. Ex: `lapply(myList, mean)`. `sapply()`: similar to `lapply`, but tries to simplify result to vector/matrix. Ex: `sapply(myList, length)`. `tapply()`: applies function to subsets of a vector, grouped by another vector (factor). Ex: `tapply(iris$Sepal.Length, iris$Species, mean)`

44.7 (7)

- Consider a task: for each number `i` from 1 to 1,000,000, if `i` is even, calculate `i/2`, if `i` is odd, calculate `3*i+1`.
- Implement this using:
 - a) A `for` loop storing results in a pre-allocated vector.
 - b) A vectorized approach using `ifelse()`.
- Compare their performance using `microbenchmark`.

```
suppressMessages(library(microbenchmark))
#> Warning: pakke 'microbenchmark' blev bygget under R version 4.3.3

fLoopApproach <- function(x) {
  vResults <- numeric(x)
  for (i in 1:x) {
    if (i %% 2 == 0) {
      vResults[i] <- i / 2
    }
  }
}
```

```

    } else {
      vResults[i] <- 3 * i + 1
    }
  }
  return(vResults)
}

fVectorizedApproach <- function(x) {
  return(ifelse(x %% 2 == 0, x / 2, 3 * x + 1))
}

microbenchmark(fLoopApproach(100000), fVectorizedApproach(100000))
#> Unit: microseconds
#>          expr      min       lq      mean   median      uq
#> fLoopApproach(1e+05) 18015.1 18497.70 19497.048 19405.25 19744.30
#> fVectorizedApproach(1e+05)      1.3      2.15      30.286       2.75      13.75
#>      max neval
#> 26860.4   100
#>  2252.5   100

```

44.8 (8)

- You have a data frame:

```

df <- data.frame(
  group = rep(c("A", "B", "C"), each = 4),
  value = rnorm(12)
)

```

- Using `tapply` (or `aggregate` or `dplyr` if allowed), calculate the range (max - min) of value for each group.

```

tapply(df$value, df$group, function(x) diff(range(x)))
#>      A      B      C
#> 2.854650 1.063354 2.165598

```

45 Exercise Set 15: Likelihood Functions and Optimization (Medium/Hard)

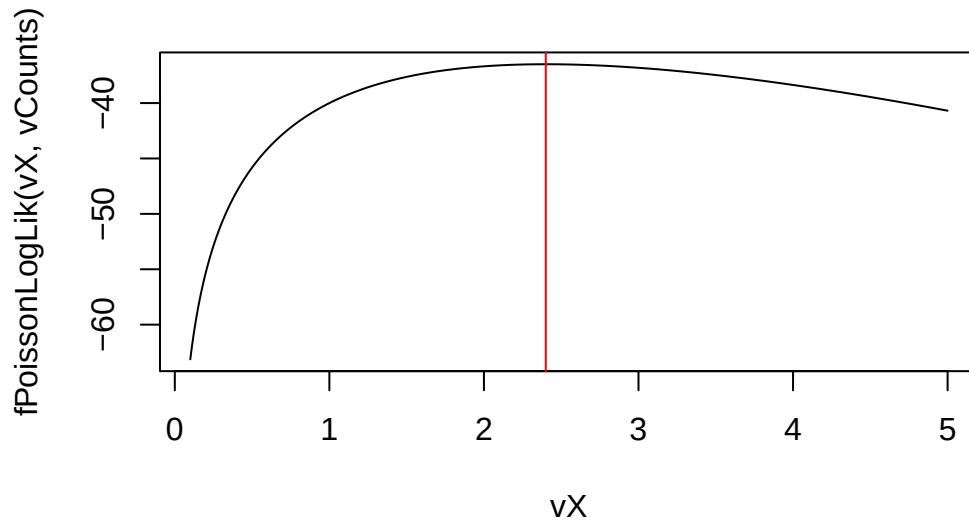
45.1 (1) Likelihood of a Poisson Process

- A Poisson process models the number of events occurring in a fixed interval of time or space, given a constant average rate λ . The probability of observing k events is $P(K = k|\lambda) = \frac{\lambda^k e^{-\lambda}}{k!}$.
- Suppose you observe the following number of events in $n = 5$ independent intervals: `vCounts = c(2, 3, 1, 2, 4)`.
- a) Write an R function `fPoissonLogLik` that takes a rate parameter `dLambda` and the vector `vCounts` as input, and returns the total log-likelihood for the observed counts.
- b) The MLE for λ is simply the sample mean of the counts. Calculate this $\hat{\lambda}_{MLE}$.
- c) Plot the log-likelihood function for $\lambda \in [0.1, 5]$. Mark the MLE on your plot.
- d) Use `optimize()` to find the value of λ that maximizes your `fPoissonLogLik` function (remember `optimize` minimizes by default). Compare it to your analytical MLE.
- e) (C++) Write a C++ function `poisson_log_lik_point_cpp(int k, double lambda)` that calculates the log-likelihood for a single count `k` and a given `lambda`. Then, write another C++ function `total_poisson_log_lik_cpp(Rcpp::IntegerVector counts, double lambda)` that iterates through the counts vector and sums the log-likelihoods using your point function. Test this against your R version.

```
fPoissonLogLik <- function(dLambda, vCounts) {
  dSum <- 0
  for (i in 1:length(vCounts)) {
    dSum <- dSum + vCounts[i] * log(dLambda) - dLambda - factorial(vCounts[i])
  }
  return(dSum)
}

vCounts <- c(2, 3, 1, 2, 4)

vX <- seq(0.1, 5, 0.01)
plot(vX, fPoissonLogLik(vX, vCounts), type = "l")
abline(v = mean(vCounts), col = "red")
```



```
mean(vCounts)
#> [1] 2.4
optimize(fPoissonLogLik, interval = c(0.1, 5), vCounts = vCounts, maximum = TRUE)
#> $maximum
#> [1] 2.400006
#>
#> $objective
#> [1] -36.49438
```

```
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
using namespace Rcpp;
using namespace arma;

// [[Rcpp::export]]
double poisson_log_lik_cpp(int k, double lambda) {
  return (k * log(lambda) - lambda - tgamma(k + 1));
}

// [[Rcpp::export]]
double total_poisson_log_lik_cpp(Rcpp::IntegerVector counts, double lambda) {
  double dSum = 0.0;
```

```

for (int i = 0; i < counts.size(); i++) {
  dSum += poisson_log_lik_cpp(counts[i], lambda);
}
return dSum;
}

```

```

suppressMessages(library(Rcpp))
#> Warning: pakke 'Rcpp' blev bygget under R version 4.3.3
sourceCpp("extra_extra_cpp.cpp")

total_poisson_log_lik_cpp(vCounts, mean(vCounts))
#> [1] -36.49438
fPoissonLogLik(mean(vCounts), vCounts)
#> [1] -36.49438

```

45.2 (2) Optimization of a Bimodal Likelihood (Reparameterization)

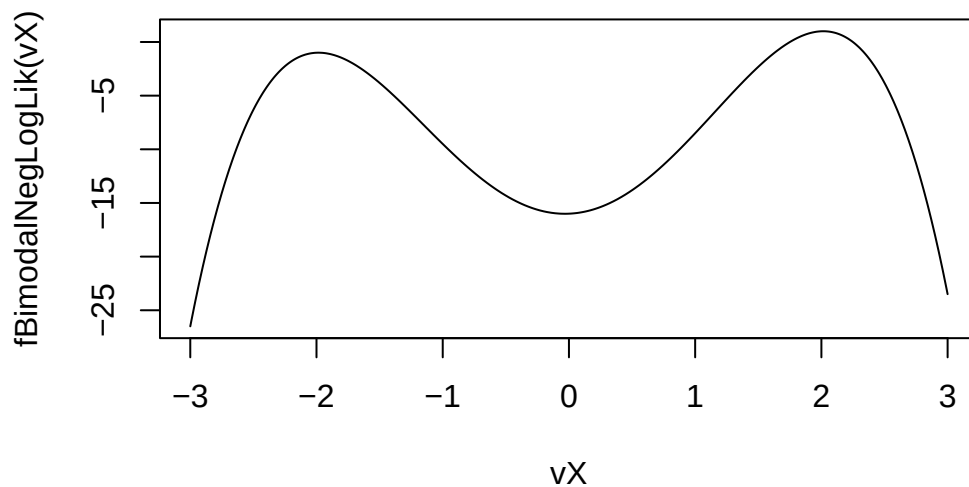
- Consider a custom likelihood function (unnormalized PDF) that is known to be bimodal for a parameter θ : $L(\theta|\text{data}) \propto \exp(-(\theta^2 - 4)^2 + 0.5\theta)$.
- a) Write an R function `fBimodalNegLogLik` for the negative log-likelihood (ignoring any normalizing constant for the data part, just use the function of θ).
- b) Plot this negative log-likelihood function for $\theta \in [-3, 3]$ to observe its shape and local minima.
- c) Use `optim()` with method “BFGS” and several different starting values (e.g., -2, 0, 2) to find local minima. Do you find different minima?
- d) Suppose you know one mode is positive and one is negative. If you want to find the positive mode, how could you use reparameterization to restrict `optim`’s search? For example, use $\theta = \exp(\tilde{\theta})$ to ensure $\theta > 0$. Define the new negative log-likelihood in terms of $\tilde{\theta}$ and optimize for $\tilde{\theta}$. Transform the result back to θ .
- e) Similarly, to find the negative mode, you might use $\theta = -\exp(\tilde{\theta})$. Implement this.

```

fBimodalNegLogLik <- function(dTheta) {
  return(-(dTheta^2 - 4)^2 + 0.5 * dTheta)
}

vX <- seq(-3, 3, 0.01)
plot(vX, fBimodalNegLogLik(vX), type = "l")

```

```

optim(-2, fBimodalNegLogLik, method = "BFGS")$par
#> [1] -1.147232e+15
optim(0, fBimodalNegLogLik, method = "BFGS")$par
#> [1] -0.03125764
optim(2, fBimodalNegLogLik, method = "BFGS")$par
#> [1] -1.160693e+34

fBimodalNegLogLik_Pos <- function(dTheta_tilde) {
  dTheta <- exp(dTheta_tilde)
  return(-(dTheta^2 - 4)^2 + 0.5 * dTheta)
}

dTheta_tilde_optim <- optim(0, fBimodalNegLogLik_Pos, method = "BFGS")$par
dTheta_optim <- exp(dTheta_tilde_optim)
dTheta_optim
#> [1] 3.726639e-06

fBimodalNegLogLik_Neg <- function(dTheta_tilde) {
  dTheta <- -exp(dTheta_tilde)
  return(-(dTheta^2 - 4)^2 + 0.5 * dTheta)
}

```

```
dTheta_tilde_optim <- optim(0, fBimodalNegLogLik_Neg, method = "BFGS")$par
dTheta_optim <- -exp(dTheta_tilde_optim)
dTheta_optim
#> [1] -1.01302e-05
```

45.3 (3) Likelihood for Linear Regression with Known Variance

- Consider a simple linear regression $y_i = \beta_0 + \beta_1 x_i + \epsilon_i$, where $\epsilon_i \sim N(0, \sigma^2)$. Assume $\sigma^2 = 1$ is known.
- Parameters to estimate are $\theta = (\beta_0, \beta_1)$.
- Data: `set.seed(123); x_data = 1:20; y_data = 0.5 + 2*x_data + rnorm(20, 0, 1);`
- a) Write an R function `fRegressNegLogLik(vBeta, vY, vX)` that calculates the negative log-likelihood. `vBeta` is a vector `c(beta0, beta1)`. (Hint: $L(\beta_0, \beta_1 | y, x, \sigma^2) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - (\beta_0 + \beta_1 x_i))^2}{2\sigma^2}\right)$).
- b) Use `optim()` (e.g., method “Nelder-Mead” or “BFGS”) to find the MLEs for β_0 and β_1 . Use starting values like `c(0,0)`.
- c) Compare your estimates with the coefficients obtained from `lm(y_data ~ x_data)`. They should be similar (though `lm` also estimates σ^2 if not fixed).
- d) (Conceptual) If σ^2 were also unknown, how would your parameter vector θ and your reparameterization strategy for σ^2 (to ensure positivity) change?

```
set.seed(12)
x_data <- 1:20
y_data <- 0.5 + 2 * x_data + rnorm(20, 0, 1)

fRegressNegLogLik <- function(vBeta, vY, vX, dSigma2 = 1) {
  dSum <- 0
  for (i in 1:length(vY)) {
    dSum <- dSum + (vY[i] - (vBeta[1] + vBeta[2] * vX[i]))^2 / (2 * dSigma2)
  }
  return(dSum)
}

optim(c(0, 0), fRegressNegLogLik, vY = y_data, vX = x_data, method = "BFGS")$par
#> [1] -0.3311065 2.0476093

coef(lm(y_data ~ x_data))
#> (Intercept)      x_data
#> -0.3311065 2.0476093
```

45.4 (4) Optimization with Constraints using `constrOptim` (or Penalty)

- Minimize $f(x, y) = (x - 1)^2 + (y - 2)^2$ subject to the linear constraint $x + y \geq 4$.
- **Method 1: `constrOptim` (if familiar or allowed as it's base R but more advanced)**
 - a) The constraint can be written as $1x + 1y - 4 \geq 0$. `constrOptim` requires constraints in the form $U\theta - C \geq 0$. Define U and C .
 - b) Use `constrOptim` to solve this. You'll need to provide the objective function `f`, its gradient (optional, `constrOptim` can approximate), `ui` (U matrix), and `ci` (C vector). Choose a starting point that satisfies the constraint e.g., $(3, 3)$.
- **Method 2: Penalty Function**
 - c) Define a penalized objective function $f_p(x, y) = f(x, y) + P \cdot \max(0, 4 - (x + y))^2$, where P is a large penalty constant (e.g., 1000).
 - d) Use `optim` with "BFGS" or "Nelder-Mead" to minimize $f_p(x, y)$ without explicit constraints.
 - e) Compare results from both methods if possible. The analytical solution is $(x, y) = (1.5, 2.5)$.

```
f <- function(vX) {  
  return((vX[1] - 1)^2 + (vX[2] - 2)^2)  
}  
  
p <- function(vX) {  
  return(max(0, 4 - (vX[1] + vX[2]))^2)  
}  
  
f_p <- function(vX, ...) {  
  return(f(vX) + 1000 * p(vX))  
}  
  
optim(c(3, 3), f_p, method = "BFGS")$par  
#> [1] 1.499802 2.499780
```

45.5 (5) Profile Likelihood

- Revisit the Poisson likelihood from Q1 (`vCounts = c(2, 3, 1, 2, 4)`). Suppose we have a more complex model with two parameters, λ_1 and λ_2 , and the log-likelihood is $LL(\lambda_1, \lambda_2)$.

- Often, we are interested in inference for one parameter (e.g., λ_1) while treating the other as a nuisance parameter. The profile log-likelihood for λ_1 is defined as $PL(\lambda_1) = \max_{\lambda_2} LL(\lambda_1, \lambda_2)$.
- Let $LL(\lambda_1, \lambda_2) = \sum_{i=1}^3 \log(\text{dpois}(vCounts[i], \lambda_1)) + \sum_{i=4}^5 \log(\text{dpois}(vCounts[i], \lambda_2))$. (This is an artificial separation for demonstration).
- a) Write an R function `fCombinedLogLik(vLambda, vC)` where `vLambda = c(lambda1, lambda2)`.
- b) Write an R function `fProfileLogLik_L1(dLambda1_fixed, vC)` that, for a fixed `dLambda1_fixed`, uses `optimize` to find the `dLambda2` that maximizes `fCombinedLogLik` (with that `dLambda1_fixed`), and returns this maximized log-likelihood value.
- c) Create a sequence of `dLambda1_fixed` values (e.g., from 0.5 to 3.5). For each value, calculate `fProfileLogLik_L1`.
- d) Plot the profile log-likelihood $PL(\lambda_1)$ against `dLambda1_fixed`. The maximum of this plot gives an estimate for λ_1 . From this plot, can you estimate an approximate confidence interval for λ_1 ? (e.g., points where $PL(\lambda_1) > \max(PL) - \text{qchisq}(0.95, 1)/2$).

```
vCounts <- c(2, 3, 1, 2, 4)

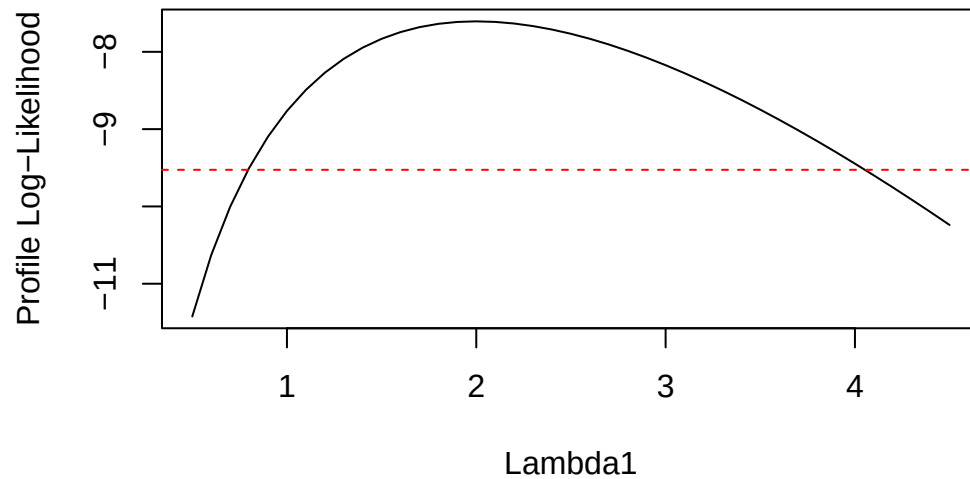
fCombinedLogLik <- function(vLambda, vC) {
  dSum1 <- 0.0
  dSum2 <- 0.0
  for (i in 1:3) {
    dSum1 <- dSum1 + log(dpois(vCounts[i], vLambda[1]))
  }
  for (i in 4:5) {
    dSum2 <- dSum2 + log(dpois(vCounts[i], vLambda[2]))
  }
  return(dSum1 + dSum2)
}

fProfileLogLik_L1 <- function(dLambda1_fixed, vC) {
  # Objective for optimize: function of lambda2
  obj_for_lambda2 <- function(dLambda2) {
    fCombinedLogLik(c(dLambda1_fixed, dLambda2), vC)
  }
  # Optimize returns list, $maximum is the value of objective function
  optimize(obj_for_lambda2, interval=c(0.01, 10), maximum=TRUE)$objective
}

lambda1_seq <- seq(0.5, 4.5, 0.1)
```

```
profile_loglik_values <- sapply(lambda1_seq, fProfileLogLik_L1, vC=vCounts)

plot(lambda1_seq, profile_loglik_values, type="l", xlab="Lambda1", ylab="Profile Log-Likelihood")
abline(h = max(profile_loglik_values) - qchisq(0.95,1)/2, col="red", lty=2)
```



46 Likelihood Exercises

47 Exercise Set 16: Advanced Econometric Likelihoods and Optimization

47.1 (1) Probit Model Likelihood (Medium-Hard)

- A Probit model for a binary outcome $y_i \in \{0, 1\}$ is given by $P(y_i = 1 | \mathbf{x}_i) = \Phi(\mathbf{x}_i' \beta)$, where $\Phi(\cdot)$ is the standard normal CDF, $\mathbf{x}_i$ is a vector of explanatory variables (including an intercept), and β is a vector of parameters.
- The log-likelihood for n observations is $LL(\beta) = \sum_{i=1}^n [y_i \log(\Phi(\mathbf{x}_i' \beta)) + (1 - y_i) \log(1 - \Phi(\mathbf{x}_i' \beta))]$.
- Data:

```
set.seed(123)
N <- 200
X1 <- rnorm(N)
X2 <- runif(N)
X_matrix <- cbind(1, X1, X2) # Design matrix with intercept
beta_true <- c(-0.5, 1.2, -0.8)
linear_predictor <- X_matrix %*% beta_true
prob_y1 <- pnorm(linear_predictor)
Y_binary <- rbinom(N, 1, prob_y1)
```

- a) Write an R function `fProbitNegLogLik(vBeta, mX, vY)` that calculates the negative log-likelihood for the Probit model. `vBeta` is the parameter vector, `mX` is the design matrix, and `vY` is the binary outcome vector.
- b) Use `optim()` with method “BFGS” to find the MLEs for β . Choose appropriate starting values (e.g., a vector of zeros).
- c) Compare your estimated β with the coefficients from `glm(Y_binary ~ X1 + X2, family = binomial(link = "probit"))`.
- d) (Hard) Analytically derive the gradient of the Probit log-likelihood with respect to β . Recall that $\frac{d\Phi(z)}{dz} = \phi(z)$ (standard normal PDF). The gradient for one observation is $\frac{y_i - \Phi(\mathbf{x}_i' \beta)}{\Phi(\mathbf{x}_i' \beta)(1 - \Phi(\mathbf{x}_i' \beta))} \phi(\mathbf{x}_i' \beta) \mathbf{x}_i$. Write an R function for this gradient (summed over observations) and use it in `optim()` via the `gr` argument. Does it converge faster or to a more precise solution?

- e) (C++) Implement the Probit negative log-likelihood calculation in C++ (e.g., `probit_neg_log_lik_cpp(arma::vec beta, arma::mat X, arma::vec Y)`). You can use `R::pnorm` and `R::dnorm` for Φ and ϕ . Compare its speed to the R version for a single evaluation.

```
#data
set.seed(123)
N <- 200
X1 <- rnorm(N)
X2 <- runif(N)
X_matrix <- cbind(1, X1, X2) # matrix with intercept
beta_true <- c(-0.5, 1.2, -0.8)
linear_predictor <- X_matrix %*% beta_true
prob_y1 <- pnorm(linear_predictor)
Y_binary <- rbinom(N, 1, prob_y1)

#a
fProbitNegLogLik <- function(vBeta, mX, vY) {
  n <- length(vY)
  dSum <- 0
  for (i in 1:n) {
    dSum <- dSum + vY[i] * log(pnorm(mX %*% vBeta))[i] + (1 - vY[i]) * log(1 - pnorm(mX %*% vBeta))
  }
  return(-dSum)
  # alternative
  # return(-sum(vY * log(pnorm(mX %*% vBeta)) + (1 - vY) * log(1 - pnorm(mX %*% vBeta))))
}

#b
optim(c(0, 0, 0), fProbitNegLogLik, mX = X_matrix, vY = Y_binary, method = "BFGS")$par
#> [1] -0.6124990 1.6593570 -0.7099176

#c
glm(Y_binary ~ X1 + X2, family = binomial(link = "probit"))$coefficients
#> (Intercept)          X1          X2
#> -0.6125001 1.6593555 -0.7099154

#d
fProbitGrad <- function(vBeta, mX, vY) {
  n <- length(vY)
  dSum <- 0
  dProb <- pnorm(mX %*% vBeta)
  for (i in 1:n) {
```



```

    dSum <- dSum + (vY[i] - dProb[i]) / (dProb[i] * (1 - dProb)[i]) * dnorm(mX %*% vBeta)[i]
  }
  return(-dSum)
}

optim(c(0, 0, 0), fProbitNegLogLik, gr = fProbitGrad, mX = X_matrix, vY = Y_binary, method =
#> [1] -0.6124989  1.6593568 -0.7099176

```

```

// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
using namespace Rcpp;
using namespace arma;

// [[Rcpp::export]]
double probit_neg_log_lik_cpp(vec beta, mat X, vec Y) {
  int n = Y.size();
  double dSum = 0.0;
  vec linear_term = X * beta;
  vec dProb = zeros<vec>(X.n_rows);

  for (int i = 0; i < n; i++) {
    dProb(i) = R::pnorm(linear_term(i), 0.0, 1.0, 1, 0);
    dSum += Y[i] * log(dProb[i]) + (1 - Y[i]) * log(1 - dProb[i]);
  }
  return(-dSum);
}

```

```

#e
suppressMessages(library(Rcpp))
#> Warning: pakke 'Rcpp' blev bygget under R version 4.3.3
suppressMessages(library(RcppArmadillo))
#> Warning: pakke 'RcppArmadillo' blev bygget under R version 4.3.3
sourceCpp("likelihood_exercises.cpp")

probit_neg_log_lik_cpp(beta_true, X_matrix, Y_binary)
#> [1] 66.87324
fProbitNegLogLik(beta_true, X_matrix, Y_binary)
#> [1] 66.87324

```

47.2 (2) GARCH(1,1) Model Likelihood (Hard)

- Recall the GARCH(1,1) model: $r_t = \sigma_t \epsilon_t$, where $\epsilon_t \sim N(0, 1)$ (iid) $\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2$
- Parameters are $\theta = (\omega, \alpha, \beta)$. Constraints: $\omega > 0, \alpha \geq 0, \beta \geq 0$, and $\alpha + \beta < 1$ (for stationarity).
- The conditional log-likelihood for r_t given past information (and σ_t^2) is $-\frac{1}{2}(\log(2\pi) + \log(\sigma_t^2) + \frac{r_t^2}{\sigma_t^2})$. The total log-likelihood is the sum over $t = 1, \dots, T$.
- For $t = 1$, an initial σ_1^2 is needed. Often, the unconditional variance $\sigma^2 = \frac{\omega}{1-\alpha-\beta}$ is used if the process is stationary, or simply r_0^2 is set to the sample variance of r_t and σ_1^2 is calculated using the GARCH equation from r_0^2 and a starting σ_0^2 (e.g., also sample variance).
- Data: Use S&P 500 daily log-returns. You can download data using `quantmod` package if allowed, or use a pre-saved dataset. For this exercise, let's simulate some GARCH-like data:

```
set.seed(456)
T_garch <- 500
omega_true <- 0.1; alpha_true <- 0.1; beta_true <- 0.85
sigma2_garch <- numeric(T_garch)
r_garch <- numeric(T_garch)
sigma2_garch[1] <- omega_true / (1 - alpha_true - beta_true) # Unconditional variance
r_garch[1] <- sqrt(sigma2_garch[1]) * rnorm(1)
for(t_idx in 2:T_garch) {
  sigma2_garch[t_idx] <- omega_true + alpha_true * r_garch[t_idx-1]^2 + beta_true * sigma2_garch[t_idx-1]
  r_garch[t_idx] <- sqrt(sigma2_garch[t_idx]) * rnorm(1)
}
```

- a) Write an R function `fGARCH11NegLogLik(vParams, vReturns, sigma2_initial)` that computes the negative log-likelihood. `vParams` is `c(omega, alpha, beta)`. Handle the recursive calculation of σ_t^2 .
- b) Reparameterize the parameters to enforce constraints:
 - $\omega = \exp(\tilde{\omega})$
 - $\alpha = \exp(\tilde{\alpha}) / (1 + \exp(\tilde{\alpha}) + \exp(\tilde{\beta}))$
 - $\beta = \exp(\tilde{\beta}) / (1 + \exp(\tilde{\alpha}) + \exp(\tilde{\beta}))$ (This ensures $\omega > 0, \alpha > 0, \beta > 0, \alpha + \beta < 1$. A simpler common reparam is $\alpha = \exp(\tilde{\alpha}) / (1 + \exp(\tilde{\alpha}))$ and $\beta = \exp(\tilde{\beta}) / (1 + \exp(\tilde{\beta}))$, then check $\alpha + \beta < 1$ inside LL and return large penalty if not met, or use L-BFGS-B on transformed α, β in $[0, 1]$ and $\omega > 0$, with an additional check for sum < 1). Modify your likelihood function to take reparameterized inputs $\tilde{\theta} = (\tilde{\omega}, \tilde{\alpha}, \tilde{\beta})$.
- c) Use `optim()` with “BFGS” to find the MLEs of $\tilde{\theta}$ using `r_garch` and an appropriate `sigma2_initial` (e.g., sample variance of `r_garch`).

- d) Transform the estimated $\tilde{\theta}^*$ back to $\theta^* = (\omega^*, \alpha^*, \beta^*)$. Compare with `omega_true`, `alpha_true`, `beta_true`.
- e) (Extremely Hard C++) Implement the GARCH(1,1) negative log-likelihood calculation in C++ using `RcppArmadillo`. This will involve a loop to calculate the σ_t^2 sequence. Compare its speed for a single evaluation against the R version.

```
#data

set.seed(456)
T_garch <- 500
omega_true <- 0.1; alpha_true <- 0.1; beta_true <- 0.85
sigma2_garch <- numeric(T_garch)
r_garch <- numeric(T_garch)
sigma2_garch[1] <- omega_true / (1 - alpha_true - beta_true) # unconditional variance
r_garch[1] <- sqrt(sigma2_garch[1]) * rnorm(1)
for (t_idx in 2:T_garch) {
  sigma2_garch[t_idx] <- omega_true + alpha_true * r_garch[t_idx-1]^2 + beta_true * sigma2_garch[t_idx-1]
  r_garch[t_idx] <- sqrt(sigma2_garch[t_idx]) * rnorm(1)
}

#a
fGARCH11NegLogLik <- function(vParams, vReturns, sigma2_initial) {
  dOmega <- vParams[1]
  dAlpha <- vParams[2]
  dBeta <- vParams[3]

  dSum <- 0
  vSigma2 <- numeric(length(vReturns))
  vSigma2[1] <- sigma2_initial
  for (t in 2:length(vReturns)) {
    vSigma2[t] <- dOmega + dAlpha * vReturns[t-1]^2 + dBeta * vSigma2[t-1]
    dSum <- dSum - 0.5*(log(2*pi) + log(vSigma2[t]) + vReturns[t]^2 / vSigma2[t])
  }
  return(-dSum)
}

#b
fGARCH11NegLogLikRepar <- function(vParams, vReturns, sigma2_initial) {
  dOmega <- exp(vParams[1])
  dAlpha <- exp(vParams[2]) / (1 + exp(vParams[2]) + exp(vParams[3]))
  dBeta <- exp(vParams[3]) / (1 + exp(vParams[2]) + exp(vParams[3]))
}
```

```

dSum <- 0
vSigma2 <- numeric(length(vReturns))
vSigma2[1] <- sigma2_initial
for (t in 2:length(vReturns)) {
  vSigma2[t] <- dOmega + dAlpha * vReturns[t-1]^2 + dBeta * vSigma2[t-1]
  dSum <- dSum - 0.5*(log(2*pi) + log(vSigma2[t]) + vReturns[t]^2 / vSigma2[t])
}
return(-dSum)
}

#c
dPar_tilde <- optim(c(0, 0, 0), fGARCH11NegLogLikRepar, sigma2_initial = var(r_garch), vRetu

#d
dOmega_star <- exp(dPar_tilde[1])
dAlpha_star <- exp(dPar_tilde[2]) / (1 + exp(dPar_tilde[2]) + exp(dPar_tilde[3]))
dBeta_star <- exp(dPar_tilde[3]) / (1 + exp(dPar_tilde[2]) + exp(dPar_tilde[3]))

print(paste0("Omega est: ", dOmega_star, " vs. true: ", omega_true))
#> [1] "Omega est: 0.188330580842483 vs. true: 0.1"
print(paste0("Alpha est: ", dAlpha_star, " vs. true: ", alpha_true))
#> [1] "Alpha est: 0.120768680865144 vs. true: 0.1"
print(paste0("Beta est: ", dBeta_star, " vs. true: ", beta_true))
#> [1] "Beta est: 0.768485839334849 vs. true: 0.85"

double fGARCH11NegLogLik_cpp(vec vParams, vec vReturns, double sigma2_initial) {
  double dOmega = vParams[0];
  double dAlpha = vParams[1];
  double dBeta = vParams[2];
  int n = vReturns.size();

  double dSum = 0;
  vec vSigma2 = zeros<vec>(vReturns.size());
  vSigma2[0] = sigma2_initial;
  double dPi = atan(1)*4;

  for (int t = 1; t < n; t++) {
    vSigma2[t] = dOmega + dAlpha * pow(vReturns[t-1], 2) + dBeta * vSigma2[t-1];
    dSum = dSum - 0.5*(log(2*dPi) + log(vSigma2[t]) + pow(vReturns[t], 2) / vSigma2[t]);
  }
  return(-dSum);
}

```

```
}
```

```
suppressMessages(library(Rcpp))
suppressMessages(library(RcppArmadillo))
sourceCpp("likelihood_exercises.cpp")

fGARCH11NegLogLik(c(omega_true, alpha_true, beta_true), r_garch, var(r_garch))
#> [1] 830.2781
fGARCH11NegLogLik_cpp(c(omega_true, alpha_true, beta_true), r_garch, var(r_garch))
#> [1] 830.2781
```

47.3 (3) Heckman Selection Model Likelihood (Extremely Hard)

- The Heckman model addresses sample selection bias. It involves two equations:
 1. Selection equation (Probit): $z_i^* = \mathbf{w}_i' \gamma + u_i$, where $z_i = 1$ if $z_i^* > 0$ (observed), 0 otherwise. $u_i \sim N(0, 1)$.
 2. Outcome equation (Linear regression, observed only if $z_i = 1$): $y_i = \mathbf{x}_i' \beta + \epsilon_i$, where $\epsilon_i \sim N(0, \sigma^2)$.
 - u_i and ϵ_i are bivariate normal with correlation ρ . $\text{corr}(u_i, \epsilon_i) = \rho$.
- The log-likelihood for one observation i (where y_i is observed if $z_i = 1$) is:
 - If $z_i = 0$: $LL_i = \log(1 - \Phi(\mathbf{w}_i' \gamma))$
 - If $z_i = 1$: $LL_i = \log \left(\frac{1}{\sigma} \phi \left(\frac{y_i - \mathbf{x}_i' \beta}{\sigma} \right) \Phi \left(\frac{\mathbf{w}_i' \gamma + \frac{\rho}{\sigma} (y_i - \mathbf{x}_i' \beta)}{\sqrt{1 - \rho^2}} \right) \right)$
- Parameters are $\theta = (\gamma', \beta', \sigma, \rho)'$. Constraints: $\sigma > 0, -1 < \rho < 1$.
- Data: You would typically simulate this.

```
set.seed(789)
N_heck <- 300
W1 <- rnorm(N_heck); W_matrix <- cbind(1, W1) # Selection vars
X1 <- rnorm(N_heck); X_matrix_outcome <- cbind(1, X1) # Outcome vars
gamma_true <- c(0.5, -1); beta_true <- c(1, 2); sigma_true <- 1.5; rho_true <- 0.6

# Simulate errors
errors <- MASS::mvrnorm(N_heck, mu=c(0,0), Sigma=matrix(c(1, rho_true*sigma_true, rho_true*sigma_true, 1), 2, 2))
u_i <- errors[,1]
epsilon_i <- errors[,2]

z_star <- W_matrix %*% gamma_true + u_i
```

```

Z_select <- ifelse(z_star > 0, 1, 0)

Y_outcome <- X_matrix_outcome %*% beta_true + epsilon_i
Y_observed <- ifelse(Z_select == 1, Y_outcome, NA) # Only observe Y if Z_select is 1
df_heckman <- data.frame(Y_observed, X1, W1, Z_select)
df_heckman_obs <- subset(df_heckman, Z_select == 1) # data for outcome eq.

```

- a) Write an R function `fHeckmanNegLogLik(vParams, dfData, mSelectionVars, mOutcomeVars)` that calculates the total negative log-likelihood. `vParams` contains all parameters. `dfData` should contain the outcome y , the selection indicator z , and covariates. `mSelectionVars` and `mOutcomeVars` are column names/indices for $\mathbf{w}$ and $\mathbf{x}$.
- b) Implement reparameterizations for $\sigma = \exp(\tilde{\sigma})$ and $\rho = \tanh(\tilde{\rho}) = \frac{e^{\tilde{\rho}} - e^{-\tilde{\rho}}}{e^{\tilde{\rho}} + e^{-\tilde{\rho}}}$. Modify your likelihood function.
- c) Use `optim()` (“BFGS” or “Nelder-Mead” due to complexity) to estimate the reparameterized parameters using the simulated data. This is notoriously difficult to optimize; choose starting values carefully (e.g., from separate Probit and OLS on selected sample, $\tilde{\sigma} = \log(\text{sd-res-ols})$, $\tilde{\rho} = 0$).
- d) Transform estimates back and compare to true values.
- (This problem is very challenging due to the complexity of the likelihood and potential for numerical instability. Focus on correctly writing the likelihood parts).

```

#data
set.seed(789)
N_heck <- 300
W1 <- rnorm(N_heck); W_matrix <- cbind(1, W1) # Selection vars
X1 <- rnorm(N_heck); X_matrix_outcome <- cbind(1, X1) # Outcome vars
gamma_true <- c(0.5, -1); beta_true <- c(1, 2); sigma_true <- 1.5; rho_true <- 0.6

# Simulate errors
errors <- MASS::mvrnorm(N_heck, mu=c(0,0), Sigma=matrix(c(1, rho_true*sigma_true, rho_true*sigma_true, 1), 2, 2))
u_i <- errors[,1]
epsilon_i <- errors[,2]

z_star <- W_matrix %*% gamma_true + u_i
Z_select <- ifelse(z_star > 0, 1, 0)

Y_outcome <- X_matrix_outcome %*% beta_true + epsilon_i
Y_observed <- ifelse(Z_select == 1, Y_outcome, NA) # Only observe Y if Z_select is 1
df_heckman <- data.frame(Y_observed, X1, W1, Z_select)
df_heckman_obs <- subset(df_heckman, Z_select == 1) # data for outcome eq.

```

```

# a + b
fHeckmanNegLogLik <- function(vParams, dfData, mSelectionVars, mOutcomeVars) {
  #vParams contains all parameters.
  #dfData should contain the outcome y, the selection indicator z, and covariates.
  #mSelectionVars and mOutcomeVars are column names/indices for w and x.
  vGamma <- vParams[1:2]
  vBeta <- vParams[3:4]
  dSigma <- exp(vParams[5])
  dRho <- (exp(vParams[6]) - exp(-vParams[6])) / (exp(vParams[6]) + exp(-vParams[6]))

  vY <- dfData[, mOutcomeVars]
  vZ <- dfData[, mSelectionVars]
  vX <- cbind(1, dfData$X1)
  vW <- cbind(1, dfData$W1)
  dLlSum <- 0
  for (i in 1:length(vY)) {
    if (vZ[i] == 0) {
      dLlSum <- dLlSum + log(1 - pnorm(vW %*% vGamma)[i])
    } else {
      dLlSum <- dLlSum + log(1 / dSigma * dnorm((vY[i] - (vX %*% vBeta)[i]) / dSigma) * pnorm(
    }
  }
  return(-dLlSum)
}

# c
vParams <- optim(c(c(0, 0), c(0, 0), 0, 0), fHeckmanNegLogLik, dfData = df_heckman, mSelectionVars = mSelectionVars, mOutcomeVars = mOutcomeVars)

# d
gamma_star <- vParams[1:2]
beta_star <- vParams[3:4]
sigma_star <- exp(vParams[5])
rho_star <- (exp(vParams[6]) - exp(-vParams[6])) / (exp(vParams[6]) + exp(-vParams[6]))

print(paste0("Gamma est: ", gamma_star, " vs. true: ", gamma_true))
#> [1] "Gamma est: 0.434869076735787 vs. true: 0.5"
#> [2] "Gamma est: -1.17799006585571 vs. true: -1"
print(paste0("Beta est: ", beta_star, " vs. true: ", beta_true))
#> [1] "Beta est: 0.964253846625409 vs. true: 1"
#> [2] "Beta est: 1.921924920477 vs. true: 2"
print(paste0("Sigma est: ", sigma_star, " vs. true: ", sigma_true))
#> [1] "Sigma est: 1.53499526849203 vs. true: 1.5"

```

```
print(paste0("Rho est: ", rho_star, " vs. true. ", rho_true))
#> [1] "Rho est: 0.597269354835849 vs. true. 0.6"
```

47.4 (4) Vector Autoregression (VAR) Likelihood (Multivariate Normal) (Hard)

- Consider a VAR(1) model for two variables y_{1t}, y_{2t} : $\mathbf{y}_t = \mathbf{c} + A\mathbf{y}_{t-1} + \epsilon_t$, where $\mathbf{y}_t = \begin{pmatrix} y_{1t} \\ y_{2t} \end{pmatrix}$, $\mathbf{c} = \begin{pmatrix} c_1 \\ c_2 \end{pmatrix}$, $A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$, and $\epsilon_t \sim N(\mathbf{0}, \Sigma)$, with $\Sigma = \begin{pmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{21} & \sigma_{22} \end{pmatrix}$ being symmetric positive definite.
- The conditional log-likelihood for one observation $\mathbf{y}_t$ given $\mathbf{y}_{t-1}$ is: $LL_t = -\frac{k}{2} \log(2\pi) - \frac{1}{2} \log(|\Sigma|) - \frac{1}{2} (\mathbf{y}_t - \mathbf{c} - A\mathbf{y}_{t-1})' \Sigma^{-1} (\mathbf{y}_t - \mathbf{c} - A\mathbf{y}_{t-1})$, where $k = 2$ (number of variables).
- Parameters: $\mathbf{c}$ (2 params), A (4 params), and unique elements of Σ (3 params: $\sigma_{11}, \sigma_{22}, \sigma_{12}$ as $\sigma_{21} = \sigma_{12}$). Total 9 parameters.
- Constraints: Σ must be positive definite. (Often ensure by parameterizing Cholesky factor L s.t. $\Sigma = LL'$).
- Data: Use `Canada` dataset from `vars` package, first two variables `e` and `prod`, differenced.

```
library(vars)
data(Canada)
Canada_diff <- data.frame(apply(Canada[,c("e","prod")], 2, diff))
Y_var_data <- as.matrix(Canada_diff)
T_var <- nrow(Y_var_data) - 1 # Number of observations for LL sum
```

(Alternatively, simulate VAR data)

- a) Write an R function `fVAR1NegLogLik(vParams, mY)` for the negative log-likelihood. `vParams` needs to be carefully unpacked into $\mathbf{c}, A, \Sigma$.
- b) For Σ , parameterize its Cholesky factor $L = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix}$. To ensure $l_{11} > 0, l_{22} > 0$, use $l_{11} = \exp(\tilde{l}_{11}), l_{22} = \exp(\tilde{l}_{22})$. l_{21} is unconstrained. $\Sigma = LL'$. The parameters in `vParams` related to Σ will be $(\tilde{l}_{11}, l_{21}, \tilde{l}_{22})$.
- c) Use `optim()` to estimate the reparameterized parameters. This will be slow and sensitive to starting values. (OLS estimates can be good starting values for $\mathbf{c}$ and A).
- d) (Conceptual) How would you derive the gradient for this likelihood? (Matrix calculus needed).

- e) (C++ RcppArmadillo) Implement the VAR(1) negative log-likelihood calculation in C++. This would involve matrix operations for which Armadillo is well-suited (`arma::solve`, `arma::log_det`).

```
# data
suppressMessages(library(vars))
#> Warning: pakke 'vars' blev bygget under R version 4.3.3
#> Warning: pakke 'strucchange' blev bygget under R version 4.3.3
#> Warning: pakke 'zoo' blev bygget under R version 4.3.3
#> Warning: pakke 'sandwich' blev bygget under R version 4.3.3
#> Warning: pakke 'urca' blev bygget under R version 4.3.3
#> Warning: pakke 'lmtest' blev bygget under R version 4.3.3
data(Canada)
Canada_diff <- data.frame(apply(Canada[,c("e","prod")], 2, diff))
Y_var_data <- as.matrix(Canada_diff)
T_var <- nrow(Y_var_data) - 1 # Number of observations for LL sum

# a + b
fVAR1NegLogLik <- function(vParams, mY) {
  vC <- vParams[1:2]
  mA <- matrix(vParams[3:6], 2, 2)
  mSigma <- matrix(c(exp(vParams[7]), vParams[8], 0, exp(vParams[9])), 2, 2)
  mSigma <- mSigma %*% t(mSigma)
  iK <- ncol(mY)

  dSum <- 0
  for (t in 2:nrow(mY)) {
    dSum <- dSum - iK / 2 * log(2 * pi) - 0.5 * log(det(mSigma)) - 0.5 * t(mY[t, ] - vC - mA
  }
  return(-dSum)
}

# c
start_c_var <- c(0,0)
start_A_var <- as.vector(t(diag(0.5, 2)))
start_L_tilde_var <- c(log(1), 0, log(1))
start_params_var <- c(start_c_var, start_A_var, start_L_tilde_var)
optim_res_var <- optim(start_params_var, fVAR1NegLogLik, mY = Y_var_data, method = "L-BFGS-B")
optim_res_var$par
#> [1] 0.10033694 0.06371892 0.67883561 0.12431749 0.19044272 0.27667404
#> [7] -0.96917510 0.01153122 -0.38346578
```

```

double fVAR1NegLogLik(vec vParams, mat mY) {
    vec vC = zeros<vec>(2);
    vC[0] = vParams[0];
    vC[1] = vParams[1];
    mat mA(2,2);
    mA(0,0) = vParams[2];
    mA(1,0) = vParams[3];
    mA(0,1) = vParams[4];
    mA(1,1) = vParams[5];
    mat mSigma(2,2);
    mSigma(0,0) = exp(vParams[6]);
    mSigma(0,1) = 0;
    mSigma(1,0) = vParams[7];
    mSigma(1,1) = exp(vParams[8]);
    mSigma = mSigma * mSigma.t();
    double iK = mY.n_cols;
    double dPi = atan(1)*4;

    double dSum = 0.0;
    for (arma::uword t = 1; t < mY.n_rows; t++) {
        vec u_t = mY.row(t).t() - vC - mA * mY.row(t-1).t();
        dSum = dSum - iK / 2.0 * log(2.0 * dPi) - 0.5 * log(det(mSigma)) - 0.5 * as_scalar(trans
    }

    return(-dSum);
}

```

```

suppressMessages(library(Rcpp))
suppressMessages(library(RcppArmadillo))
sourceCpp("likelihood_exercises.cpp")

```

48 Exercise Set 17: Integrated Econometric Modeling Problems

48.1 (1) Logistic Regression: Full Implementation (Hard)

- A Logistic model for a binary outcome $y_i \in \{0, 1\}$ is $P(y_i = 1 | \mathbf{x}_i) = \Lambda(\mathbf{x}_i' \beta) = \frac{\exp(\mathbf{x}_i' \beta)}{1 + \exp(\mathbf{x}_i' \beta)}$.
- The log-likelihood is $LL(\beta) = \sum_{i=1}^n [y_i (\mathbf{x}_i' \beta) - \log(1 + \exp(\mathbf{x}_i' \beta))]$.
- Data (same as Probit example):

```
set.seed(123)
N_log <- 200
X1_log <- rnorm(N_log)
X2_log <- runif(N_log)
X_matrix_log <- cbind(1, X1_log, X2_log)
beta_true_log <- c(-0.5, 1.2, -0.8)
linear_predictor_log <- X_matrix_log %*% beta_true_log
prob_y1_log <- exp(linear_predictor_log) / (1 + exp(linear_predictor_log))
Y_binary_log <- rbinom(N_log, 1, prob_y1_log)
```

- **Tasks:**

- a) **R Likelihood:** Write an R function `fLogisticNegLogLik(vBeta, mX, vY)` for the negative log-likelihood.
- b) **R Gradient & Hessian:** Analytically derive the gradient vector and Hessian matrix of the log-likelihood.
 - * Gradient: $\nabla LL(\beta) = \sum_{i=1}^n (y_i - \Lambda(\mathbf{x}_i' \beta)) \mathbf{x}_i$.
 - * Hessian: $H(\beta) = -\sum_{i=1}^n \Lambda(\mathbf{x}_i' \beta)(1 - \Lambda(\mathbf{x}_i' \beta)) \mathbf{x}_i \mathbf{x}_i'$. Write R functions `fLogisticGradient(vBeta, mX, vY)` and `fLogisticHessian(vBeta, mX, vY)`. For the Hessian, return the negative of it if your Newton-Raphson is set up for maximization using $H^{-1}g$.
- c) **R Newton-Raphson:** Implement the Newton-Raphson algorithm (for maximization) using your functions from (a)-(b) to find the MLEs for β . Your Newton function should take the target function (negative LL for minimization, or LL for maximization), gradient, Hessian, starting values, tolerance, and max

iterations. It should return a list with optimal parameters, function value, and convergence status.

- d) **optim() Comparison:** Use `optim()` with method “BFGS” and your negative log-likelihood from (a) and gradient from (b) (negated if `optim` minimizes). Compare the results and number of iterations/evaluations with your Newton-Raphson.
- e) **C++ Likelihood:** Implement `fLogisticNegLogLik` in C++ (`logistic_neg_log_lik_cpp`) using `RcppArmadillo`. Use `arma::log1p(exp(Xb))` for $\log(1 + \exp(\mathbf{x}_i'\beta))$ for numerical stability if $\mathbf{x}_i'\beta$ is large. Compare speed of one evaluation.
- f) **R Package:** Create a small R package named “MyLogistic” containing your R functions `fLogisticNegLogLik`, `fLogisticGradient`, `fLogisticHessian`, your Newton-Raphson solver, and the C++ likelihood function (with an R wrapper). Document the main estimation function (e.g., your Newton-Raphson solver). Ensure C++ dependencies are correctly specified in `DESCRIPTION`. Build and test the package.

```
# data
set.seed(123)
N_log <- 200
X1_log <- rnorm(N_log)
X2_log <- runif(N_log)
X_matrix_log <- cbind(1, X1_log, X2_log)
beta_true_log <- c(-0.5, 1.2, -0.8)
linear_predictor_log <- X_matrix_log %*% beta_true_log
prob_y1_log <- exp(linear_predictor_log) / (1 + exp(linear_predictor_log))
Y_binary_log <- rbinom(N_log, 1, prob_y1_log)

#a
fLogisticLogLik <- function(vBeta, mX, vY) {
  dSum <- 0
  for (i in 1:length(vY)) {
    dSum <- dSum + (vY[i] * (mX %*% vBeta)[i] - log(1 + exp(mX %*% vBeta)[i]))
  }
  return(dSum)

  # alternatively
  #return(-sum(vY * mX %*% vBeta - log(1 + exp(mX %*% vBeta))))
}

#b
fLogisticGradient <- function(vBeta, mX, vY) {
  dProb <- exp(mX %*% vBeta) / (1 + exp(mX %*% vBeta))
```

```

dSum <- numeric(length(vBeta))
for (i in 1:length(vY)) {
  dSum <- dSum + (vY[i] - dProb[i]) * mX[i, ]
}
return(dSum)

# alternatively
# return(t(mX) %*% (vY - dProb))
}

fLogisticHessian <- function(vBeta, mX, vY) {
  dProb <- exp(mX %*% vBeta) / (1 + exp(mX %*% vBeta))
  mHessianSum <- matrix(0, ncol(mX), ncol(mX))
  for (i in 1:length(vY)) {
    mHessianSum <- mHessianSum + dProb[i] * (1 - dProb[i]) * mX[i, ] %*% t(mX[i, ])
  }
  return(-mHessianSum)

  # alternatively
  # return(- t(mX) %*% diag(as.numeric(dProb * (1 - dProb))) %*% mX)
  # return(-crossprod(mX, mX * as.numeric(dProb * (1 - dProb))))
}

#c
fNewton <- function(f, fScore, fHessian, vY, mX, add.constant = TRUE, init.vals = NULL, max.iter = 100) {
  i <- 0
  vB <- init.vals

  # Keep updating until stopping criterion or max iterations reached
  while ((max(abs(fScore(vB, mX, vY))) > dTol) && (i < max.iter)) {
    # Newton-Raphson updating
    vB <- vB - solve(fHessian(vB, mX, vY), fScore(vB, mX, vY))
    i <- i + 1
    #cat("At iteration", n, "the value of the parameter is:", dParam, "\n")
  }

  if (i == max.iter) {
    return(list(
      beta_hat = NULL,
      log_likelihoood_opt = NULL,
      score_opt = NULL,
      hessian_opt = NULL,

```

```

    log_likelihood_null = NULL,
    #predicted_probabilities = NULL,
    iterations = i,
    msg = "Algorithm failed to converge. Maximum iterations reached.")
  )
} else {
  return(list(
    beta_hat = vB,
    log_likelihood_opt = f(vB, mX, vY),
    score_opt = fScore(vB, mX, vY),
    hessian_opt = fHessian(vB, mX, vY),
    log_likelihood_null = f(vB, mX, vY),
    #predicted_probabilities = exp(mX %*% vB) / (1 + exp(mX %*% vB)),
    iterations = i,
    msg = "Algorithm converged")
  )
}
}

fNewton(fLogisticLogLik, fLogisticGradient, fLogisticHessian, vY = Y_binary_log, mX = X_matr
#> $beta_hat
#>           X1_log      X2_log
#> -0.313394  1.609719 -0.590626
#>
#> $log_likelihood_opt
#> [1] -100.5938
#>
#> $score_opt
#>           X1_log      X2_log
#> -1.963759e-10  6.407941e-10 -1.425498e-10
#>
#> $hessian_opt
#>           X1_log      X2_log
#> [1,] -33.374735  -4.528574 -16.508856
#> [2,]  -4.528574 -16.413238  -2.941622
#> [3,] -16.508856  -2.941622 -10.913058
#>
#> $log_likelihood_null
#> [1] -100.5938
#>
#> $iterations
#> [1] 5

```

```

#>
#> $msg
#> [1] "Algorithm converged"

#d
optim(c(0, 0, 0), fLogisticLogLik, gr = fLogisticGradient, mX = X_matrix_log, vY = Y_binary_log)
#> $par
#> [1] -0.3133828  1.6097293 -0.5906087
#>
#> $value
#> [1] -100.5938
#>
#> $counts
#> function gradient
#>      15      8
#>
#> $convergence
#> [1] 0
#>
#> $message
#> NULL

```

```

double logistic_neg_log_lik_cpp(vec vBeta, mat mX, vec vY) {
  double dSum = 0.0;
  vec linear_term = mX * vBeta;
  int n = vY.size();
  for (int i = 0; i < n; i++) {
    dSum += (vY(i) * linear_term(i)) - log(1 + exp(linear_term(i)));
  }
  return(dSum);
}

f(vB, mX, vY)

```

```

suppressMessages(library(Rcpp))
suppressMessages(library(RcppArmadillo))
sourceCpp("likelihood_exercises.cpp")

fLogisticLogLik(c(-0.313394, 1.609719, -0.590626), X_matrix_log, Y_binary_log)
#> [1] -100.5938
logistic_neg_log_lik_cpp(c(-0.313394, 1.609719, -0.590626), X_matrix_log, Y_binary_log)
#> [1] -100.5938

```

48.2 (2) AR(1) Model with Student's t-errors (Extremely Hard)

- Consider an AR(1) model: $y_t = \phi y_{t-1} + \epsilon_t$, where $\epsilon_t \sim \text{i.i.d. Standardized Student's } t(\nu)$.
- The PDF of a standardized Student's t-distribution with ν degrees of freedom is $f(\epsilon_t|\nu) = \frac{\Gamma((\nu+1)/2)}{\sqrt{\pi(\nu-2)}\Gamma(\nu/2)} \left(1 + \frac{\epsilon_t^2}{\nu-2}\right)^{-(\nu+1)/2}$. (Note: σ (scale) is implicitly 1 here for the standardized version, or rather, scale is $\sqrt{(\nu-2)/\nu}$ times the scale of a non-standardized t to make variance 1. The formula given is for a t-dist scaled to have variance 1.)
- Parameters are $\theta = (\phi, \nu)$. Constraints: $|\phi| < 1$ (stationarity), $\nu > 2$ (for defined variance).
- The conditional log-likelihood for y_t given y_{t-1} is $\log f(y_t - \phi y_{t-1}|\nu)$. The total log-likelihood sums these over $t = 2, \dots, T$ (conditioning on y_1).
- Data:

```
set.seed(800)
T_ar <- 300
phi_true <- 0.7
nu_true <- 5 # Degrees of freedom
# Simulate standardized t errors (variance = 1)
errors_t <- rt(T_ar, df = nu_true) * sqrt((nu_true - 2) / nu_true)
y_ar <- numeric(T_ar)
y_ar[1] <- errors_t[1] / sqrt(1 - phi_true^2) # Start from unconditional variance
for(t_idx in 2:T_ar) {
  y_ar[t_idx] <- phi_true * y_ar[t_idx-1] + errors_t[t_idx]
}
```

- **Tasks:**
 - a) **R Likelihood:** Write an R function `fAR1tNegLogLik(vParams, vY)` for the negative log-likelihood. `vParams = c(phi, nu)`. Use `lgamma()` for $\log(\Gamma(\cdot))$.
 - b) **Reparameterization:**
 - * $\phi = \tanh(\tilde{\phi})$ (ensures $|\phi| < 1$)
 - * $\nu = 2 + \exp(\tilde{\nu})$ (ensures $\nu > 2$) Modify your likelihood to take $\tilde{\theta} = (\tilde{\phi}, \tilde{\nu})$.
 - c) **Numerical Derivatives:** Use `numDeriv::grad()` and `numDeriv::hessian()` (if allowed/available) to compute the gradient and Hessian of the *reparameterized* negative log-likelihood at a test point (e.g., $\tilde{\phi} = 0, \tilde{\nu} = \log(3)$ which means $\phi = 0, \nu = 5$).
 - d) **Custom Newton-Raphson (on reparameterized LL):** Implement Newton-Raphson using the numerical derivatives from (c) to estimate $\tilde{\theta}$. If numerical derivatives are too slow or unstable for the Hessian within the loop, you might use BFGS update for Hessian or just use `optim`.

- e) **optim() with Numerical Gradient:** Use `optim()` with method “BFGS” and your *reparameterized* negative log-likelihood. Provide the gradient numerically using `numDeriv::grad()` in the `gr` argument of `optim`.
- f) **Results:** Transform estimates back to ϕ^*, ν^* and compare with true values.
- g) **C++ Likelihood (Challenge):** Implement the core calculation $\log f(\epsilon_t | \nu)$ in C++. You will need `R::lgammafn` and `R::dt` (be careful with `R::dt`’s arguments, it might not be for standardized `t` directly, you might need to implement the PDF formula). Sum these in an outer C++ loop. Compare evaluation speed.
- h) **Simulation Study (Conceptual):** How would you set up a small Monte Carlo simulation study to evaluate the performance (bias, RMSE) of your MLE estimator for ϕ and ν using, say, 100 replications of data like the one generated? (This part is about describing the loop and storage, not full implementation unless time allows). How could parallel processing help here?

```
#data
set.seed(800)
T_ar <- 300
phi_true <- 0.7
nu_true <- 5 # Degrees of freedom
# Simulate standardized t errors (variance = 1)
errors_t <- rt(T_ar, df = nu_true) * sqrt((nu_true - 2) / nu_true)
y_ar <- numeric(T_ar)
y_ar[1] <- errors_t[1] / sqrt(1 - phi_true^2) # Start from unconditional variance
for(t_idx in 2:T_ar) {
  y_ar[t_idx] <- phi_true * y_ar[t_idx-1] + errors_t[t_idx]
}

#a + b
fAR1tNegLogLik <- function(vParams, vY) {
  dPhi <- tanh(vParams[1])
  dNu <- 2 + exp(vParams[2])
  vEps <- numeric(length(vY))

  dSum <- 0
  for (t in 2:length(vY)) {
    vEps[t] <- vY[t] - dPhi * vY[t-1]
    dSum <- dSum + lgamma((dNu+1)/2) - log(sqrt(pi * (dNu-2))) - lgamma(dNu/2) - (dNu + 1) /
  }
  return(-dSum)
}
```

```

#c
suppressMessages(library(numDeriv))
fAR1tGrad <- function(f, vParams, ...) {
  return(grad(f, vParams, ...))
}
fAR1tHess <- function(f, vParams, ...) {
  return(hessian(f, vParams, ...))
}

fAR1tGrad(fAR1tNegLogLik, c(0, log(3)), vY = y_ar)
#> [1] -353.44783 -12.69966
fAR1tHess(fAR1tNegLogLik, c(0, log(3)), vY = y_ar)
#>      [,1]      [,2]
#> [1,] 205.88262  3.80849
#> [2,]   3.80849 23.23789

#d
fNewton <- function(f, fScore, fHessian, init.vals = NULL, max.iter = 200, dTol = 1e-9, ...)
  i <- 0
  vPars <- init.vals

  while ((max(abs(fScore(f, vPars, ...))) > dTol) && (i < max.iter)) {
    vPars <- vPars - solve(fHessian(f, vPars, ...), fScore(f, vPars, ...))
    i <- i + 1
  }

  if (i == max.iter) {
    return(list(
      curr_params = vPars,
      curr_log_likelihood_opt = f(vPars, ...),
      curr_score_opt = fScore(f, vPars, ...),
      curr_hessian_opt = fHessian(f, vPars, ...),
      iterations = i,
      msg = "Algorithm failed to converge. Maximum iterations reached.")
    )
  } else {
    return(list(
      params = vPars,
      log_likelihood_opt = f(vPars, ...),
      score_opt = fScore(f, vPars, ...),
      hessian_opt = fHessian(f, vPars, ...),
      iterations = i,

```

```

    msg = "Algorithm converged")
  )
}
}

fNewton(fAR1tNegLogLik, fAR1tGrad, fAR1tHess, vY = y_ar, init.vals = c(0.2, log(3)))
#> $params
#> [1] 0.8557711 1.1124668
#>
#> $log_likelihood_opt
#> [1] 410.3974
#>
#> $score_opt
#> [1] -2.601885e-10 5.501685e-10
#>
#> $hessian_opt
#>      [,1]      [,2]
#> [1,] 200.2957152 -0.5986651
#> [2,] -0.5986651 9.6342894
#>
#> $iterations
#> [1] 92
#>
#> $msg
#> [1] "Algorithm converged"
vPars_temp <- fNewton(fAR1tNegLogLik, fAR1tGrad, fAR1tHess, vY = y_ar, init.vals = c(0.2, log(3)))

dPhi_star <- tanh(vPars_temp[1])
dNu_star <- 2 + exp(vPars_temp[2])

print(paste0("Phi-est: ", dPhi_star, " vs true: ", phi_true))
#> [1] "Phi-est: 0.694072436111299 vs true: 0.7"
print(paste0("Nu-est: ", dNu_star, " vs true: ", nu_true))
#> [1] "Nu-est: 5.04185272789097 vs true: 5"

# e
vPars_temp <- optim(c(0.2, log(3)), fn = fAR1tNegLogLik, gr = function(par, ...) { fAR1tGrad

# f
dPhi_star <- tanh(vPars_temp[1])
dNu_star <- 2 + exp(vPars_temp[2])

```

```
print(paste0("Phi-est: ", dPhi_star, " vs true: ", phi_true))
#> [1] "Phi-est: 1 vs true: 0.7"
print(paste0("Nu-est: ", dNu_star, " vs true: ", nu_true))
#> [1] "Nu-est: 6.97158263449599 vs true: 5"
```

48.3 (3) Zero-Inflated Poisson (ZIP) Regression Likelihood (Extremely Hard)

- A ZIP model assumes that zero outcomes $y_i = 0$ can come from two sources:
 1. A “certain zero” state (e.g., never uses a service), with probability π_i .
 2. A Poisson count process that happens to produce a zero, with probability $(1 - \pi_i) \cdot P(\text{Count} = 0 | \lambda_i)$.
- The probability mass function is: $P(y_i = k) = \pi_i \cdot I(k = 0) + (1 - \pi_i) \cdot \frac{e^{-\lambda_i} \lambda_i^k}{k!}$ for $k = 0, 1, 2, \dots$ where $I(k = 0)$ is an indicator function (1 if $k = 0$, 0 otherwise).
- Often, π_i and λ_i are modeled with covariates:
 - $\text{logit}(\pi_i) = \mathbf{w}_i' \gamma$ (Logistic regression for zero-inflation probability)
 - $\log(\lambda_i) = \mathbf{x}_i' \beta$ (Log-linear model for Poisson mean)
- Parameters: $\theta = (\gamma', \beta')$.
- Data Simulation:

```
set.seed(999)
N_zip <- 250
W1_zip <- rnorm(N_zip); W_matrix_zip <- cbind(1, W1_zip) # For pi
X1_zip <- runif(N_zip); X_matrix_zip_lambda <- cbind(1, X1_zip) # For lambda

gamma_true_zip <- c(-1, 0.8) # logit(pi) params
beta_true_zip <- c(0.5, 1.5) # log(lambda) params

logit_pi_vals <- W_matrix_zip %*% gamma_true_zip
pi_vals <- exp(logit_pi_vals) / (1 + exp(logit_pi_vals))

log_lambda_vals <- X_matrix_zip_lambda %*% beta_true_zip
lambda_vals <- exp(log_lambda_vals)

Y_zip <- numeric(N_zip)
for(i_zip in 1:N_zip) {
  if (runif(1) < pi_vals[i_zip]) {
    Y_zip[i_zip] <- 0 # Certain zero
```

```

} else {
  Y_zip[i_zip] <- rpois(1, lambda_vals[i_zip]) # From Poisson
}
}
# hist(Y_zip, breaks=-0.5:(max(Y_zip)+0.5)) # Lots of zeros?

```

• **Tasks:**

- a) **R Likelihood:** Write an R function `fZIPNegLogLik(vParams, vY, mW, mX)` for the negative log-likelihood. `vParams` will contain both γ and β . You need to unpack them carefully.
- b) **Optimization:** Use `optim()` (“BFGS” or “Nelder-Mead”) to find the MLEs for γ and β . This is complex; good starting values are crucial (e.g., γ from a logit model predicting $y_i = 0$ vs $y_i > 0$, and β from a Poisson regression on $y_i[y_i > 0]$).
- c) **C++ Likelihood (Point calculation):** Implement a C++ function `zip_log_lik_point_cpp(int yi, double logit_pi_i, double log_lambda_i)` that calculates the log-likelihood for a single observation y_i , given its specific logit(π_i) and $\log(\lambda_i)$. Use `R::pnorm` (for logit to prob), `R::dpois`.
- d) **Full C++ Likelihood (Loop):** Write a C++ function `fZIPNegLogLikCpp(arma::vec vGamma, arma::vec vBeta, arma::ivec vY, arma::mat mW, arma::mat mX)` that calculates the total negative log-likelihood by iterating through observations and calling a C++ point likelihood (or calculating directly). This will require computing $\mathbf{w}_i'\gamma$ and $\mathbf{x}_i'\beta$ for each i .
- e) **Comparison:** Compare the speed of a single evaluation of your R and C++ full likelihood functions.
- f) **Package (Conceptual):** If you were to package this, what would be the main user-facing R function? What arguments would it take? What would it return? How would you handle the C++ code?

```

#data
set.seed(999)
N_zip <- 250
W1_zip <- rnorm(N_zip); W_matrix_zip <- cbind(1, W1_zip) # For pi
X1_zip <- runif(N_zip); X_matrix_zip_lambda <- cbind(1, X1_zip) # For lambda

gamma_true_zip <- c(-1, 0.8) # logit(pi) params
beta_true_zip <- c(0.5, 1.5)  # log(lambda) params

logit_pi_vals <- W_matrix_zip %*% gamma_true_zip
pi_vals <- exp(logit_pi_vals) / (1 + exp(logit_pi_vals))

```

```

log_lambda_vals <- X_matrix_zip_lambda %%% beta_true_zip
lambda_vals <- exp(log_lambda_vals)

Y_zip <- numeric(N_zip)
for(i_zip in 1:N_zip) {
  if (runif(1) < pi_vals[i_zip]) {
    Y_zip[i_zip] <- 0 # Certain zero
  } else {
    Y_zip[i_zip] <- rpois(1, lambda_vals[i_zip]) # From Poisson
  }
}

#a
fZIPNegLogLik <- function(vParams, vY, mW, mX) {
  vGamma <- vParams[1:ncol(mW)]
  vBeta <- vParams[(ncol(mW) + 1):(ncol(mW) + ncol(mX)))]

  logit_pi_i <- mW %%% vGamma
  pi_i <- plogis(logit_pi_i)
  log_lambda_i <- mX %%% vBeta
  lambda_i <- exp(log_lambda_i)

  dSum <- 0
  for (i in 1:length(vY)) {
    if (vY[i] == 0) {
      dSum <- dSum + log(pi_i[i] + (1 - pi_i[i]) * ((exp(-lambda_i[i]) * lambda_i[i]^vY[i]) / factorial(vY[i]))
    } else {
      dSum <- dSum + log((1 - pi_i[i]) * ((exp(-lambda_i[i]) * lambda_i[i]^vY[i]) / factorial(vY[i]))
    }
  }
  return(-dSum)
}

optim(c(0, 0, 0, 0), fZIPNegLogLik, vY = Y_zip, mW = W_matrix_zip, mX = X_matrix_zip_lambda,
#> $par
#> [1] -1.0449956  0.6241883  0.5144958  1.5029647
#>
#> $value
#> [1] 490.5832
#>
#> $counts
#> function gradient

```

```
#>      26      10
#>
#> $convergence
#> [1] 0
#>
#> $message
#> NULL
```